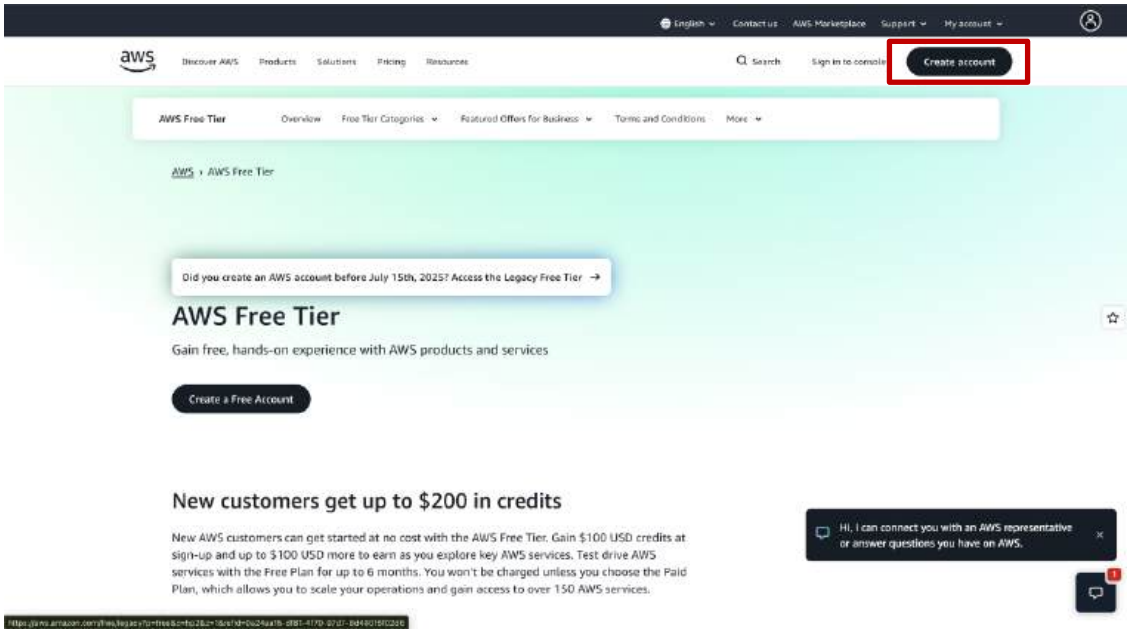
ASSIGNMENT 1

Problem Statement: Create an account in AWS and configure a budget.

Procedure: (a) Create an account in AWS

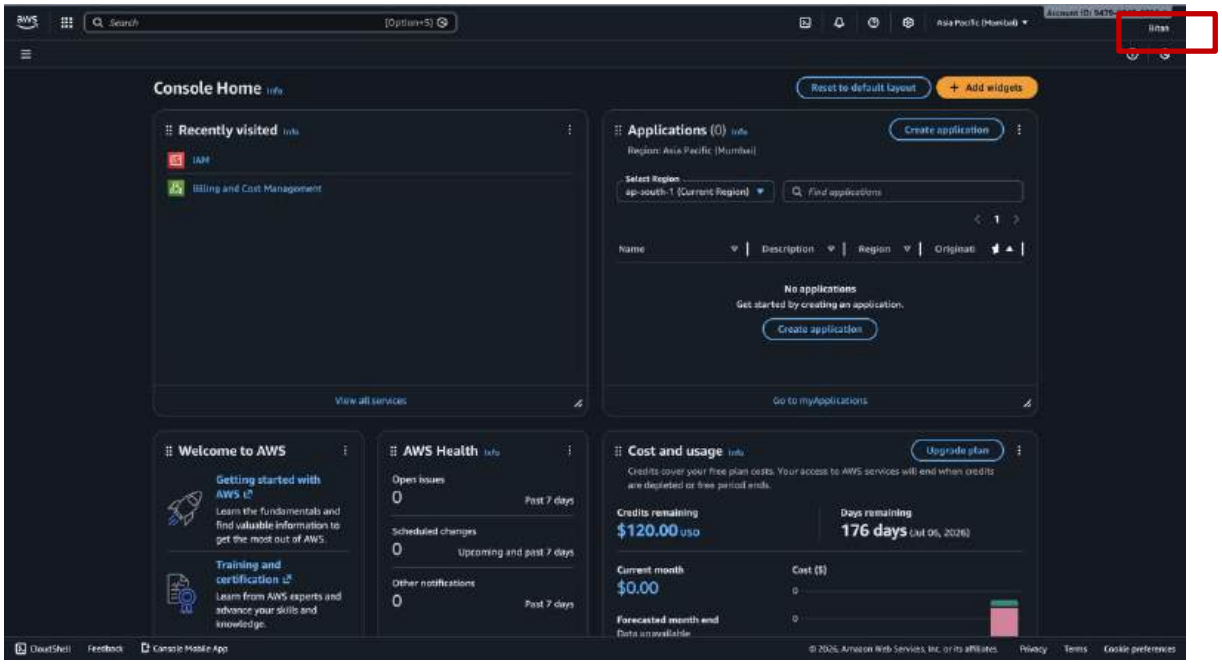


1. Open the Amazon Web Services home page .
2. Choose **Create an AWS account**. Make sure to create the account as a R**oot user** to unlock full functionality of the newly created AWS account.
3. Enter our account information, and then choose Verify Email . Be sure that we enter our account information correctly, especially email address.
4. Choose **personal** account.
5. Enter our personal information.
6. Read and accept the **AWS Customer Agreement**.
7. Choose Create Account and Continue.
8. On the Payment Information page we can use card payment or upi , and then choose **Verify and Add**. We can't proceed with the sign-up process until we do valid payment method.
9. Next, we must **verify our phone number**. Choose our country or region code from the list, and enter a phone number .
10. Enter the code displayed in the CAPTCHA, and then submit.
11. On the **Select a Support Plan** page, **choose the free tier**.
12. Finally, wait for your new account to be **activated**. This usually takes a few minutes but can take up to **24 hours**.
13. When our account is **fully activated**,we receive a **confirmation email message**. Check email and spam folder for the confirmation message. **After receive this email message, we have full access to all AWS services**.

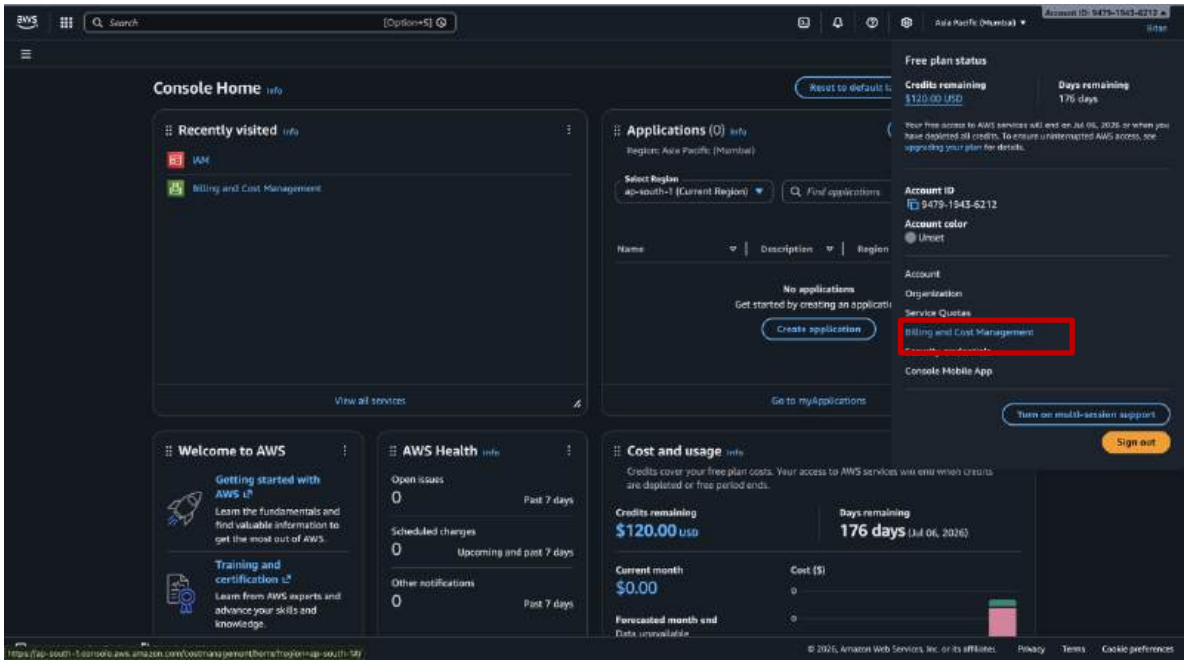
(b)Configure a Budget

1. Sign in to the console using root email for newly created account.

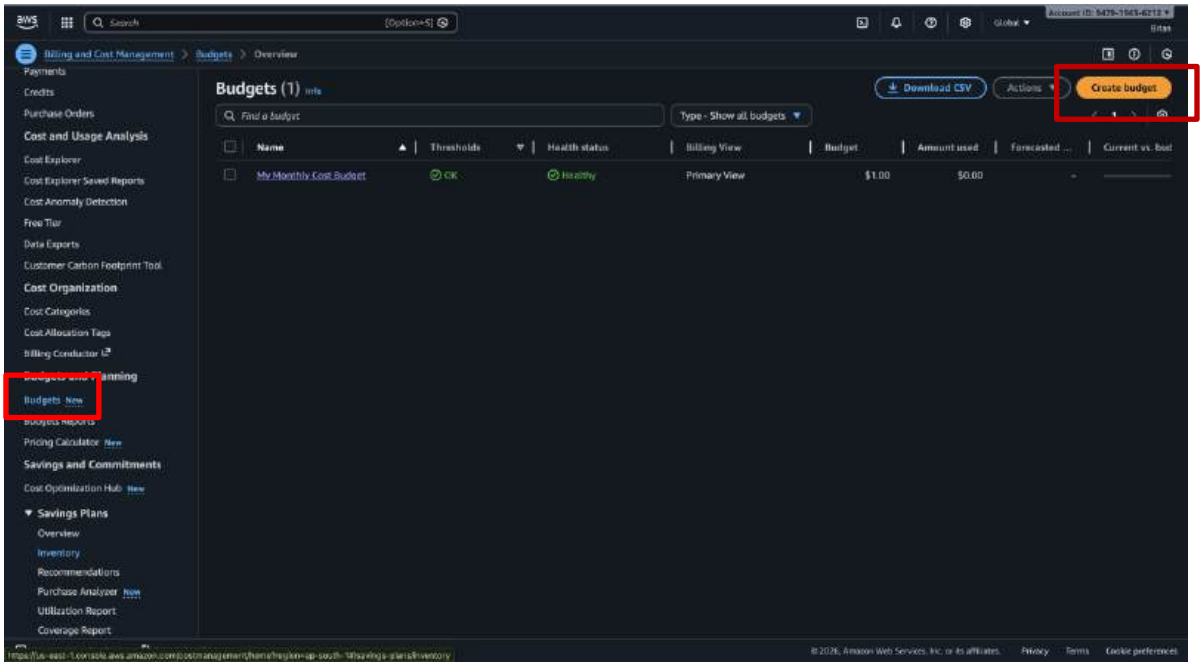
2. Now after successfully signing in we will arrive at the home page. Then click on down arrow beside your account name on the top right corner of the home page.



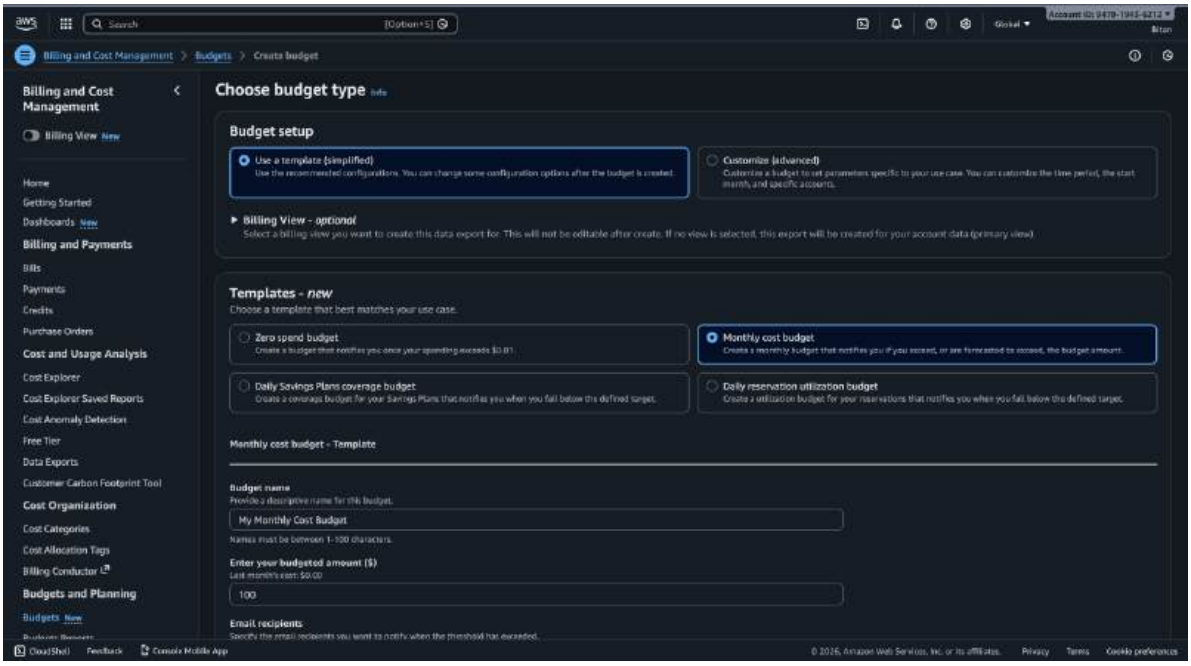
3. A drop down will appear and select **Billing Dashboard** from the list.



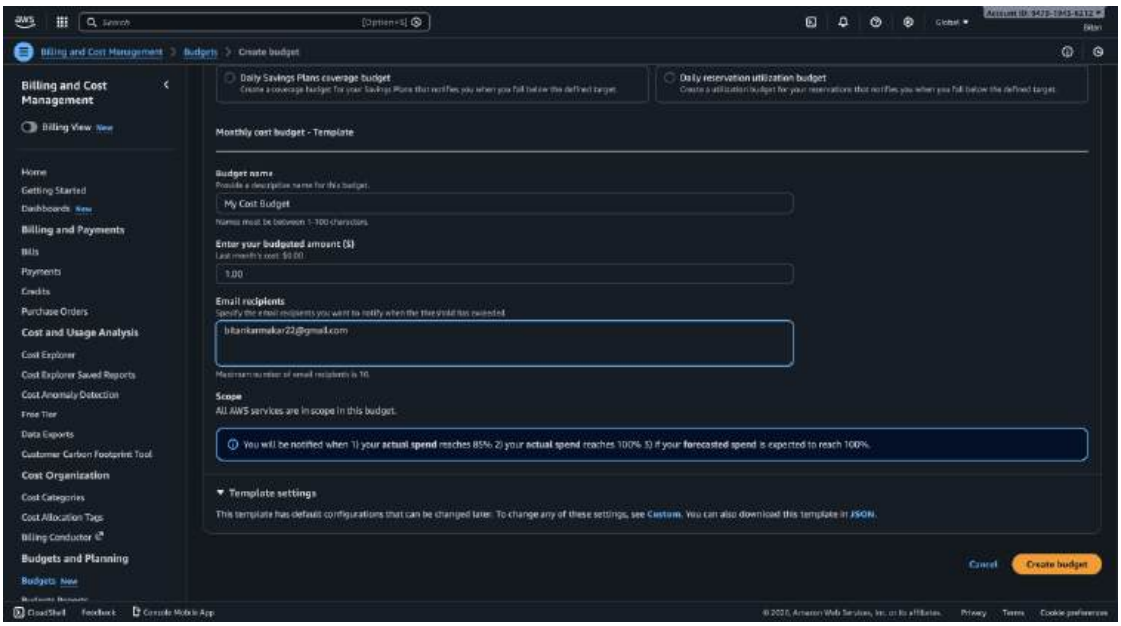
4. After arriving in Billing Dashboard, go to the **Budgets** section on the left side panel under cost management. Click on the **Create Budget** button in the **overview page** to start creating a new budget for your AWS account.



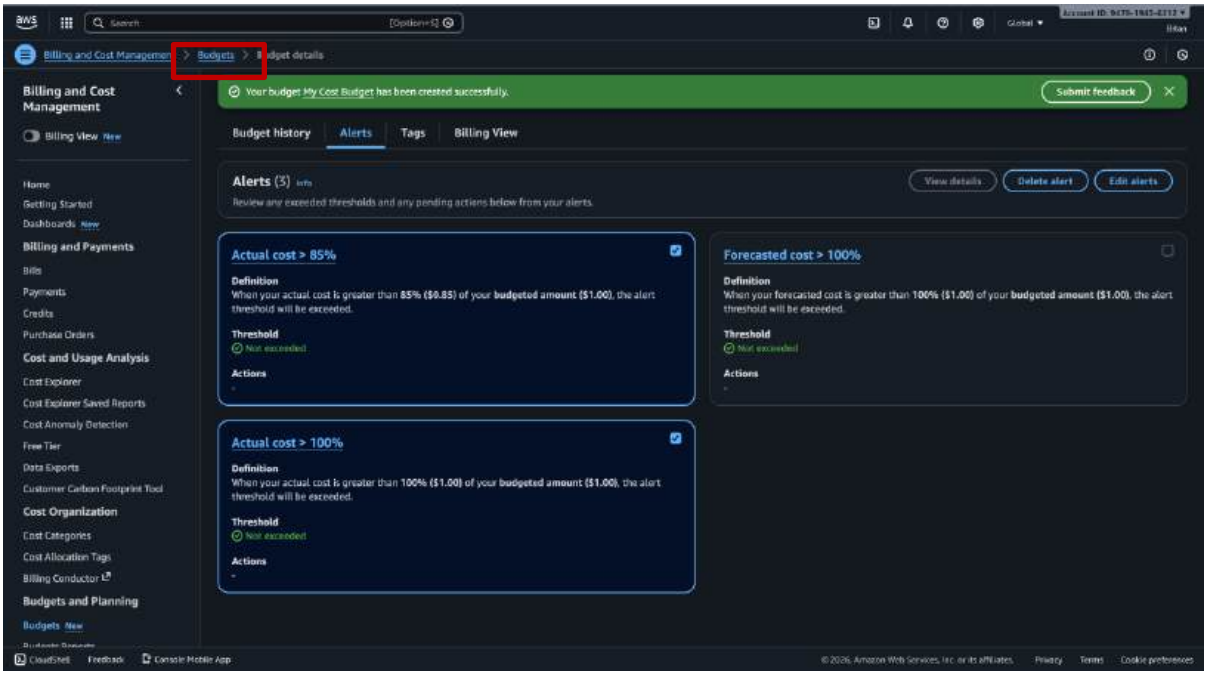
5. Select Use a template(simplified) in Budget Setup section and Monthly **Cost Budget** in the Templates-new section.



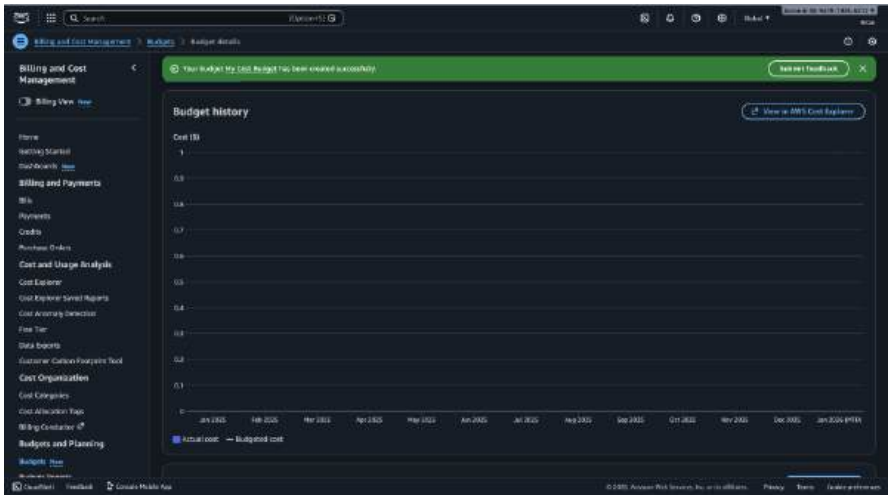
6. Enter **Budget Name** and Enter your Budgeted amount. For best free usage enter 1 in the box. (All amounts are automatically considered in dollars “$”)No need to change any other options in the page. After that enter the Email recipients to recieve email about my budget . So, now we click on Create budget.

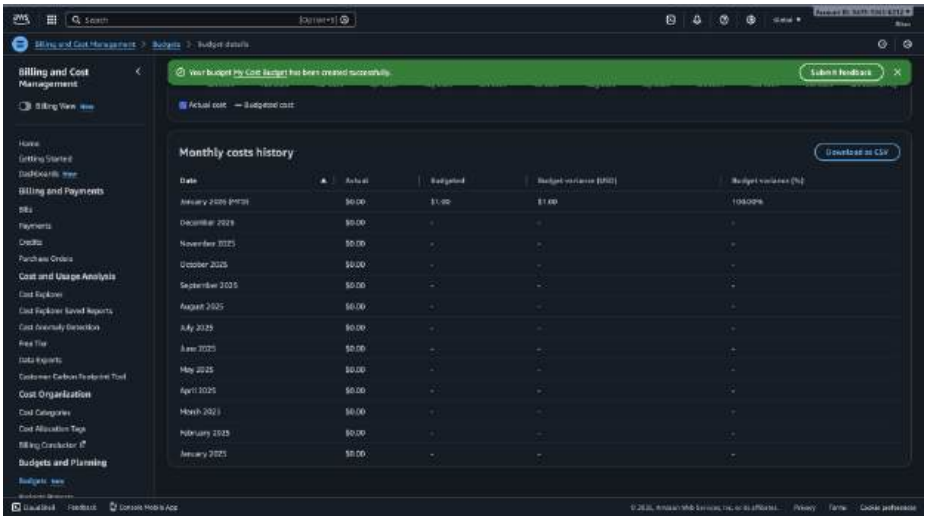


7. Next, again we click the My Cost Budget . Then go to Alerts section and select Actual cost > 85% and Actual cost > 100%. We can also select the Forecasted cost > 100% . Then click the Budgets to exit from that .

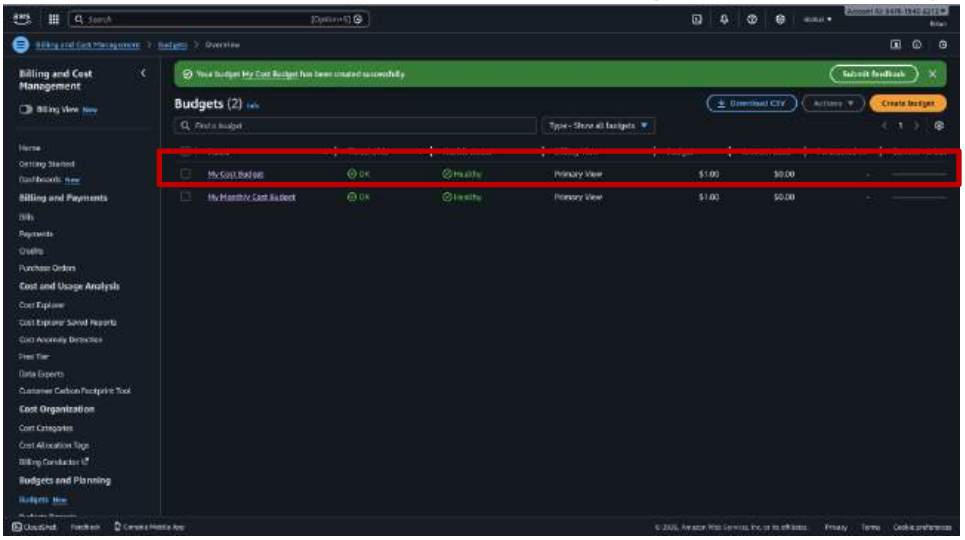


8. Again, click on My Cost Budget and enter into it .
9. Now, we are finally at the review or summary page of the whole budget . We can see the Budget history and Monthly cost history





10. After this step we will be redirected to the **overview page** where all budgets will be shown. We can see your newly created budget in the table format with various information related to it. Our Budget creation is complete!

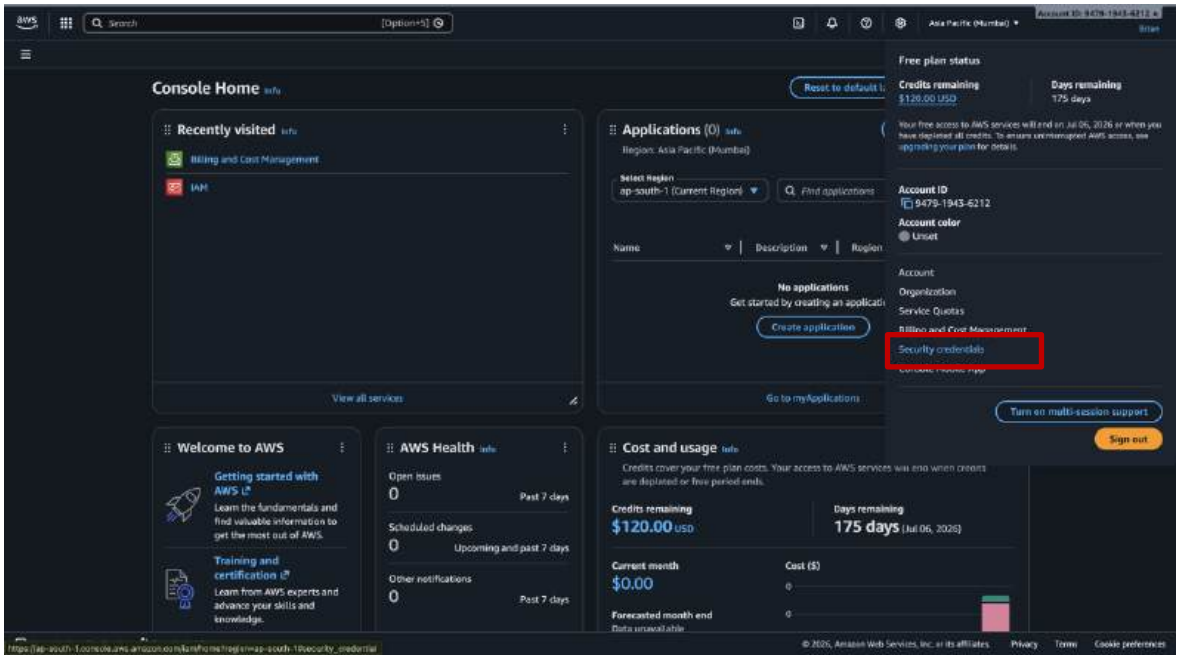


ASSIGNMENT 2

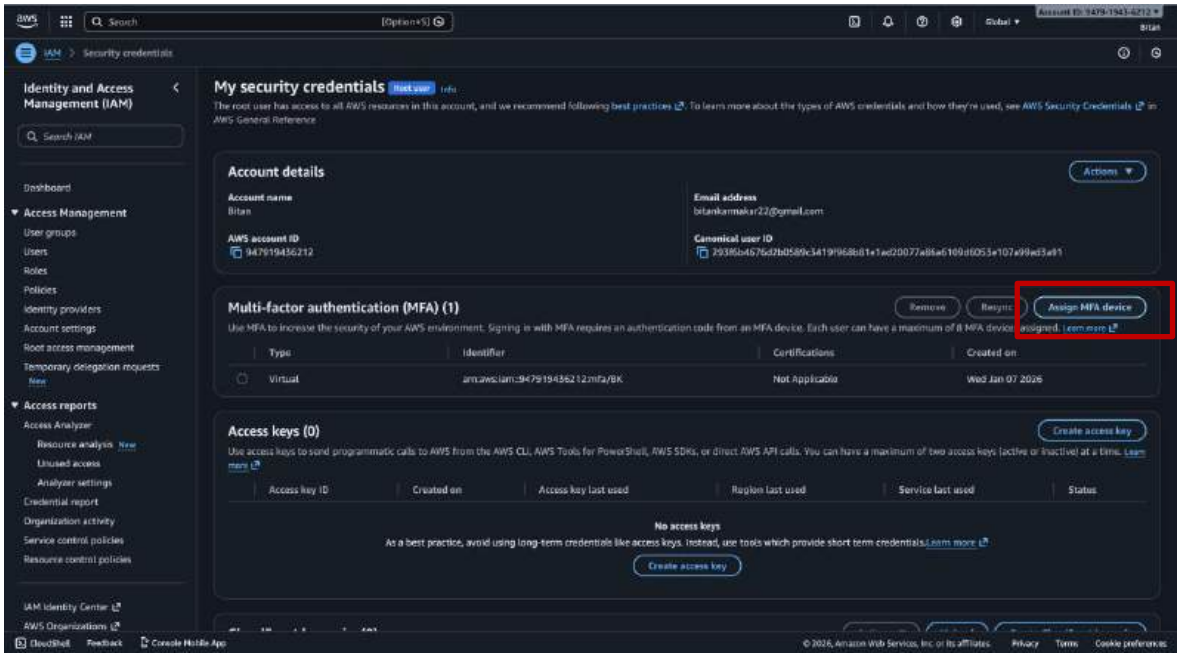
Problem Statement: Create Multi Factor Authentication(MFA) for account authentication.

Procedure:

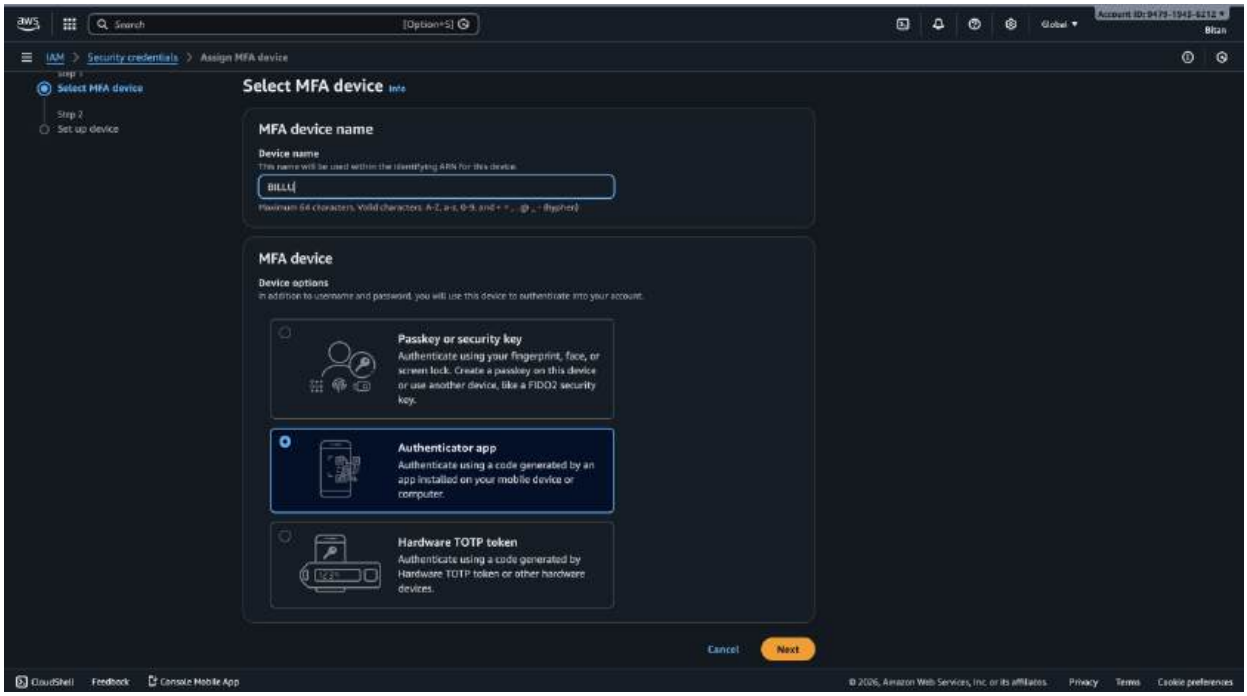
1. Sign-in to AWS console. Then select your account name in the top right of the page. Now select the Security credentials option in the drop-down menu .



2. Now after arriving in My Security Credentials page, now scroll down to **Multi-Factor Authentication (MFA)** section. Click on the **Assign MFA device** button.

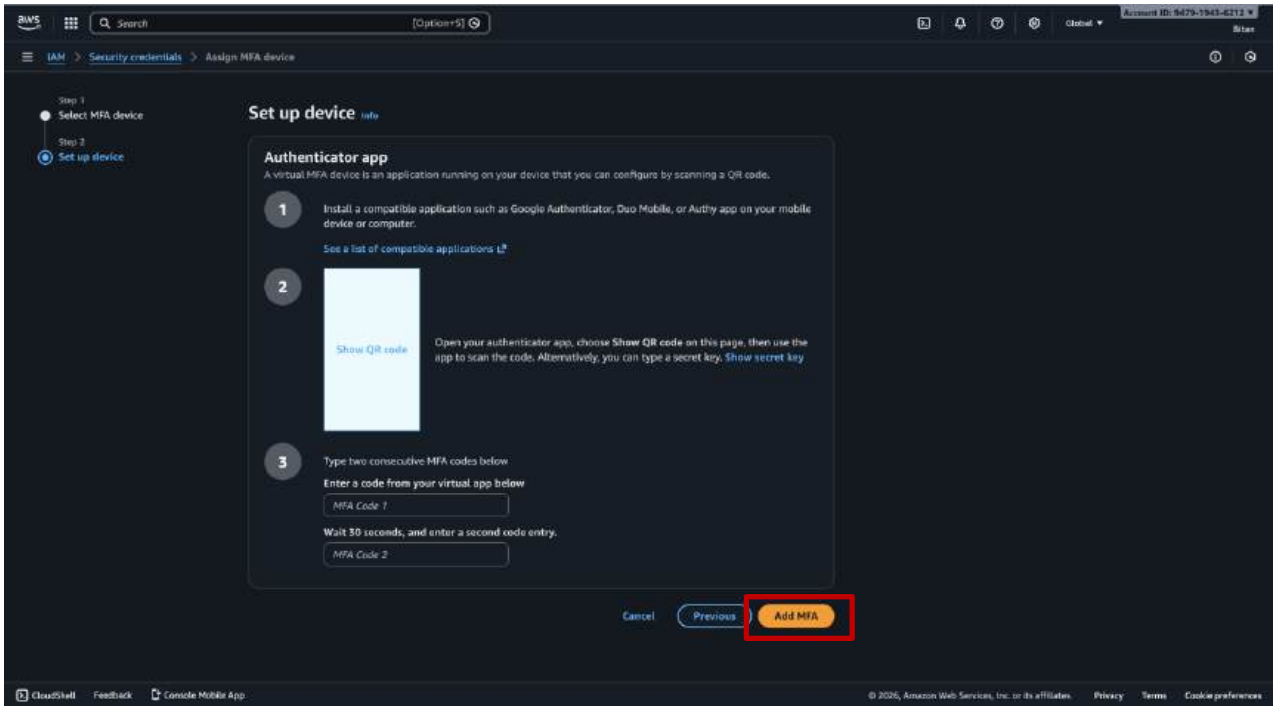


3. Next Assign a **unique device name** (very important and needs to be unique). This name is actually the name of the account that shows up in your authenticator app with the security codes and hence is essential for identification later. Select **Authenticator app** in **Select MFA device** section.

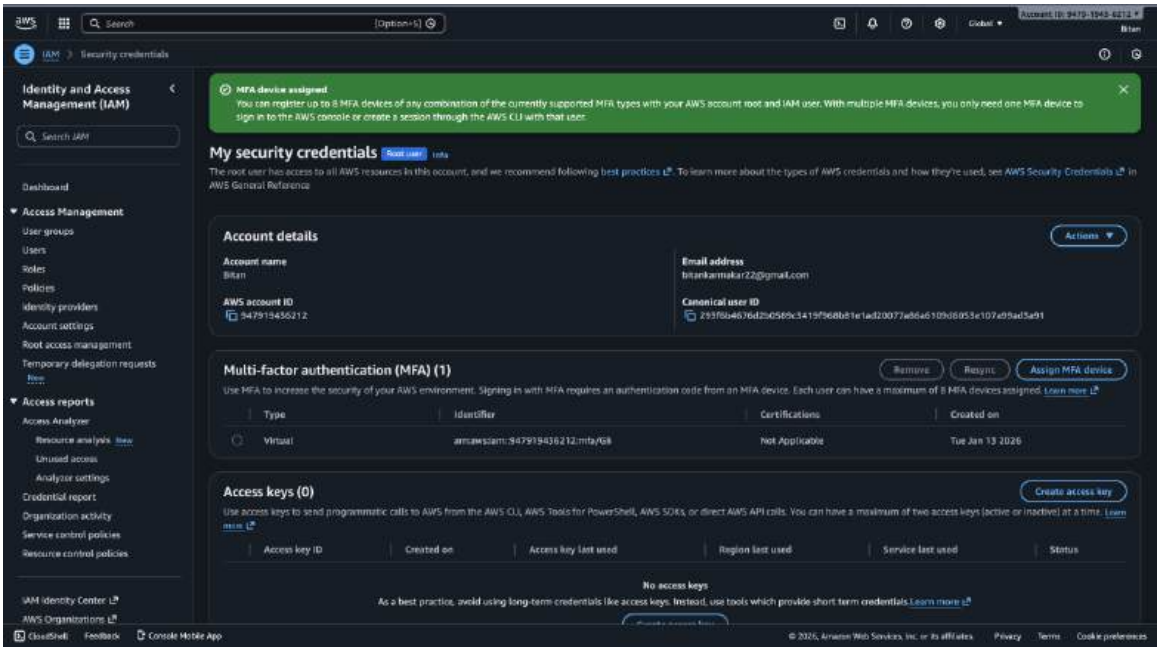


4. Click on **Next.**

5. After that you have to make sure you download an authenticator app from in your android/iOS device. Google/Microsoft authenticator is preferred. After installation, click on the **Show QR code box** here in the website. Scan the QR code with your authenticator app. Your device name given will show up in your authenticator main page.We will see a certain unique combination appearing against our given device name for our AWS account and stays only for 30-60 seconds. Enter 2 consecutive codes appearing in the given box in the website to authenticate your MFA. After successfully verifying your MFA, click on the Add MFA button.



6. You will be redirected to the security credentials page and see your newly added MFA method with your given device name for your account.



7. Hence, we have successfully added an MFA device. Now sign-out and re-login to the console using Root user.

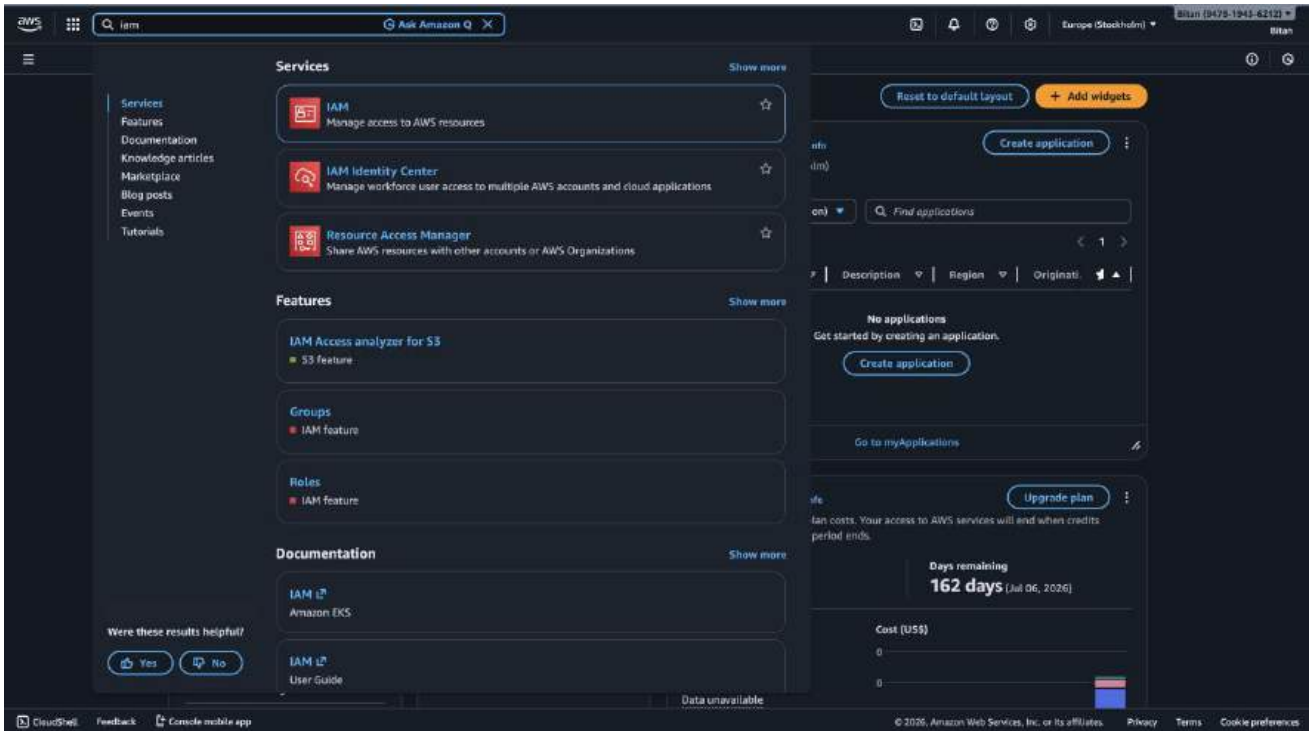
8. Now after providing user email and password, from now on you have to enter the MFA code. The code changes every 30-60 seconds and we have to enter the current or existing one which has not expired.

ASSIGNMENT 3

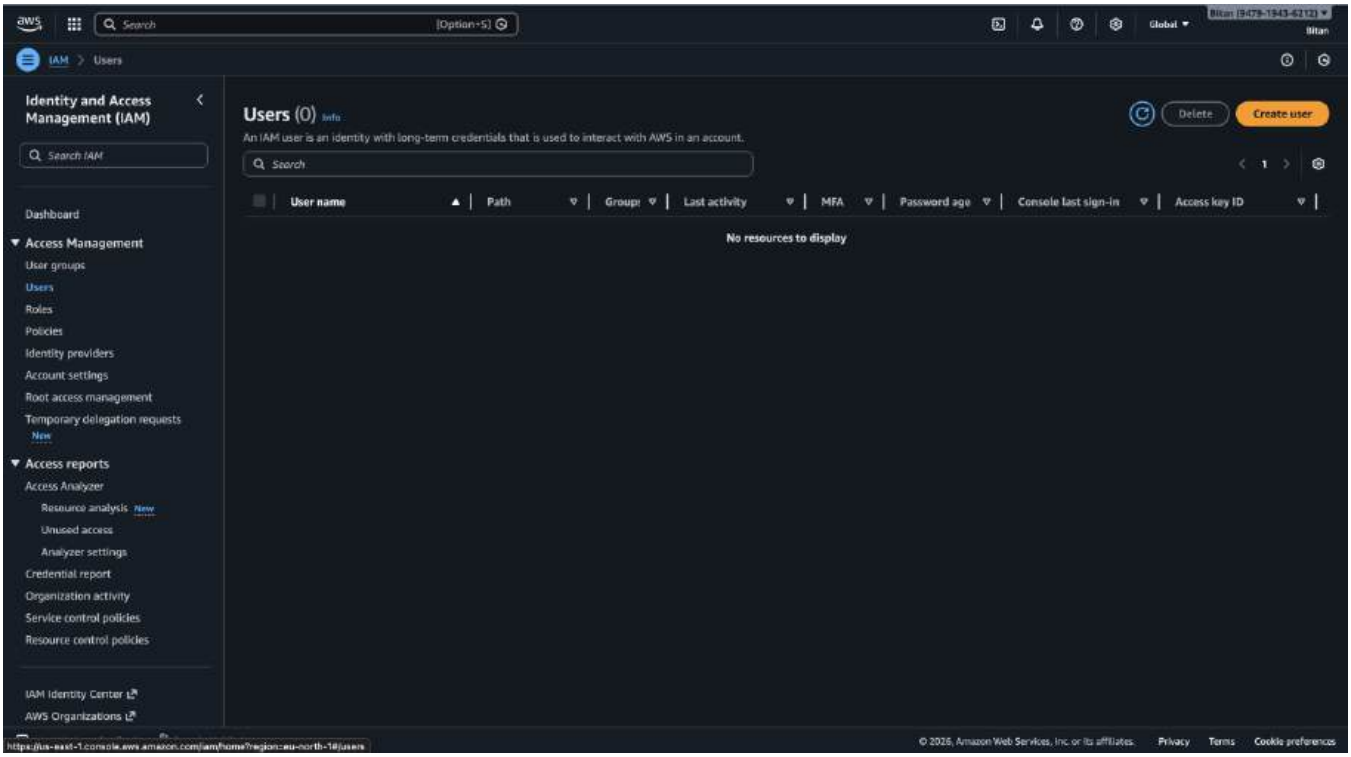
Problem Statement: Creating an IAM User and give full S3 access.

Procedure:

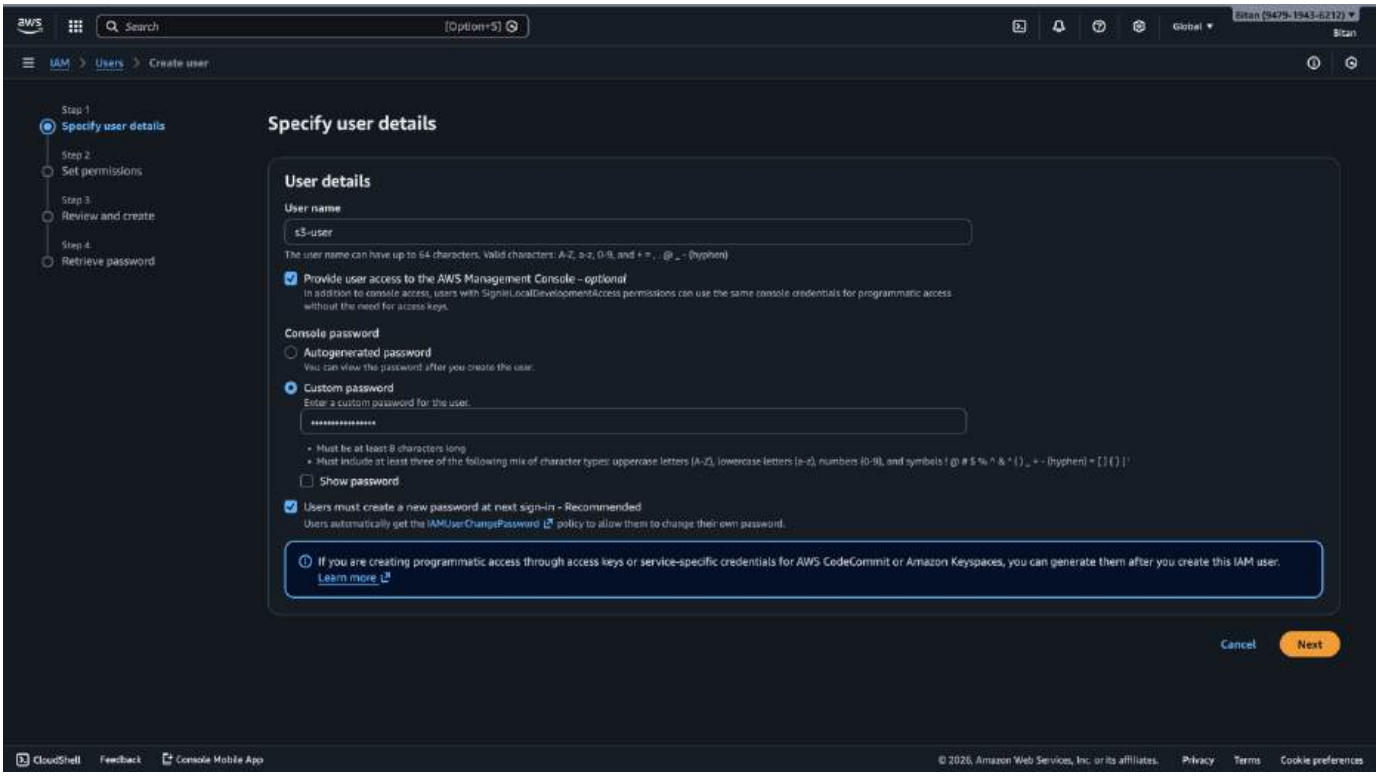
Step 1: Log in to the **AWS Management Console** as the Root user (or an Admin).Search for and select **IAM**



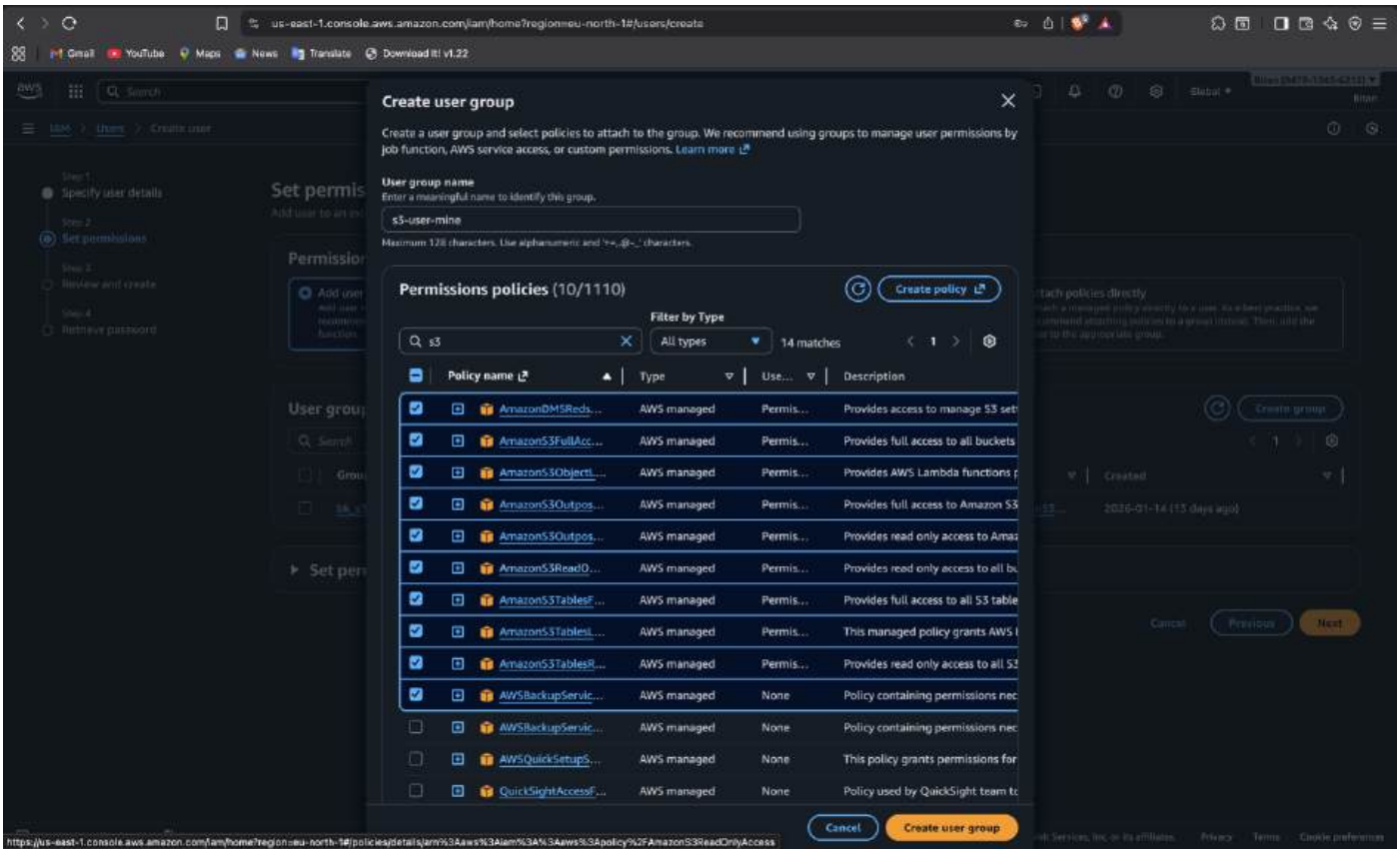
Step 2: In the left sidebar, click **Users**. Click the orange **Create user** button.

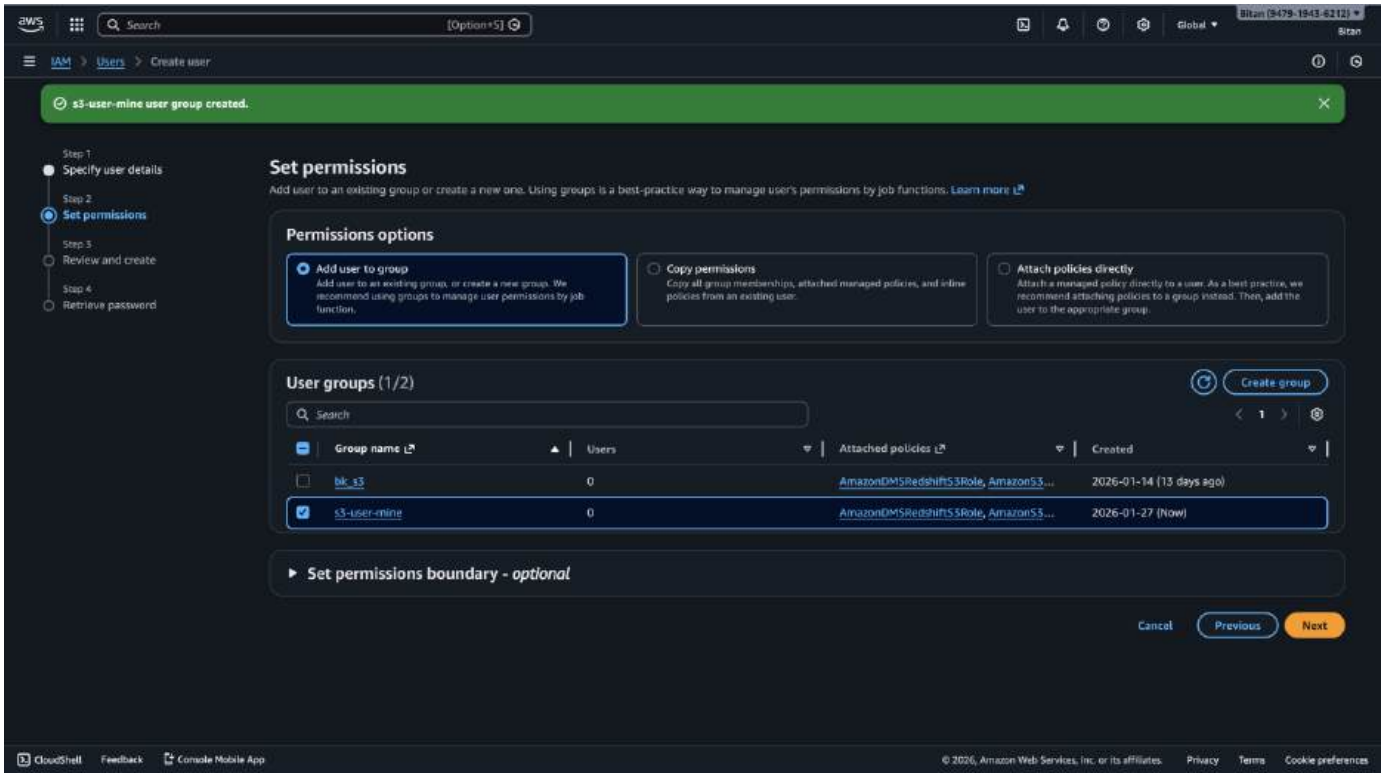


Step 3:Enter a **User name** . Check **Provide user access to the AWS Management Console**. Select **I want to create an IAM user**. Set a custom password and uncheck "Users must create a new password" Click **Next**.

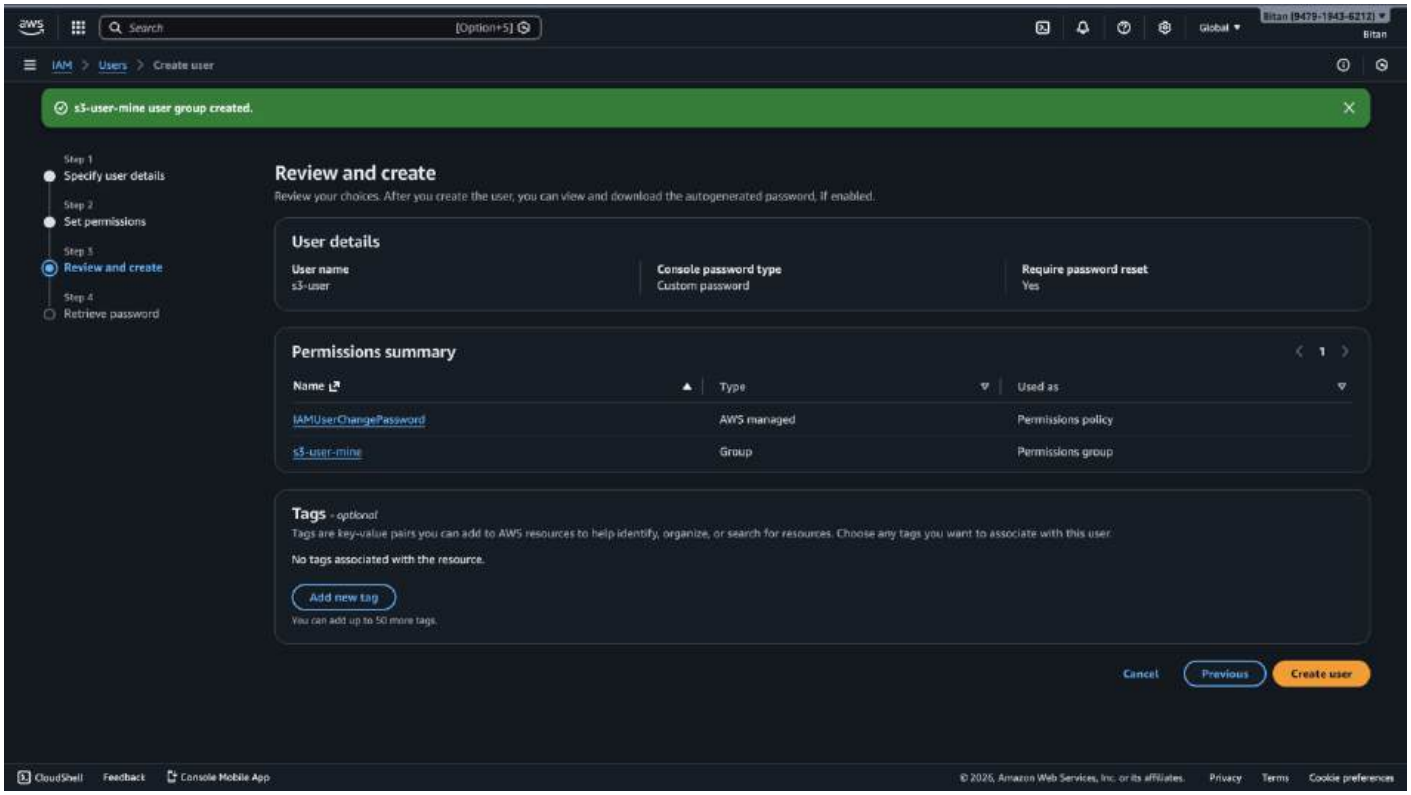


Step 4: Under **Permission options**, select **Add user to group**. Click **Create group**. Enter a group name . Search for and check **AmazonS3FullAccess**. Click **Create user group**. Back on the list, ensure the new group is **checked**. Click **Next**.



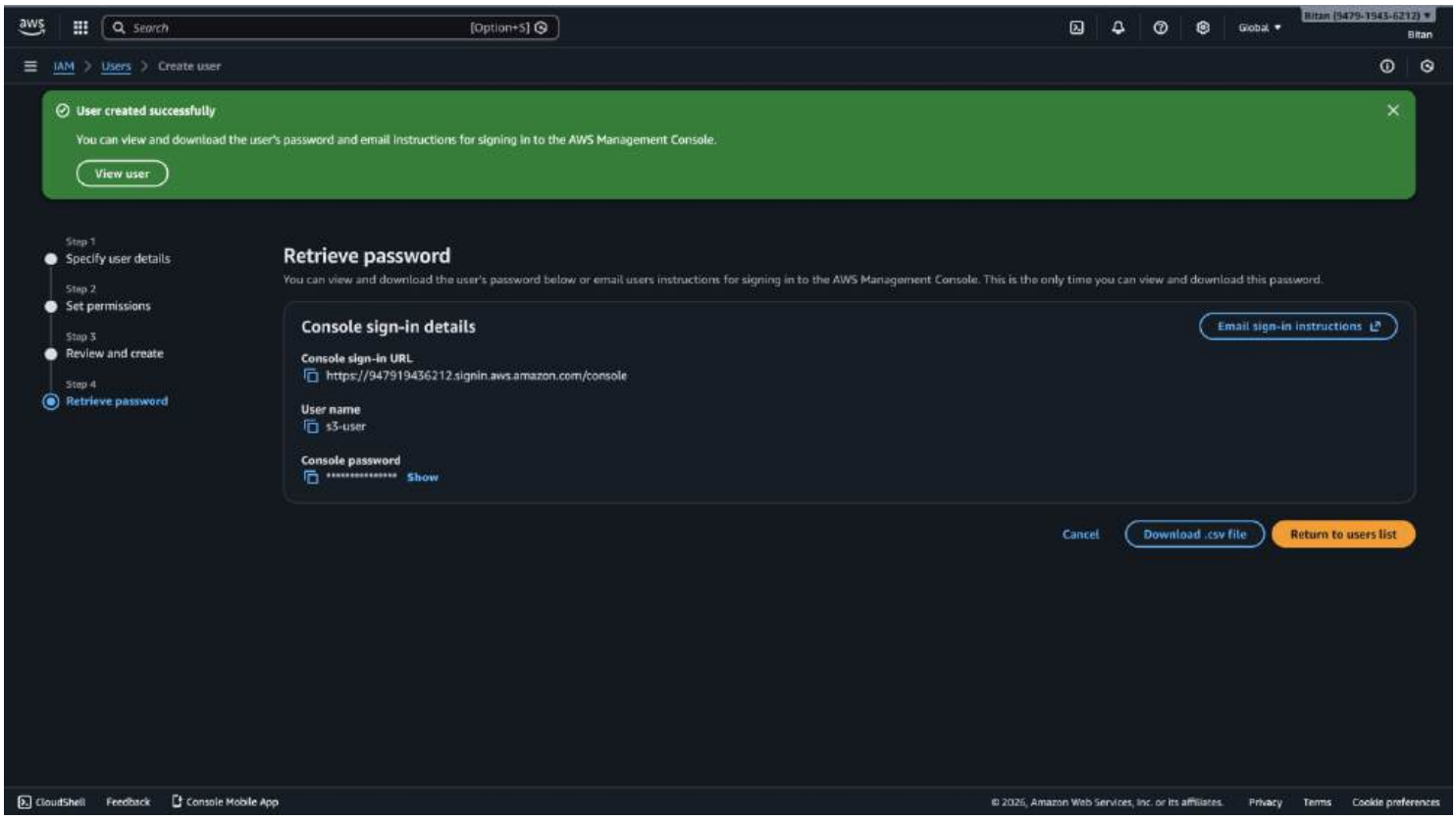


Step 5: On the **Review** page, confirm the entered details. Click Create user.

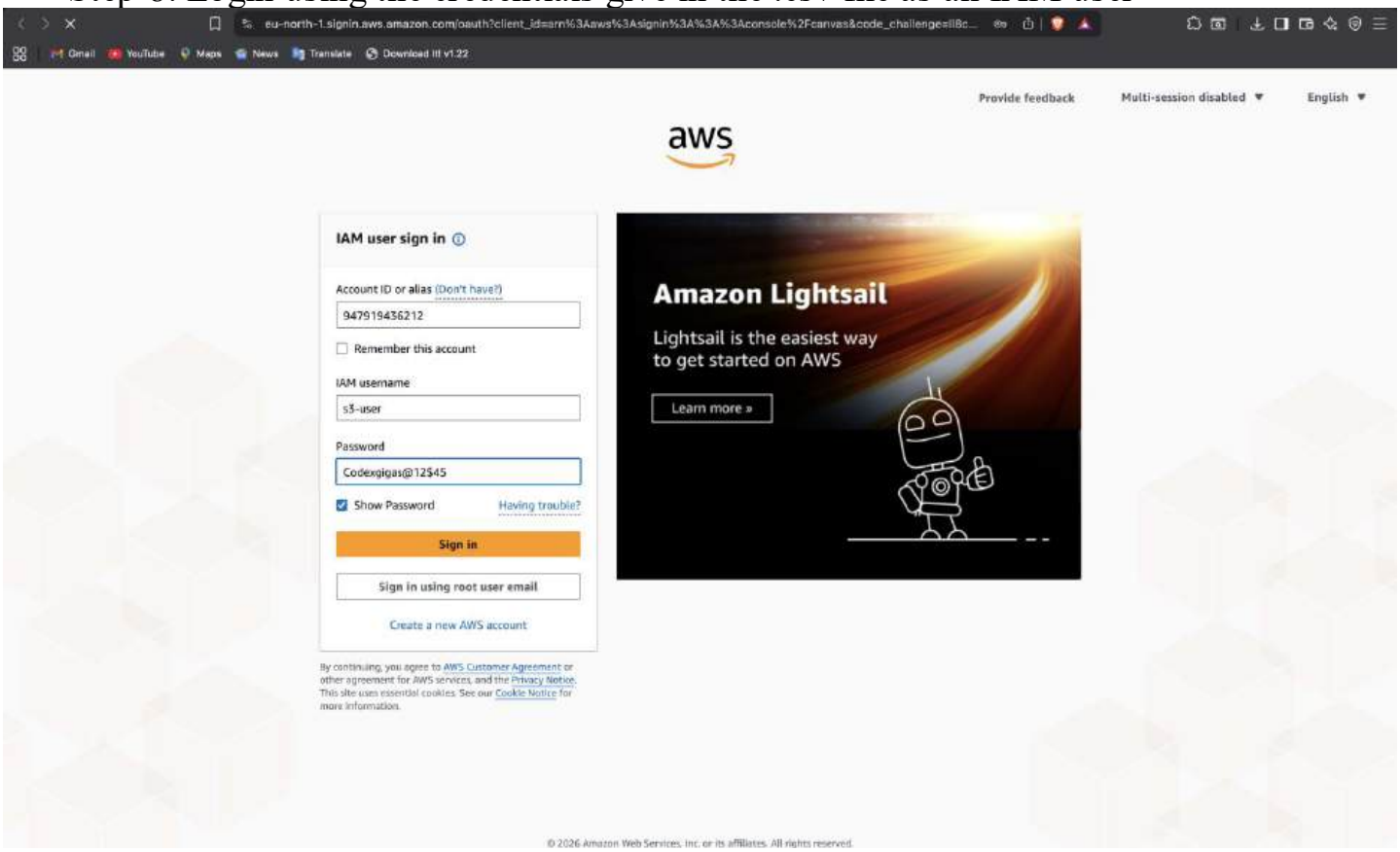


Step 6: Click Download the .csv to save the login details. Click **Return to users list**

Step 7: Newly created user will show up here



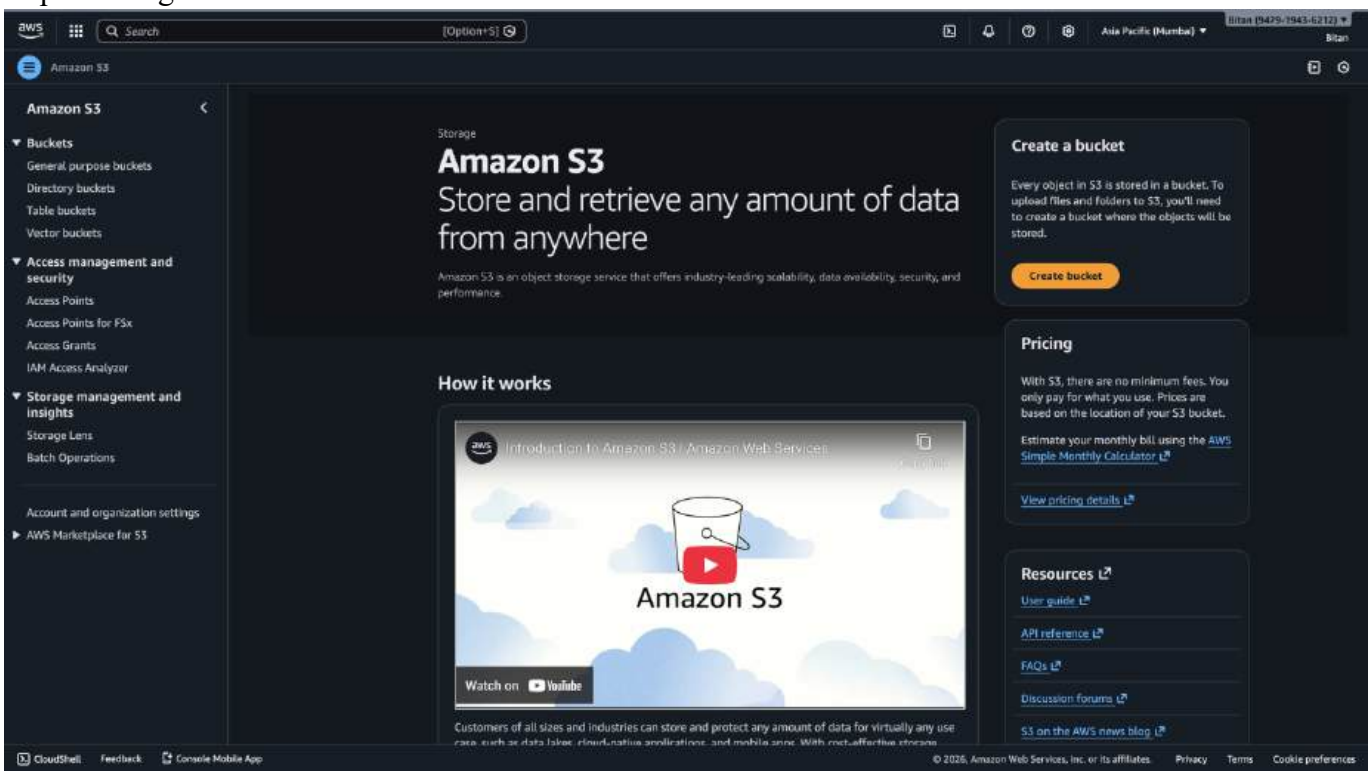
Step 8: Login using the credentials give in the .csv file as an IAM user



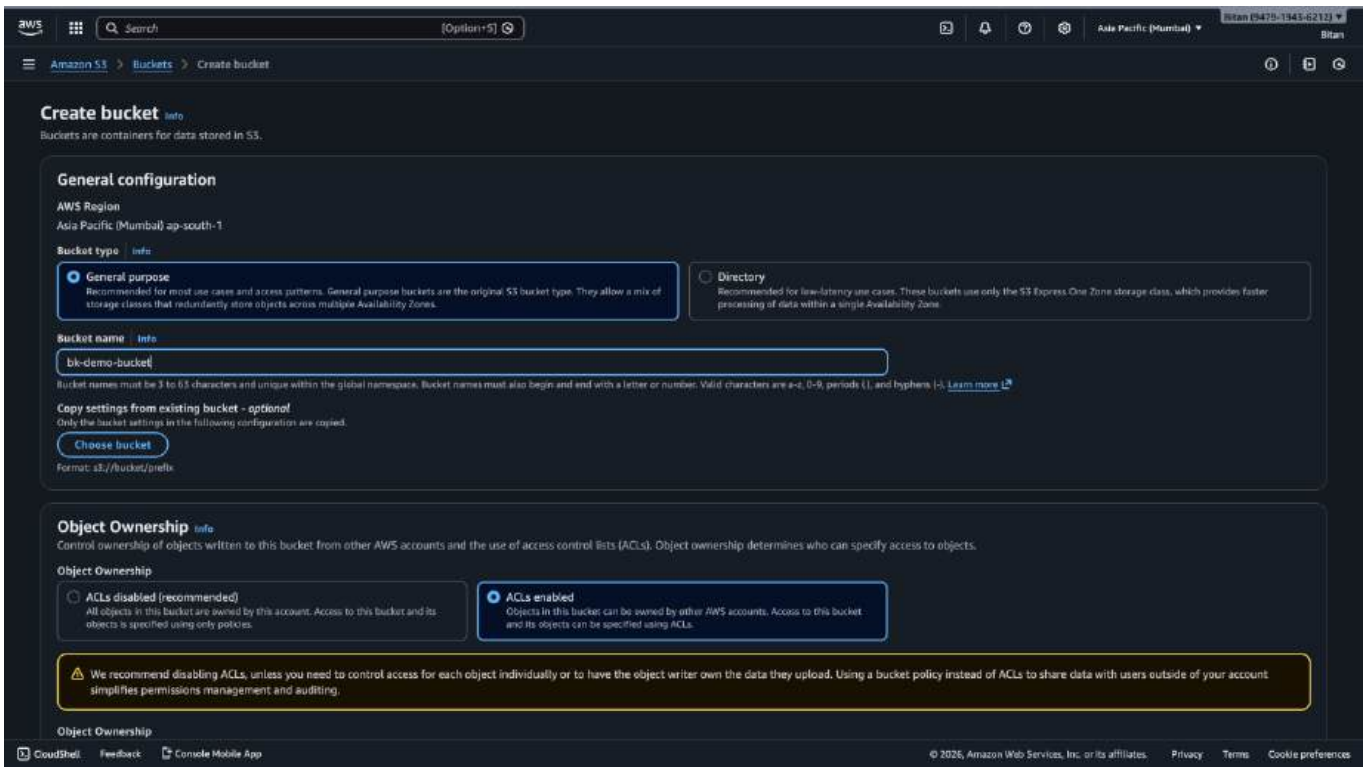
ASSIGNMENT 4

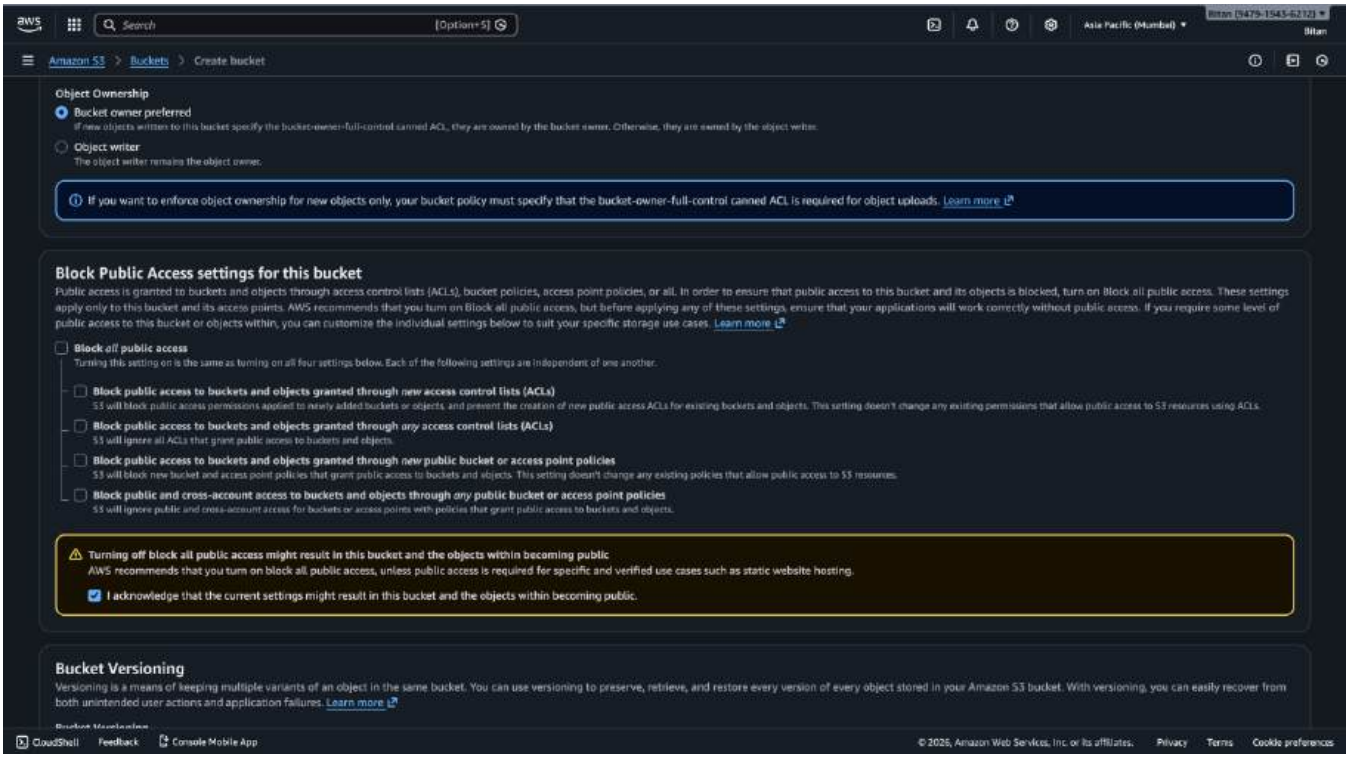
Problem Statement: Create a public bucket in AWS. Upload a file and give the necessary permission to check the file URL is working or not.

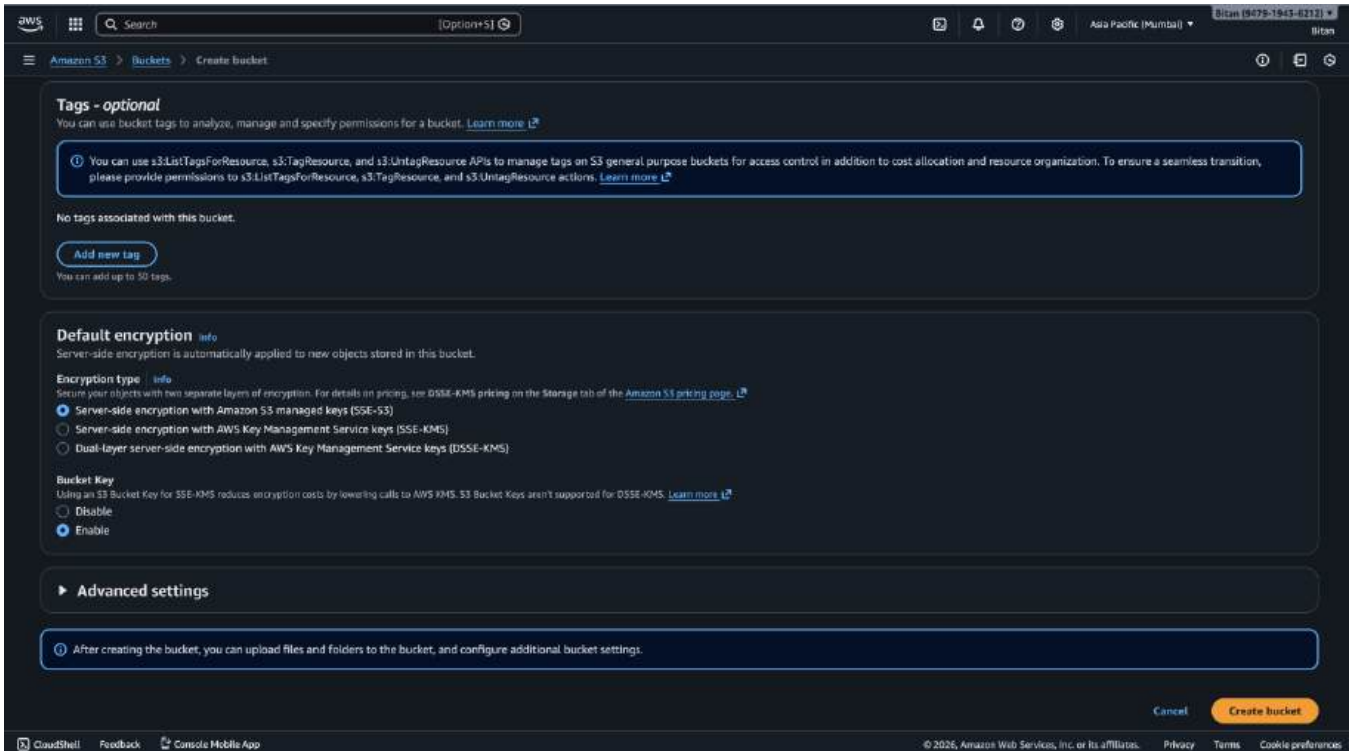
Procedure:
Step 1: Navigate to **Buckets** in **Amazon S3 Console** then select **Create bucket**



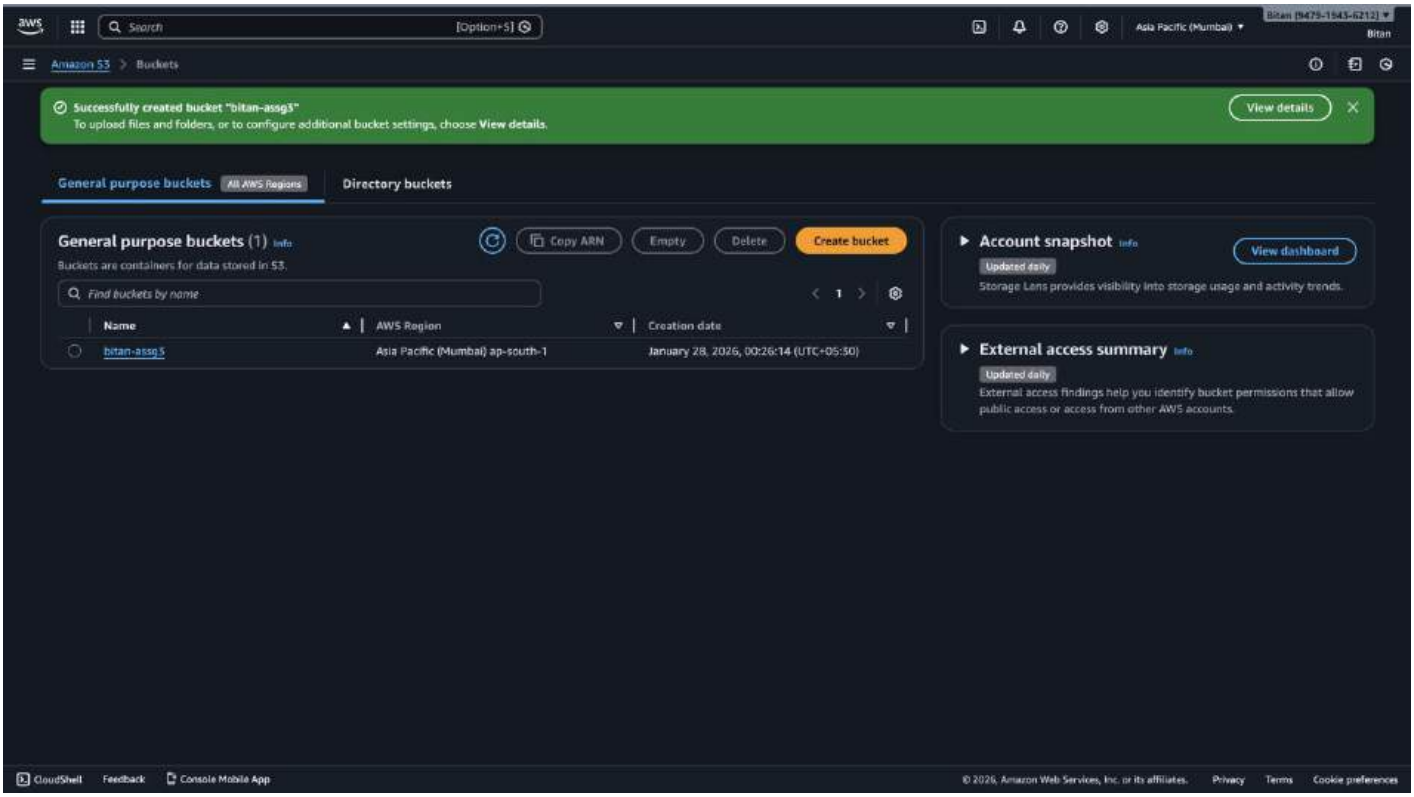
Step 2: Enter a unique bucket name. **Object Ownership**: Select ACLs enabled (and "Bucket owner preferred"). Uncheck **Block all Public Access.** Check the acknowledgment box. Lastly **Create bucket**.



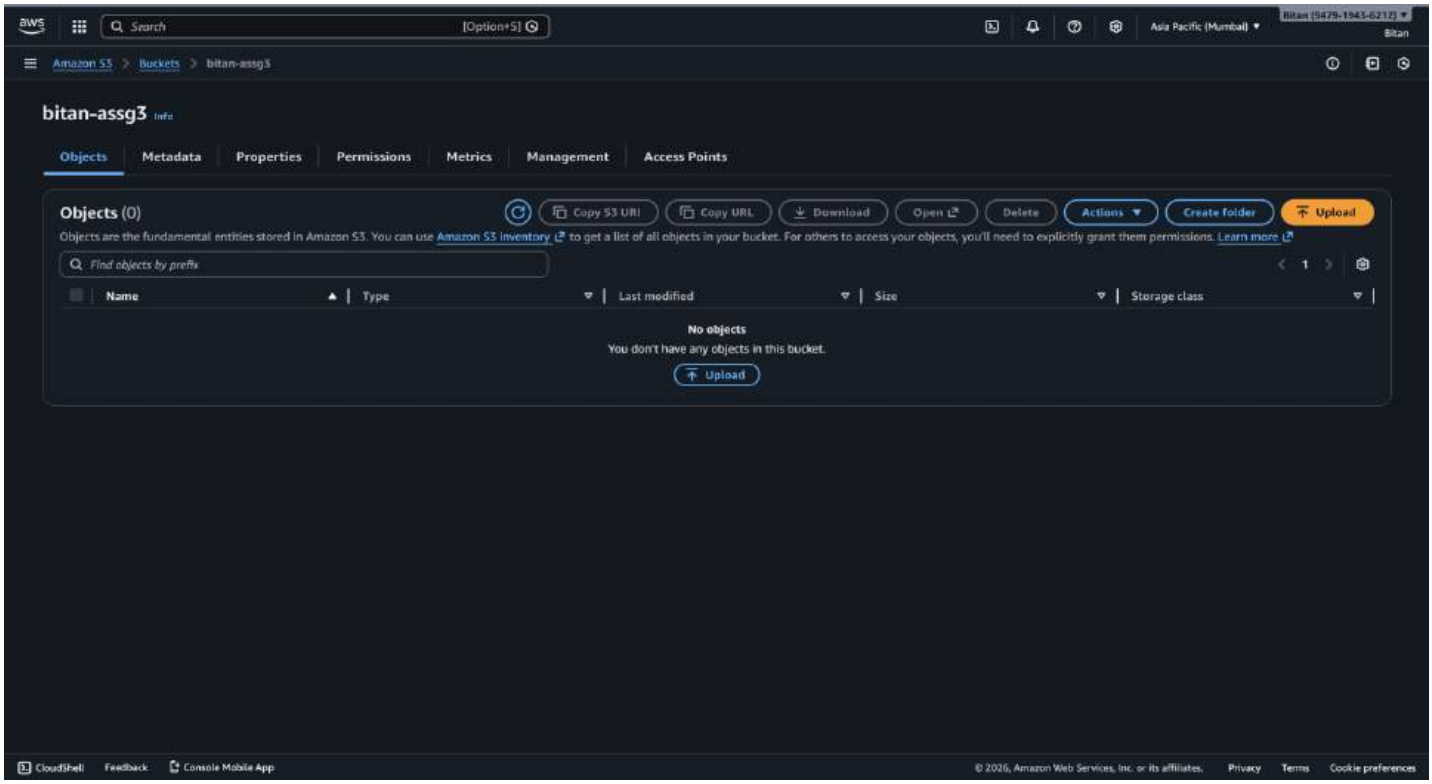




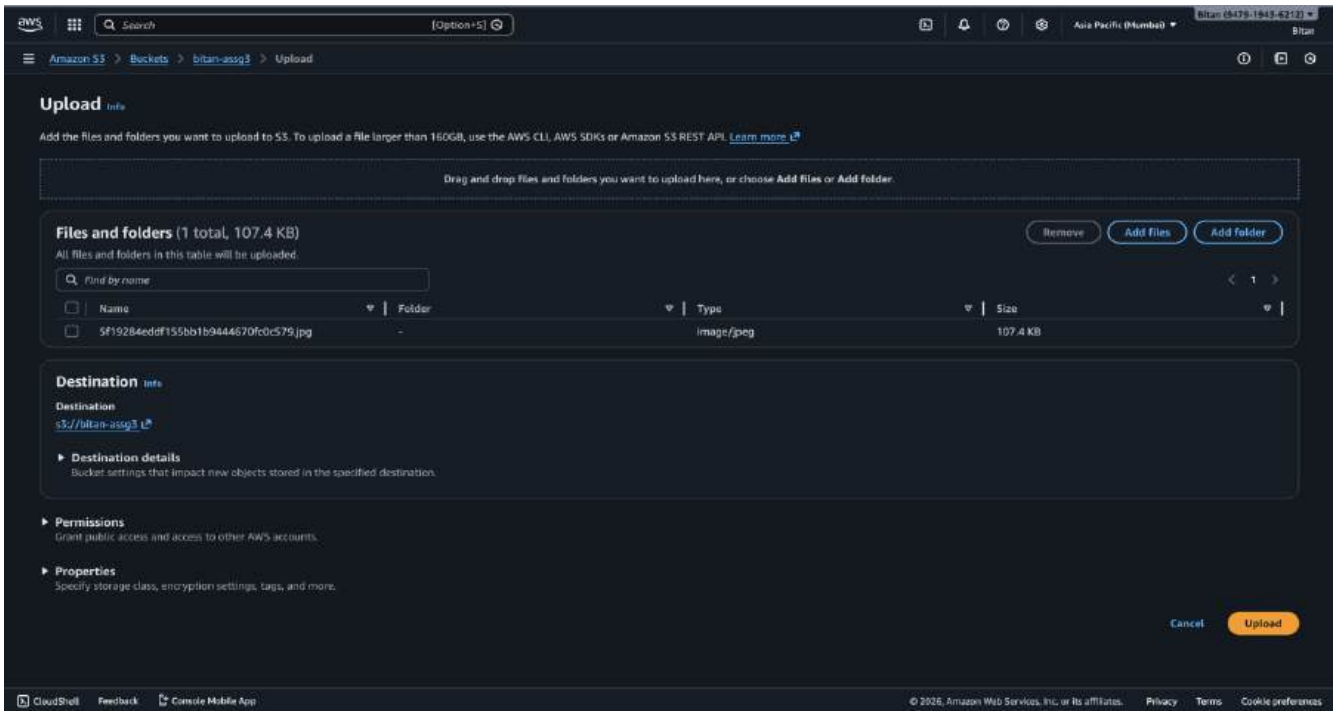
Step 3: A new bucket will be created. Click on the newly created bucket

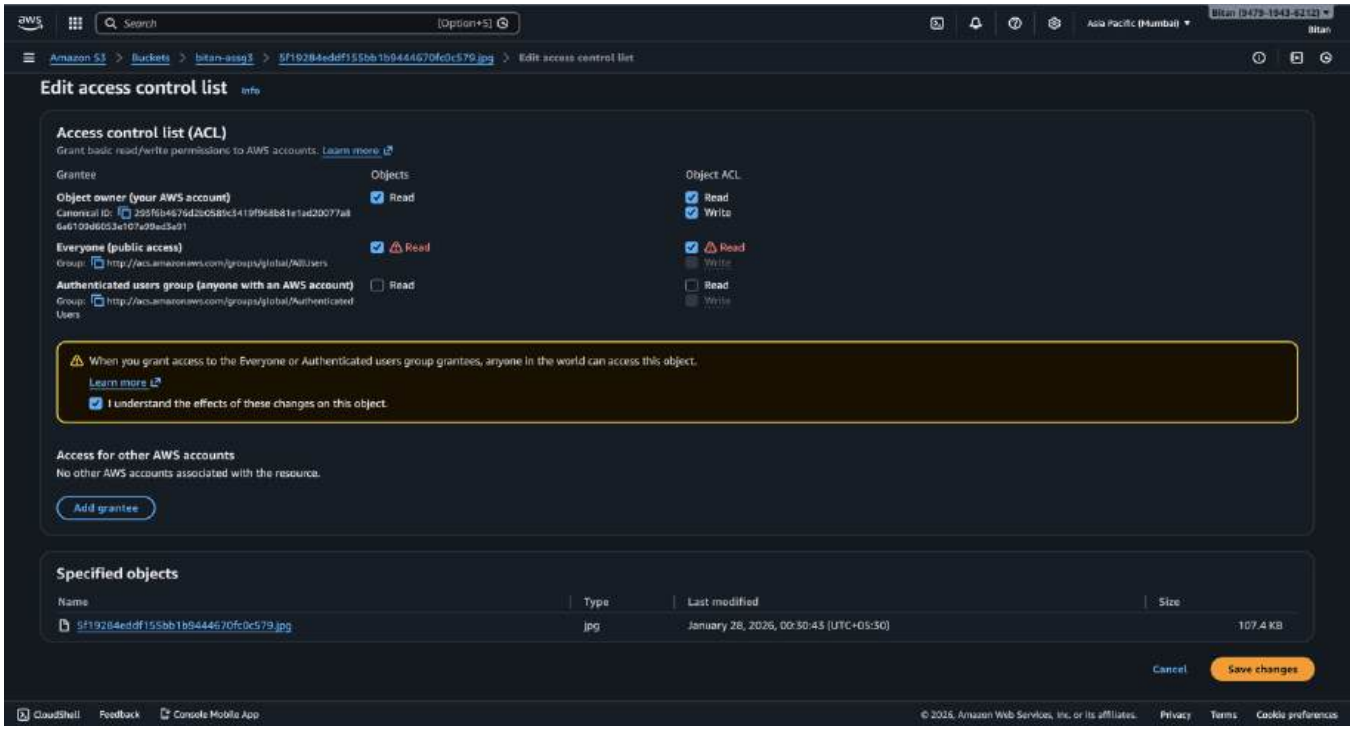


Step 4: Within the bucket, click **Upload**.

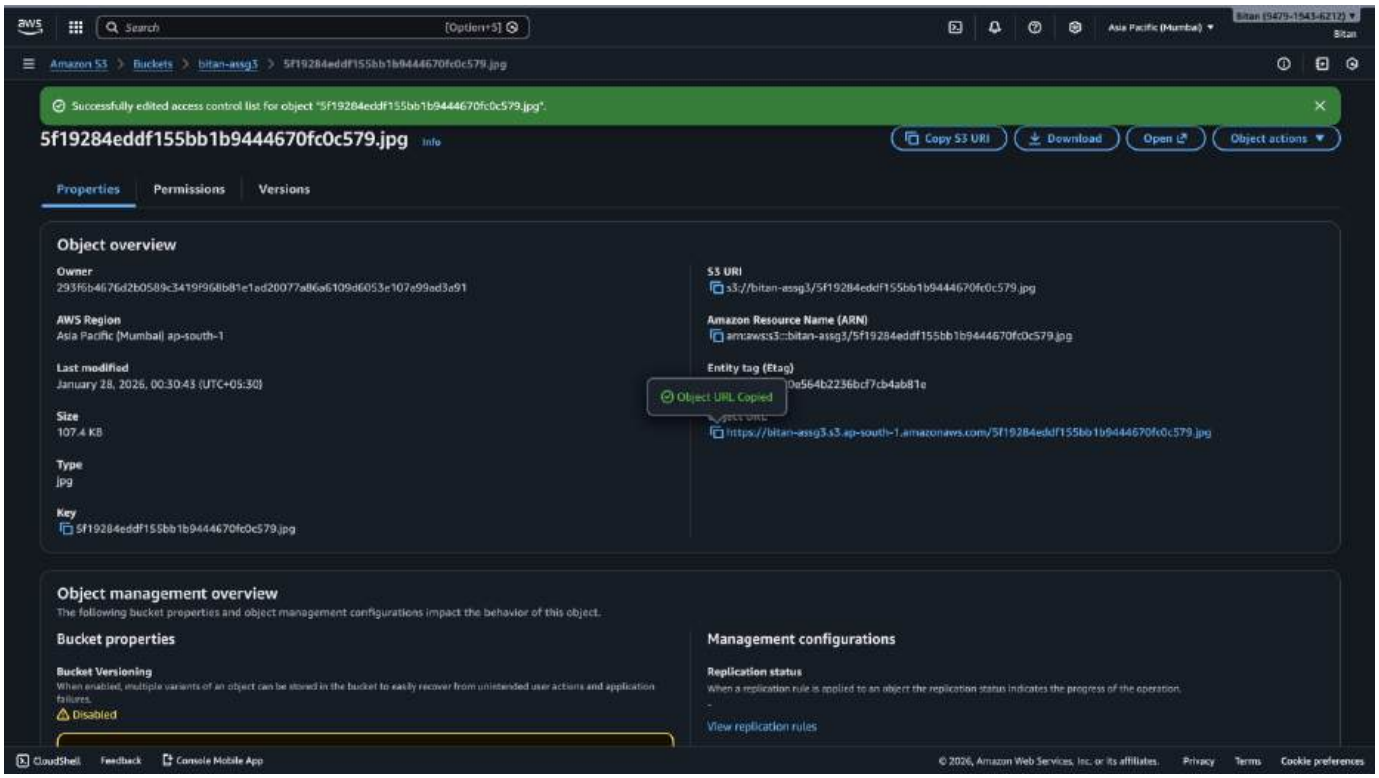


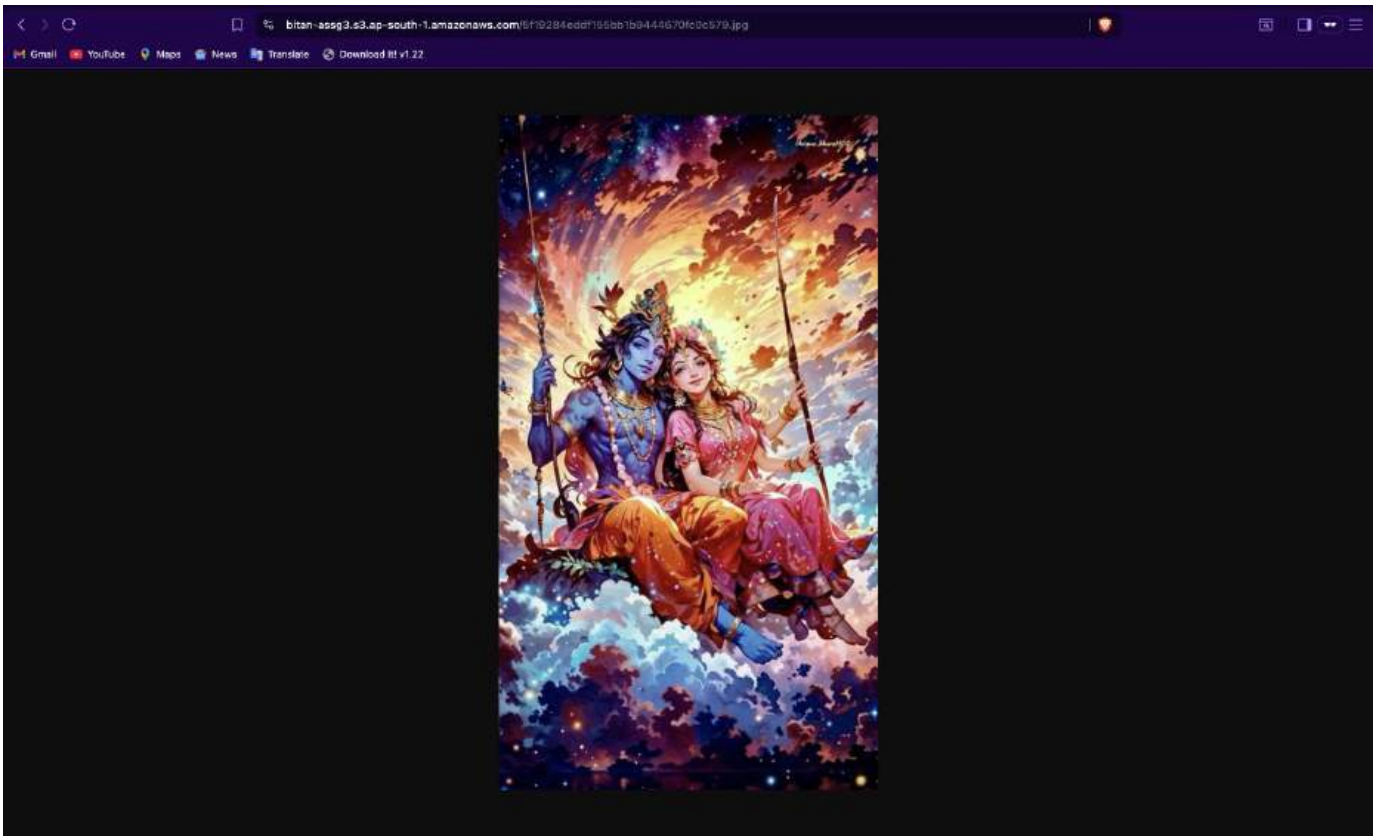
Step 5: On the **upload** page, click Add files and select the file(s) to upload. Under **Permissions** select **Specify individual ACL permissions.** Then Grant Public Access. Check the Read permission boxes for Object and Object's ACL under **Everyone (public access)**. Tick the acknowledgment box confirming you understand the changes. Click **Upload** to complete the process.





Step 6: Go to the Properties tab of the uploaded file. Copy the Object URL. Paste the URL into a another web browser or incognito window. The file should be publicly accessible.


Search
[Option+S]
Asia Pacific (Mumbai)
Amazon S3 > Buckets > bitan-assg3 > 5f19284eddf155bb1b9444670fc0c579.jpg
Successfully edited access control list for object "5f19284eddf155bb1b9444670fc0c579.jpg".
5f19284eddf155bb1b9444670fc0c579.jpg Info
Copy S3 URI
Download
Open
Object actions
Properties
Permissions
Versions
Object overview
Owner
293f6b4676d2b0589c3419f968b81e1ad20077a86a6109d6053e107a99ad3a91
AWS Region
Asia Pacific (Mumbai) ap-south-1
Last modified
January 28, 2026, 00:30:43 (UTC+05:30)
Size
107.4 KB
Type
jpg
Key
5f19284eddf155bb1b9444670fc0c579.jpg
S3 URI
s3://bitan-assg3/5f19284eddf155bb1b9444670fc0c579.jpg
Amazon Resource Name (ARN)
arn:aws:s3:::bitan-assg3/5f19284eddf155bb1b9444670fc0c579.jpg
Entity tag (Etag)
Object URL Copied
https://bitan-assg3.s3.ap-south-1.amazonaws.com/5f19284eddf155bb1b9444670fc0c579.jpg
Object management overview
The following bucket properties and object management configurations impact the behavior of this object.
Bucket properties
Bucket Versioning
When enabled, multiple variants of an object can be stored in the bucket to easily recover from unintended user actions and application failures.
Disabled
Management configurations
Replication status
When a replication rule is applied to an object the replication status indicates the progress of the operation.
View replication rules
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


bitan-assg3.s3.ap-south-1.amazonaws.com/5f19284eddf155bb1b9444670fc0c579.jpg
Gmail
YouTube
Maps
News
Translate
Download It! v1.22

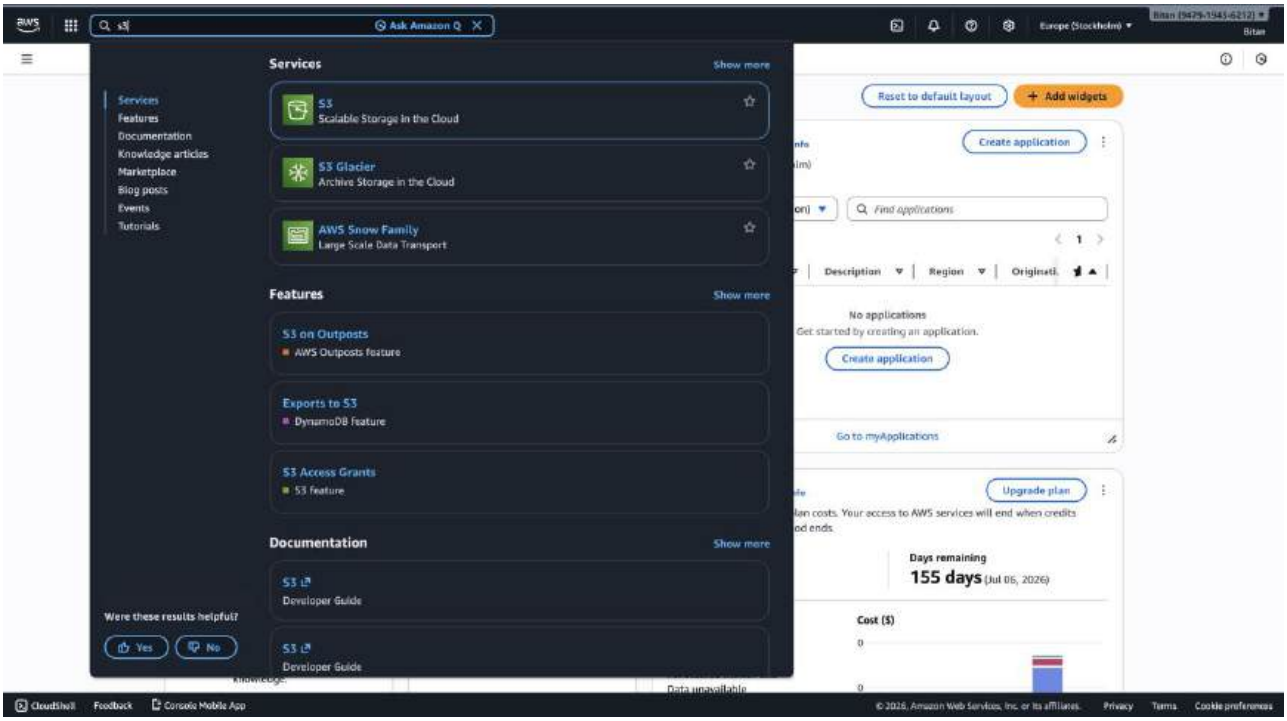
Assignment No.:5

Problem Statement: Create a private bucket in AWS. Upload a file and check by reassigned URL whether you can access the file or not.

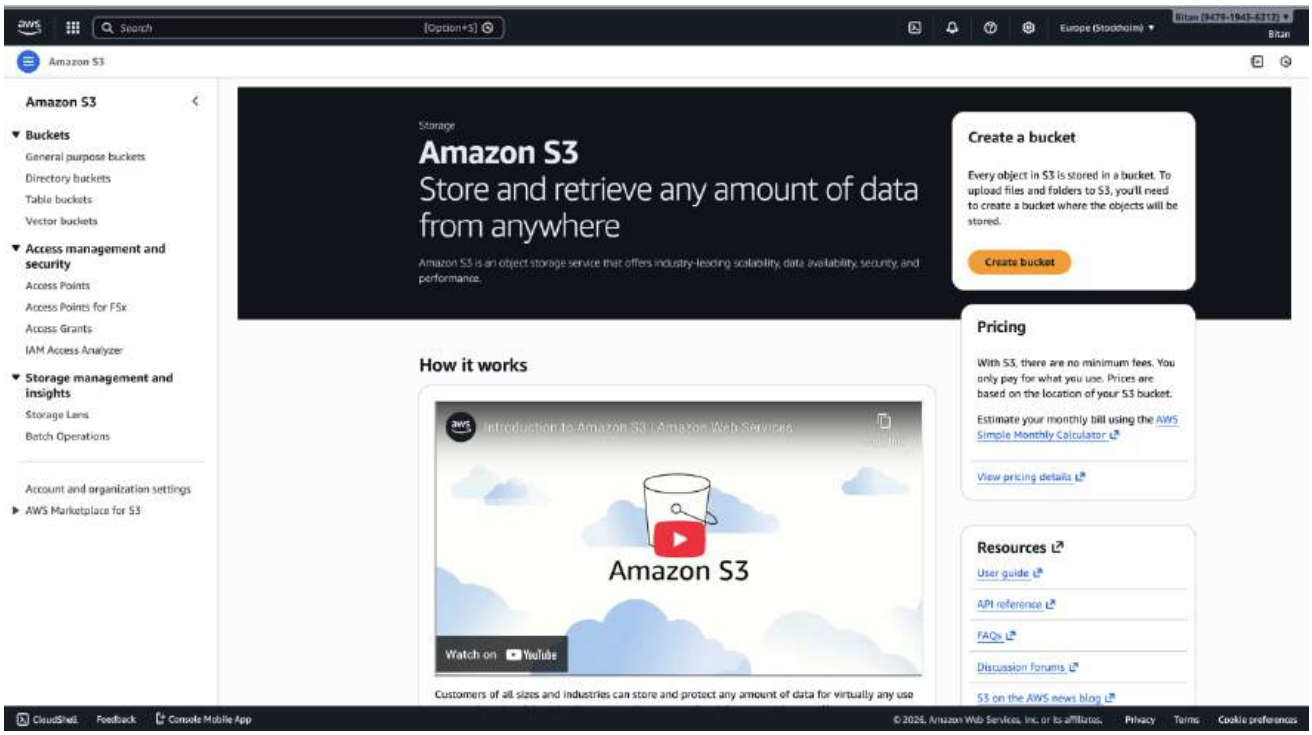
Procedure:

The steps are as follows: -

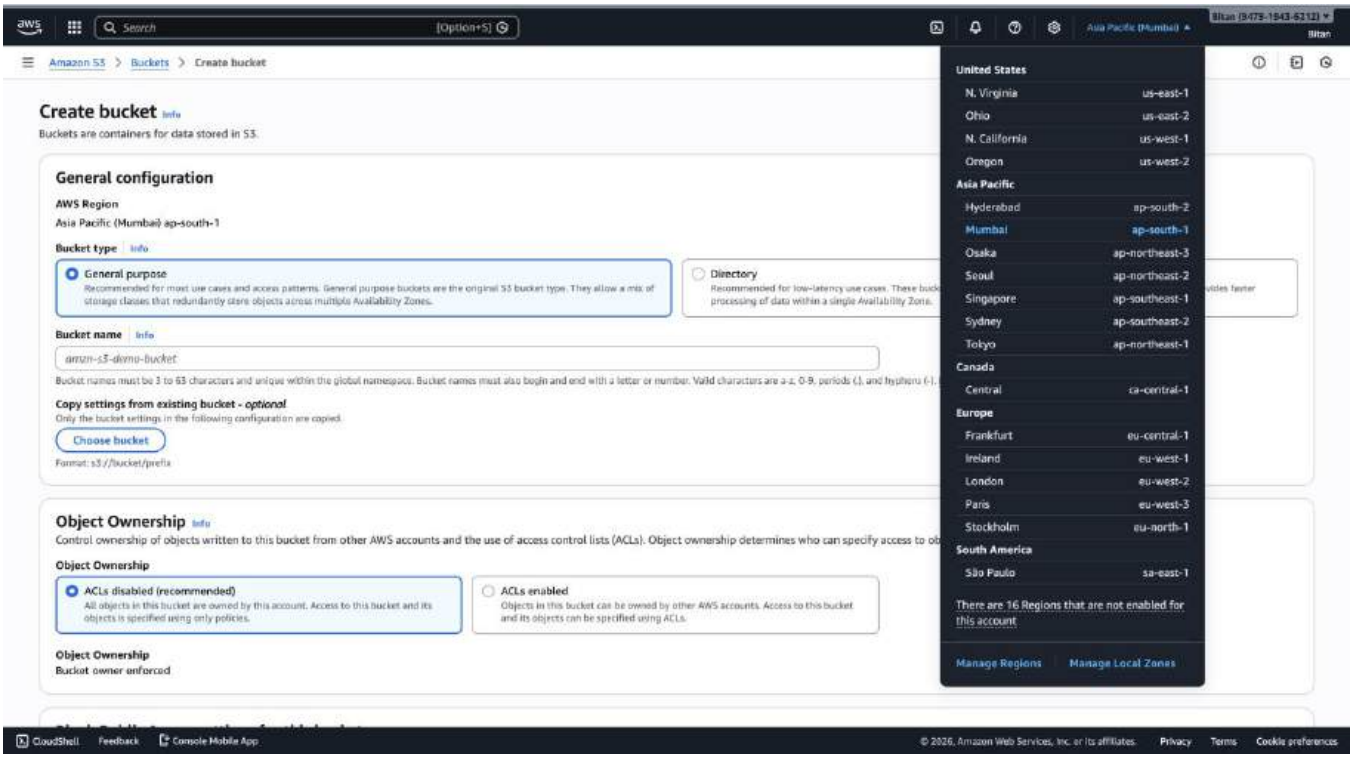
1. Access the AWS console, search for S3, and select the top option from the search results.

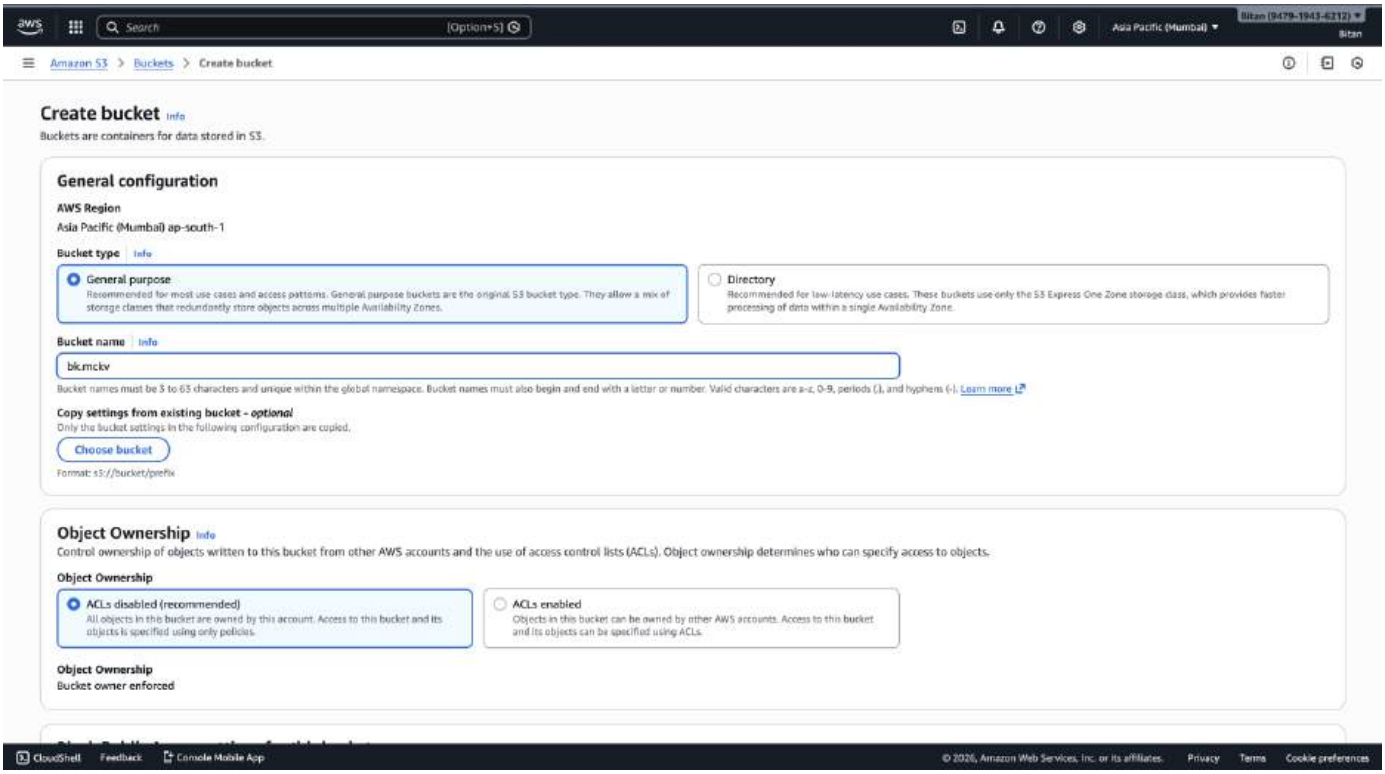


2. Click on 'Create bucket'.

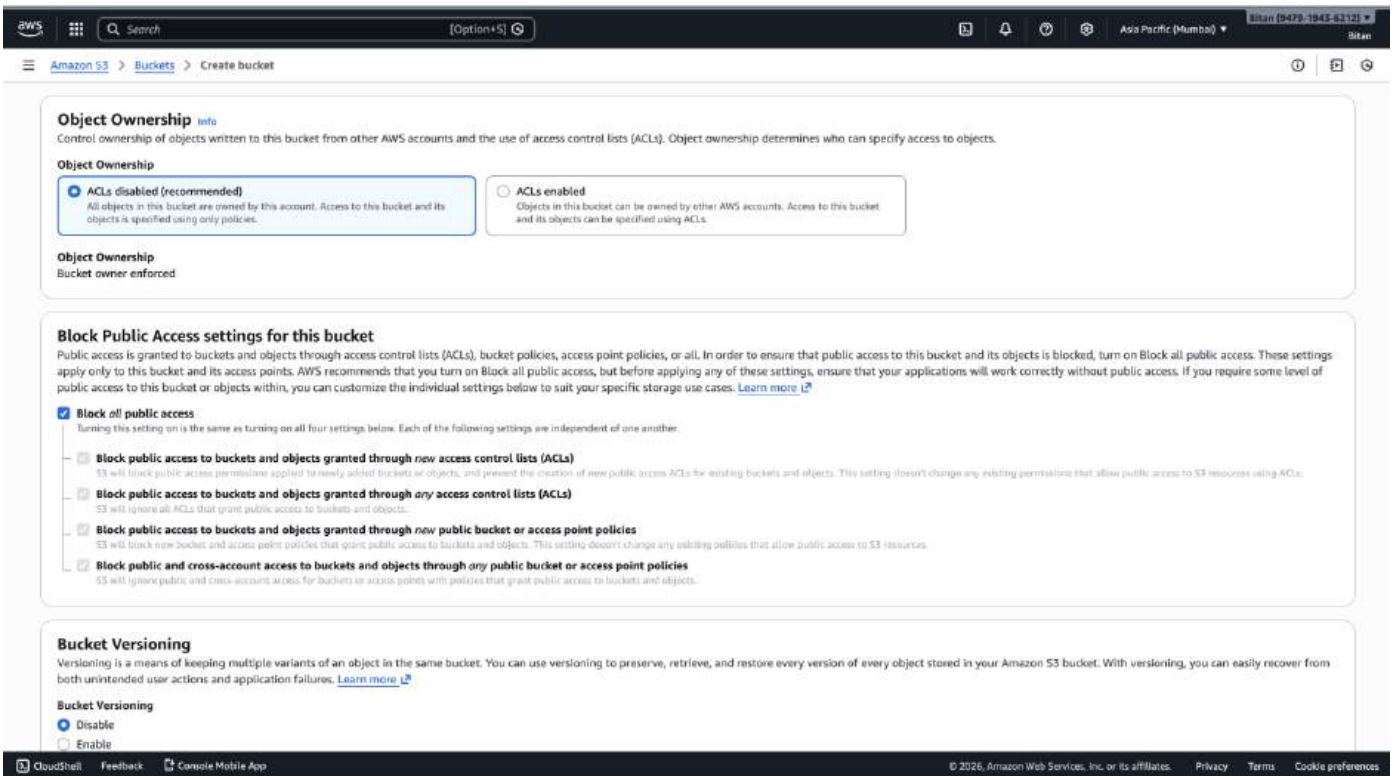


3. Subsequent window appears. From the drop-down menu, select 'Asia Pacific (Mumbai) ap-south-1' as the AWS Region. Next, specify a bucket name , such as ‘bk.mckv’.

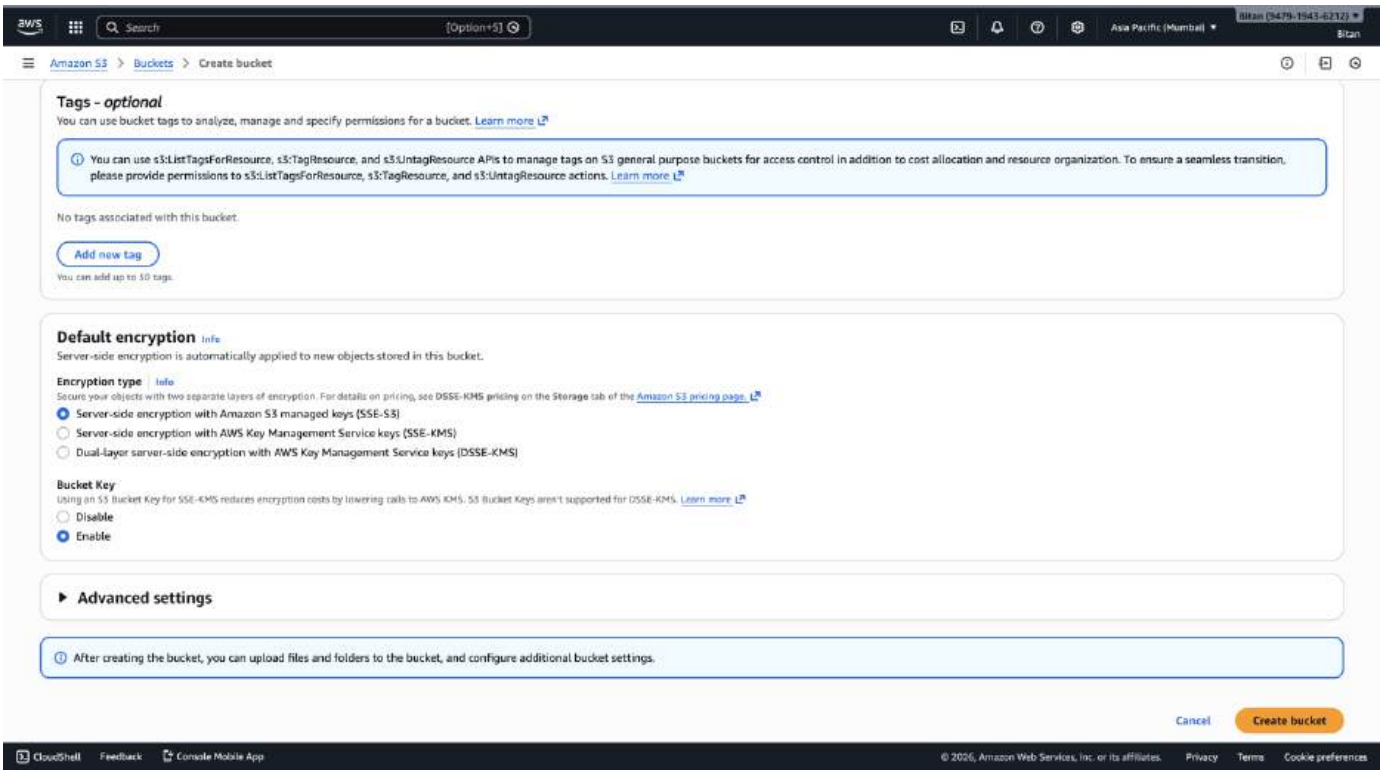




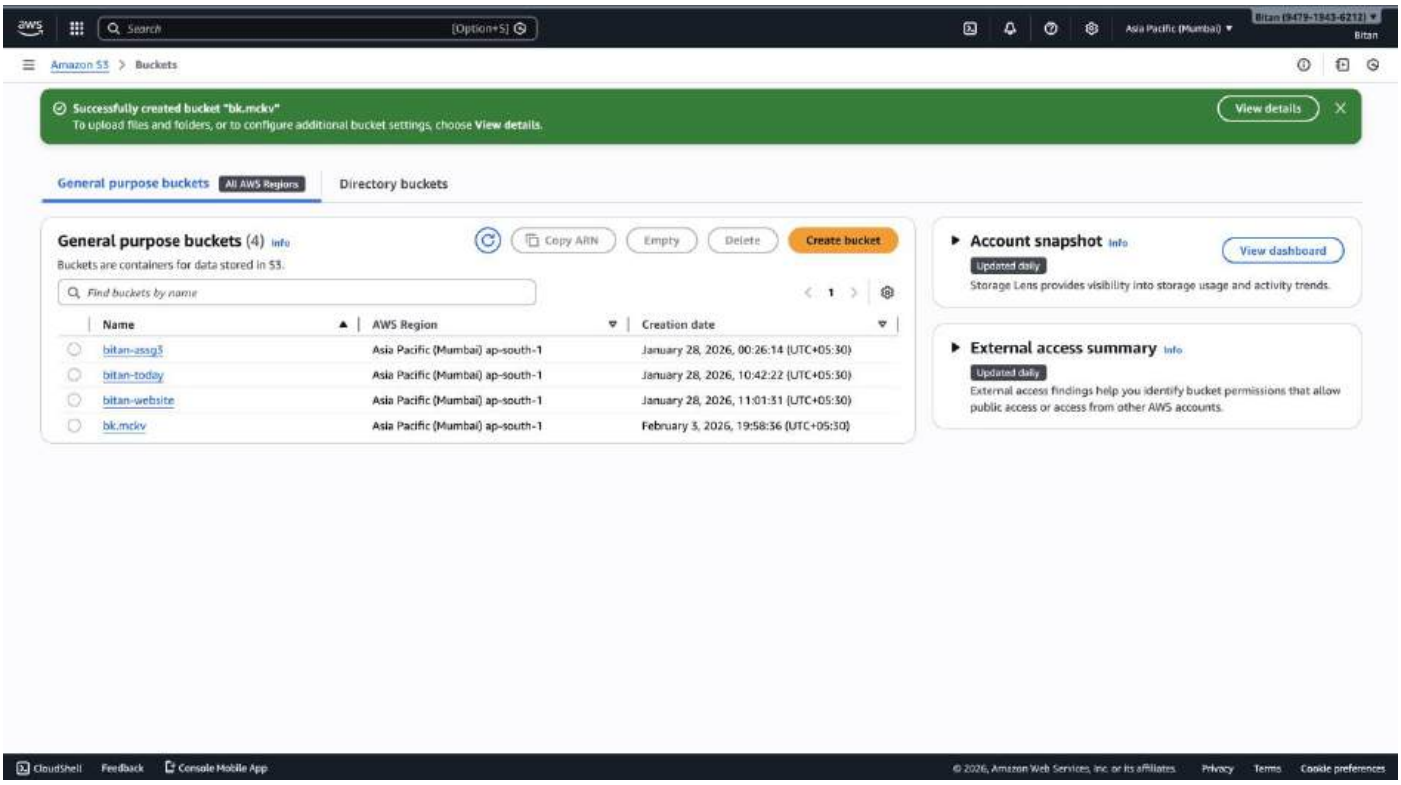
4. By default, the Object Ownership is set to **'ACLs disabled'** in this window. We will not make any adjustments as we are creating a private bucket. Similarly, we will leave the settings unchanged as the **'Block all public access'** checkbox is already selected by default, aligning with our intention to create a private bucket.



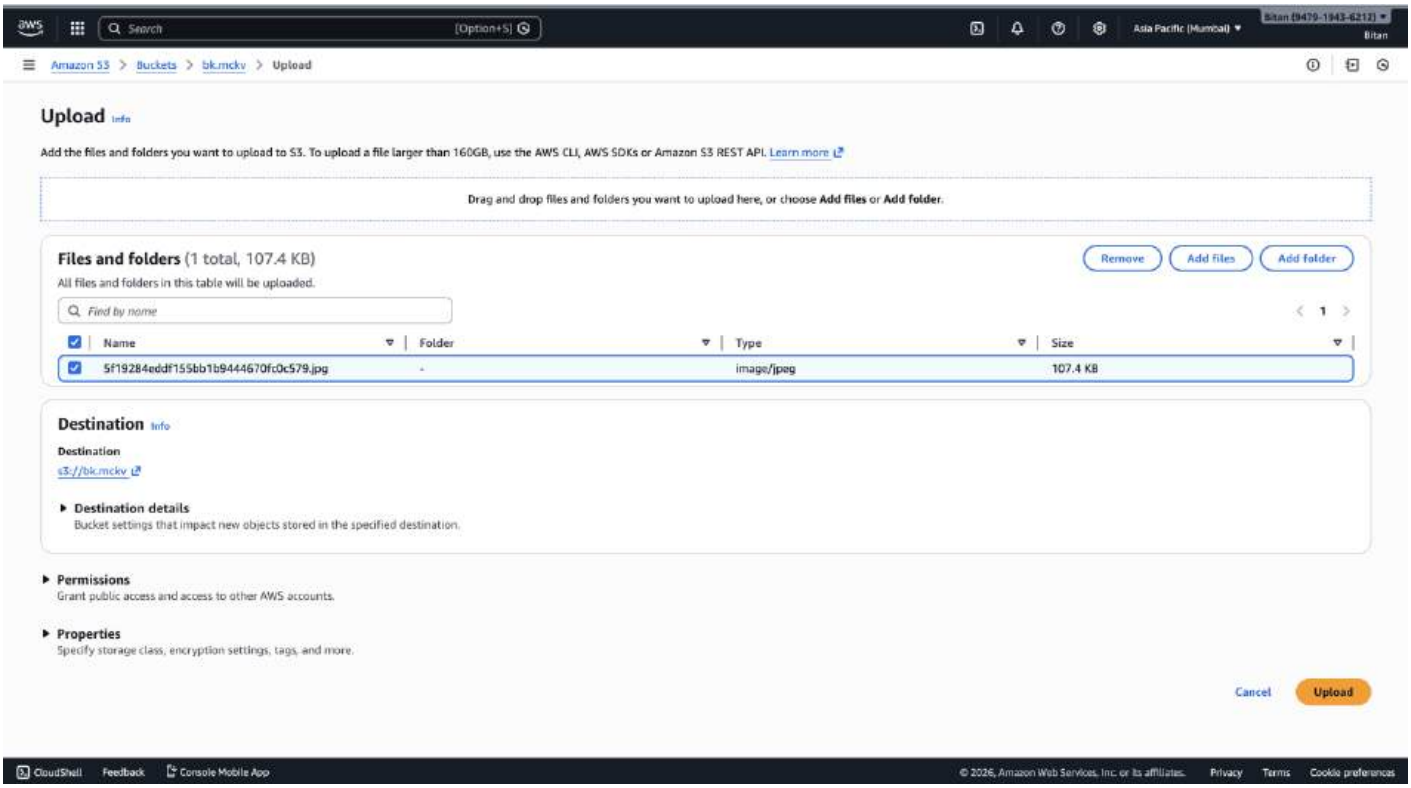
6. Scroll down without any further modifications and proceed to click on 'Create Bucket'.



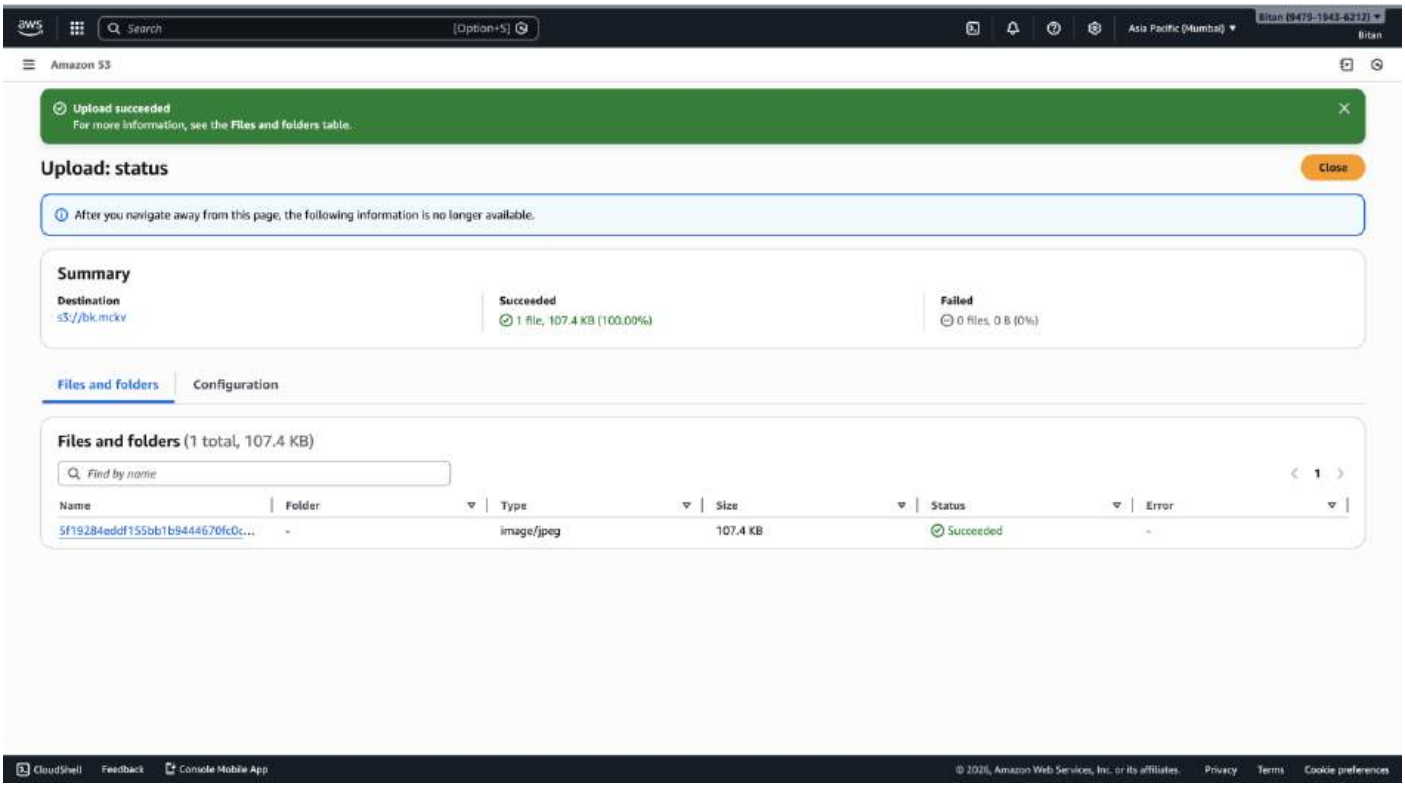
7. It's evident that the bucket has been successfully created.



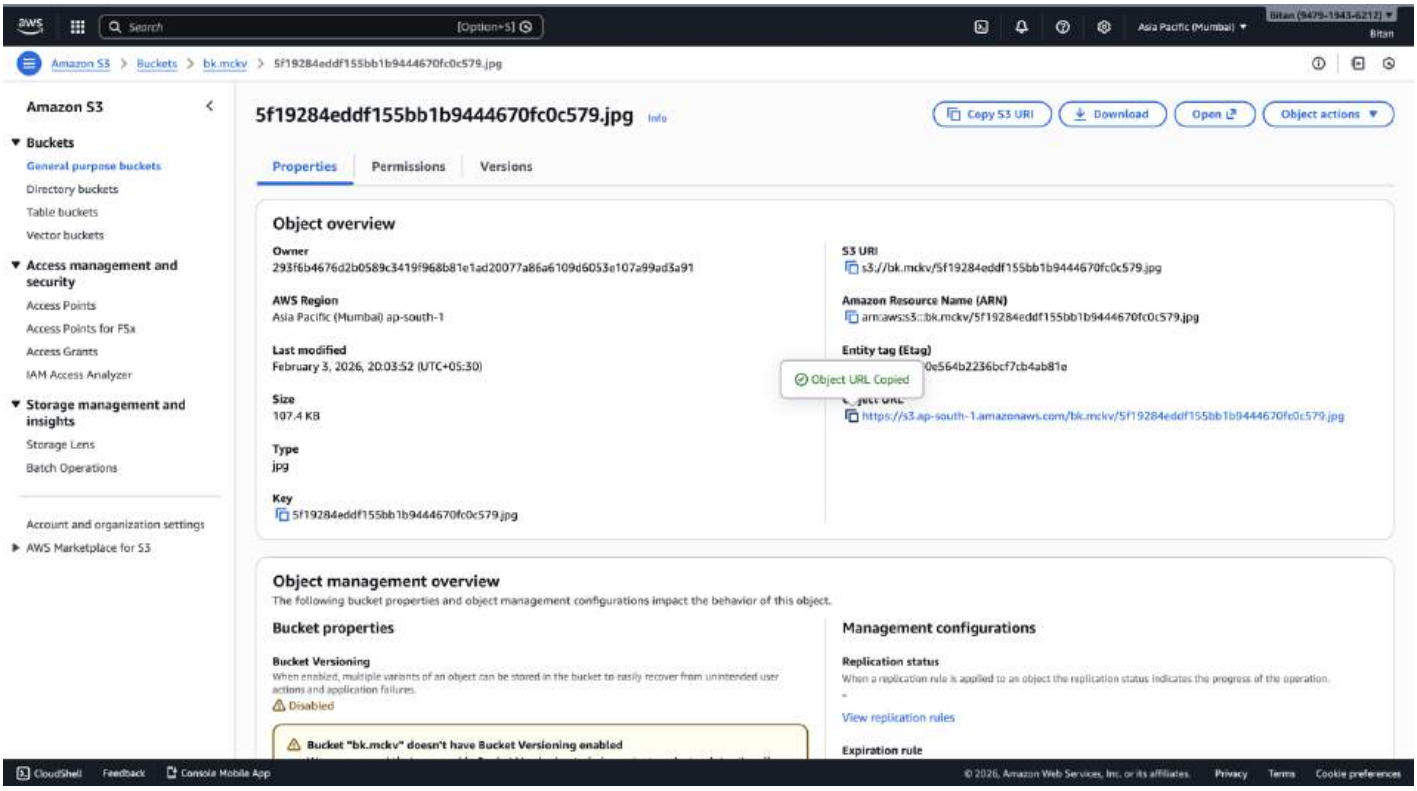
8. Now click on the bucket name. Choose the **'Upload'** option, which will lead to the opening of the subsequent window. From there, click on **'Add files'** and proceed to upload your desired file(s) into the selected bucket.



9. Scroll down and proceed to click on 'Upload'. The notification confirms that the file has been successfully uploaded.

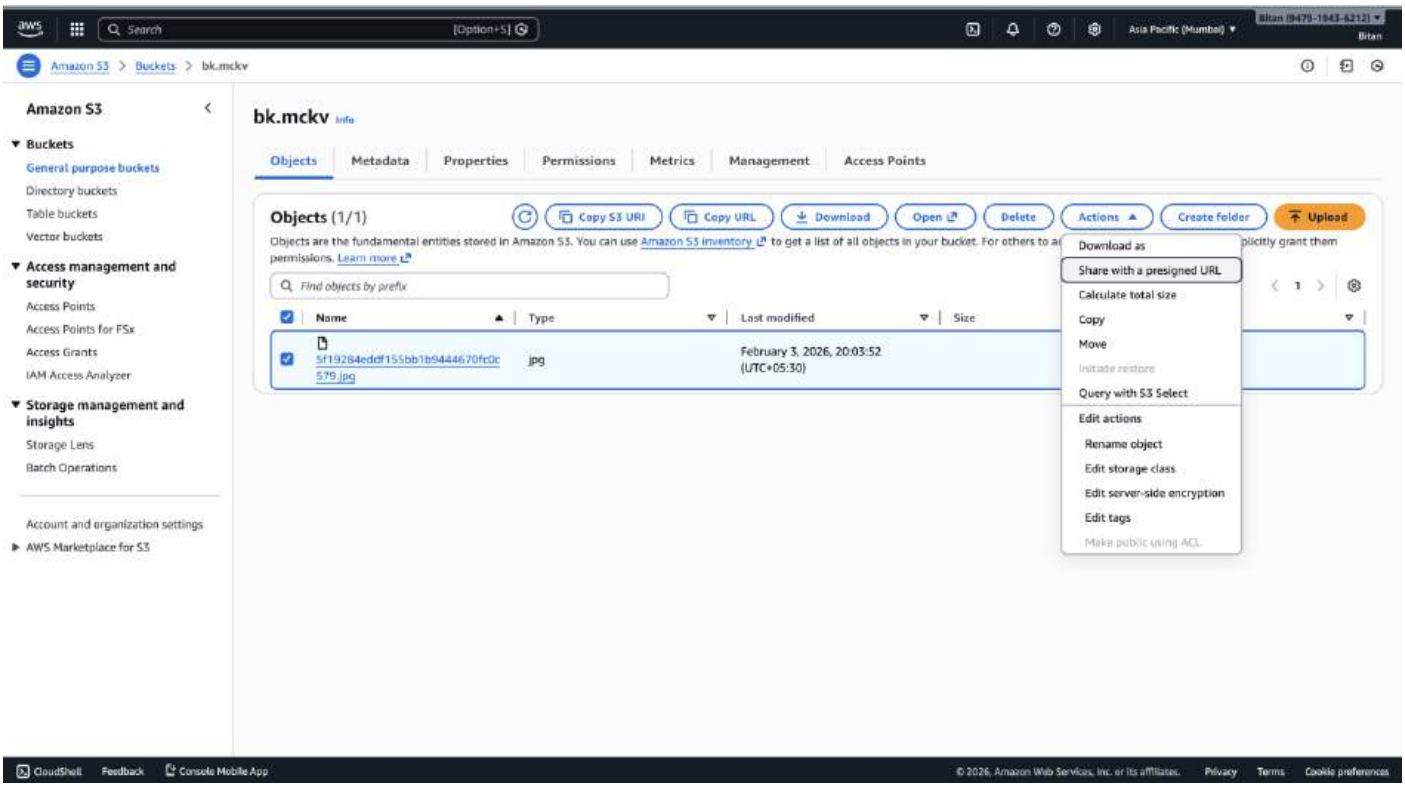


10. Now, click on one of the uploaded file, then 'Copy URL'.

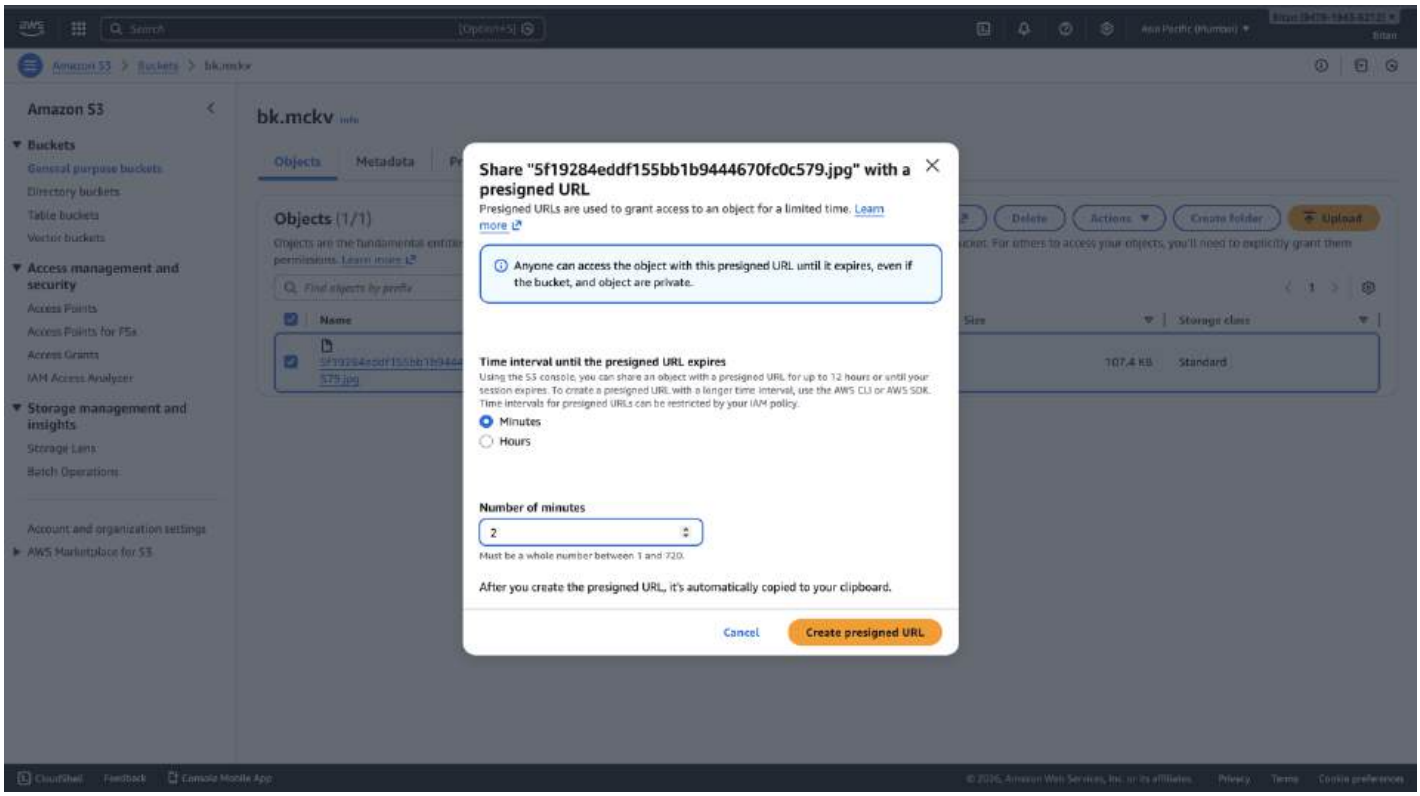


11. Navigate on a new window and paste this URL. An error appears indicating that access to the contents is restricted.

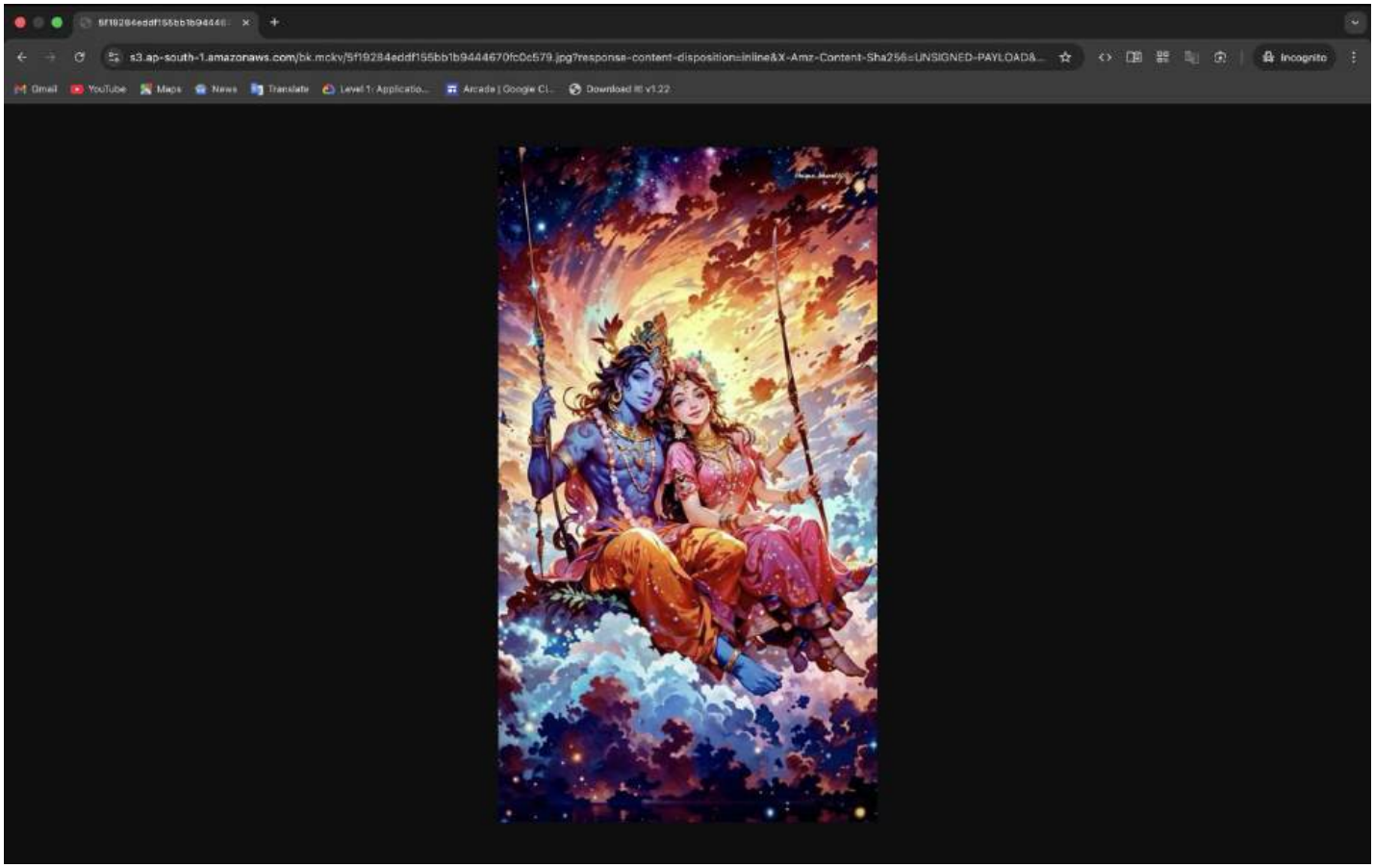
12. Close the current window, then navigate to **'Actions'** and choose **'Share with a presigned URL'**.



13. Select the desired duration until you want the presigned URL to expire.



14. Copy the pre-assigned URL and paste it on a new window. Now, you can view the uploaded file.



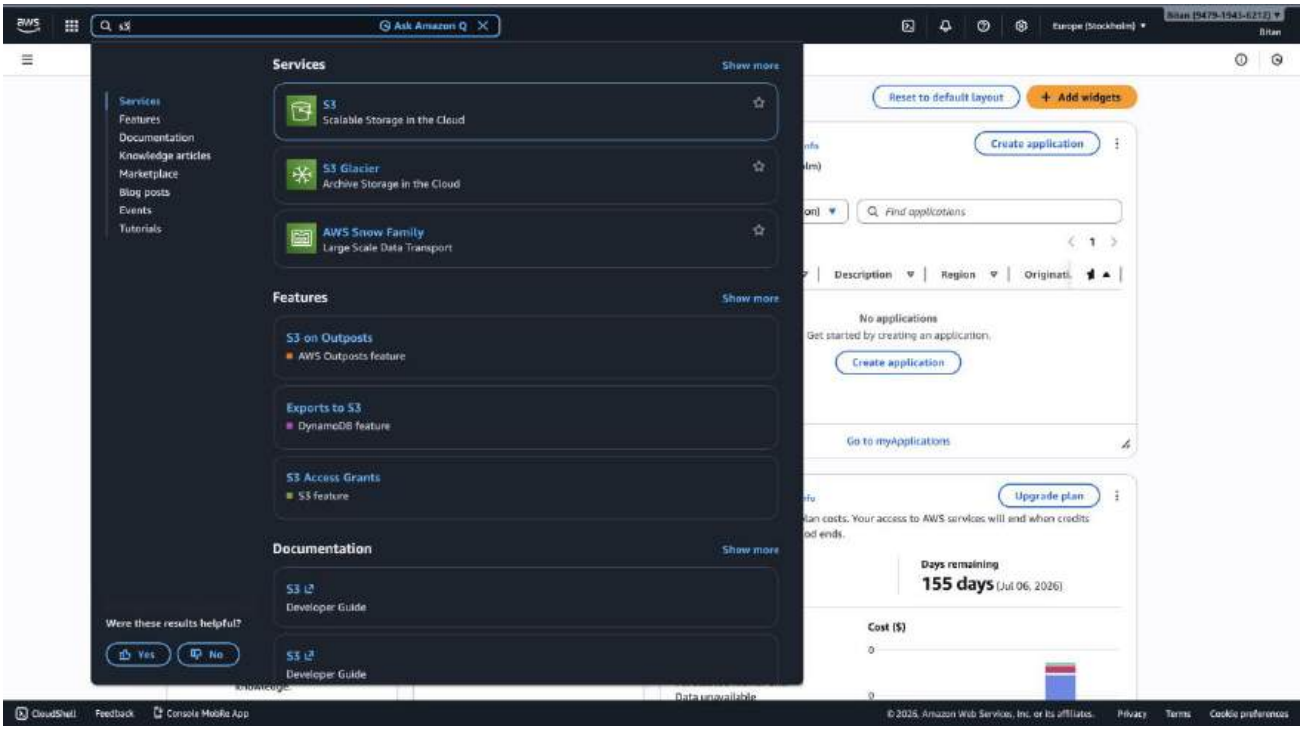
Assignment No.:6

Problem Statement: Upload a static website on S3.

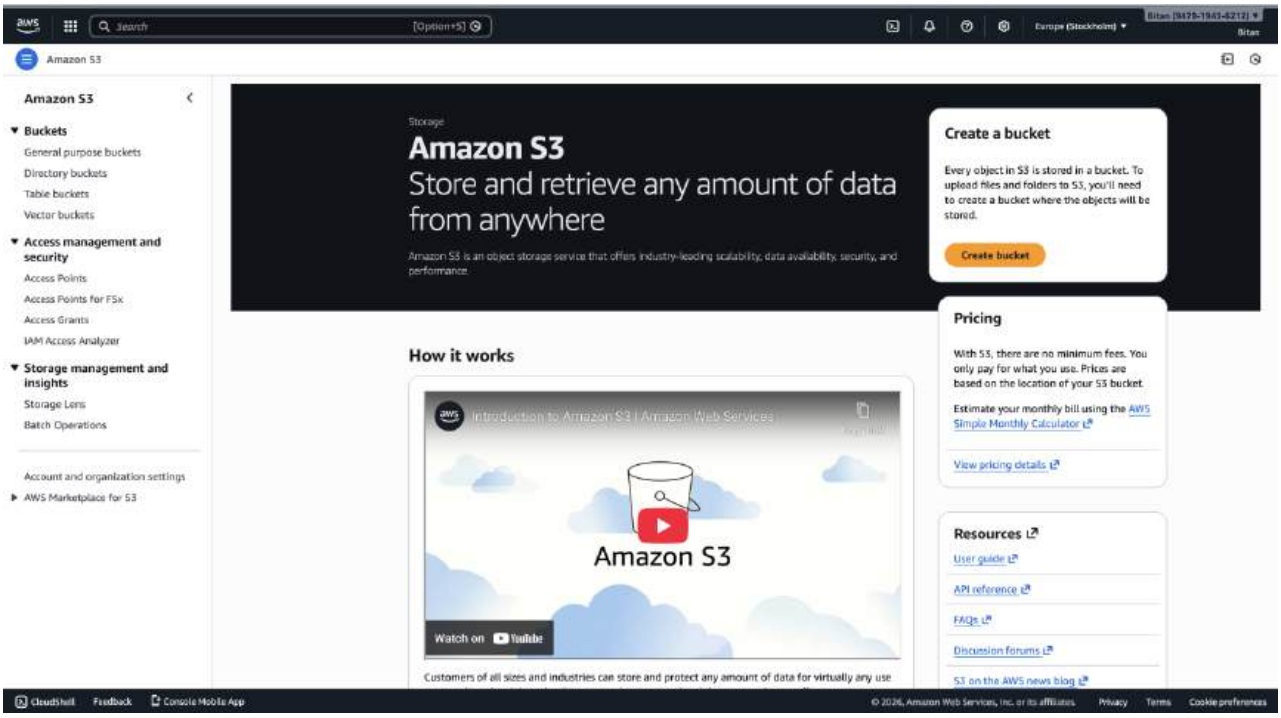
Procedure:

The steps are as follows: -

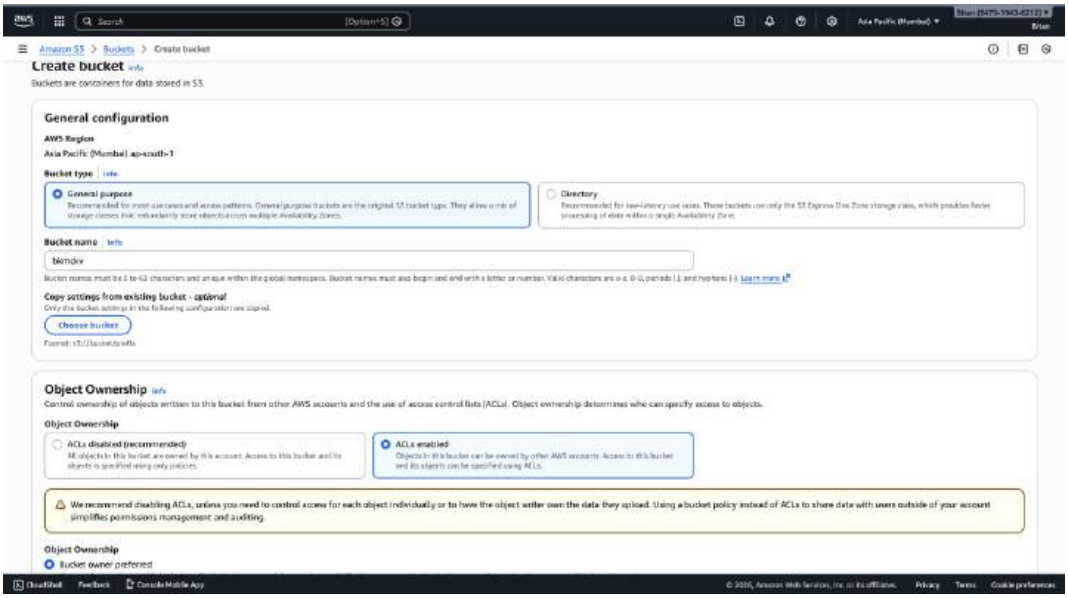
1. Access the AWS console, search for S3, and select the top option from the search results.

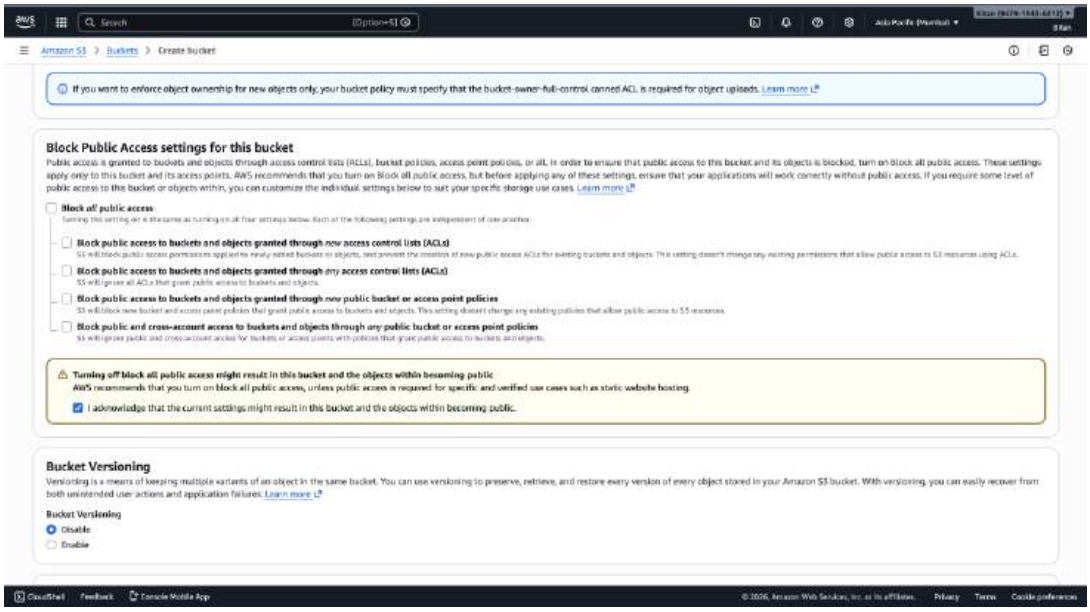


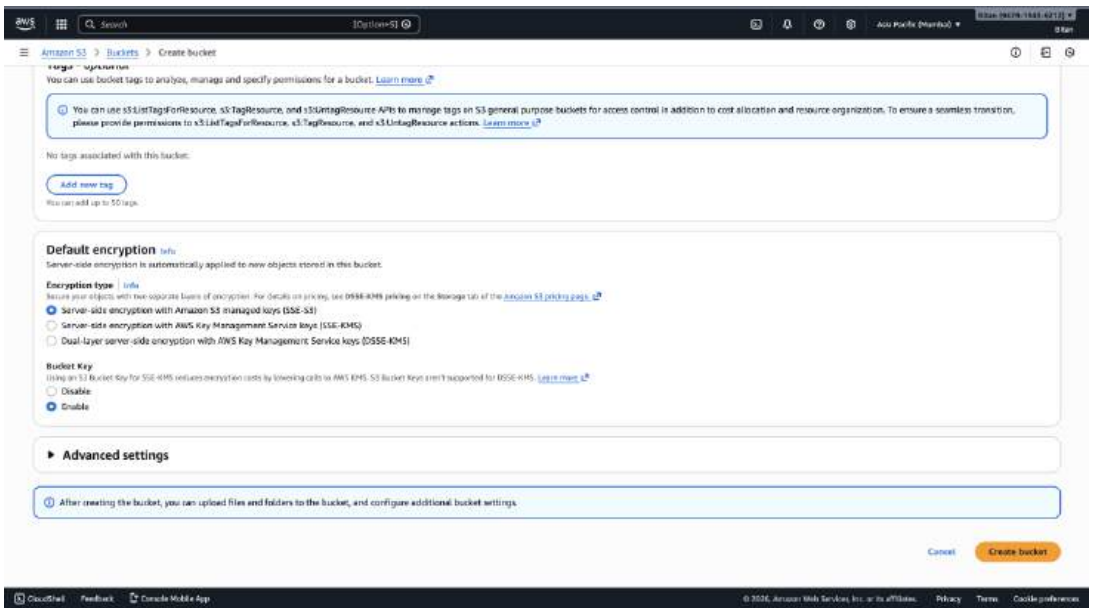
2. Click on 'Create bucket'.



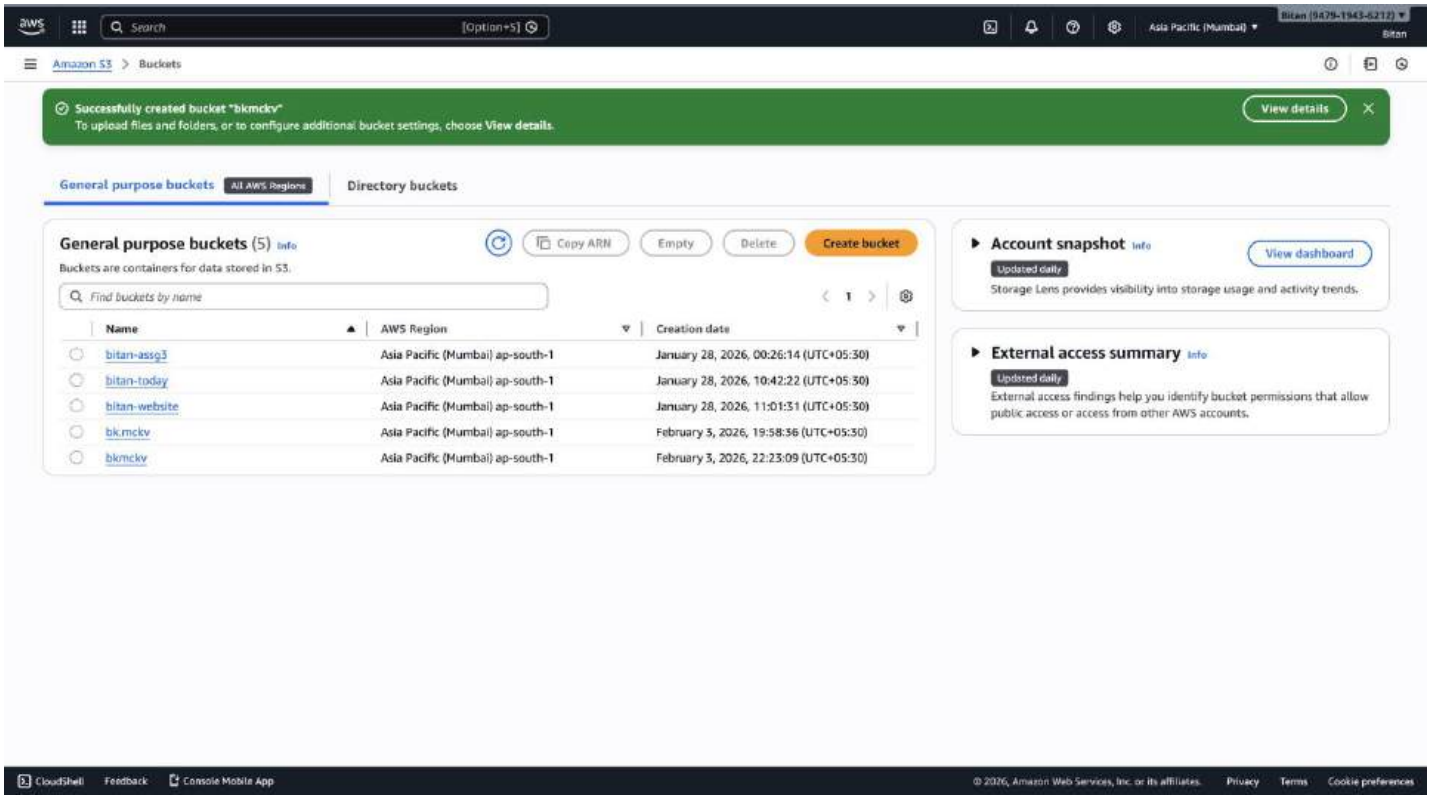
3. Fill up the required details: ’Aws Region’, ’Bucket name’. Click on ‘ACLs enabled’, uncheck ‘Block all public access’, tick ‘I acknowledge…’ and then click on ‘Create bucket’.



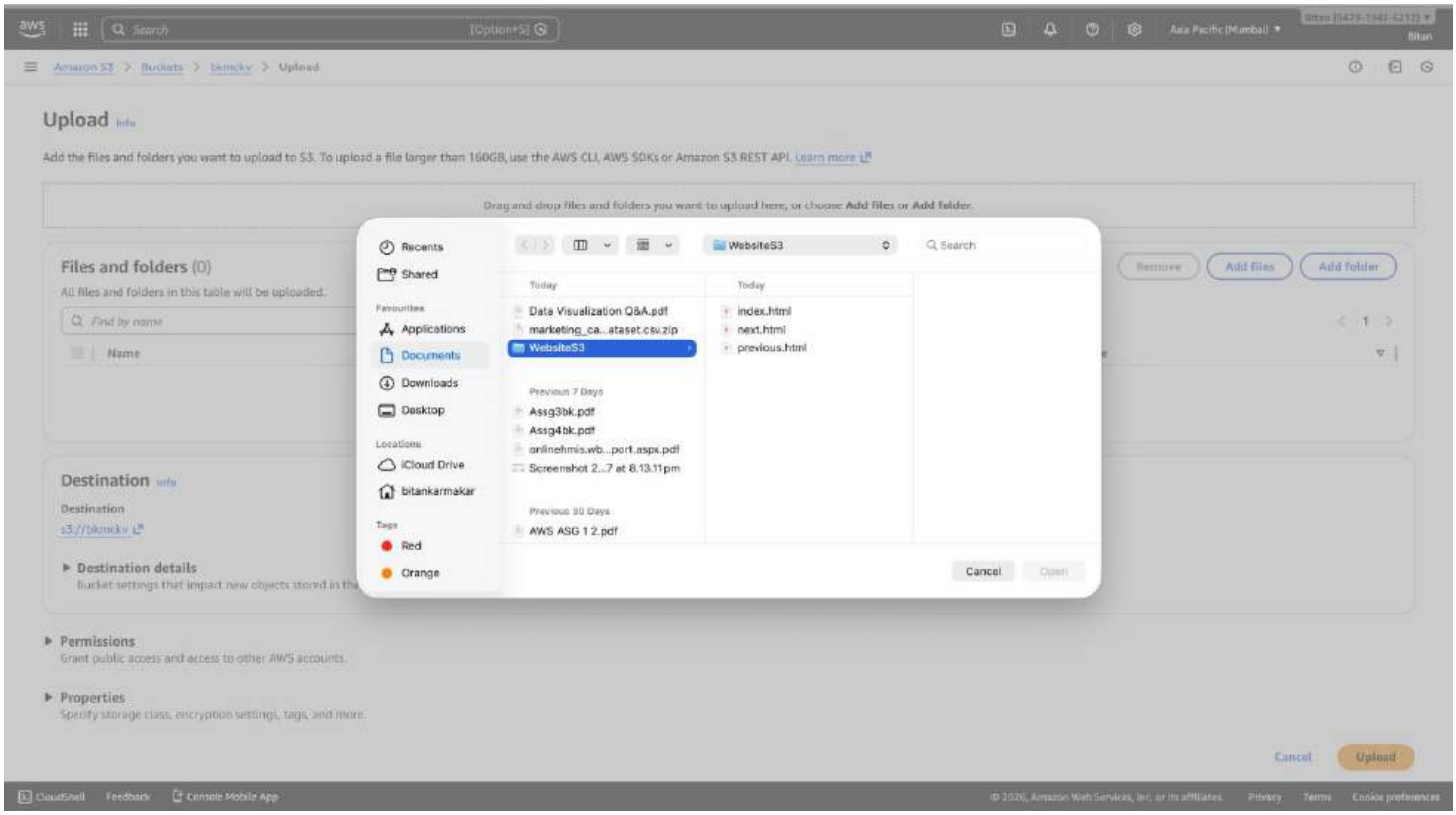




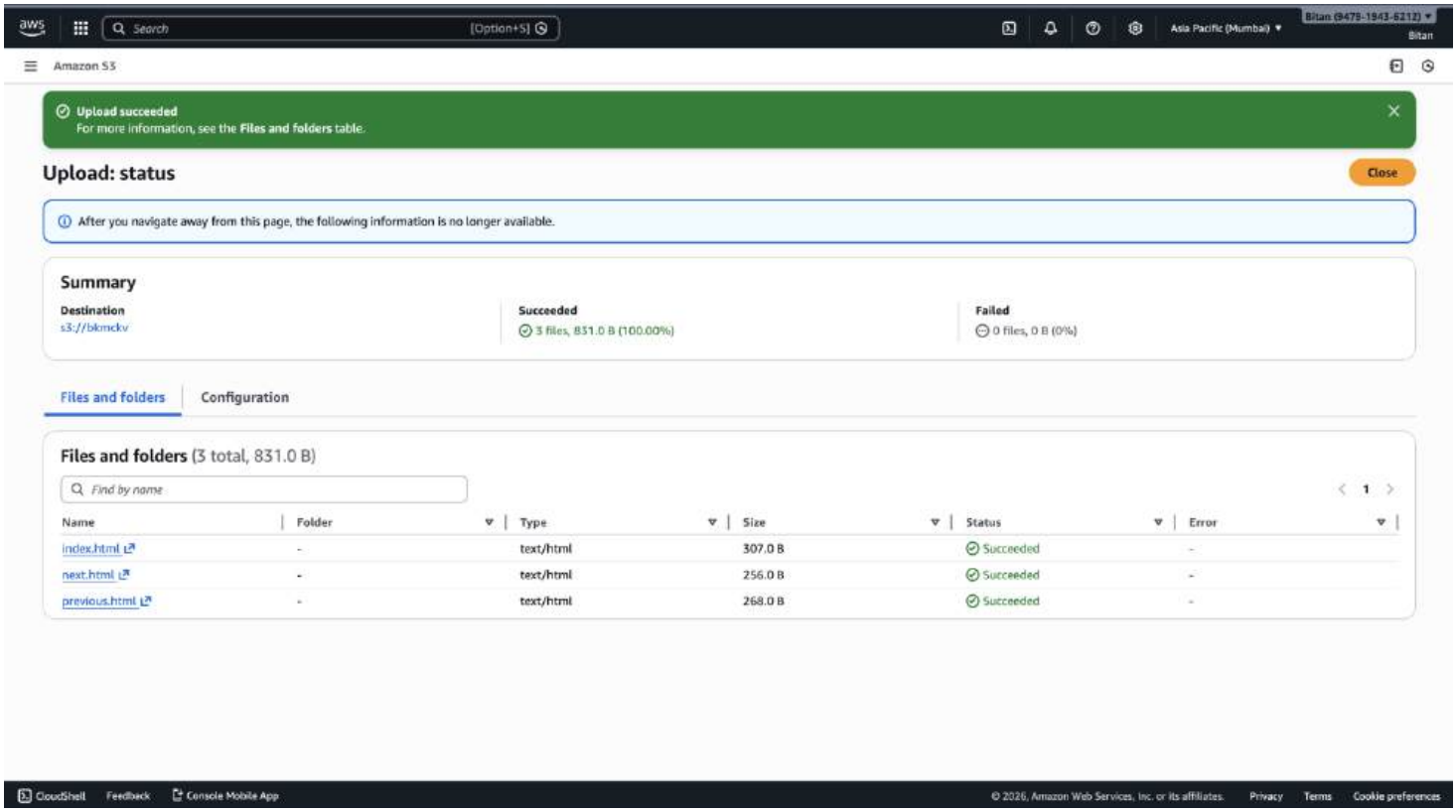
4. It's evident that the bucket has been successfully created. Click on the bucket name.



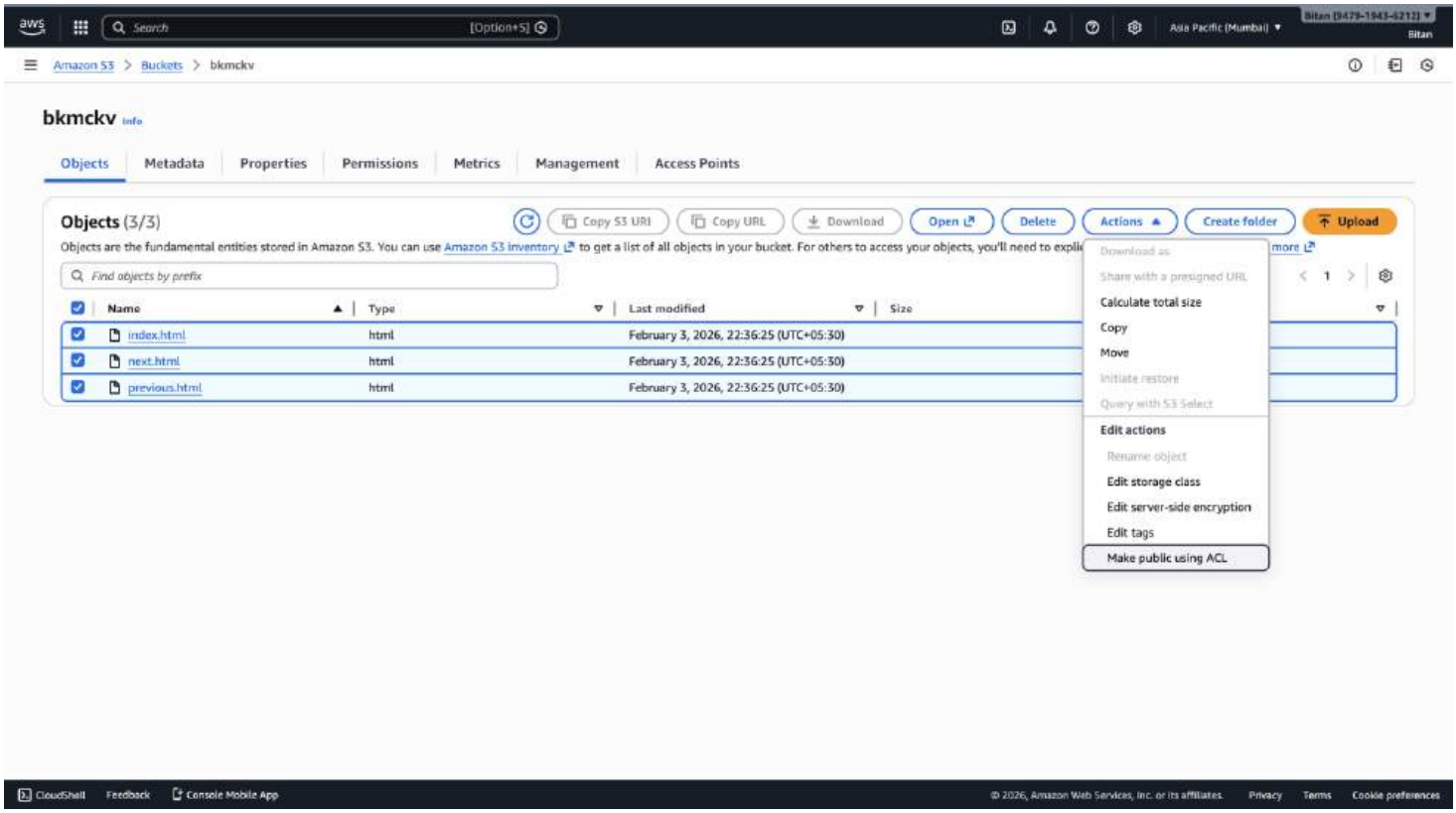
5. Click on 'Upload' then choose the three html files and upload those.

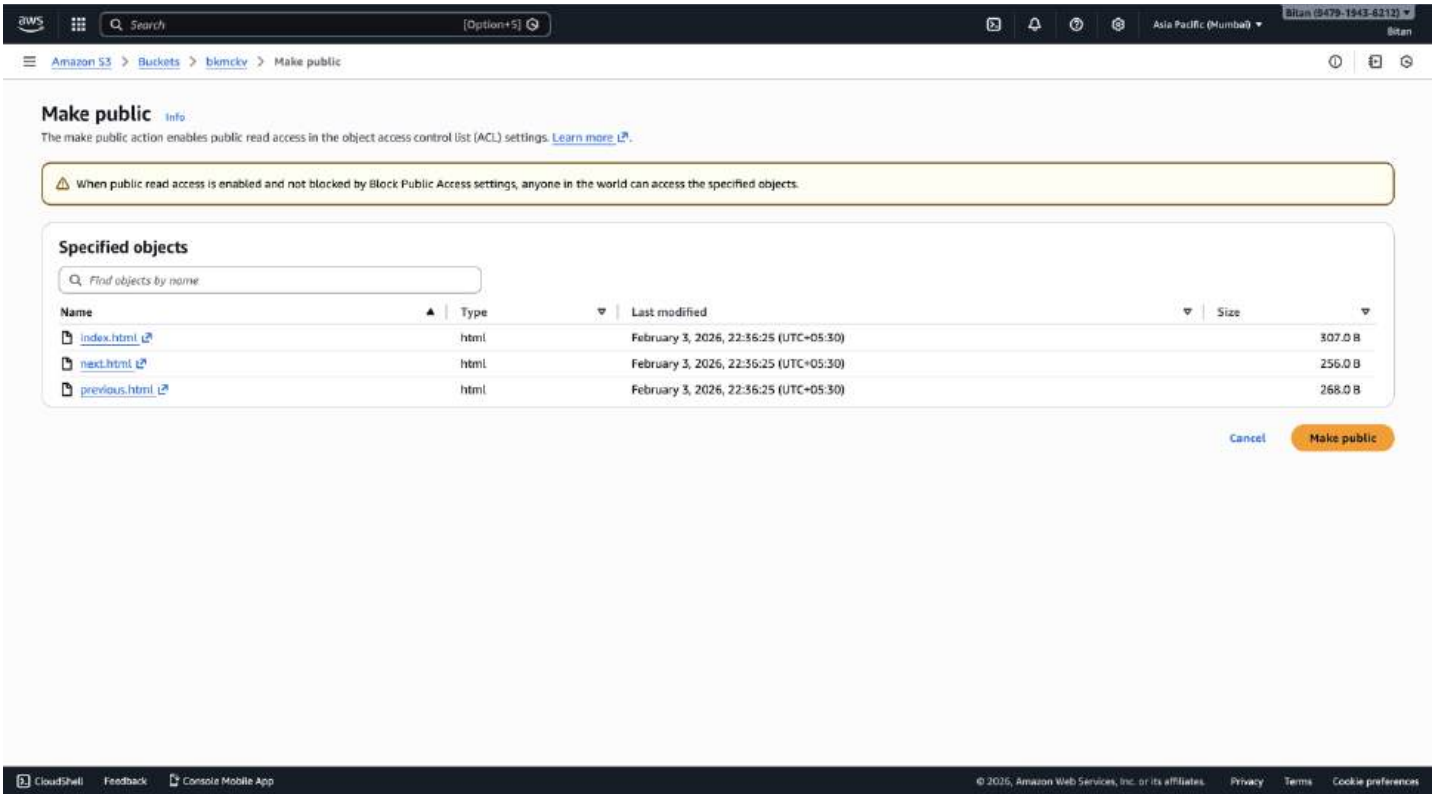


6. We can see that the three html files are successfully uploaded.

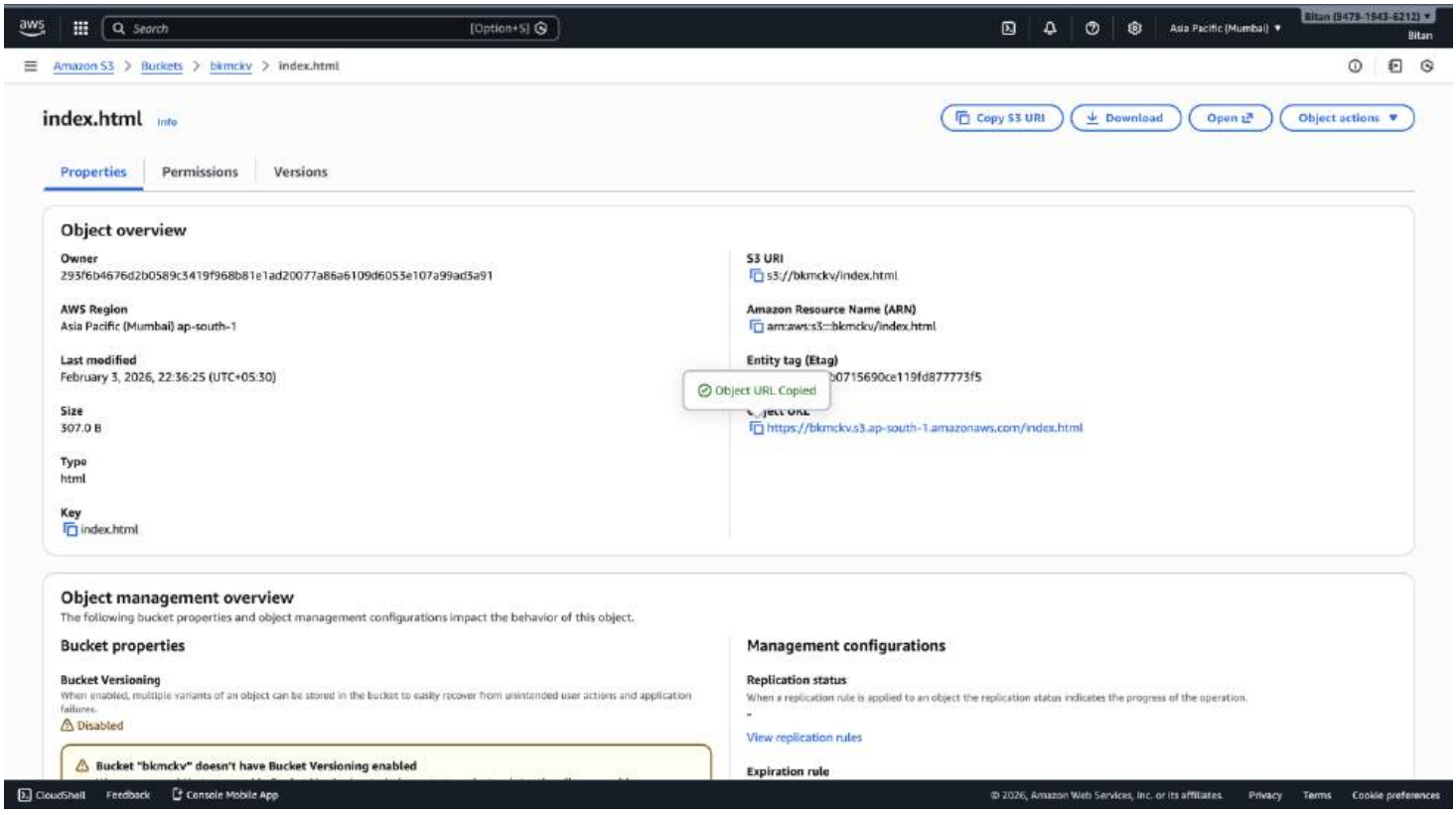


7. Now select all the three html files and click on ‘Actions’ Button and then ‘Make Public Using ACL’. Then click on ‘Make Public’ option in the next screen.

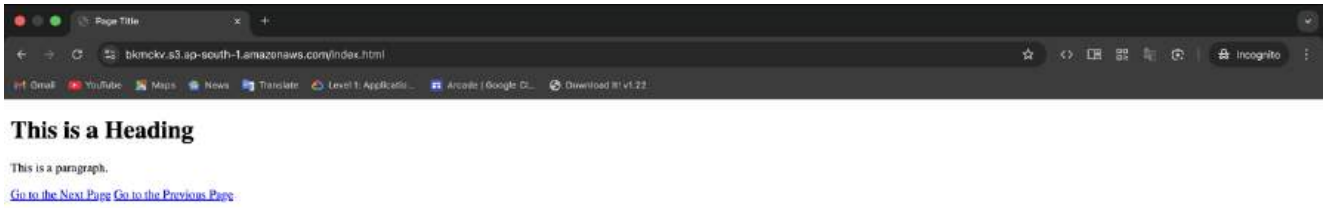




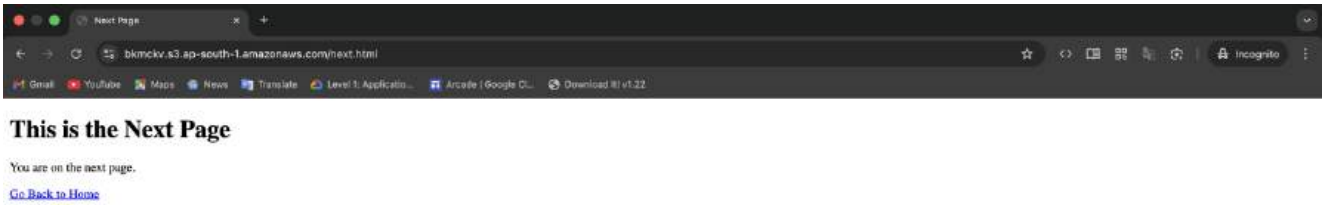
8. Now click on the main html file which is index.html here in this case. Then copy the URL of the index.html file.



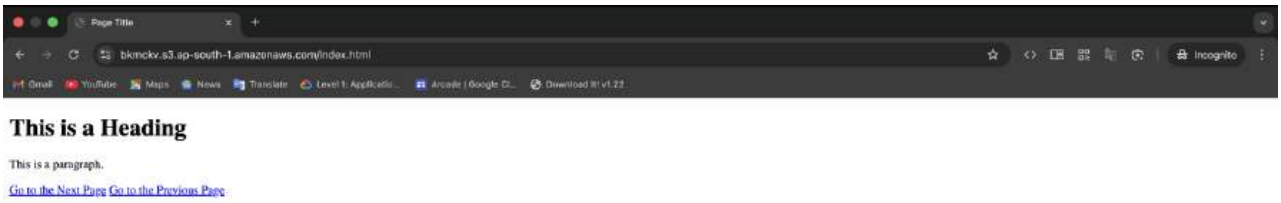
9. Paste the URL copied in a new inprivate window. You will see the expected results as follows. Click on the hyperlink ‘Go to the Next page’.



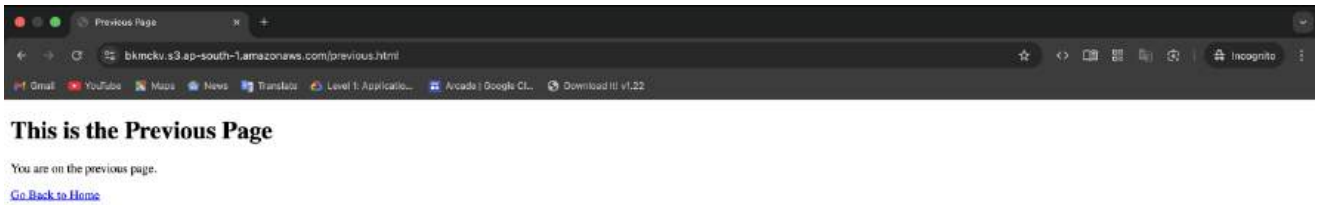
10. Go back to next page by clicking on the hyperlink.



11. Now click on the next hyperlink .



12. Again go back to previous page by clicking on the hyperlink shown in the page.



13. So by above steps we have successfully uploaded a static website on S3.

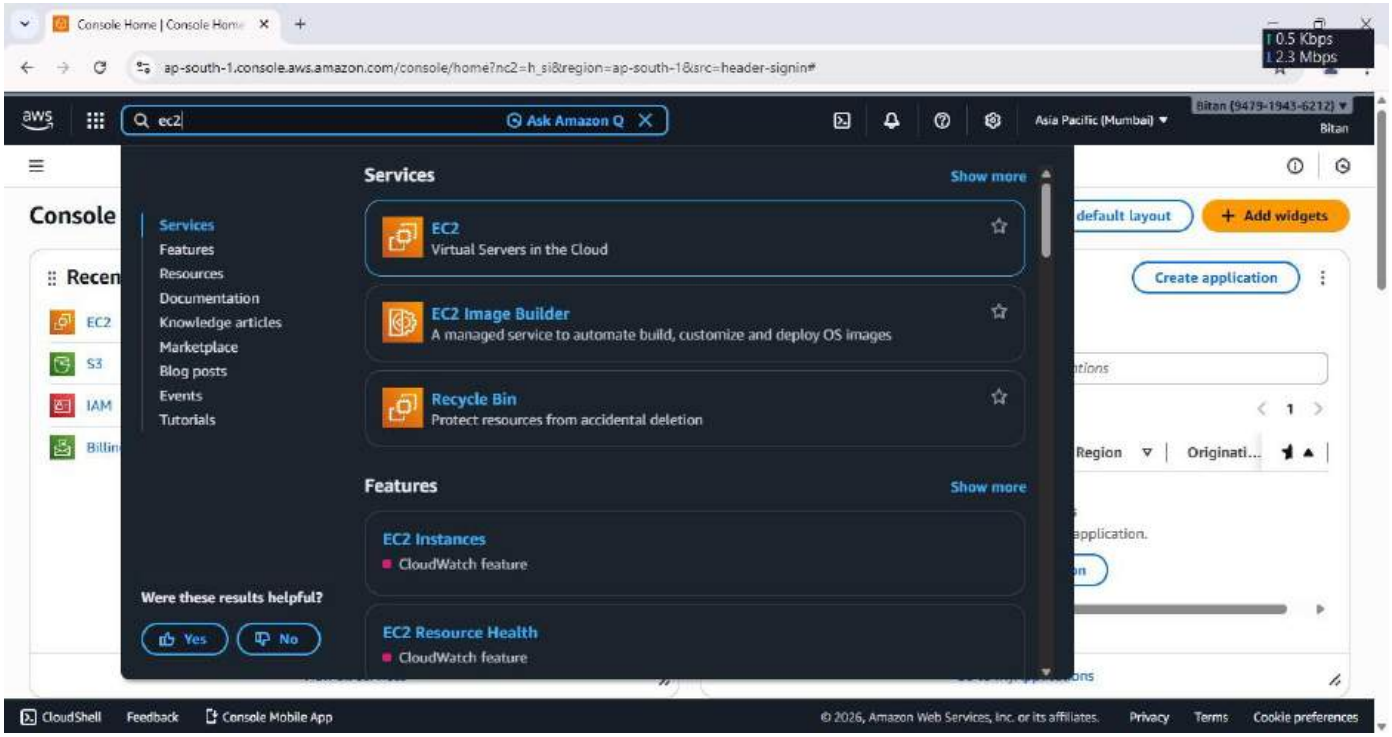
Assignment No.:7

Problem Statement: Hosting a website on EC2.

Procedure:

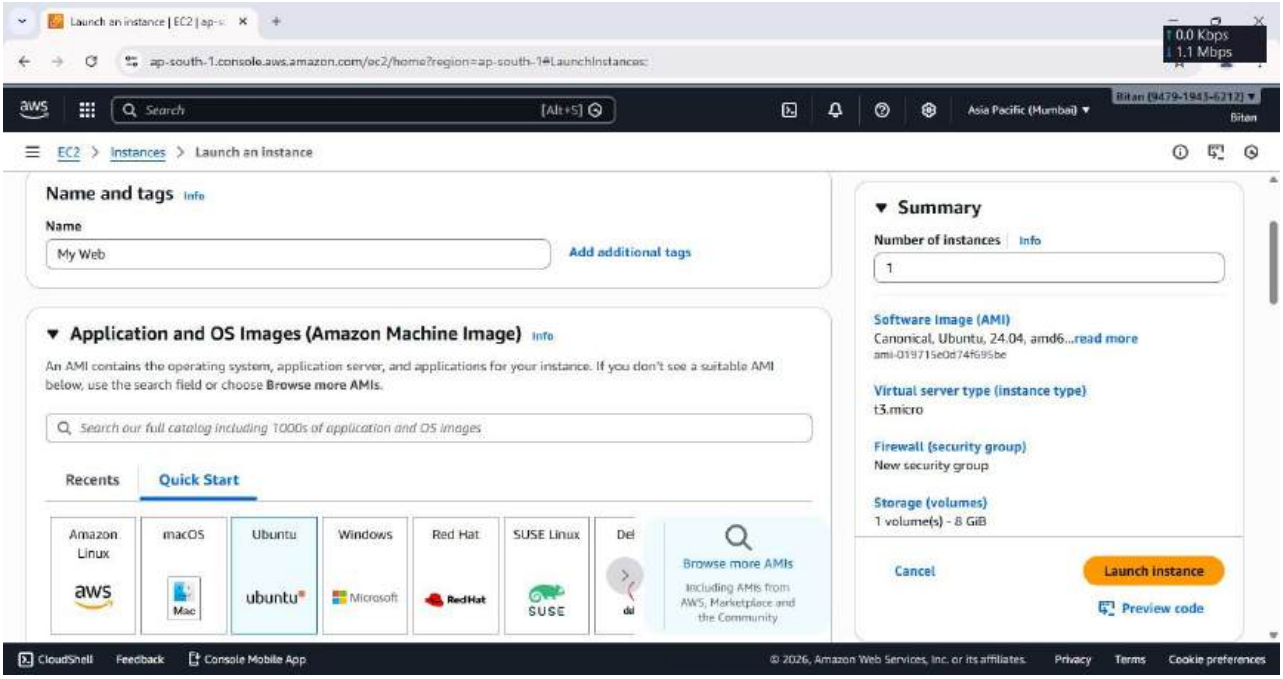
The steps are as follows: -

1. Sign up for an AWS account, search for ‘EC2’ then click on it.

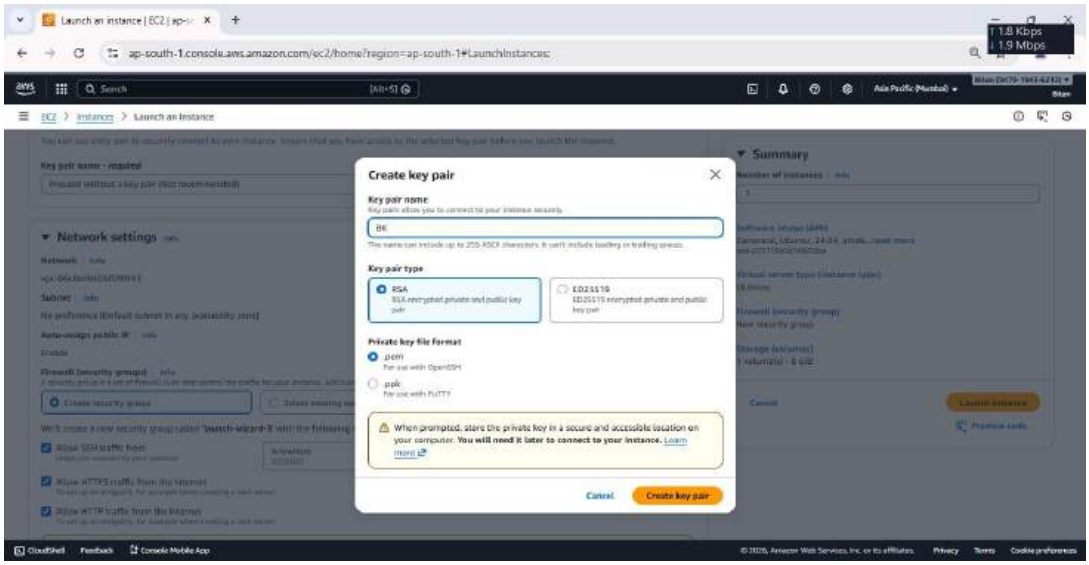


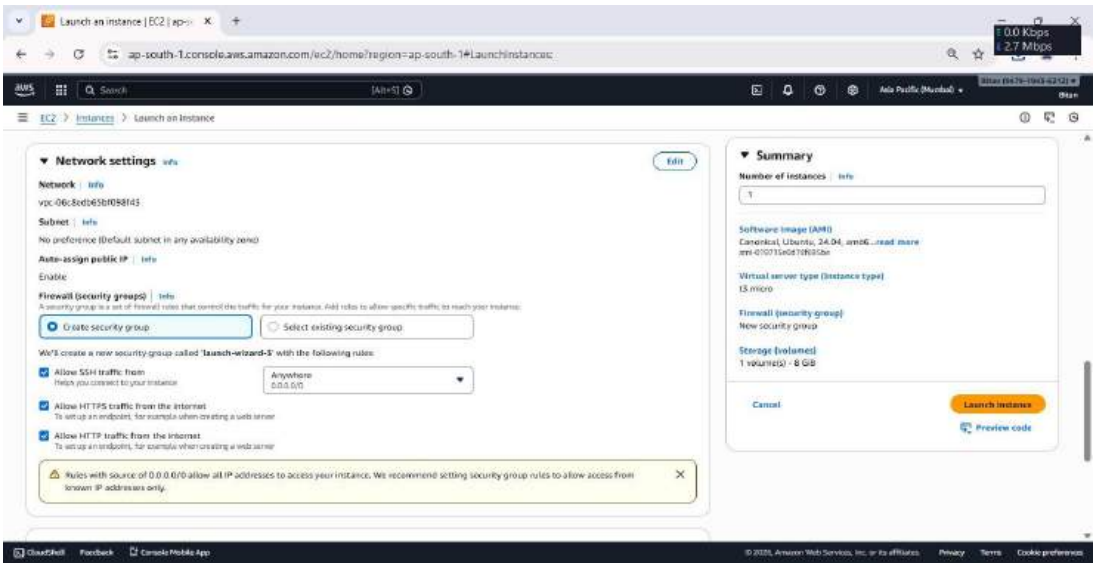
2. Click on ‘Launch instance’.

3. Fill up the required details->'Name' and under 'Application and OS Images' select 'ubuntu'.

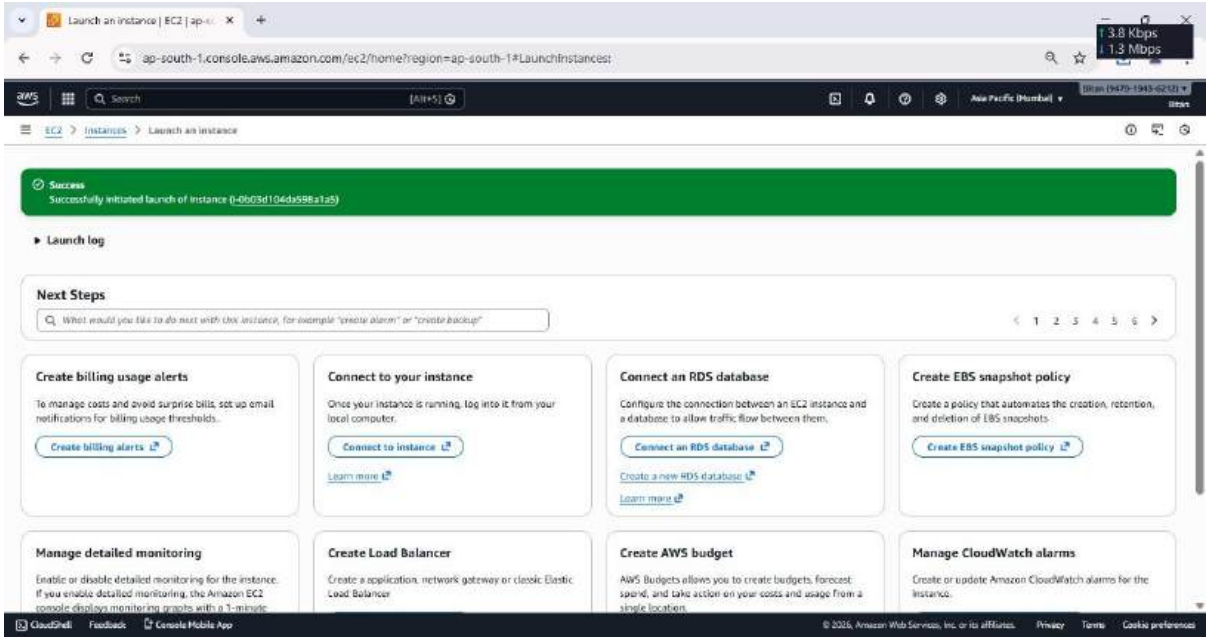


4.In key pair(login)', click on 'Create new key pair', give 'key pair name' and create it. Under 'Network Settings' tick on all checkboxes. Launch instance.

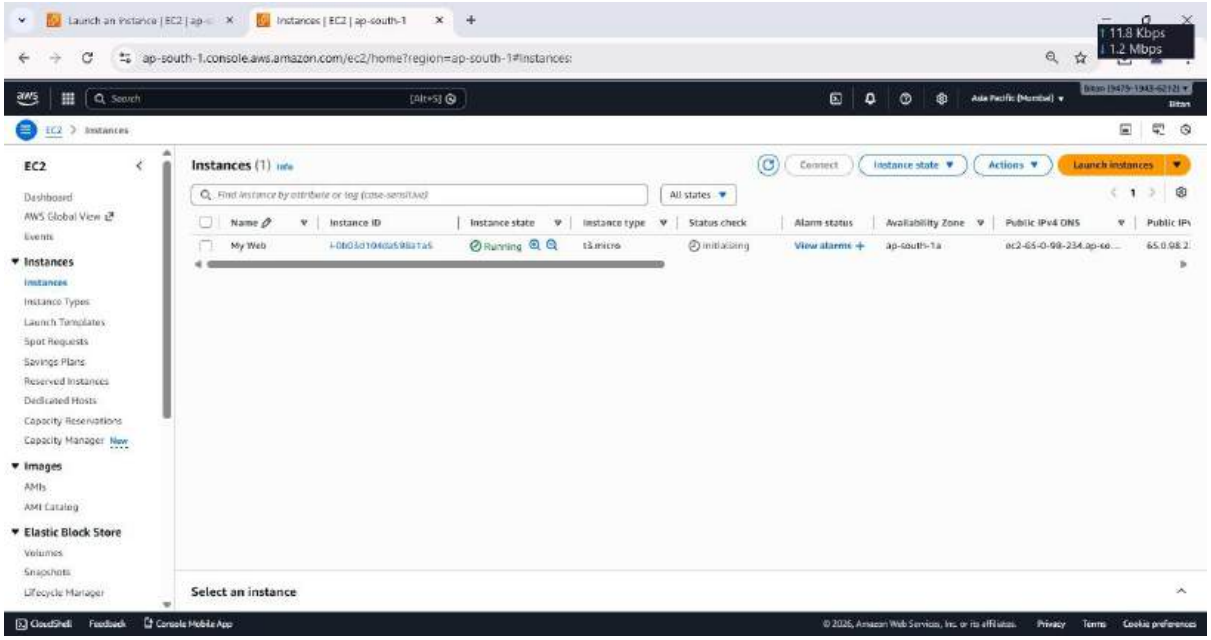




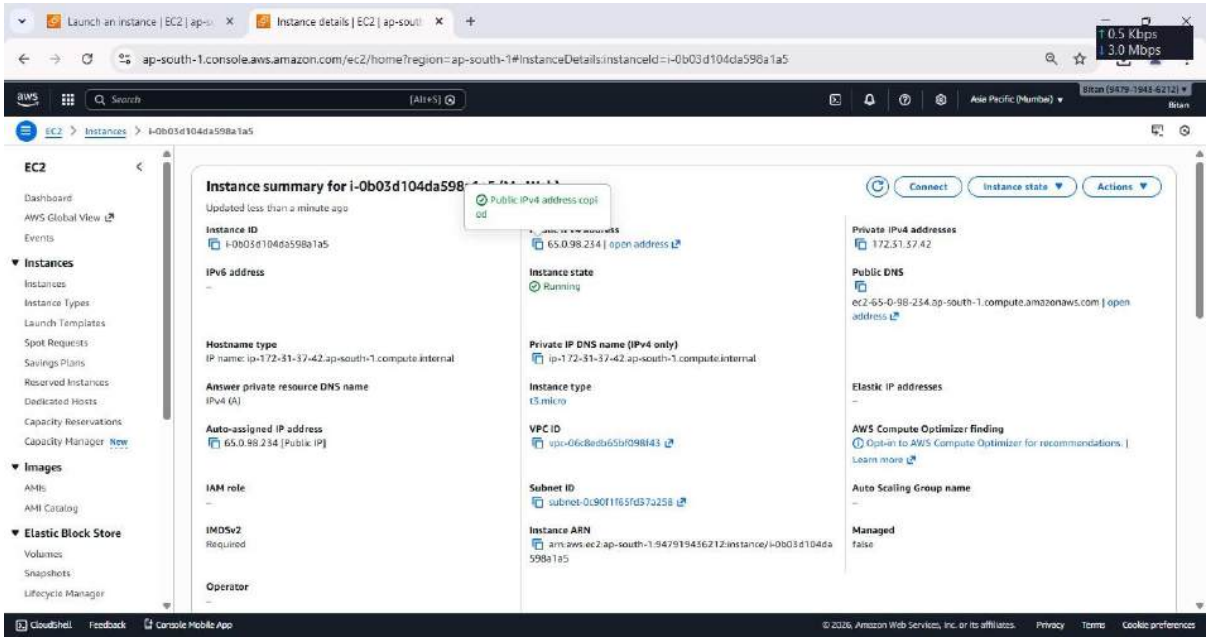
5. ‘My Web’ instance is created successfully. Then click on ‘Instances’.



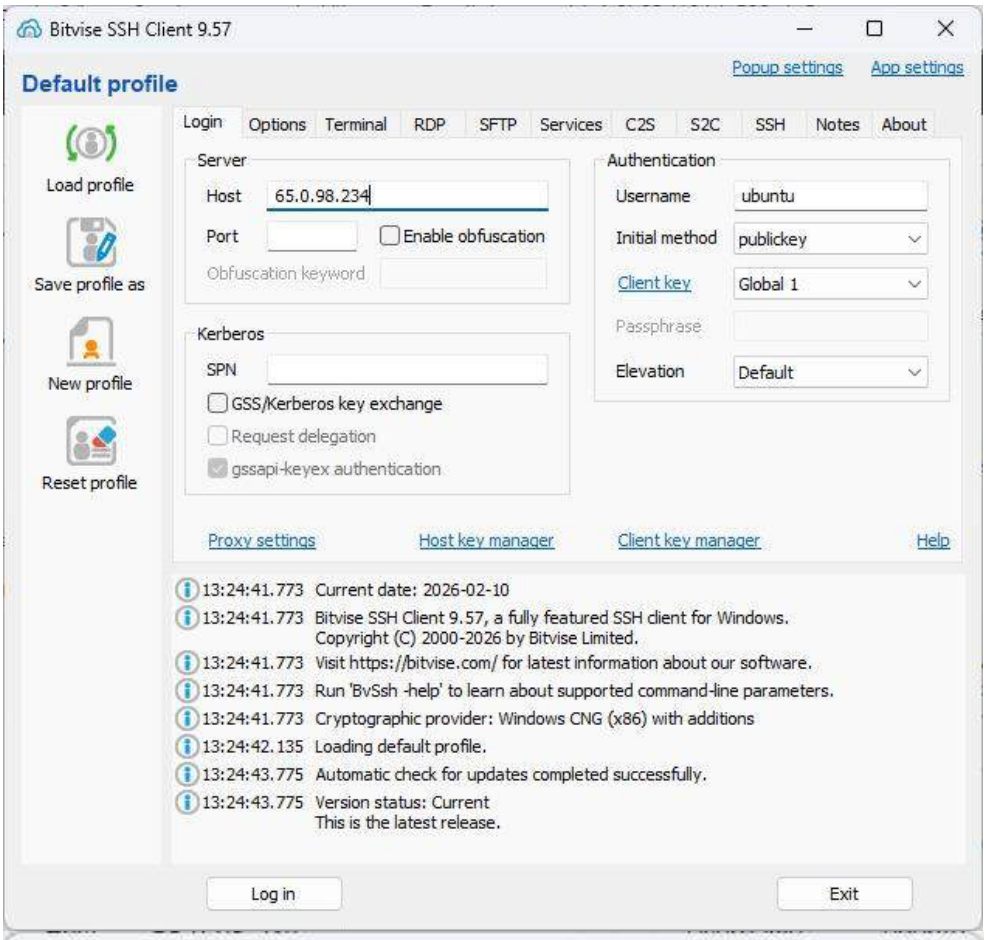
6. Now click on ‘Instance ID’.



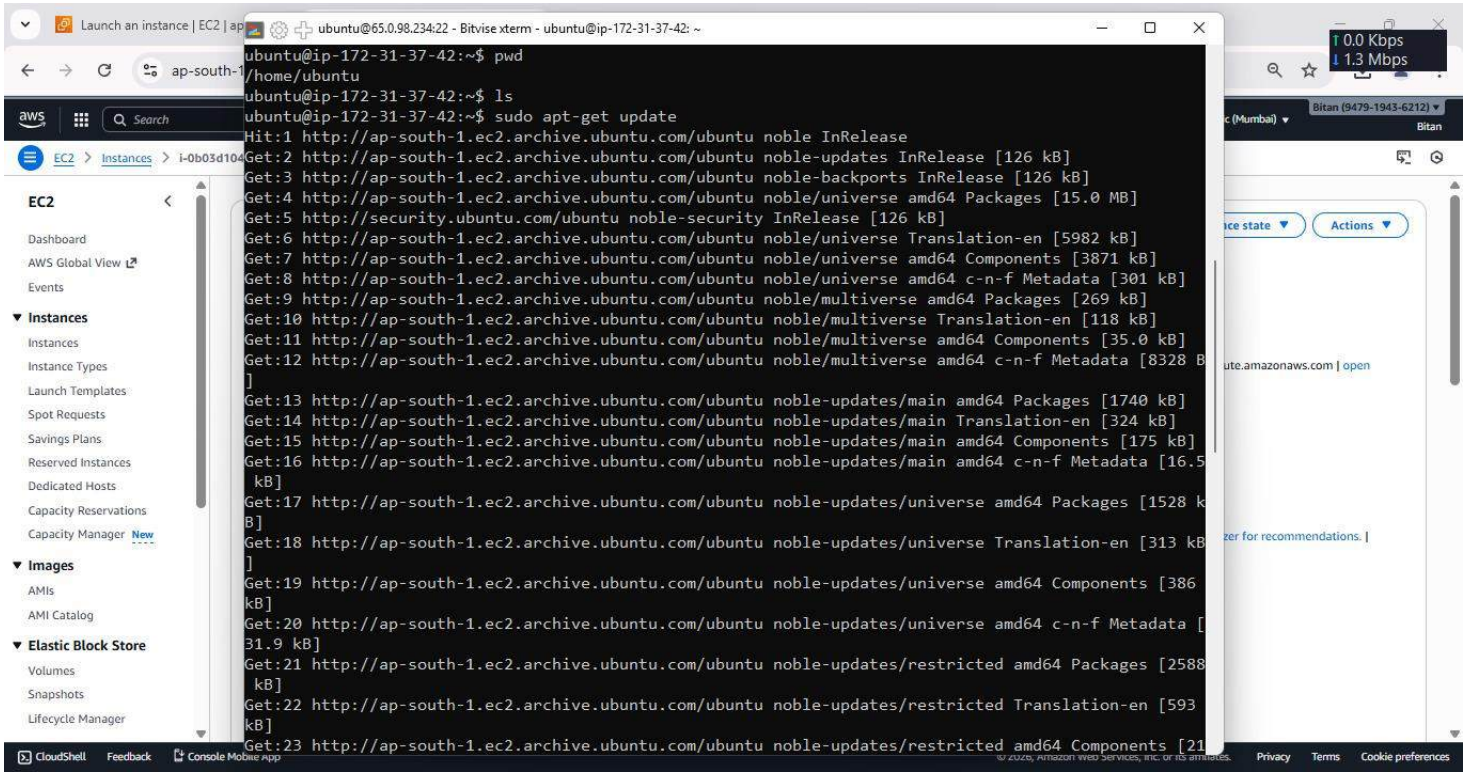
7. Click on ‘Public IPV4 address’ & copy it then open ‘Bitvise SSH Client’.

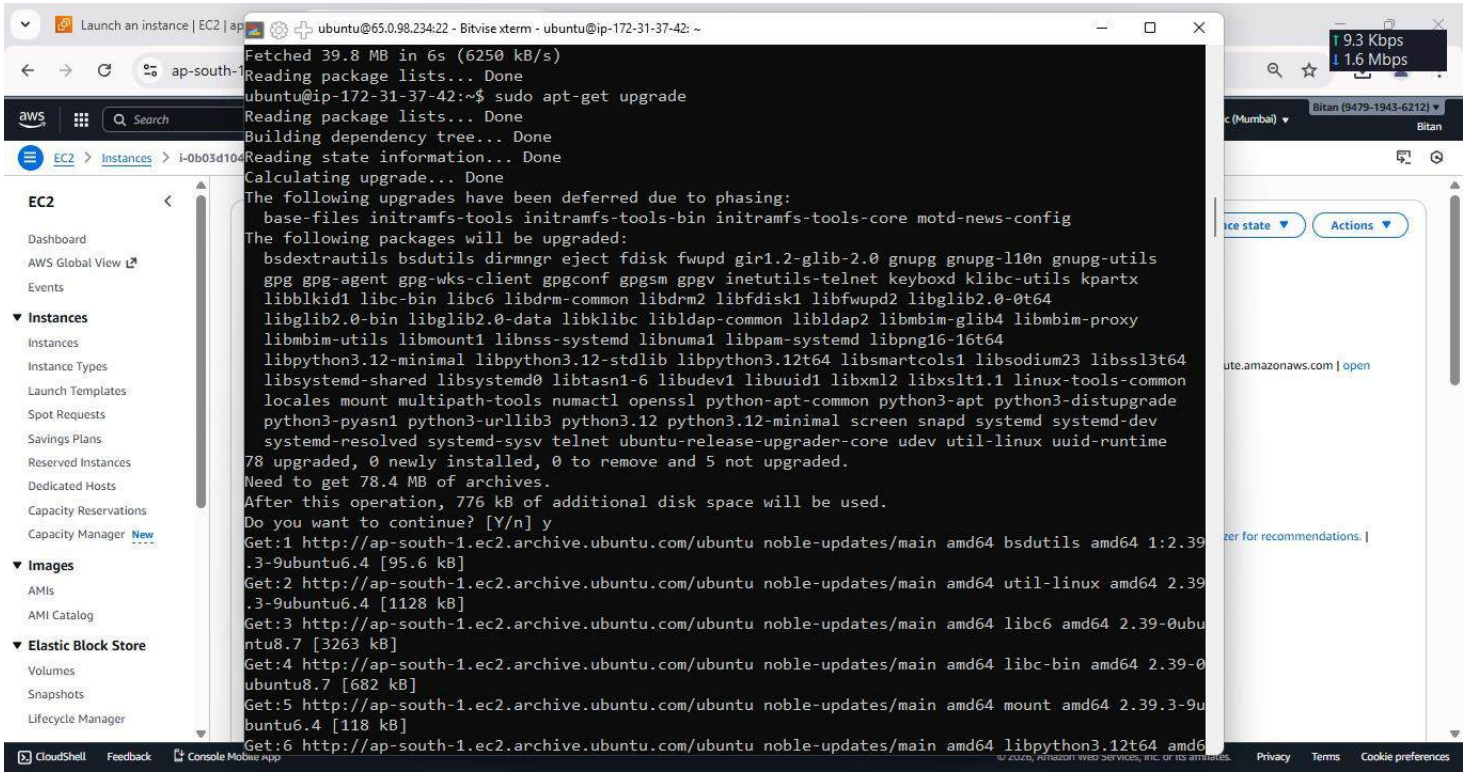


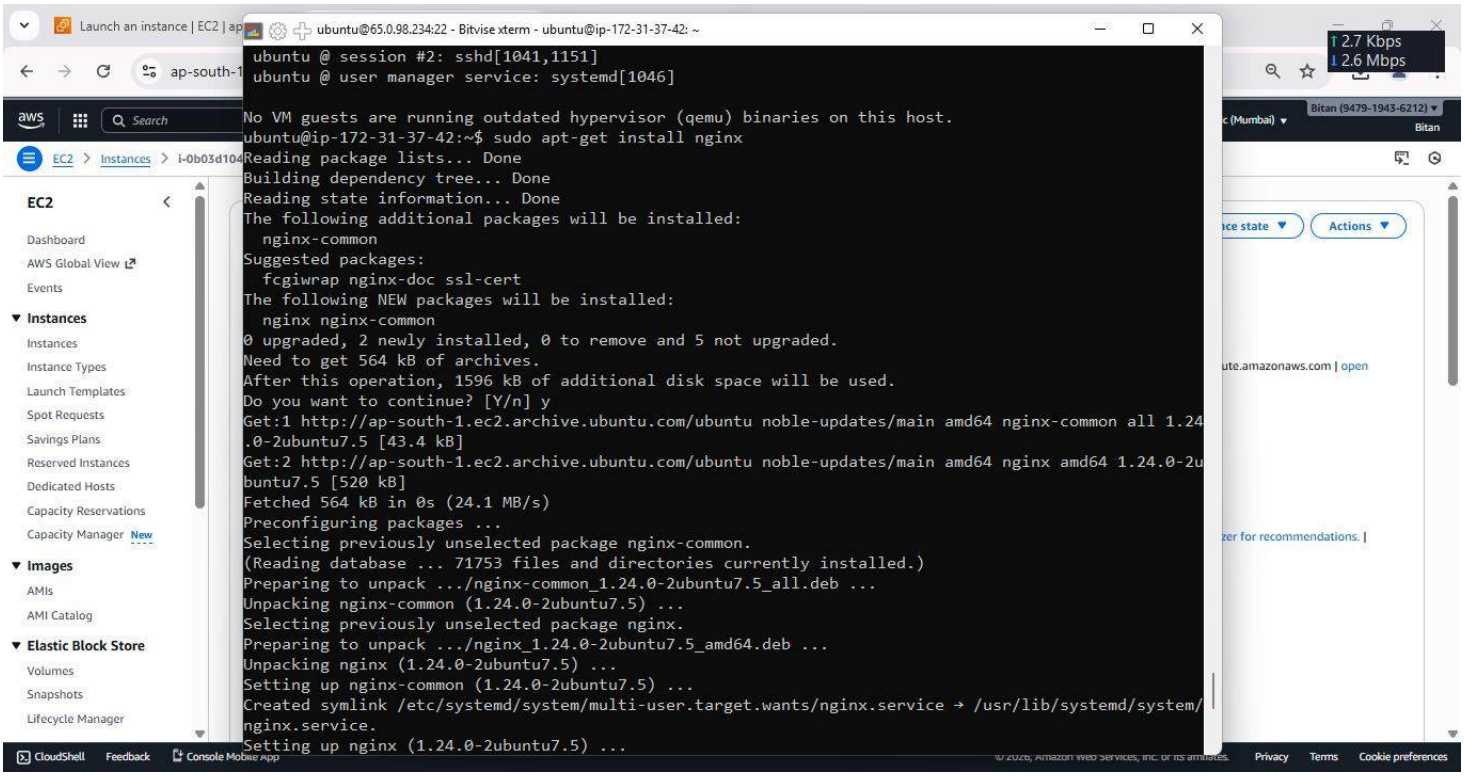
8. In Bitvise SSH Client, paste the ‘Public IPV4 address’ 7 click on ‘Client key Manager’. Under ‘Client key manager’, if there is any existing key then remove it and click on ‘Import’ , select the created key ‘BK’ then again click on ‘Import’ and then the key is successfully imported.Now under ‘Default profile’ give the ‘Username’ as ‘ubuntu’ then select Global1 from ‘Client Key Manager’ then click on ‘Log in’ & ‘Accept and save’.



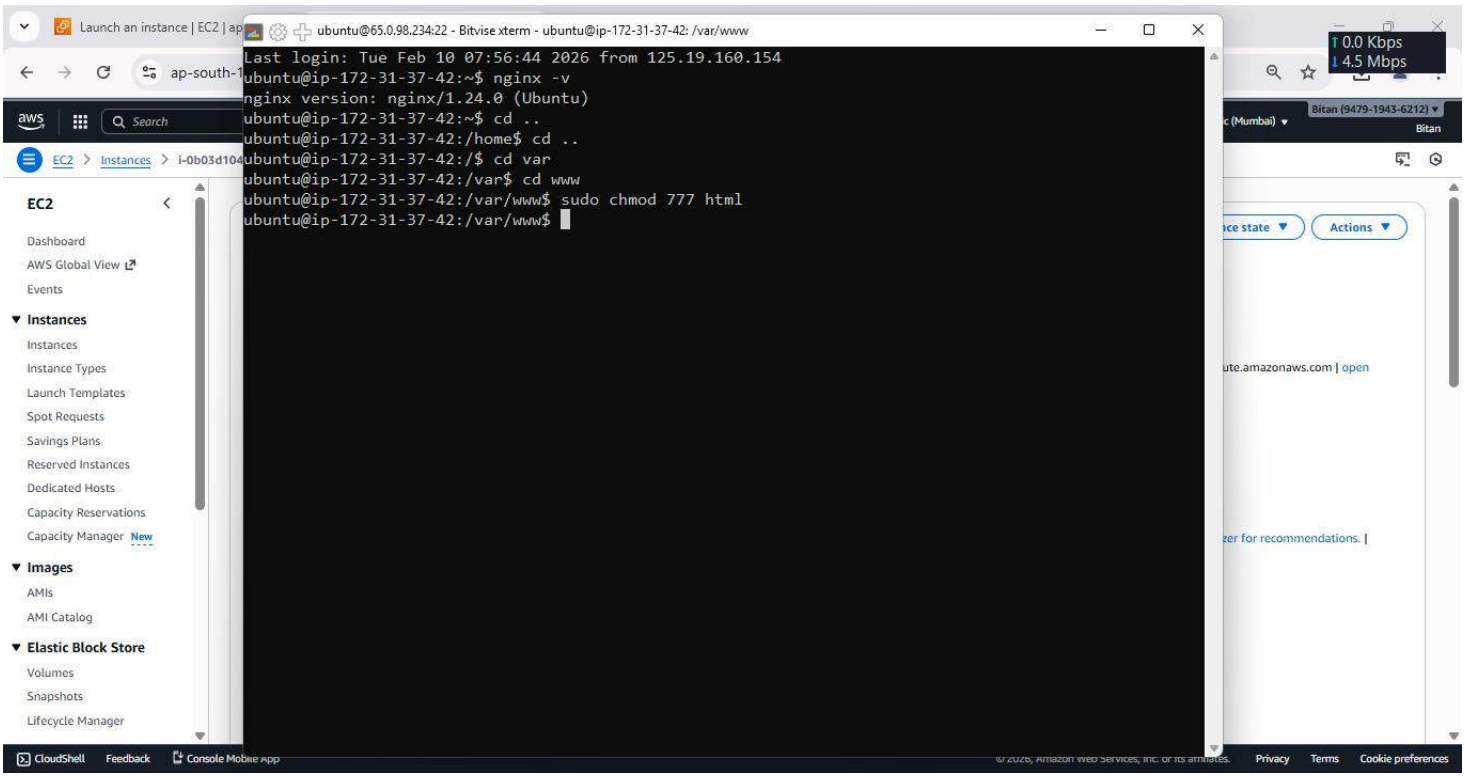
9. In ‘Bitvise SSH client’, under ‘Default profile’ click on ‘New terminal console’ and there run ‘pwd’ command to check where we are. In terminal console, type the following commands.-> ‘sudo apt-get update’ , ‘sudo apt-get upgrade’ , ‘sudo apt-get install nginx’.



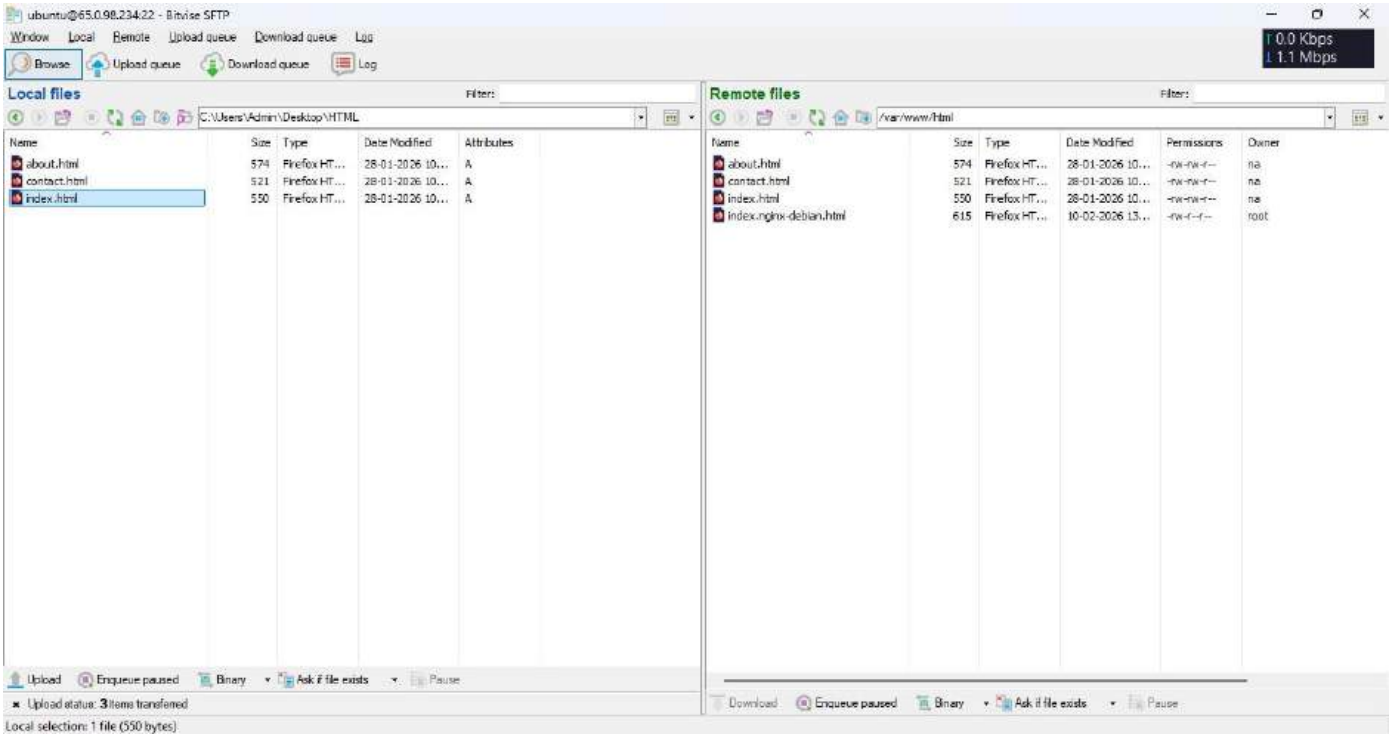




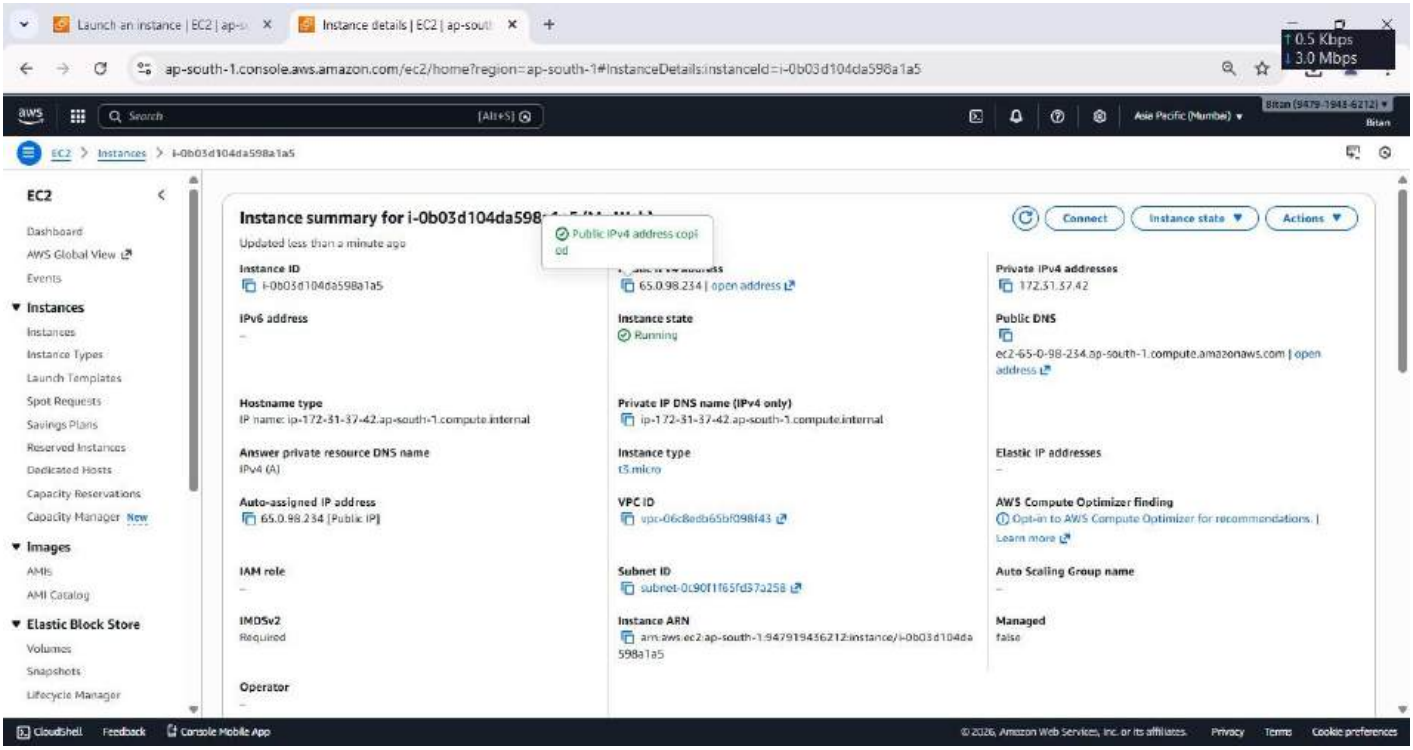
10. Check if www folder appears or not by using 'ls' command inside 'var' folder Now to give the file access permissions to it go back to the 'www' directory, and type the command 'sudo chmod 777 html' and press 'Enter'.



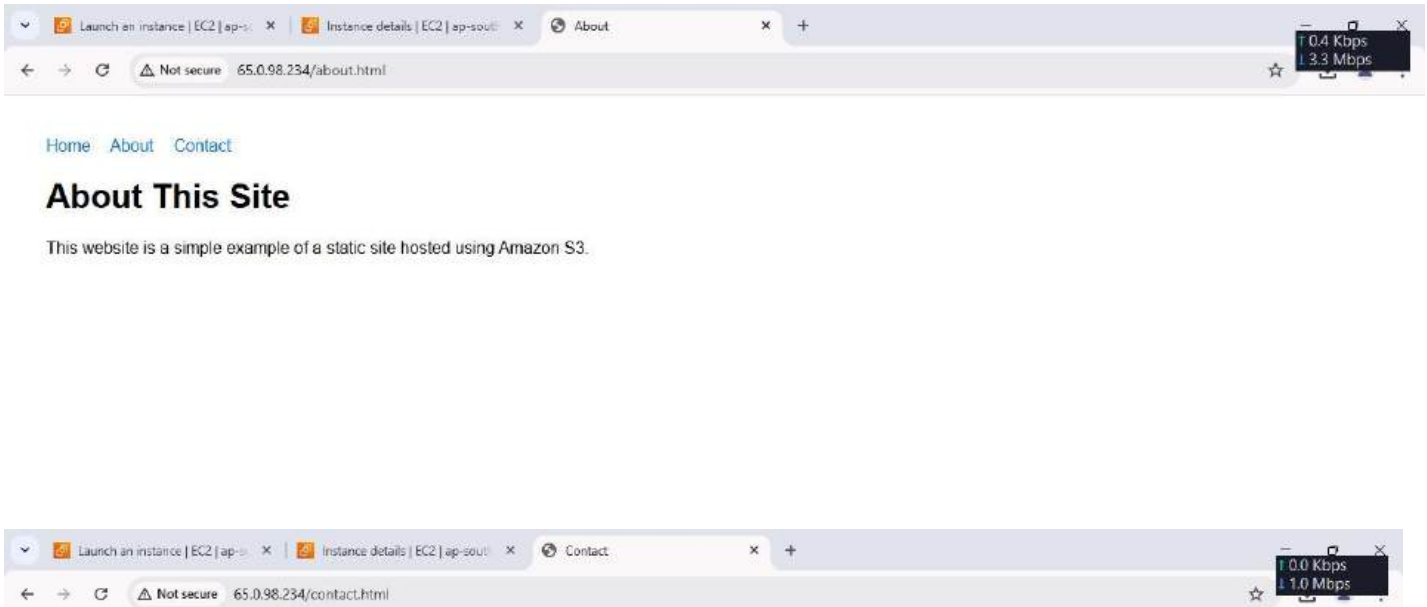
11. Now going back to the ‘SFTP Window’ under the ‘Remote Files’ open the HTML directory and drag & drop the HTML files.



12. Now go back to the ‘AWS Window’ and copy the ‘IPV4 address’.



13. In a new window paste the ‘IPV4 address’.



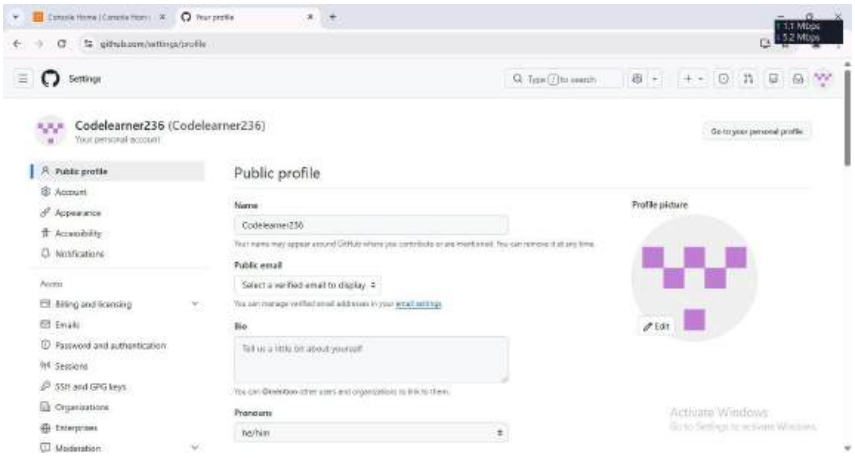
Assignment No:8

Problem Statement:

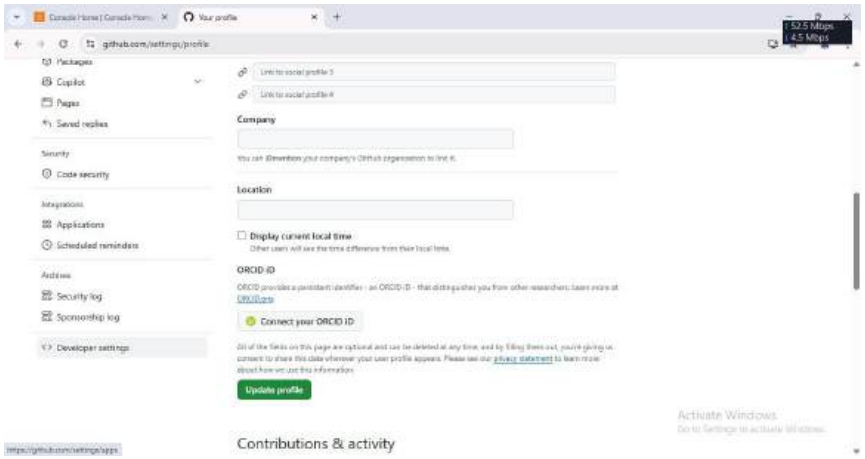
Deploy a project from a local machine to GitHub and vice versa.

Procedure:

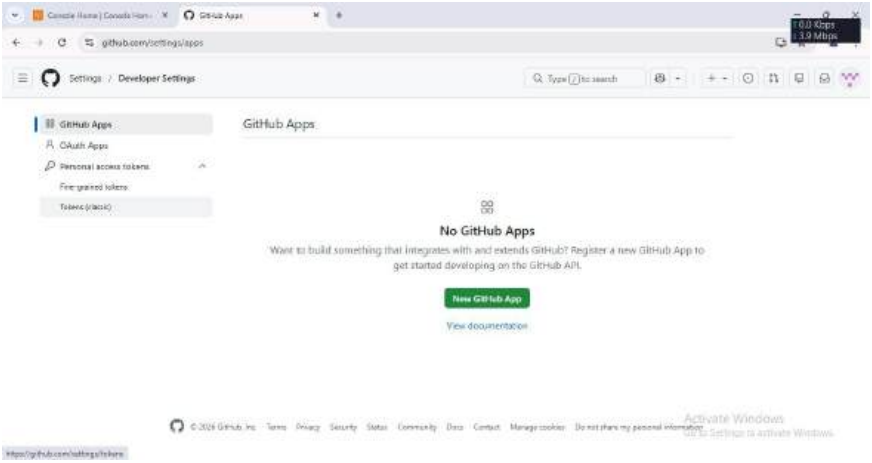
1. **Login to the GitHub account and from there click on the profile picture and from there click on "Settings"**



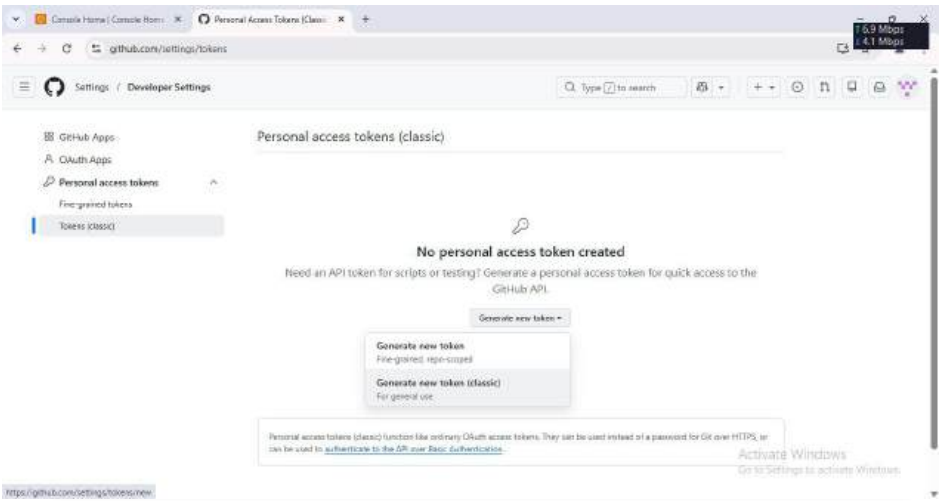
2. **And then scroll down then click on "Developer Settings"**



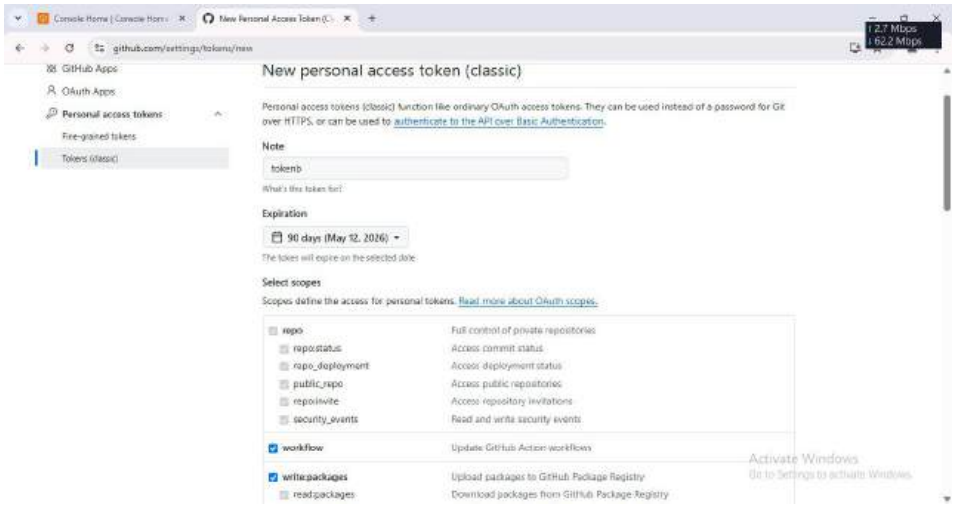
3. **Now click on "Personal access tokens" drop down then click on "Tokens(classic)" and then click on “New GitHub app”**



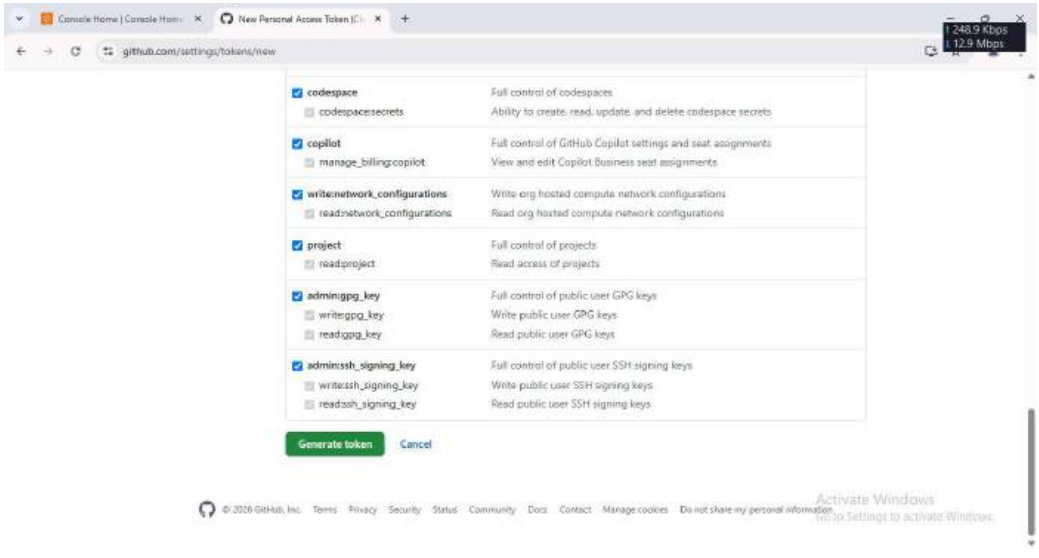
4. **Now click on "Generate new token" drop down then click on "Generate new token(classic)".**



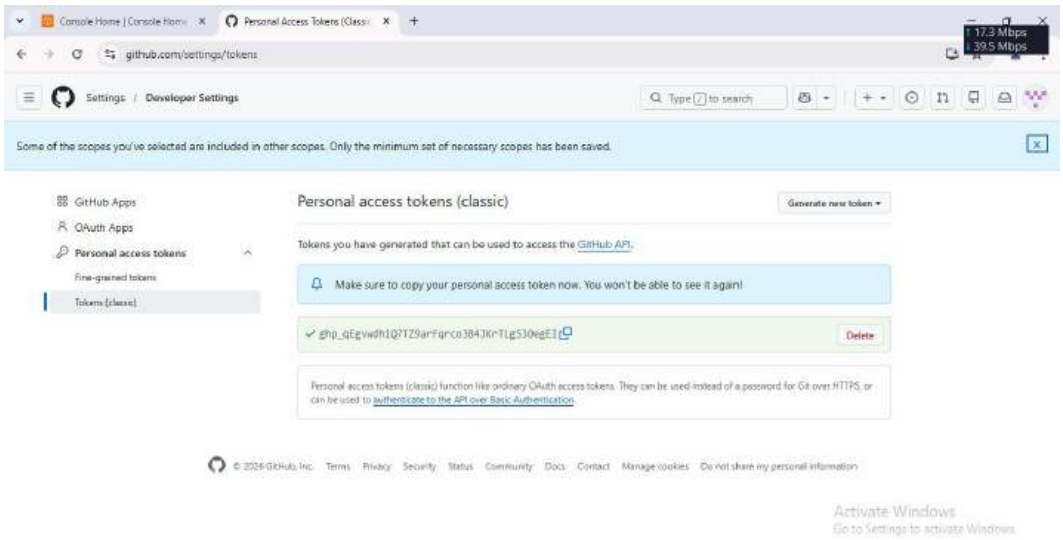
5. **Now enter the token name and select "Expiration" as "90 days". Here the name is "tokenb"**



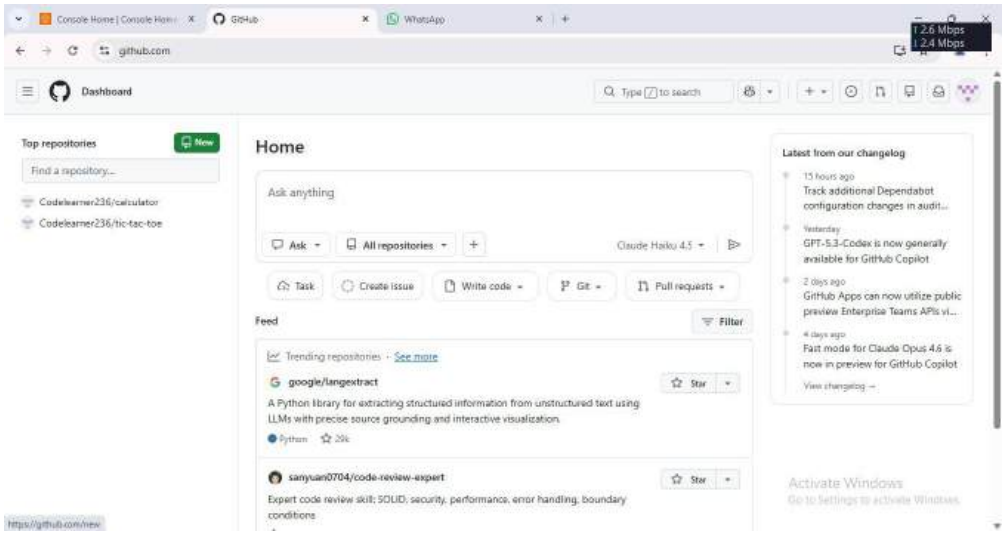
6. **Now check all the check-boxes present there and click on "Generate token".**



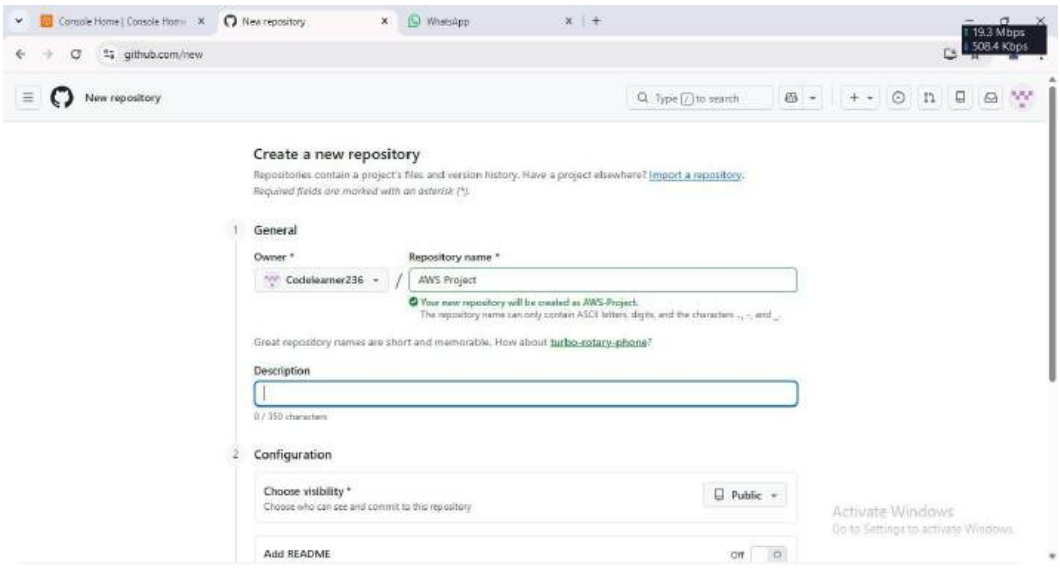
7. **Now copy the token address and paste it into the text file**



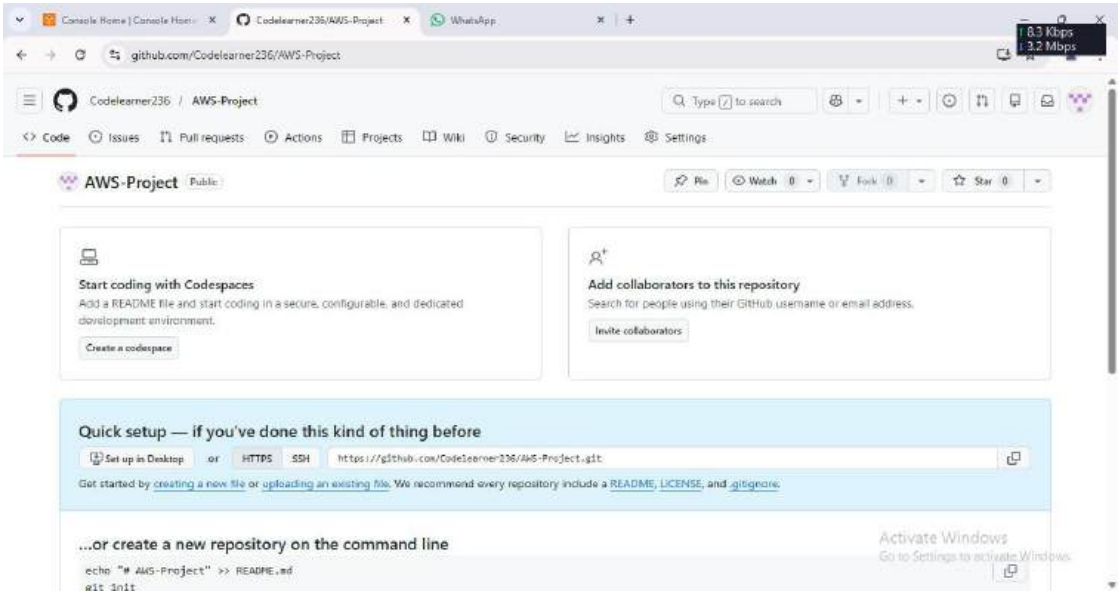
8. **Now again come back to the GitHub dashboard and from there click on "Create Repository".**



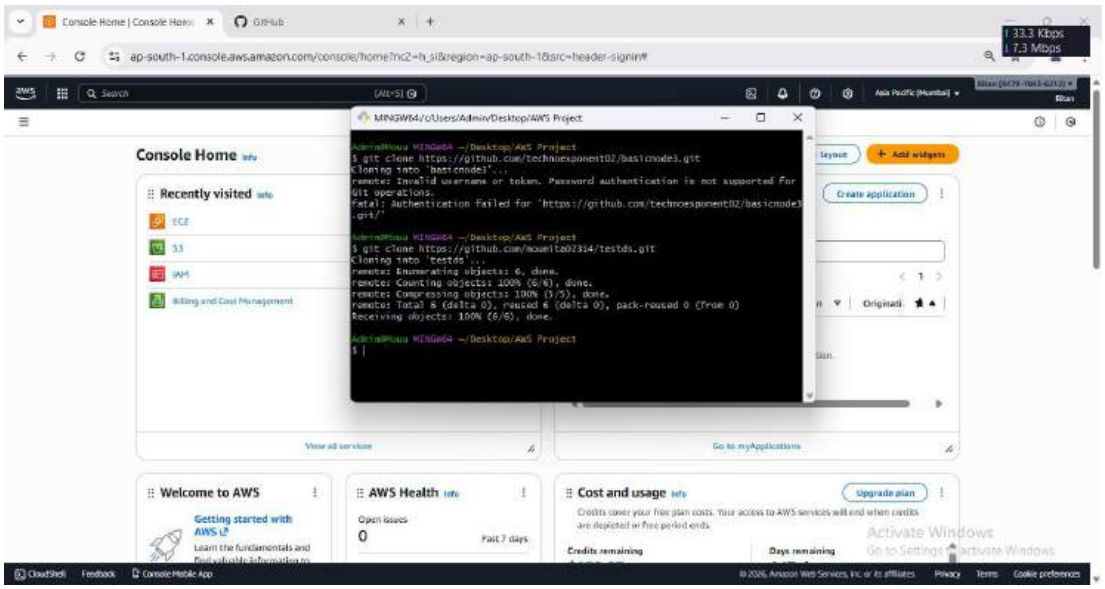
9. **Now enter the Repository name and click on "Create repository". Here the repository name is "AWS Project".**



10 Now copy the Repository Address"https://github.com/Codelearner236/AWS-Project.git"and paste it into the same text file. Now we are ready to deploy any project from Github to our local machine.



11 So to do so, open the folder that we have created and from there we will open Git Bash. Here the folder name is "AWS Project".

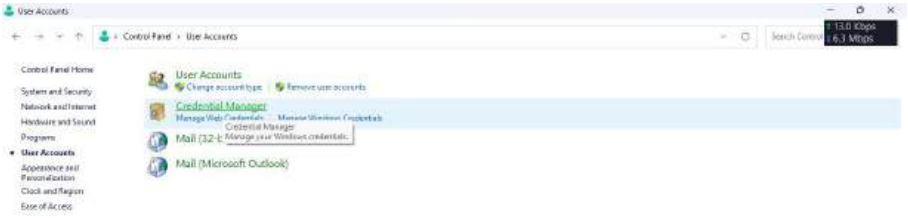


12 Clone "testds" repository from GitHub of user "moumita02314" to the local machine using the following command "git clone https://github.com/moumita02314 /testds.git".

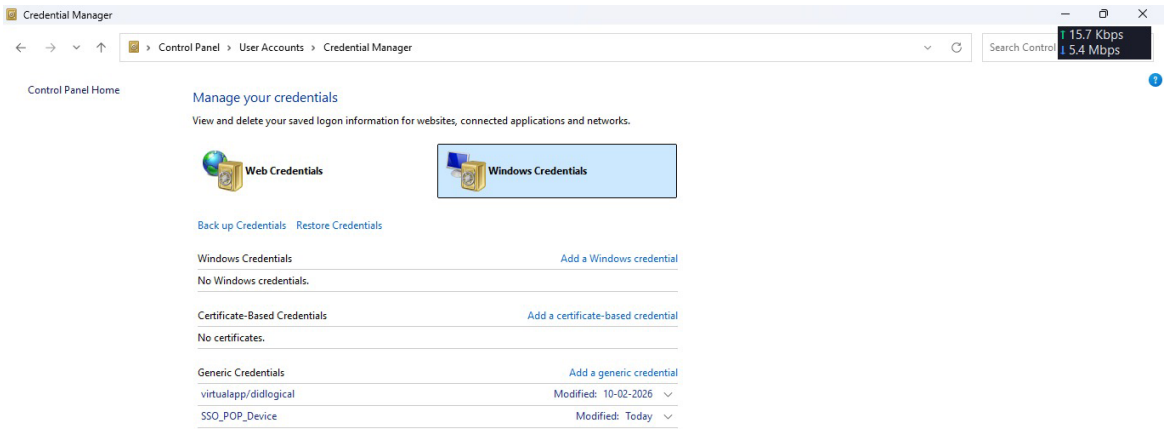
```
Admin@Mouu MINGW64 ~/Desktop/AWS Project
$ git clone https://github.com/moumita02314/testds.git
Cloning into 'testds'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

13 Now we will open Control Panel and navigate to User Accounts.

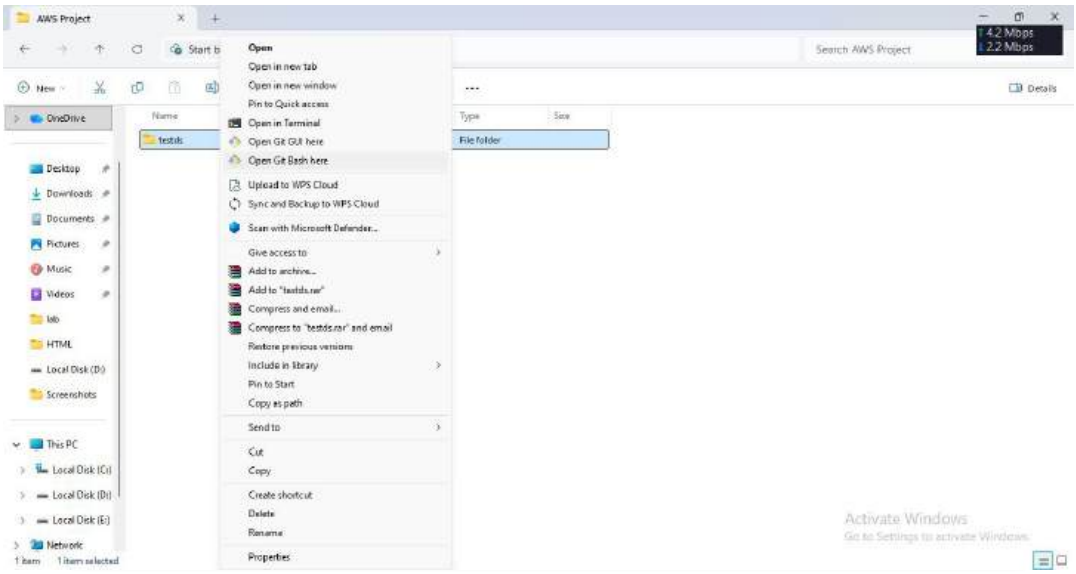
14 Then we will open Credential Manager and select Windows Credentials.



15 If any existing GitHub credentials are present, then we will remove them to ensure fresh authentication during the Git push process. Now we will not face any type issue during Authentication.



16 As we can see we have successfully deployed the project from GitHub to our local machine , Now we will deploy the contents of “testds” from our local machine to GitHub repo. To do we will open the "testds" folder and delete the ".gitignore" file and now again open Git Bash from this "testds" folder only.



17 Now run the given commands one by one :

(git config --global --list) -->

(git config --global user.name <user name>) -->

(git config --global user.name <user email>) -->

(git config --global --list) -->

(git init) -->

(git add .) -->

(git status) -->

(git commit -m "Done") -->

(git remote add origin <Repository Address>) -->

(git push -u origin master).

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global --list
user.name=pausali-sengupta
user.email=pausali2004@gmail.com
use.email=pausali2004@gmail.com
```

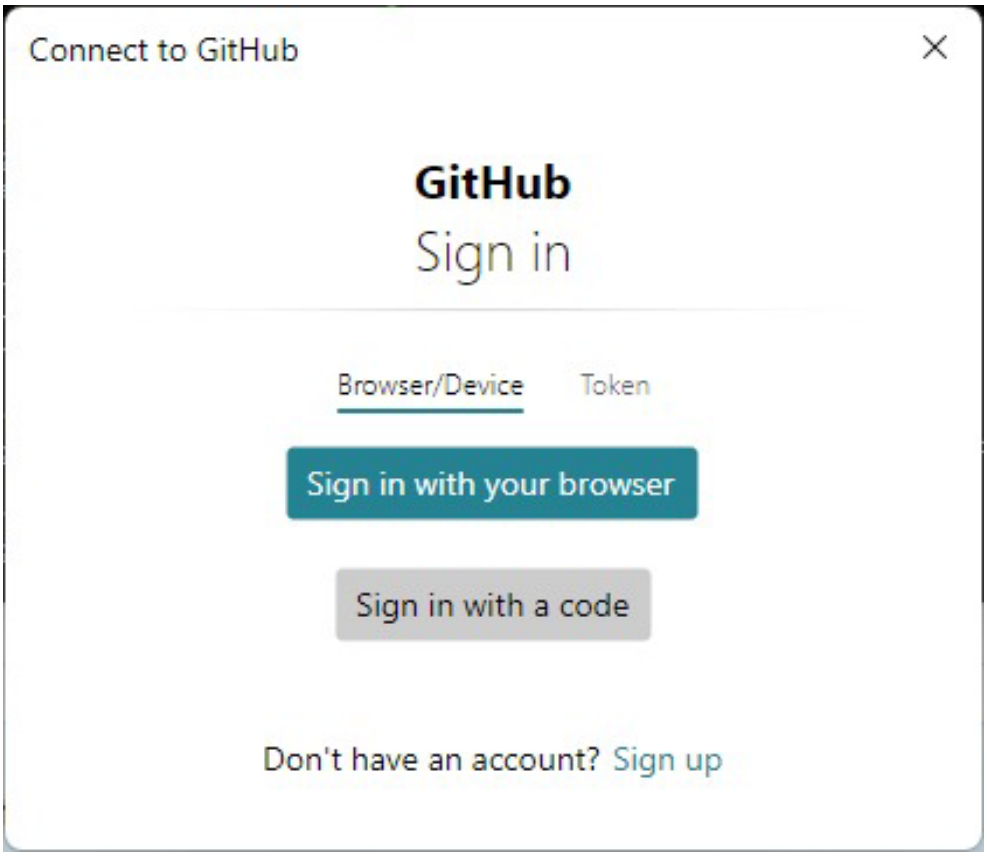
```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global user.name "Codelearner236"

Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global user.email "bitankarmakar22@gmail.com"
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git config --global --list
user.name=Codelearner236
user.email=bitankarmakar22@gmail.com
```

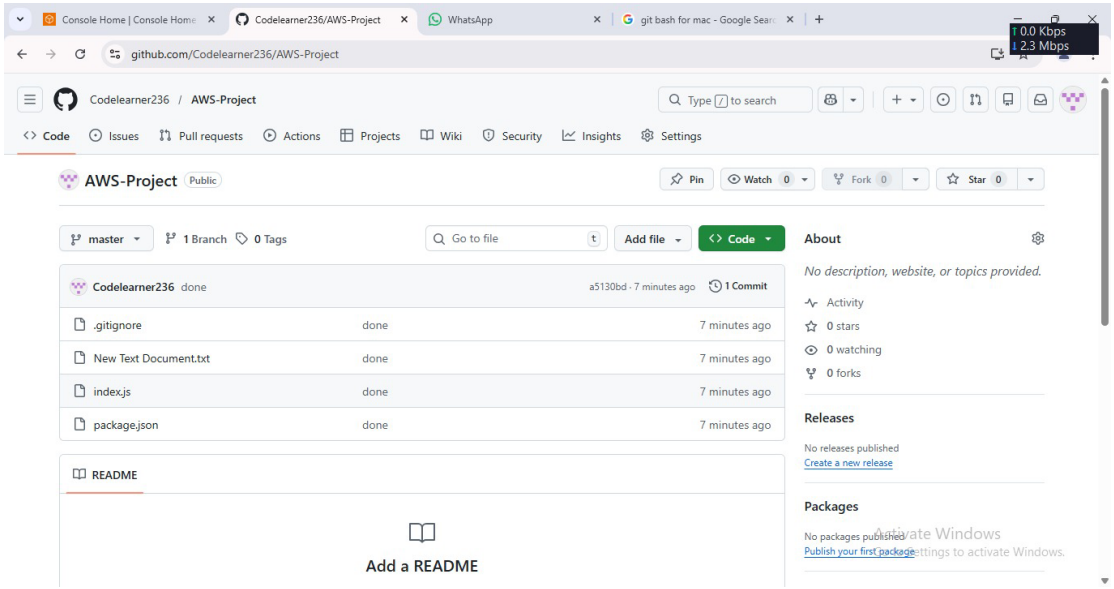
```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds
$ git init
Initialized empty Git repository in C:/Users/Admin/Desktop/AWS Project/testds/.g
it/

Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git add .

Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   New Text Document.txt
        new file:   index.js
        new file:   package.json
```

```
Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git commit -m "done"
[master (root-commit) a5130bd] done
 4 files changed, 53 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 New Text Document.txt
 create mode 100644 index.js
 create mode 100644 package.json

Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git remote add origin https://github.com/Codelearner236/AWS-Project.git

Admin@Mouu MINGW64 ~/Desktop/AWS Project/testds (master)
$ git push -u origin master
```

18 After running all the commands "GitHub OAuth Authorization Window" will open, paste the token address that we have created during token generation and sign in.



19 As we can see after successful authentication, the files were deployed and displayed in the GitHub repository.



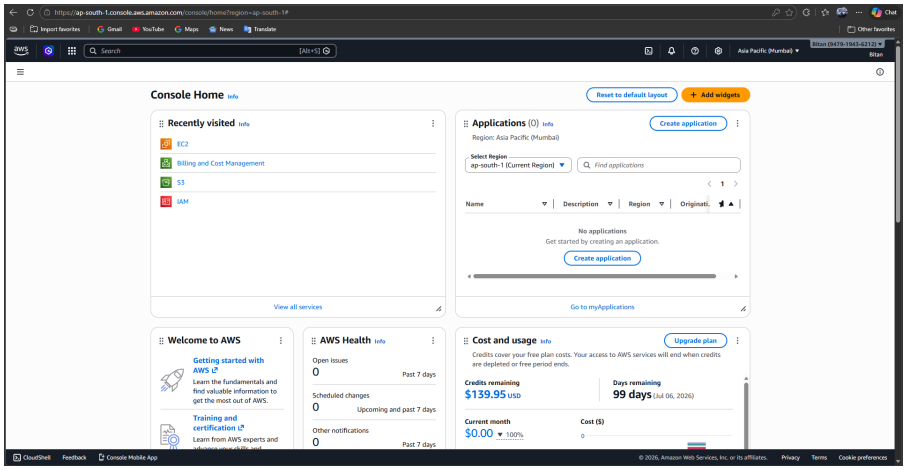
ASSIGNMENT – 9

PROBLEM STATEMENT:

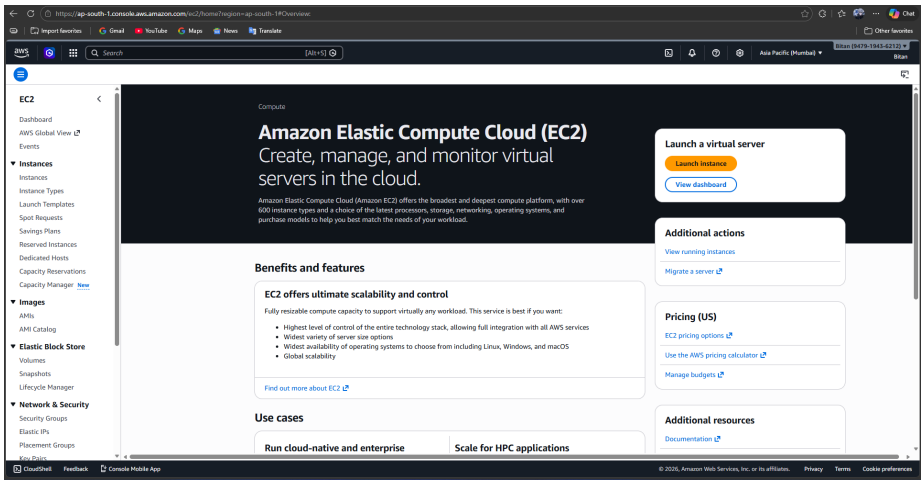
Deploy a project from Github to EC2

Steps:

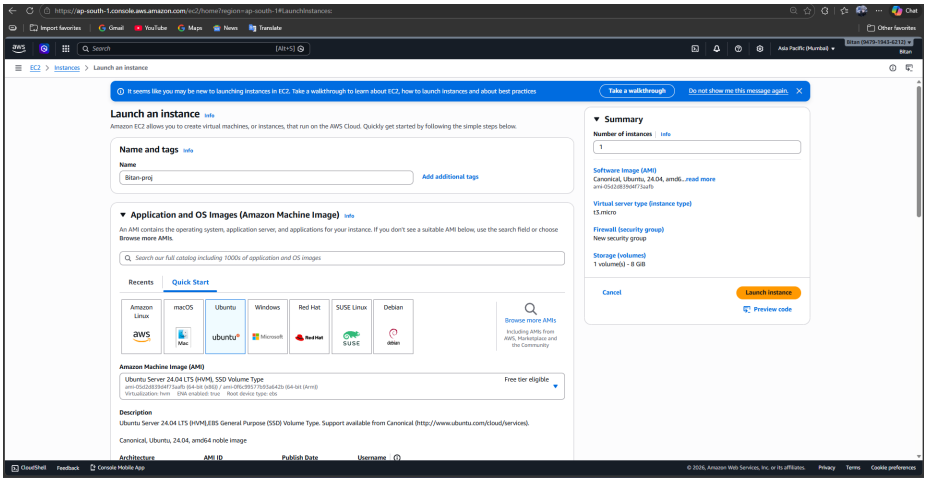
1. **Login to the AWS Management Console and from the home page click on "EC2".**



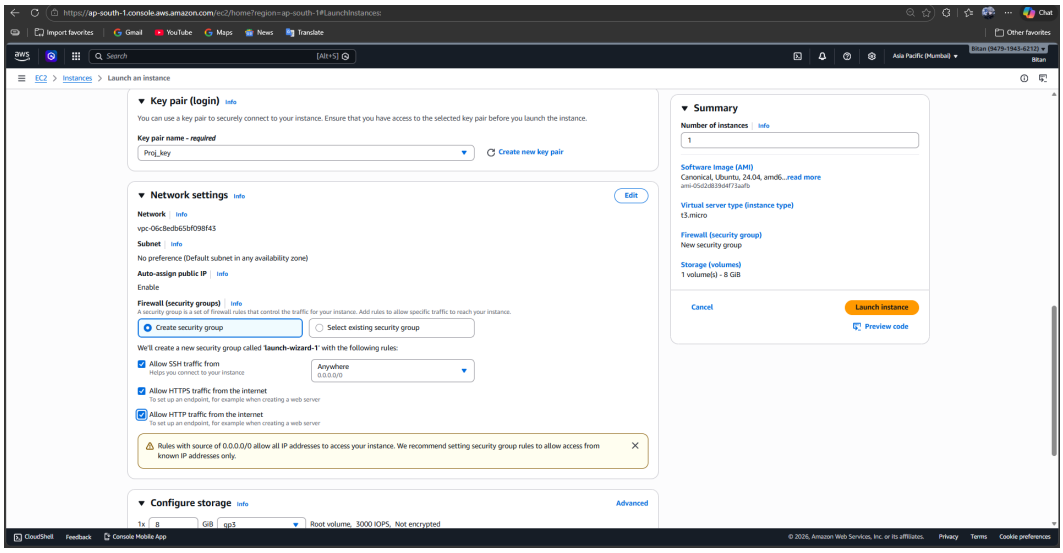
2. **Now click on "Launch instance"**



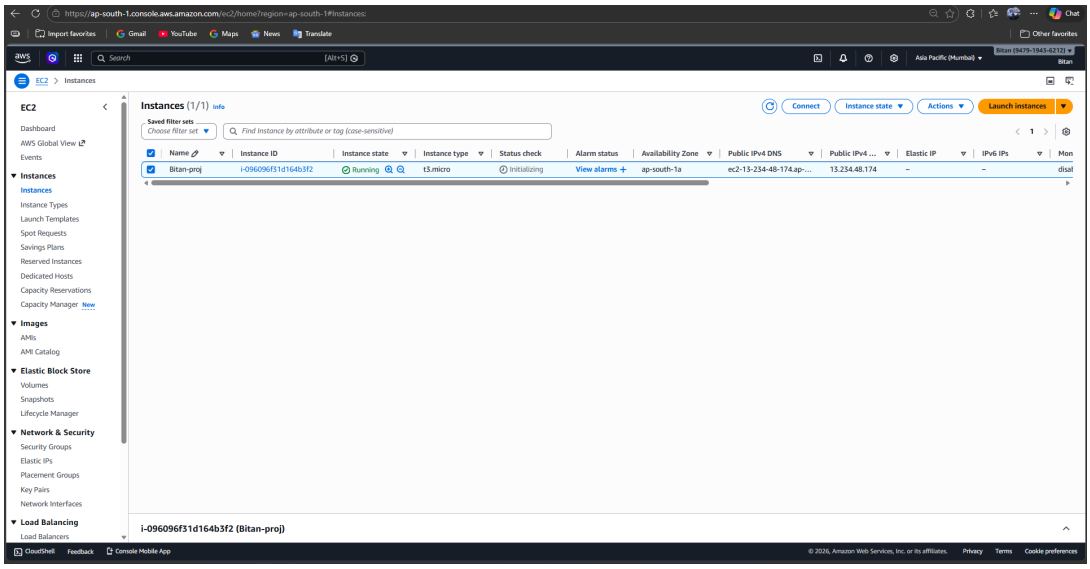
3. **Now enter any name of the instance, Here it is "Bitan-proj", and select "Ubuntu" from Application and OS Images.**



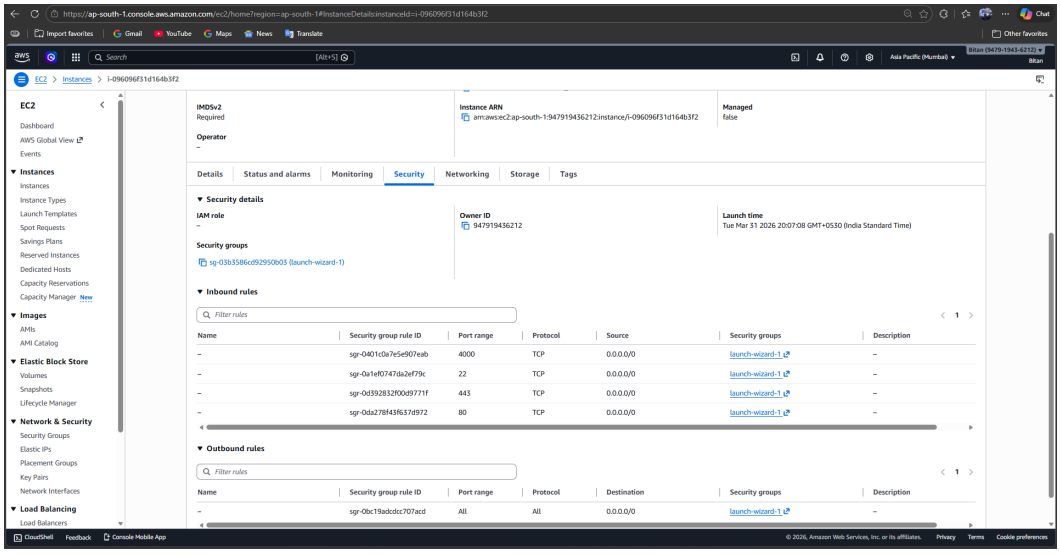
4. Now enter key pair, Here it is “Proj_key” and select all three check boxes from the Network settings that are “Allow SSH traffic from”, “Allow HTTPS traffic from the internet” and “Allow HTTP traffic from the internet” and click on “Launch instance”.



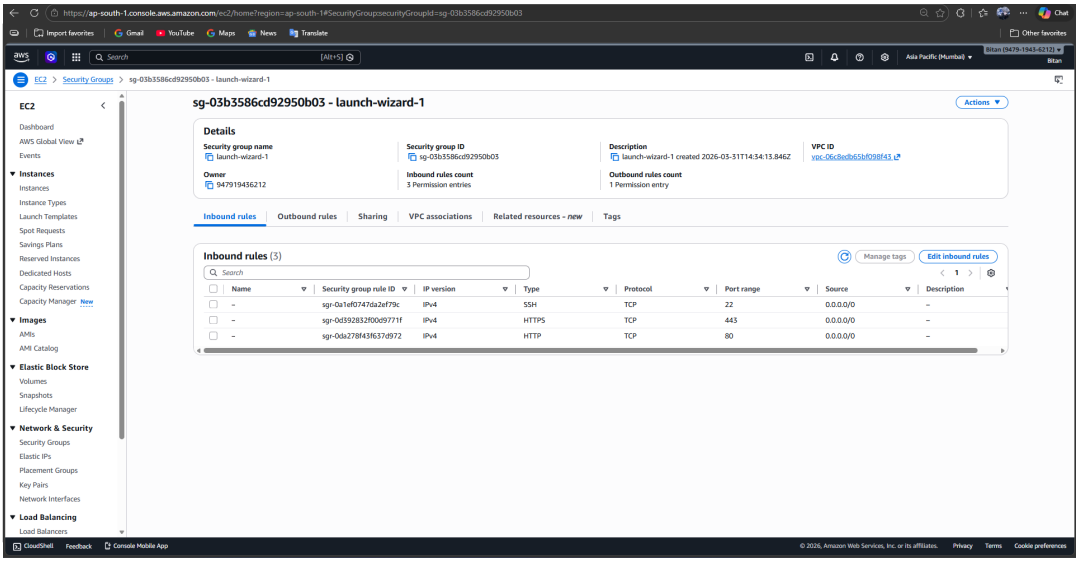
5. Now move back to the instances and, from there click on instance id.



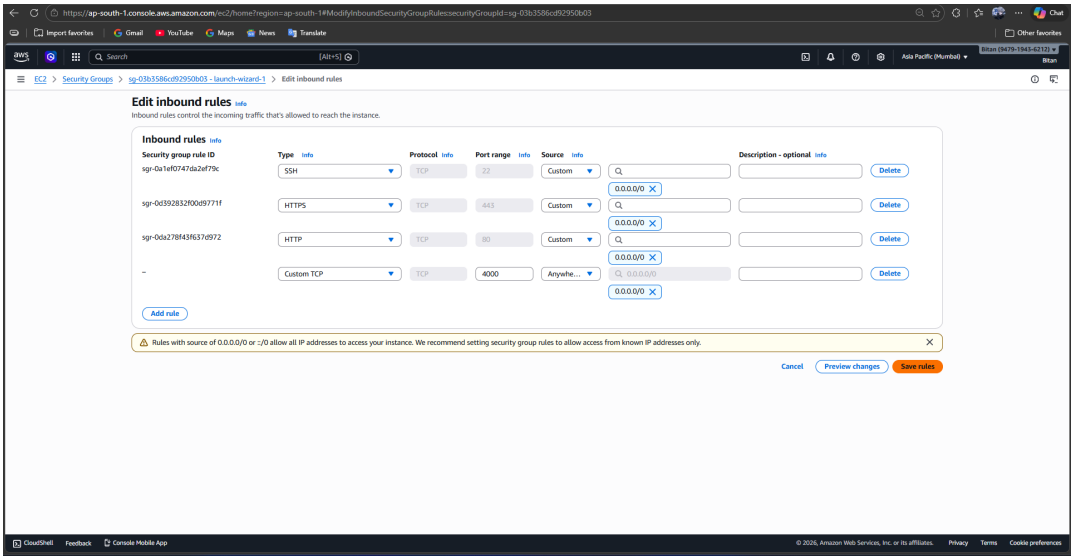
6. Now click on “security” tab from there, the click on “Security groups”.



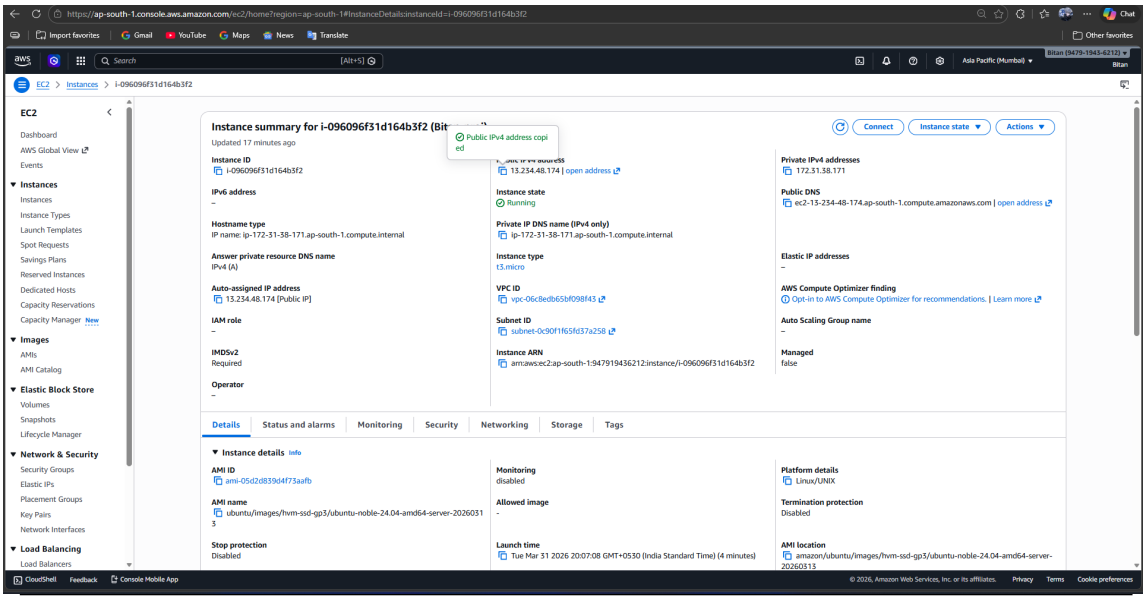
7. **Now click on “Edit Inbound rules”**



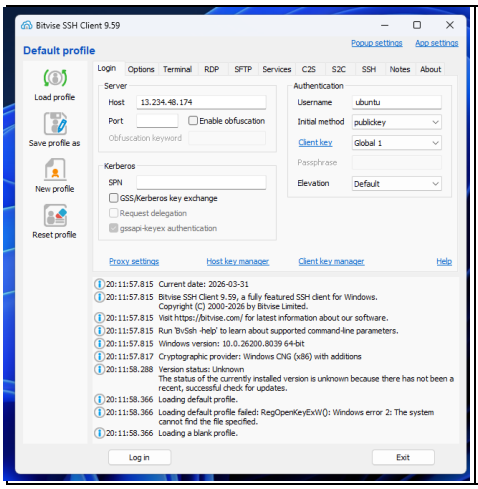
8. **Now click on “add rule” button, then a new section named as : “Custom TCP” will open in that section, enter (Port range = 4000), and enter 0.0.0.0/0 in the search pannel.**

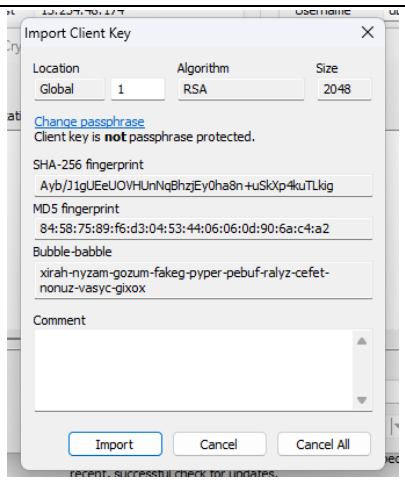


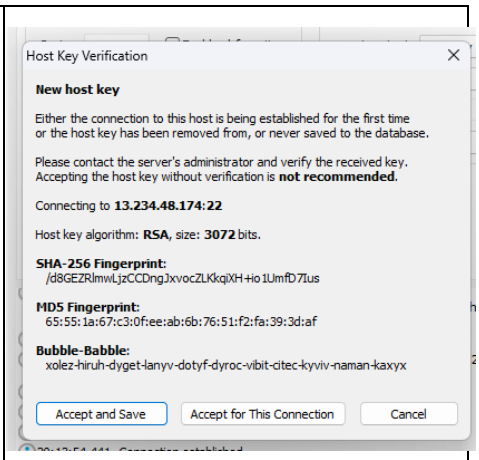
9. **Now copy the “Public IPV4 Address”**



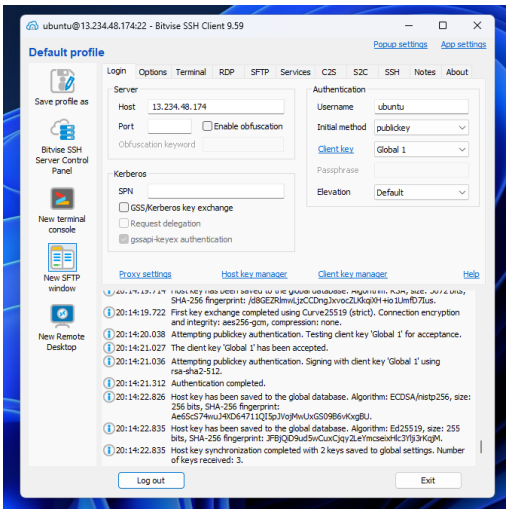
10 Now open “Bitvise SSH Client” and paste the Public IPV4 address that we have just copied from the instance, and then click on “Client key manager” to import the key pair and finally we can login.



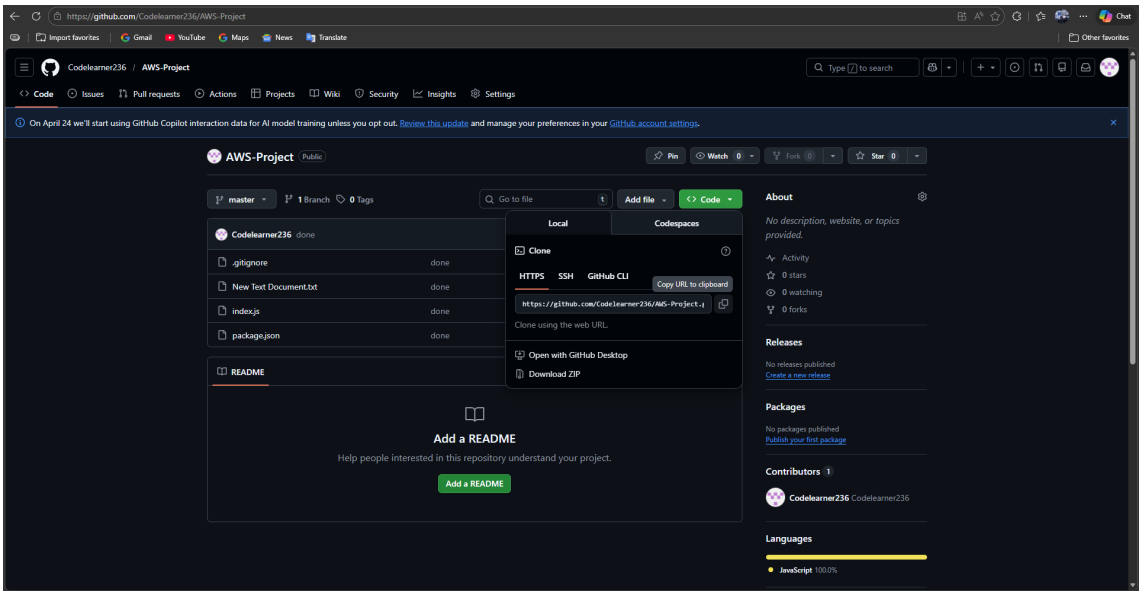




11 Now click on “New Terminal Console”



12 Now login to the GitHub account and goto the repository where the projects are stored, now from there click on “code” button and copy the link.



13 Now we run the given commands one by one using the teminal :

(sudo apt-get update)

(sudo apt-get upgrade)

(sudo apt install nginx)

(curl -SL https://deb.nodesource.com/setup_16.x|sudo -E bash -)

(sudo apt install nodejs)

(git clone “Paste the project repository link”)

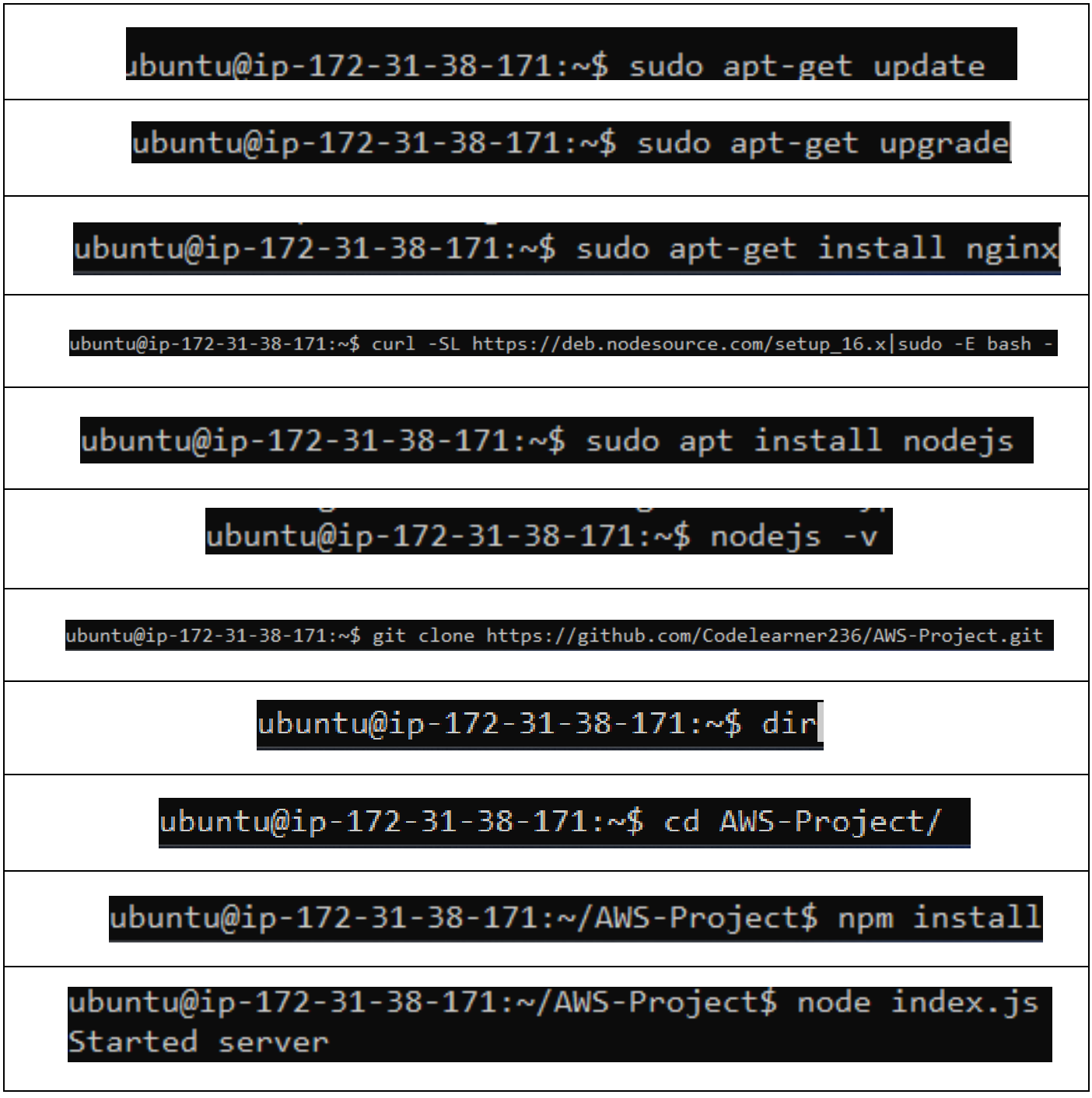
(dir)

(cd “repository name”)

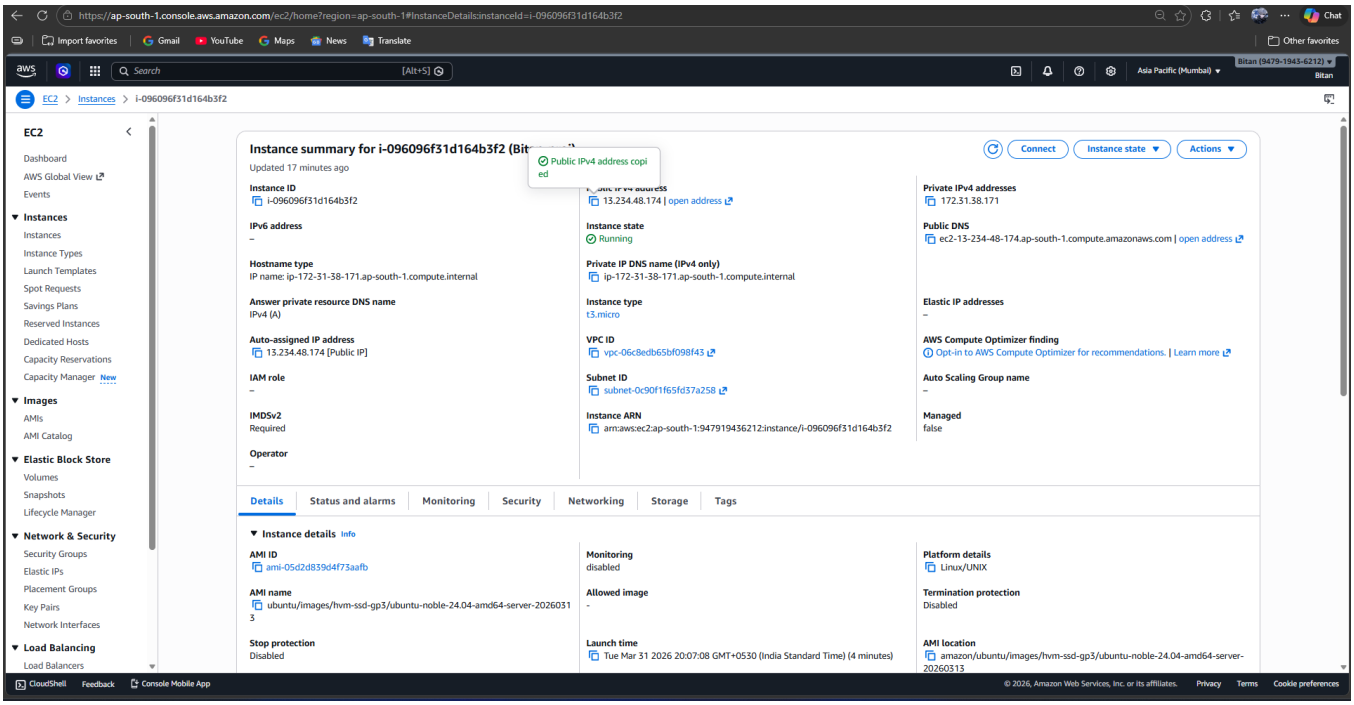
(npm install)

(node index.js)

To start the server.



14 Now copy the “Public IPV4 Address” again from the instance.



15 And paste it into a new tab and press enter,

As we can see the content of the project is displayed on the screen i.e.

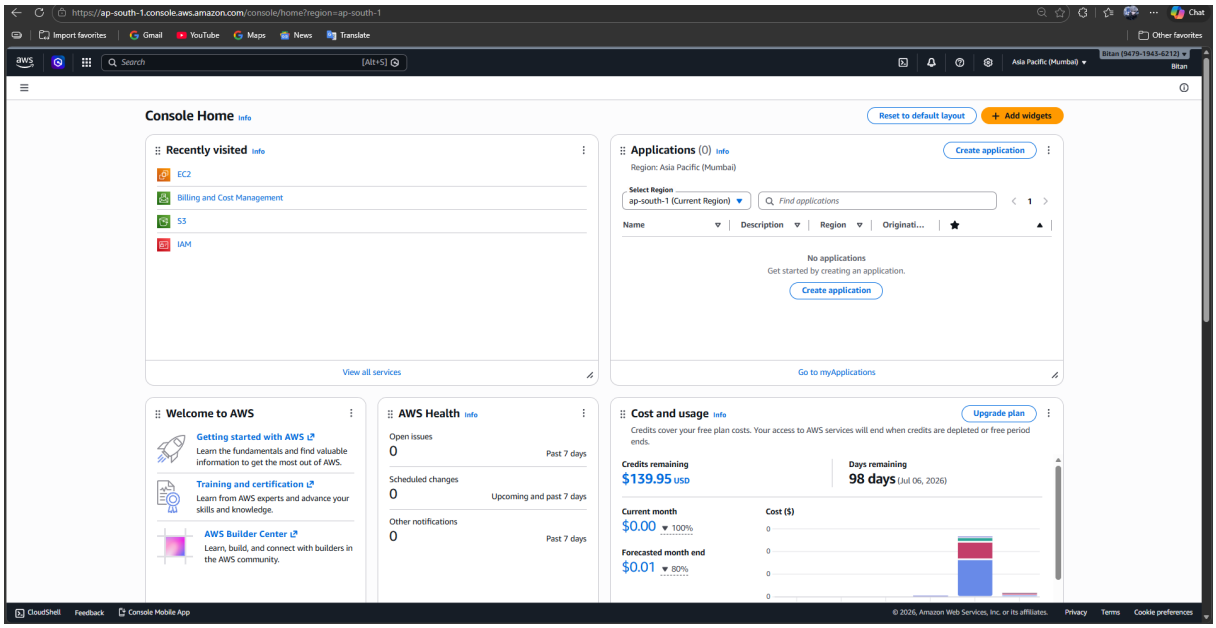
“Hello mckv”.

ASSIGNMENT – 10

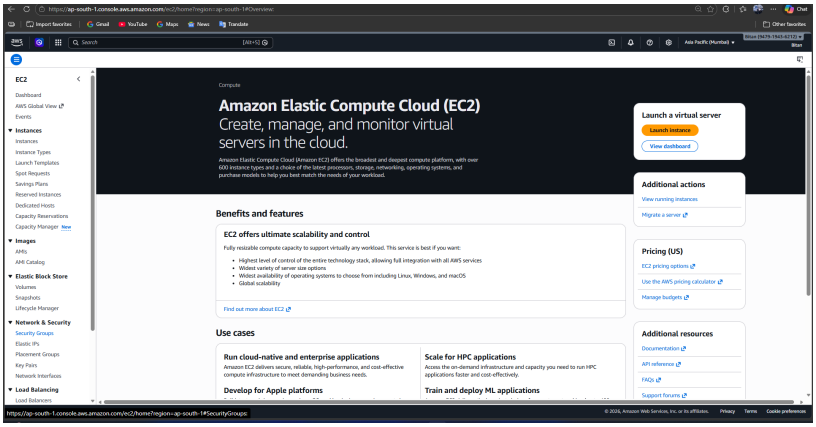
PROBLEM STATEMENT: Deploy a project from GitHub to EC2 by creating a new security group and user data.

Steps:

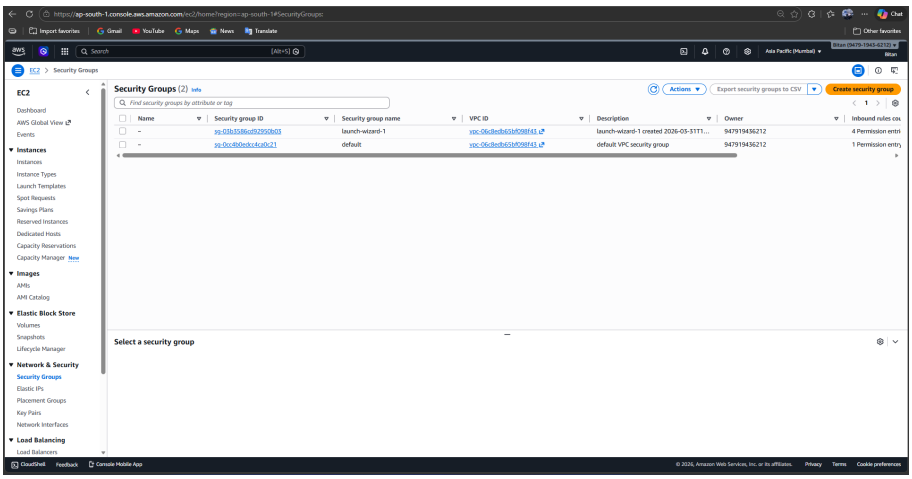
1. Login to the AWS Management Console and from the home page click on "EC2".



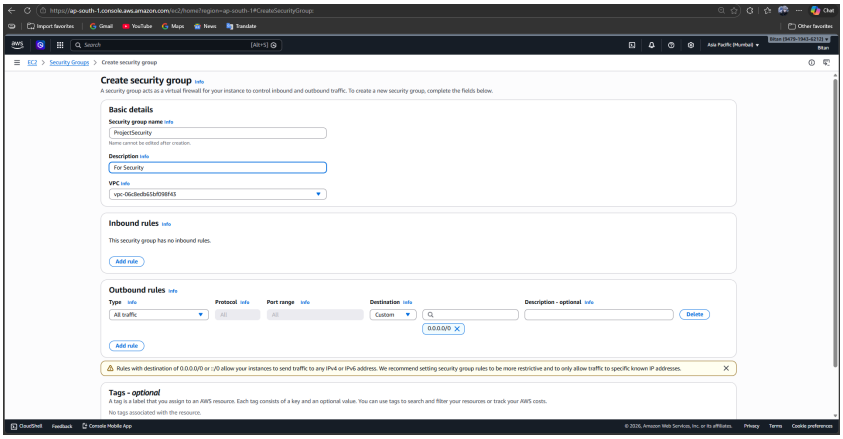
2. Now click on "Security groups"



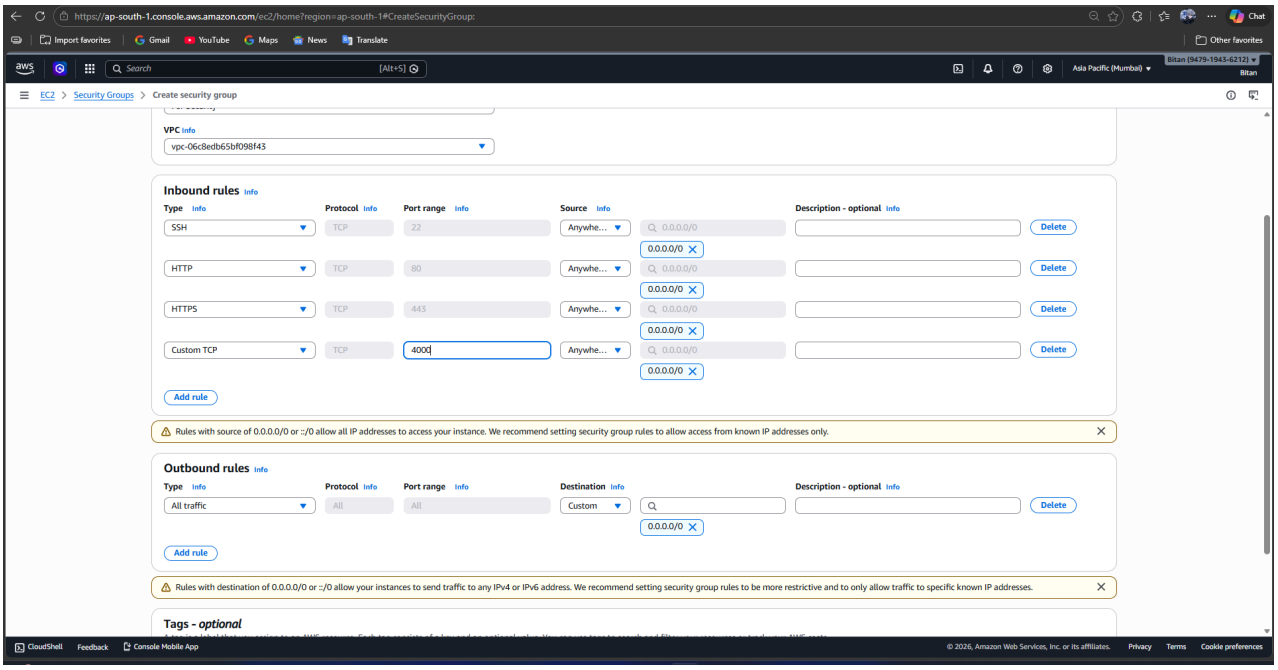
3. Now delete all the existing security groups except the "default" one. Then click on "Create security group" from the top right corner.



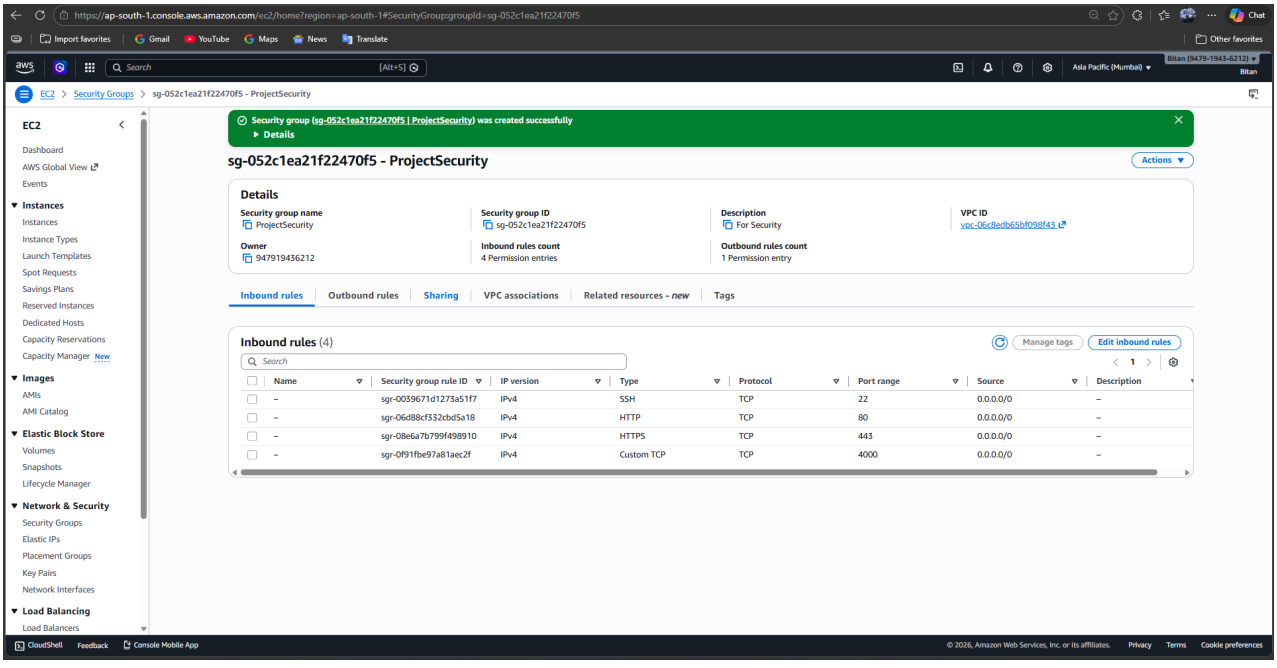
4. Now enter the basic details of the security group. Here security group name is “ProjectSecurity” and Description is "For Security".



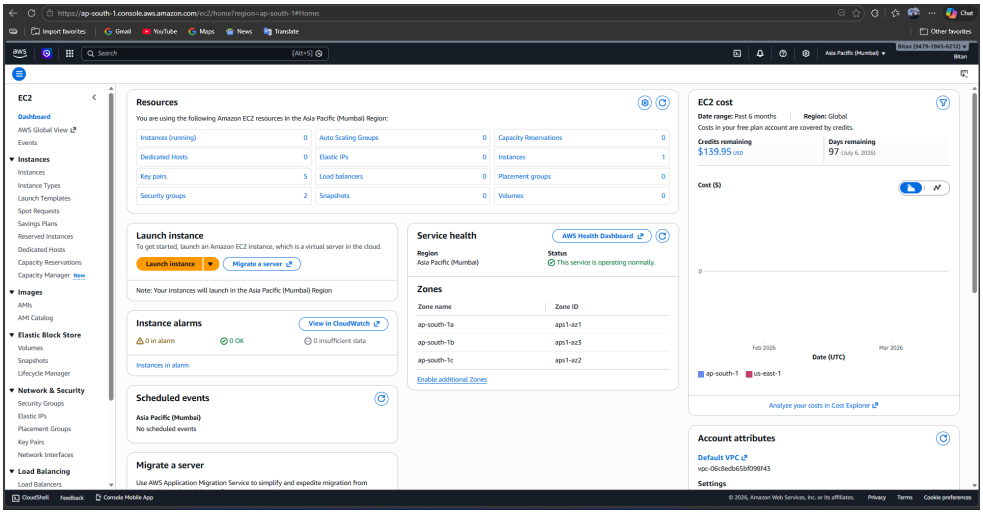
5. Scroll down and add the four inbound rules as shown below. Do not change anything in Outbound rules section. Finally click on “Create Security Group”.



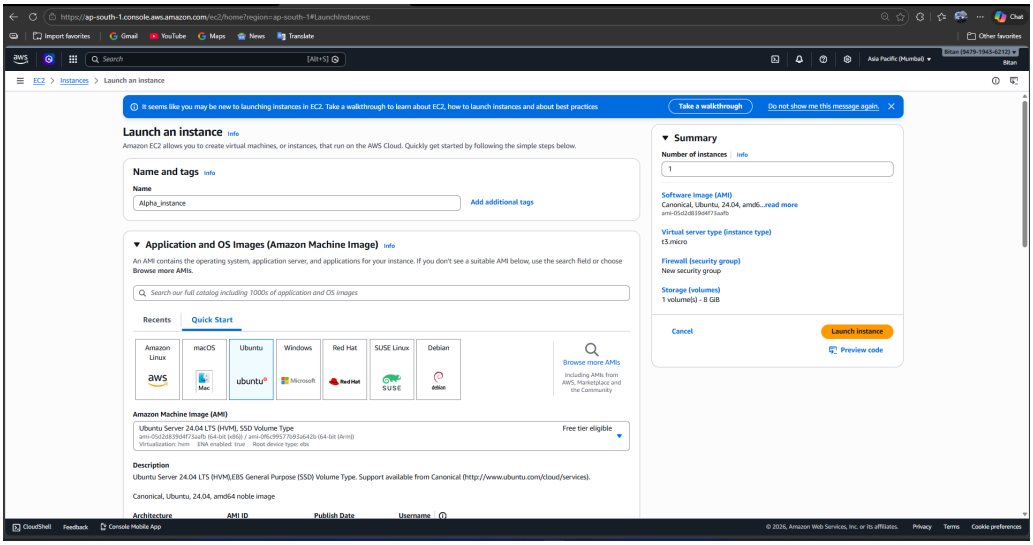
6. The new security group " ProjectSecurity" is created successfully.



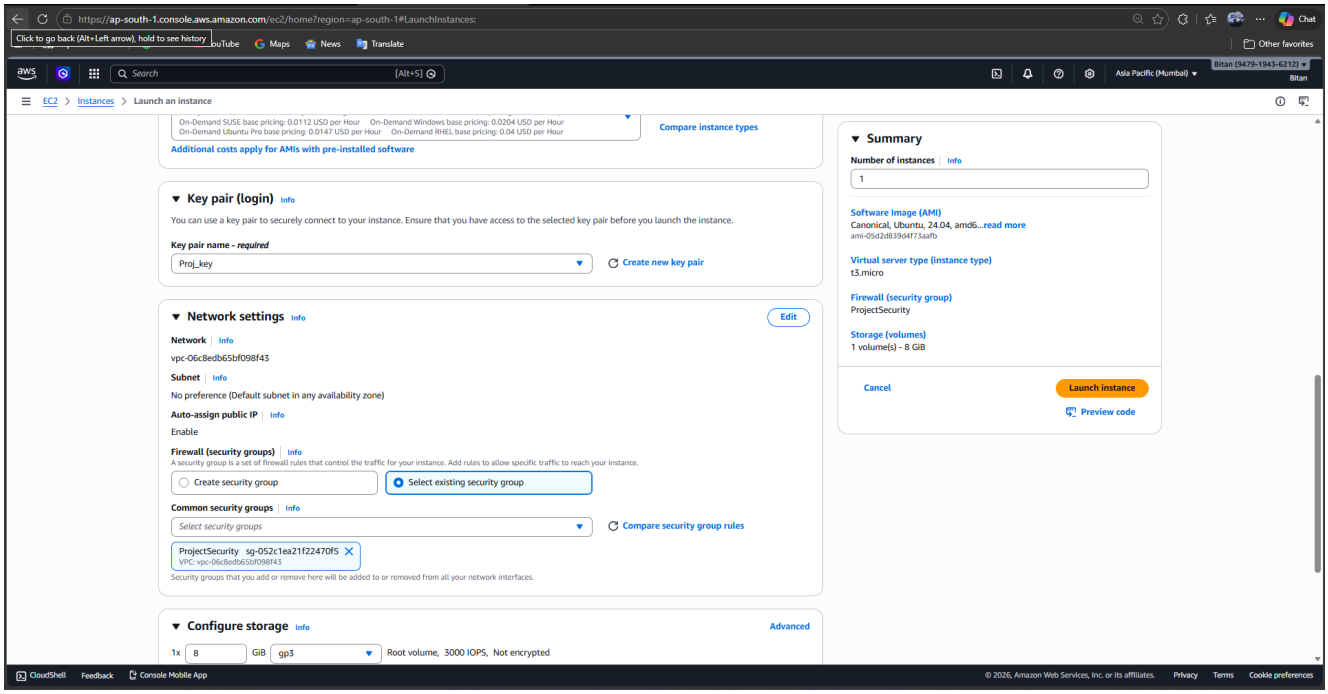
7. Now go back to the EC2 dashboard and click on “Launch Instance”



8. Enter the name of the instance (here Alpha_instance) and select the UBUNTU as the operating system.



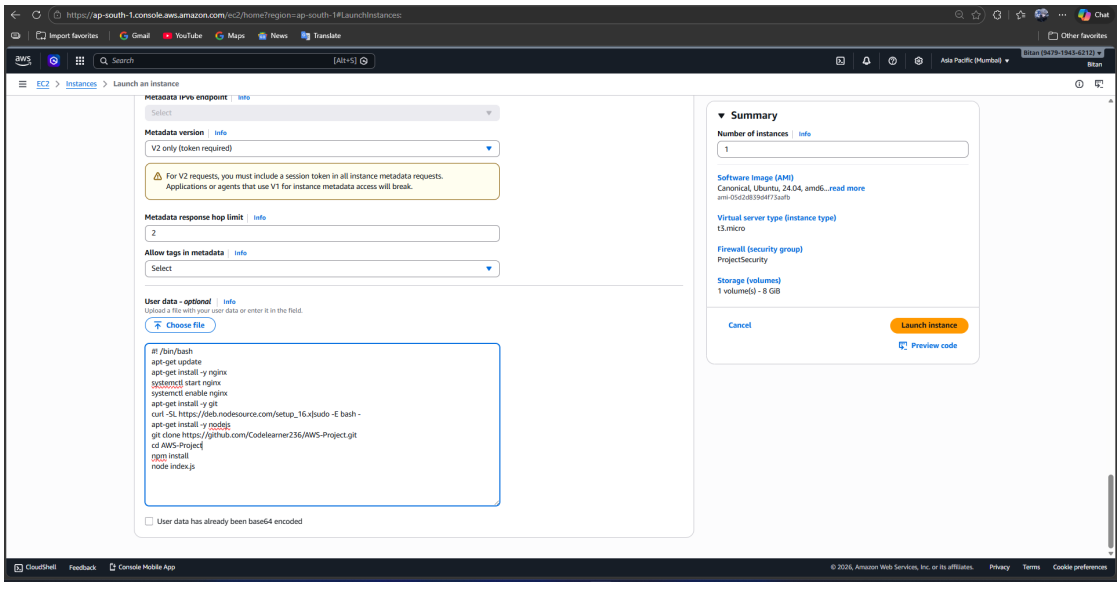
9. Scroll down and select the keypair which is present in your device. If not then create a new
keypair. Here keypair selected is “Proj_key” and click on “Select existing security group”.

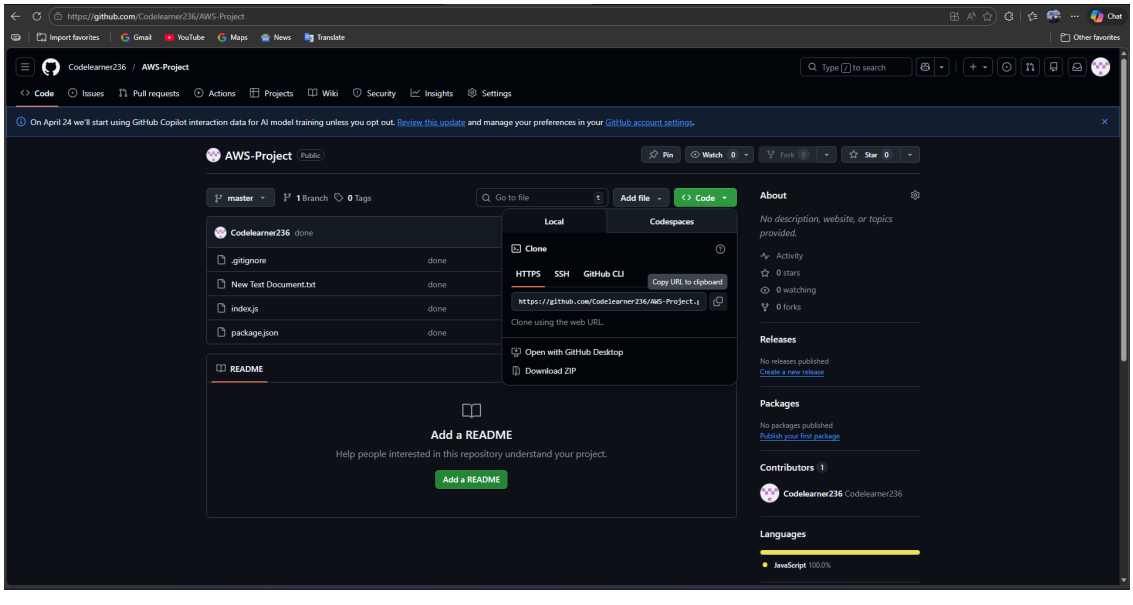


10. Go to Advanced settings and scroll down to the end. Copy paste the given code in the “User data-optional” blue box. After that click on Launch Instance.

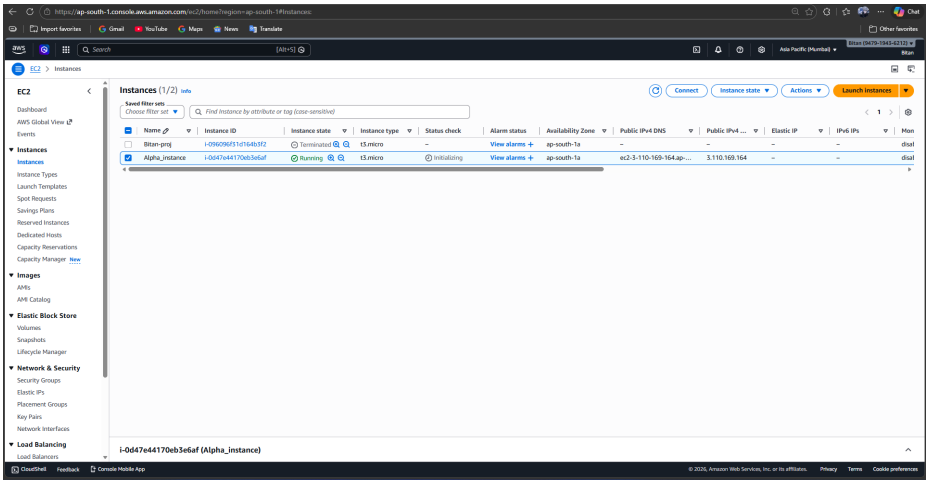
```
#! /bin/bash
apt-get update
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -SL https://deb.nodesource.com/setup_16.x|sudo -E bash -
apt-get install -y nodejs
git clone <your repository address>
cd <your repository folder name>
npm install
node index.js
```

Login to the GitHub account and go to the repository where the projects are stored, now from there click on “code” button and copy the link.

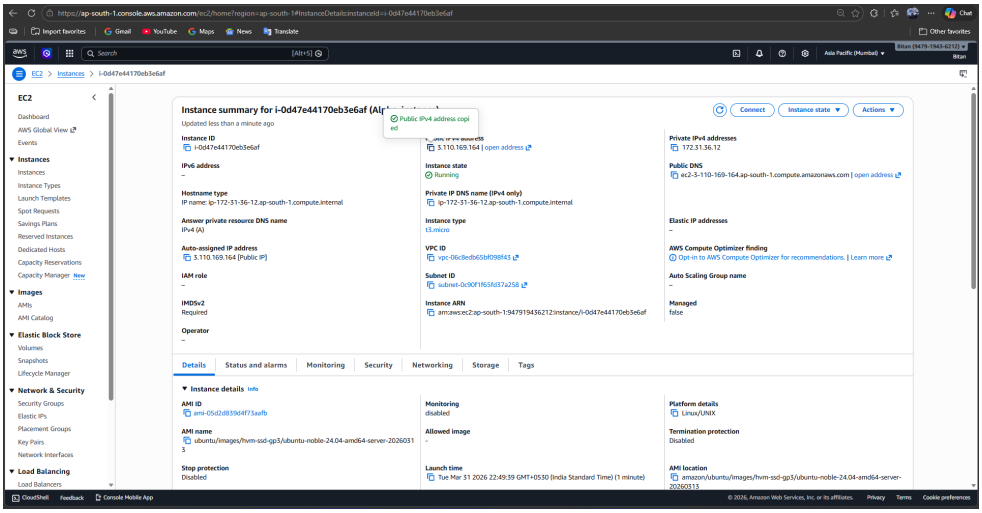




11. Instance (here Alpha_instance) is created successfully.



12. Click on the Instance ID and then copy the Public IPv4 address.

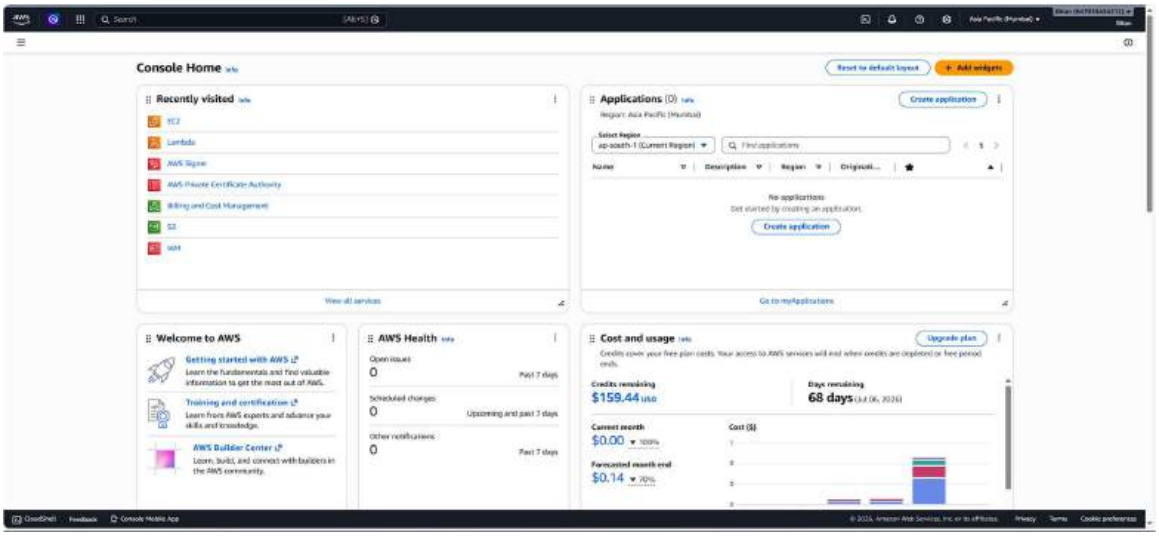


13. And paste it into a new tab add ":4000" to it, then press enter, we can see the content of the project is displayed on the screen i.e."Hello mckv".

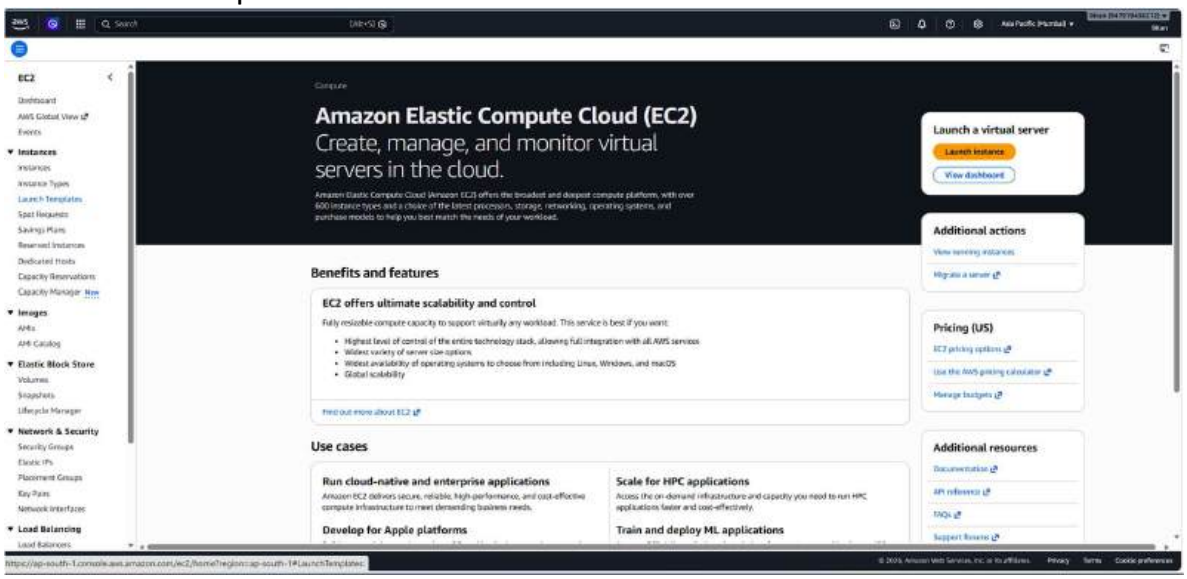
"Thus, seeing the final output, we can say our Project has been deployed from the GitHub to EC2 by creating a new security group (here ProjectSecurity) and the user data (the code which we copied)"

<u>ASSIGNMENT 11 :</u> Build scaling plans in AWS that balance load on different EC2 instances

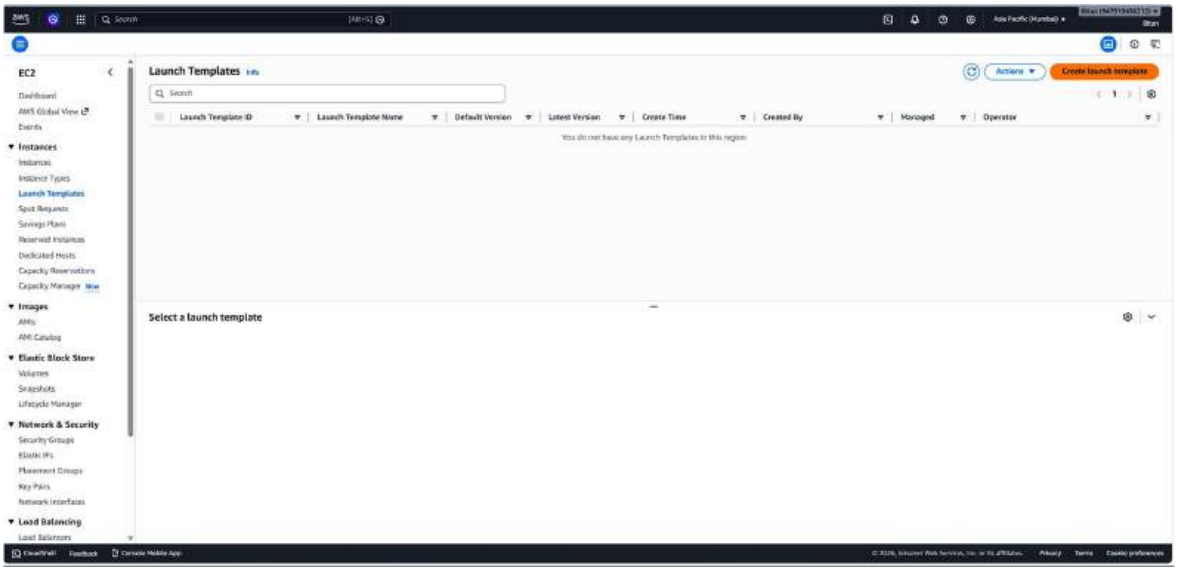
1. Log into AWS console and click on “EC2”.



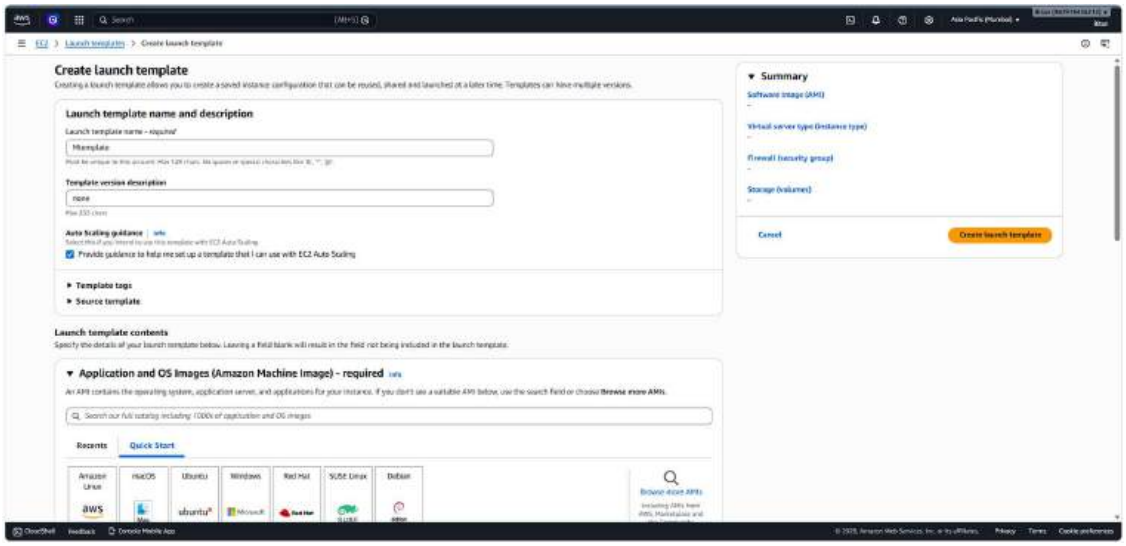
2. Click on ‘Launch Templates’.



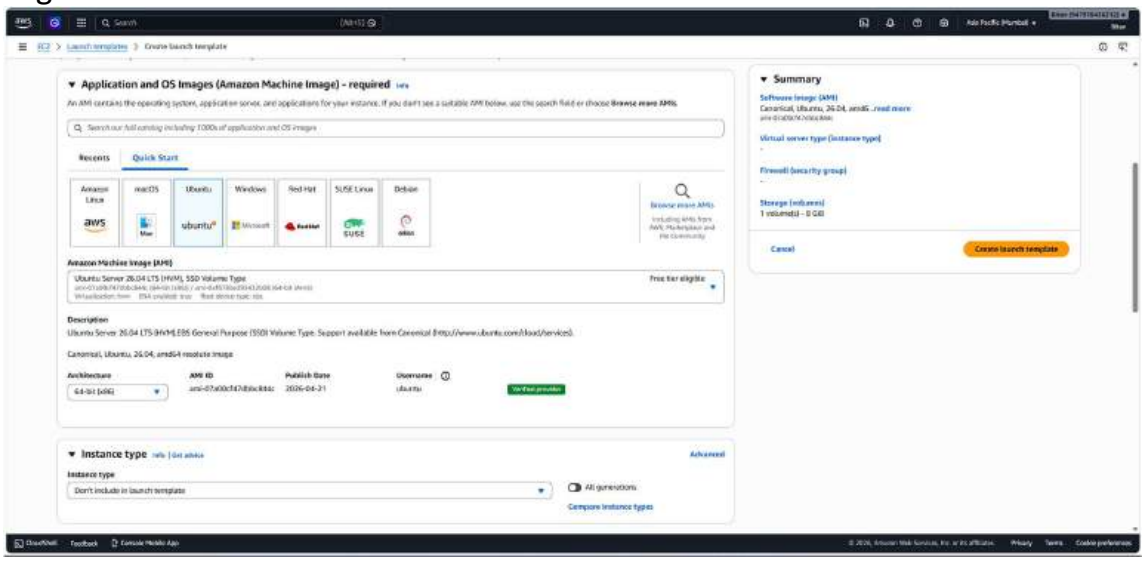
3. Create launch templates.



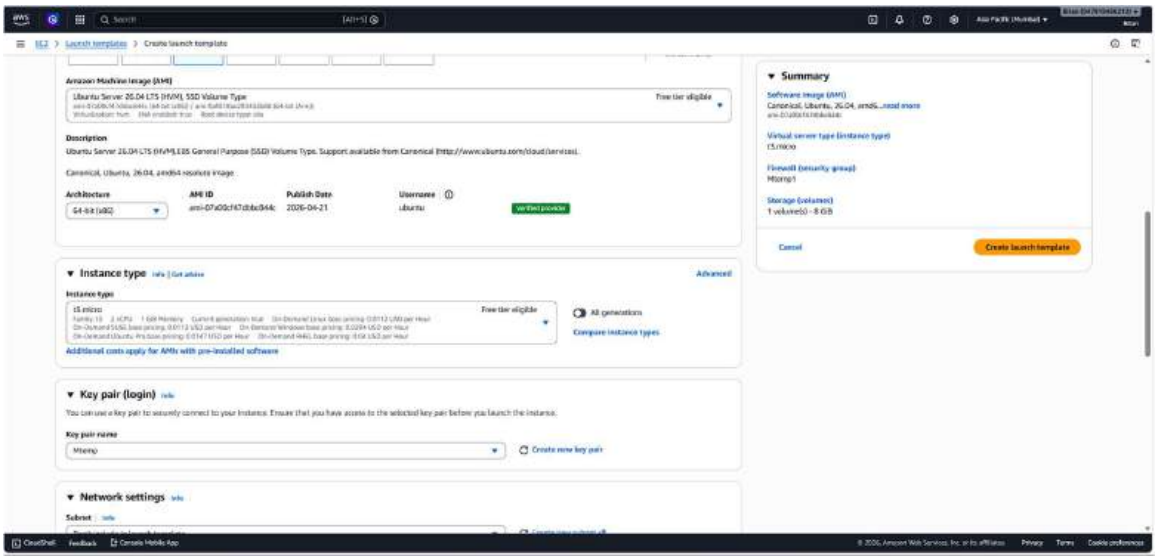
4. Enter necessary details. In ‘Launch templet name and description’ give a name and description to your launch templet and select the check box for ‘Provide guidance to help me set up a template that I can use with EC2 Auto Scaling’



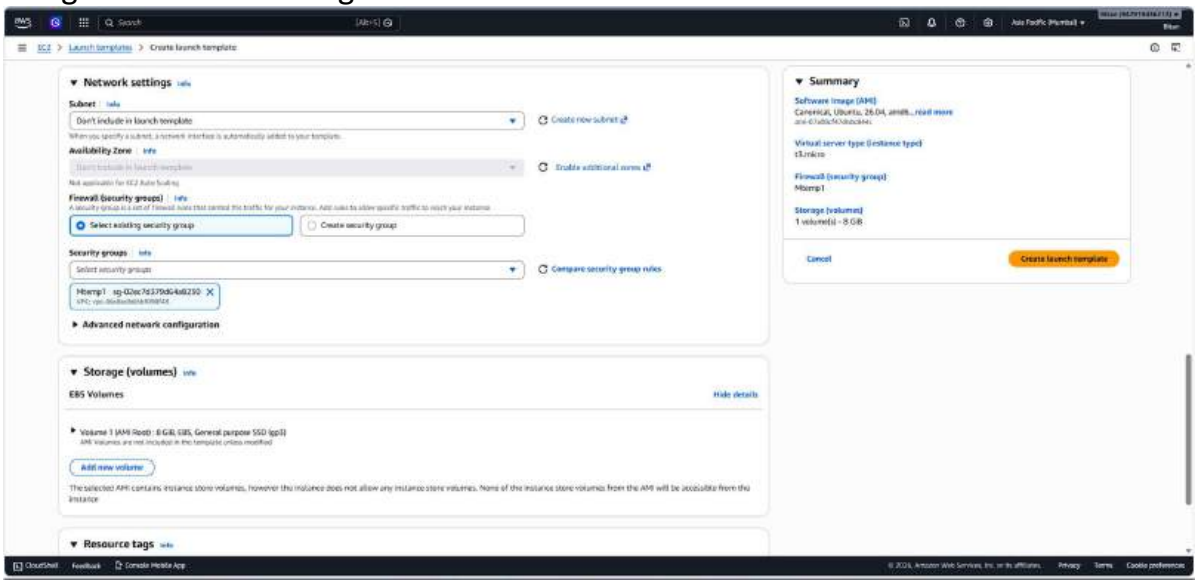
For OS Image select ‘Ubuntu Server’



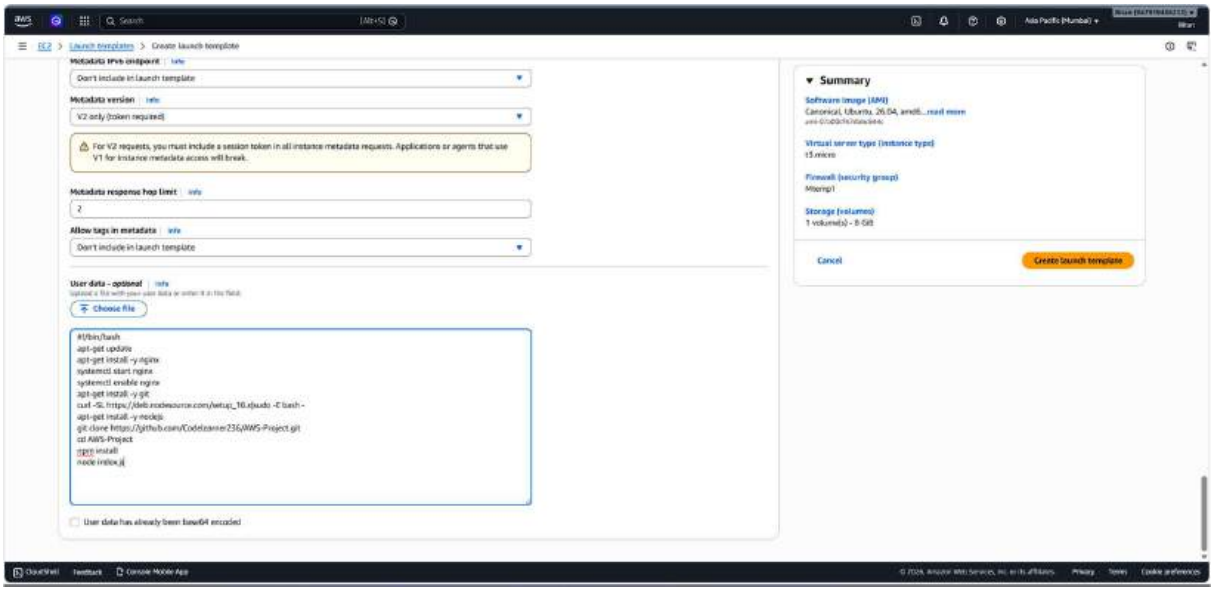
For instance type, select ‘t2.micro’ or any other free tire eligible instance type. For key pair, select an existing key pair or crate a new key pair.



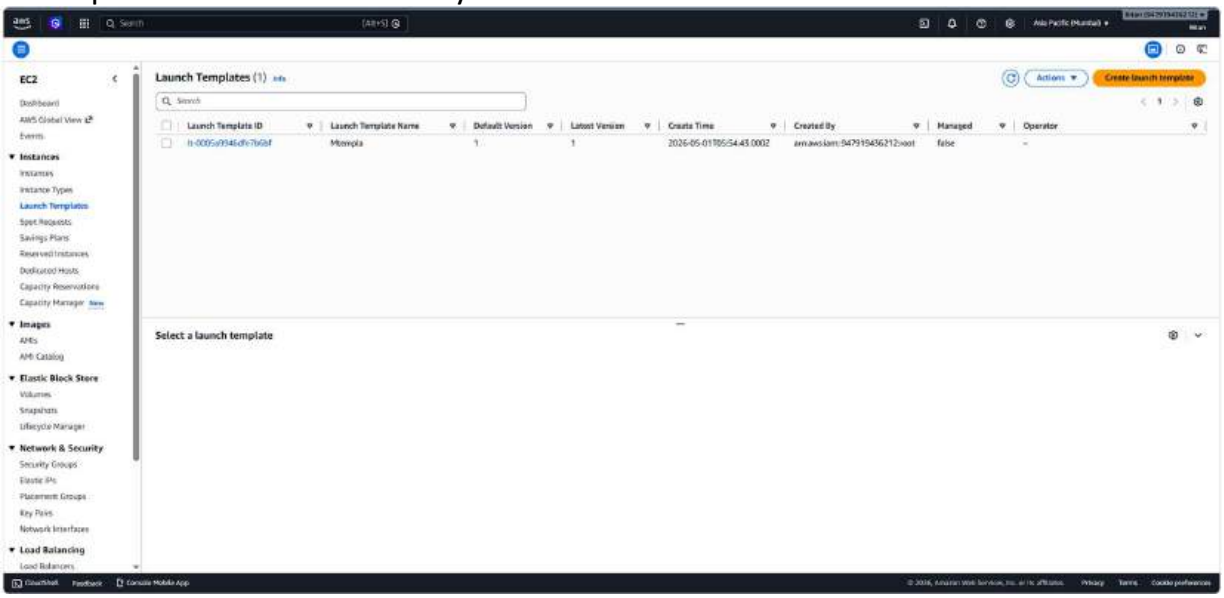
For security group select an existing security group where port 4000 is open for incoming traffic. Keep Storage and Resource Tag as default.



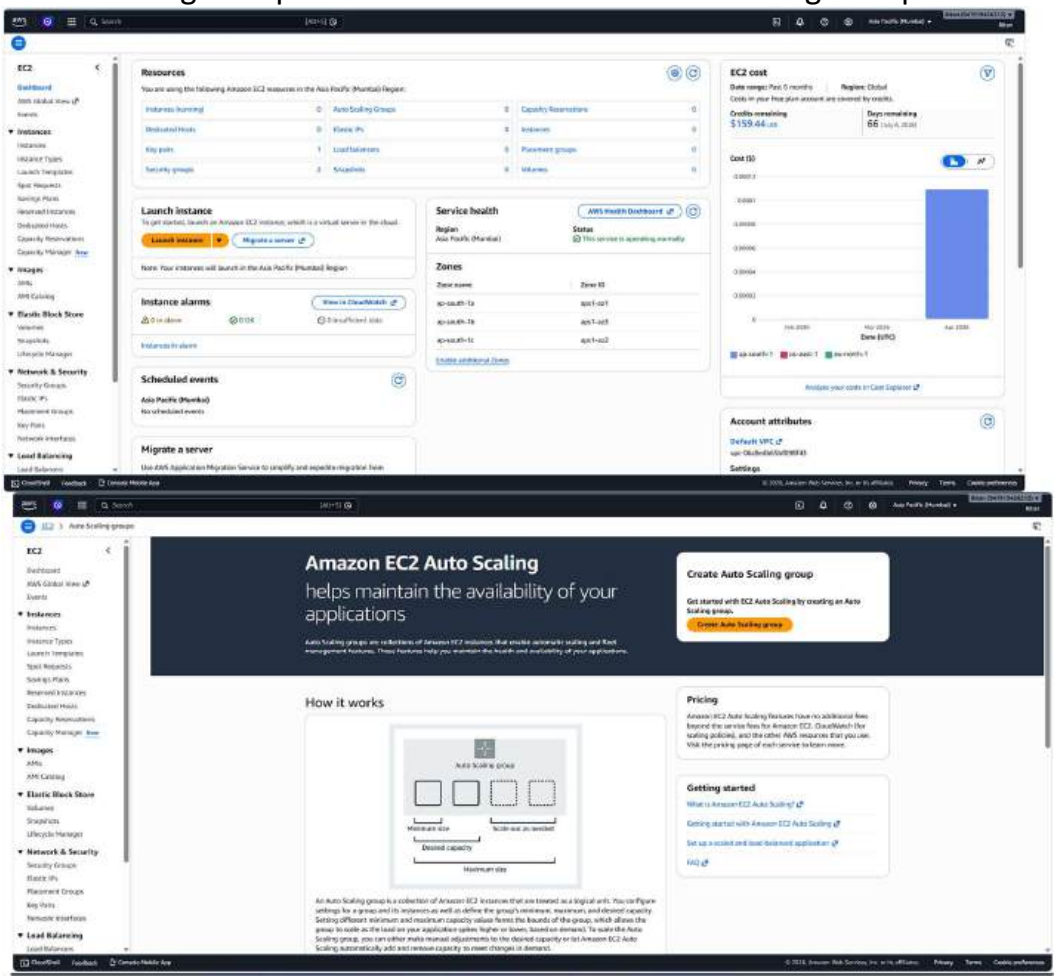
In Advanced details scroll down to the bottom and add the text to the user data. Then click on Create launch template.



Launch Templates created successfully.



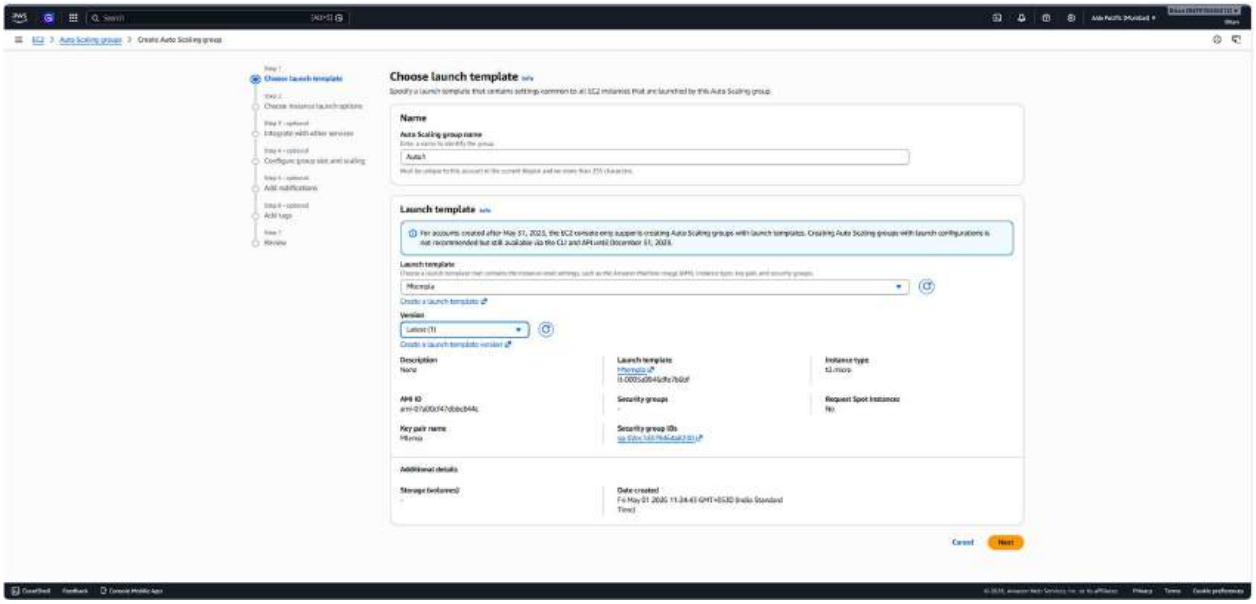
6. Now go to ‘Auto Scaling Groups’ . Then click on ‘Create Auto Scaling Group’.



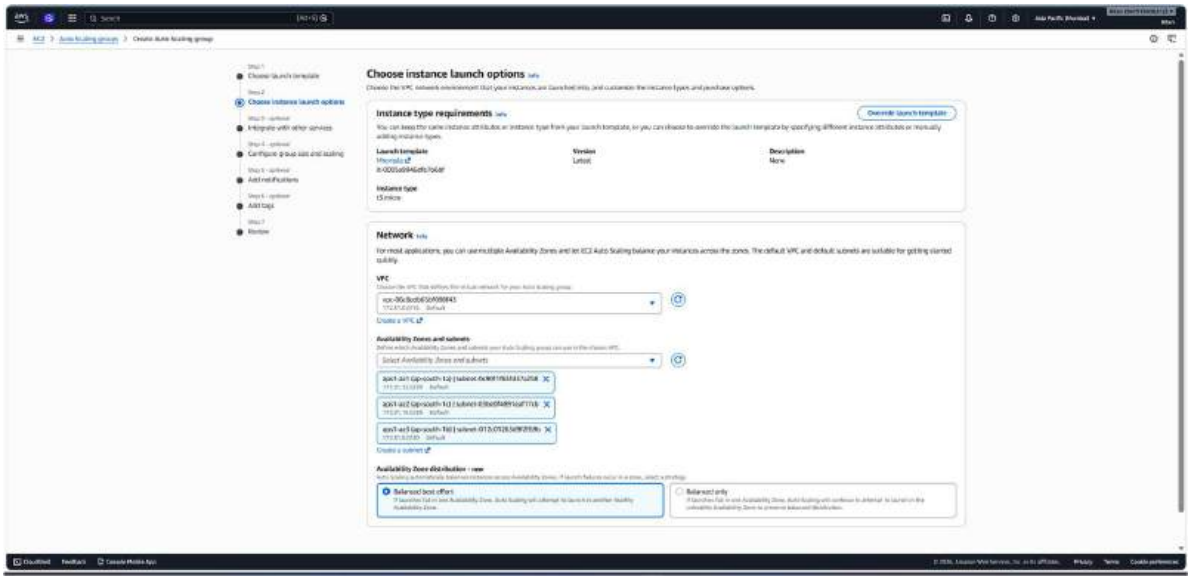
7. Add Details for the Auto Scaling Group.

Step 1: Choose launch templates- Give an name to the Auto Scaling Group and choose the Launch template you just created as the launch template.

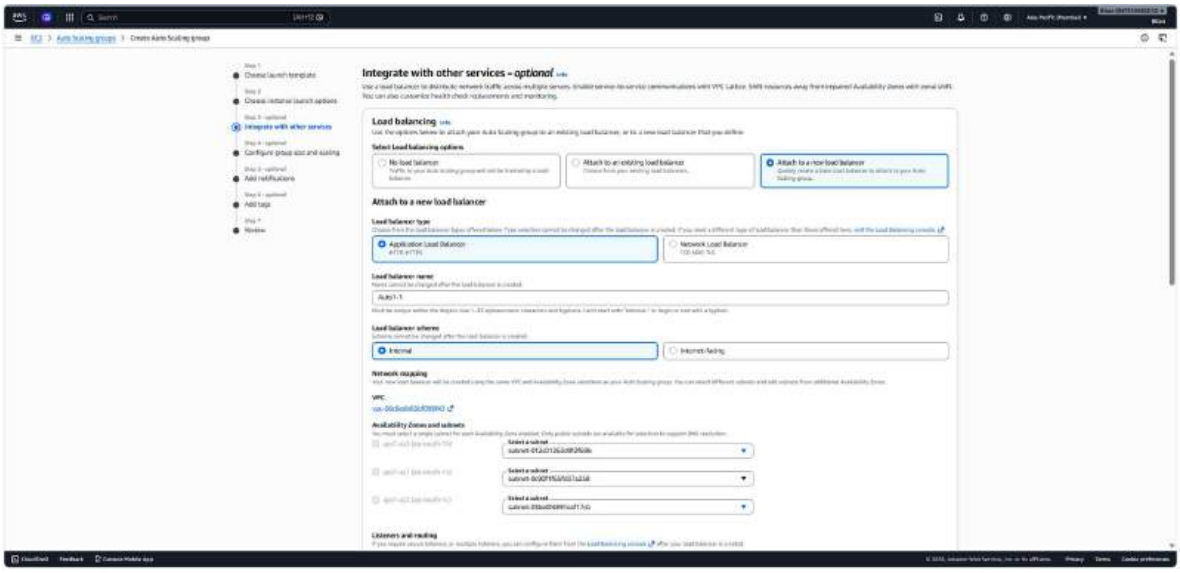
Click on Next.



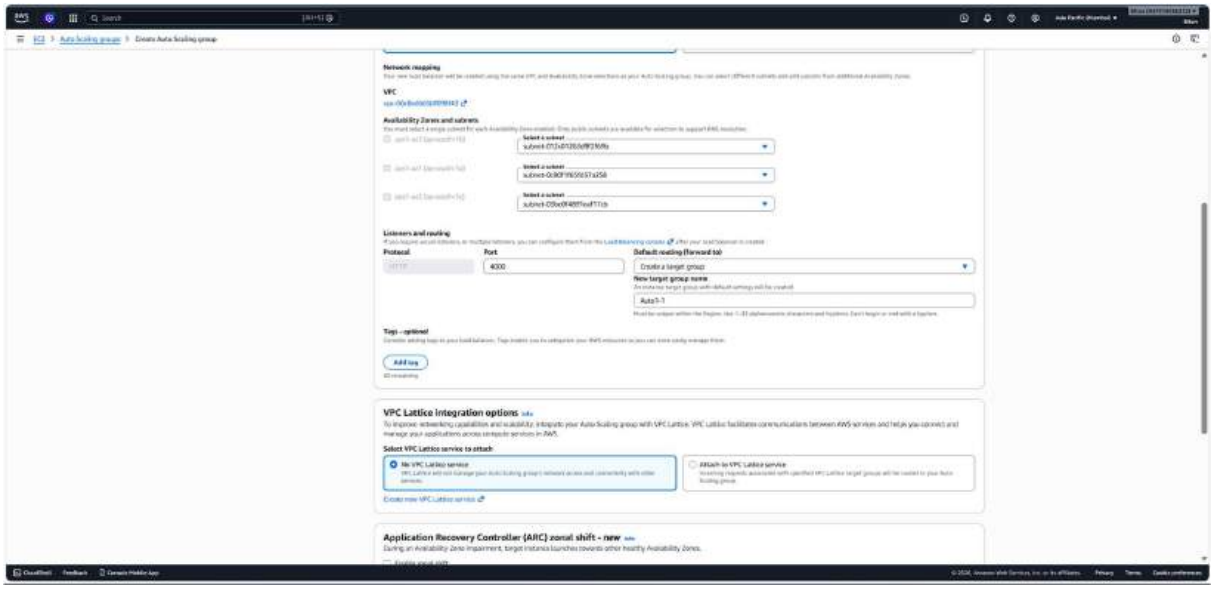
Step 2: Choose instance launch options- Choose all the ‘Availability Zones’ available. Click on Next.



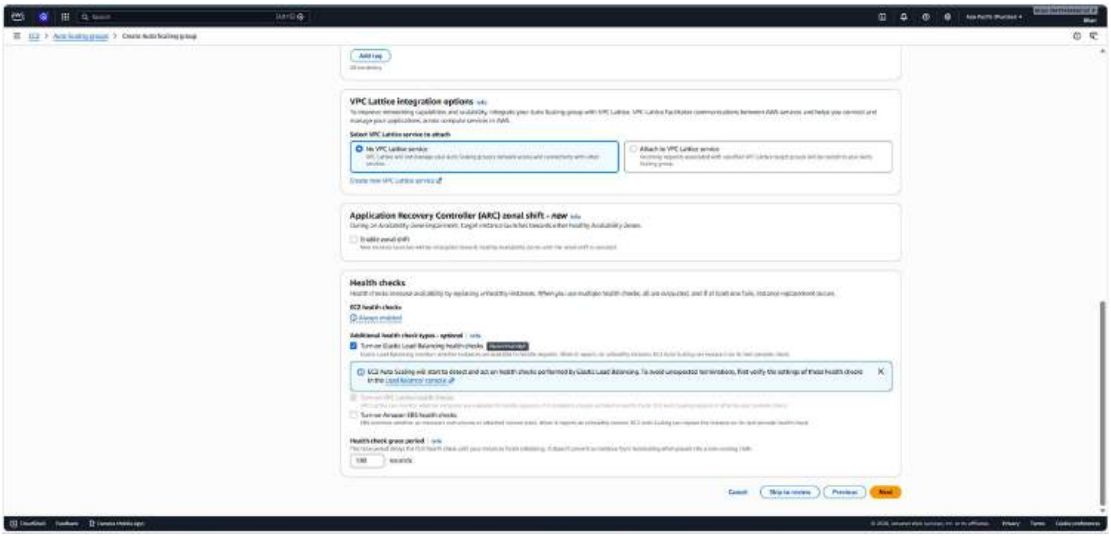
Step 3: Configure advanced options- For Load balancing select ‘Attach to a new load balancer’



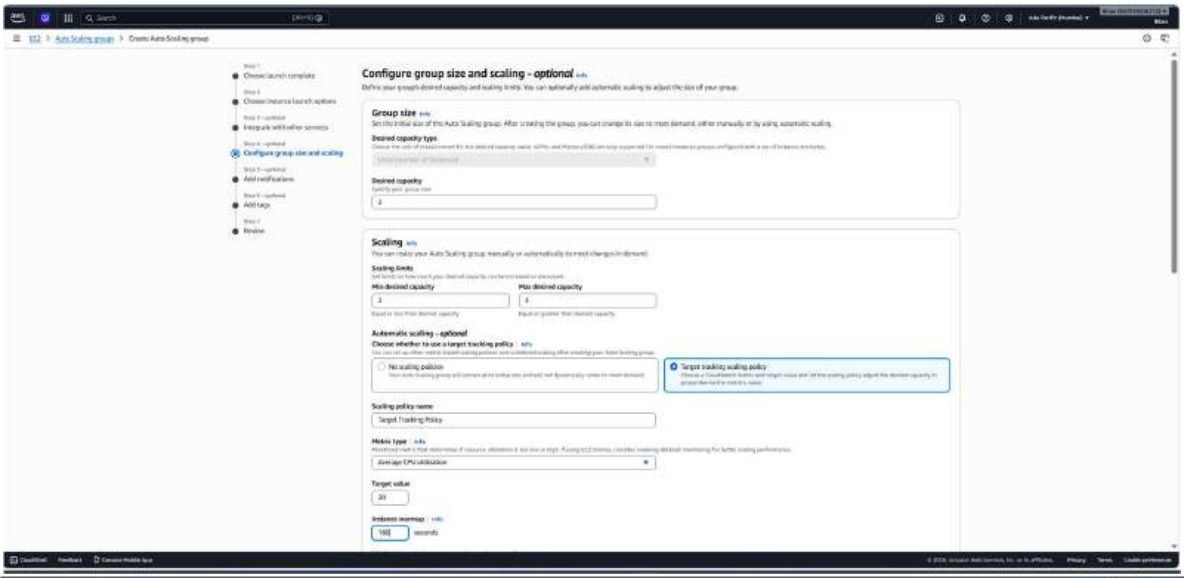
For new load balancer configuration select ‘Application Load Balancer’, ‘Internet-facing’ and make HTTP port from 80 to 4000 while creating a target group.

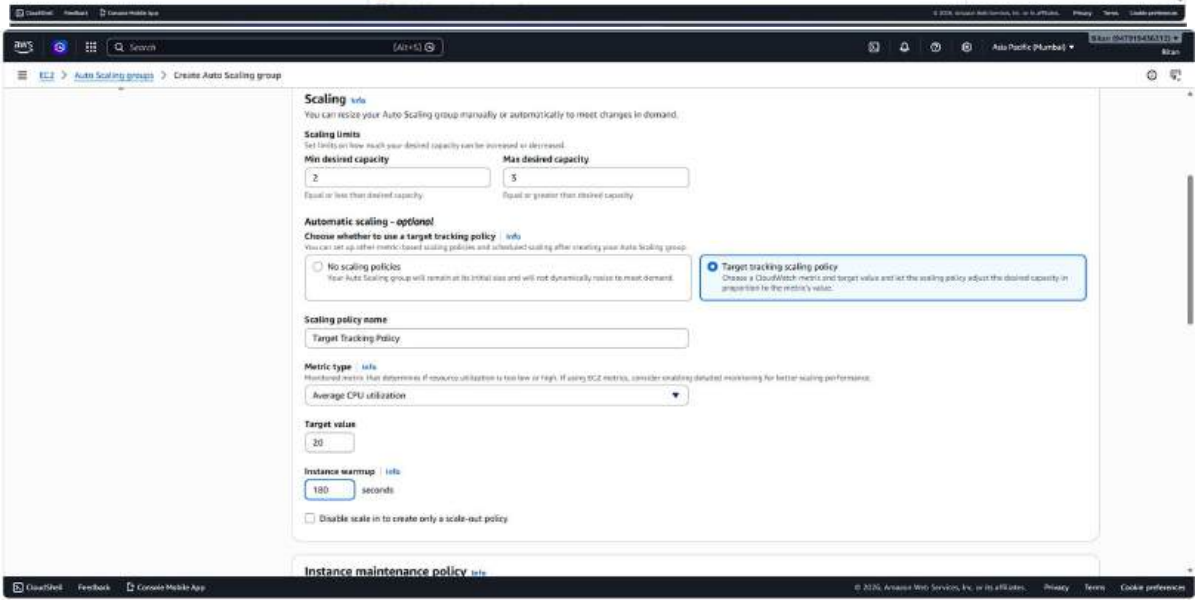


Health checks tick the check box for ‘Turn on Elastic Load Balancing health checks’ and make the health check grace period 180 seconds. Click on Next.

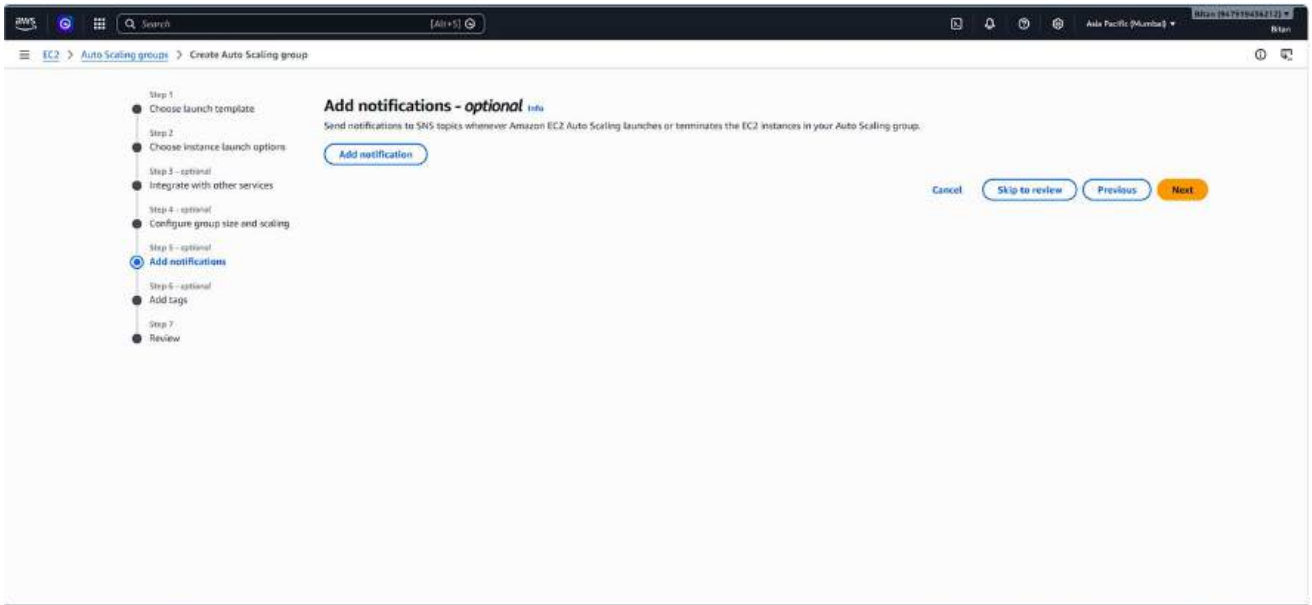


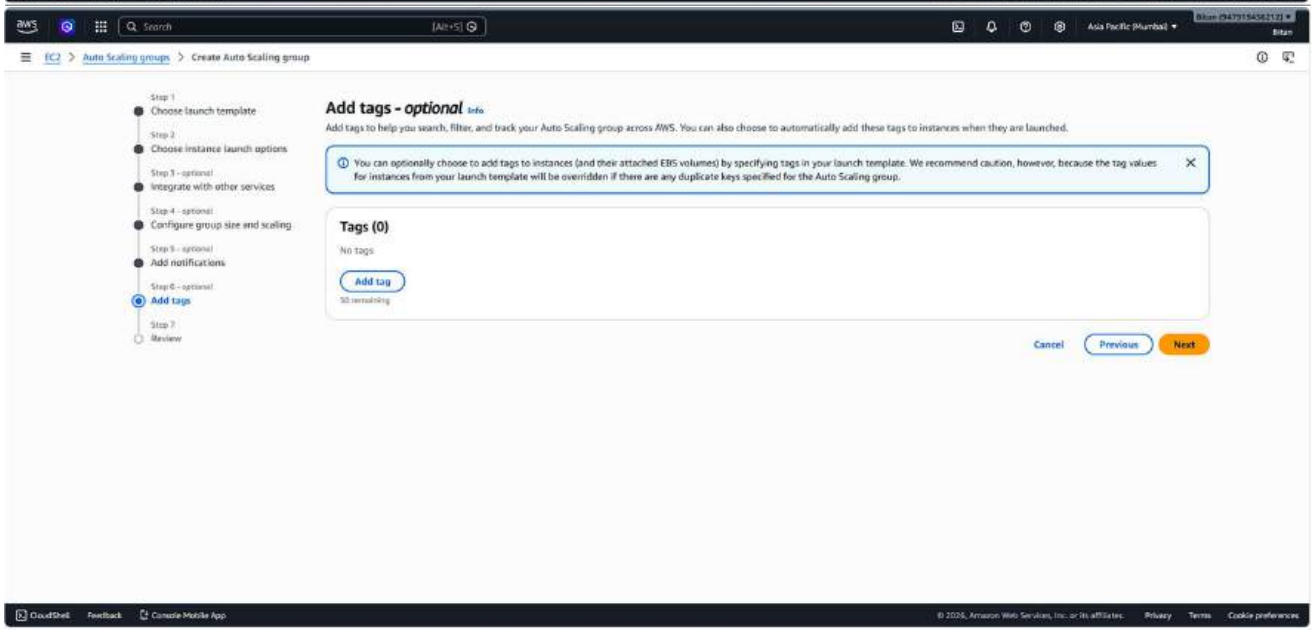
Step 4: Configure group size and scaling- In Group size set Desired capacity as 2. In Scaling set Min desired capacity as 2 and Max desired capacity as 3. In Automatic scaling choose target scaling policy and set the instance warmup as 180 second. Keeping everything else as default, Click on Next



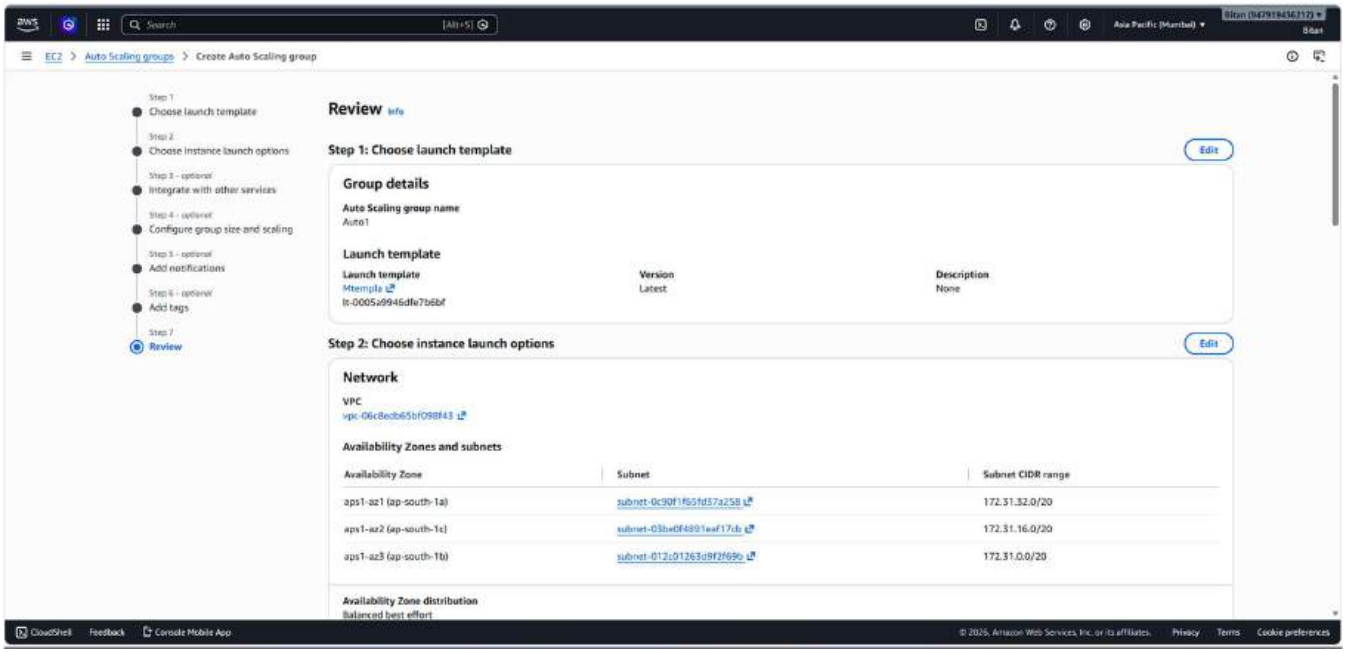


Skip through Step 5 and Step 6 by clicking Next.

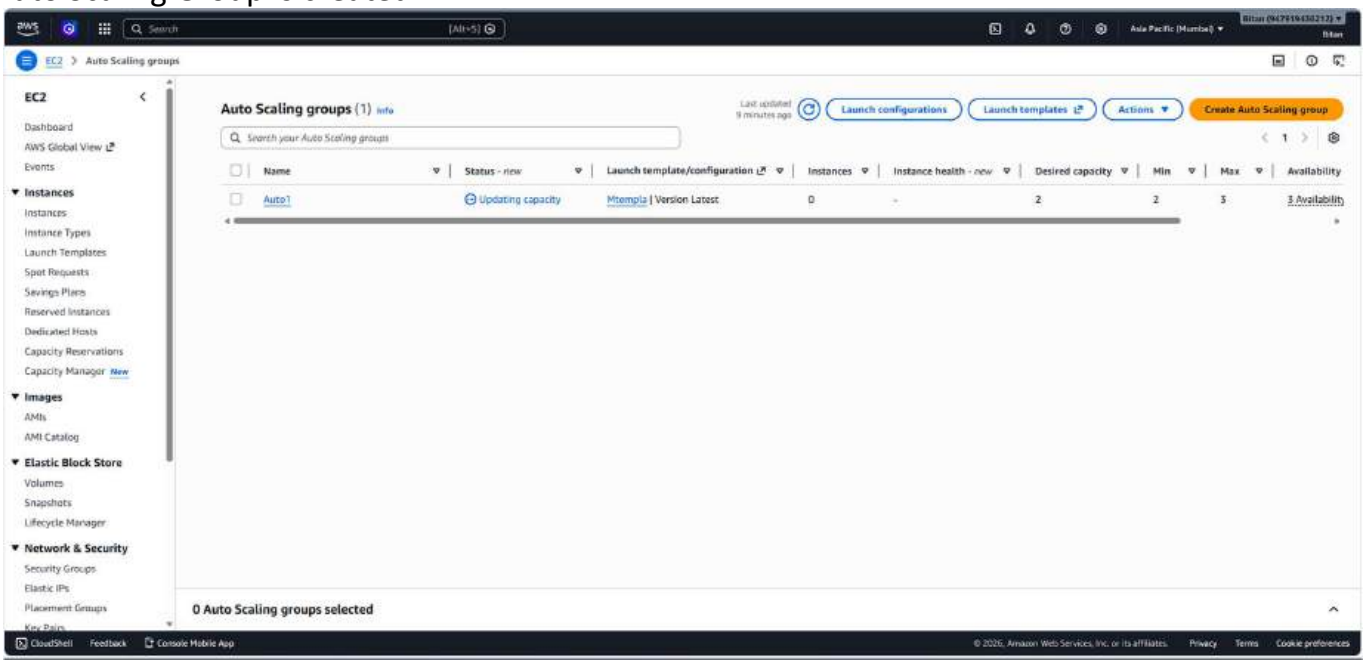




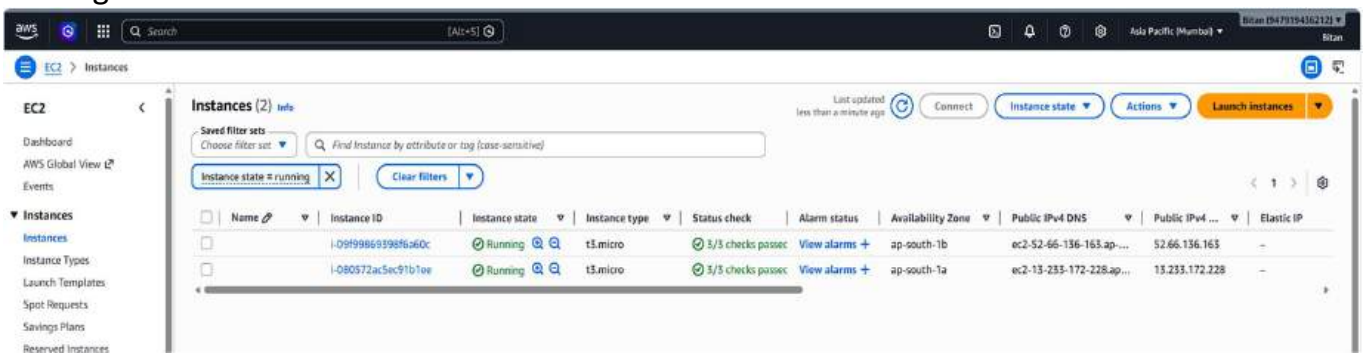
Finally in Step 7 Click on Create Auto Scaling Group.



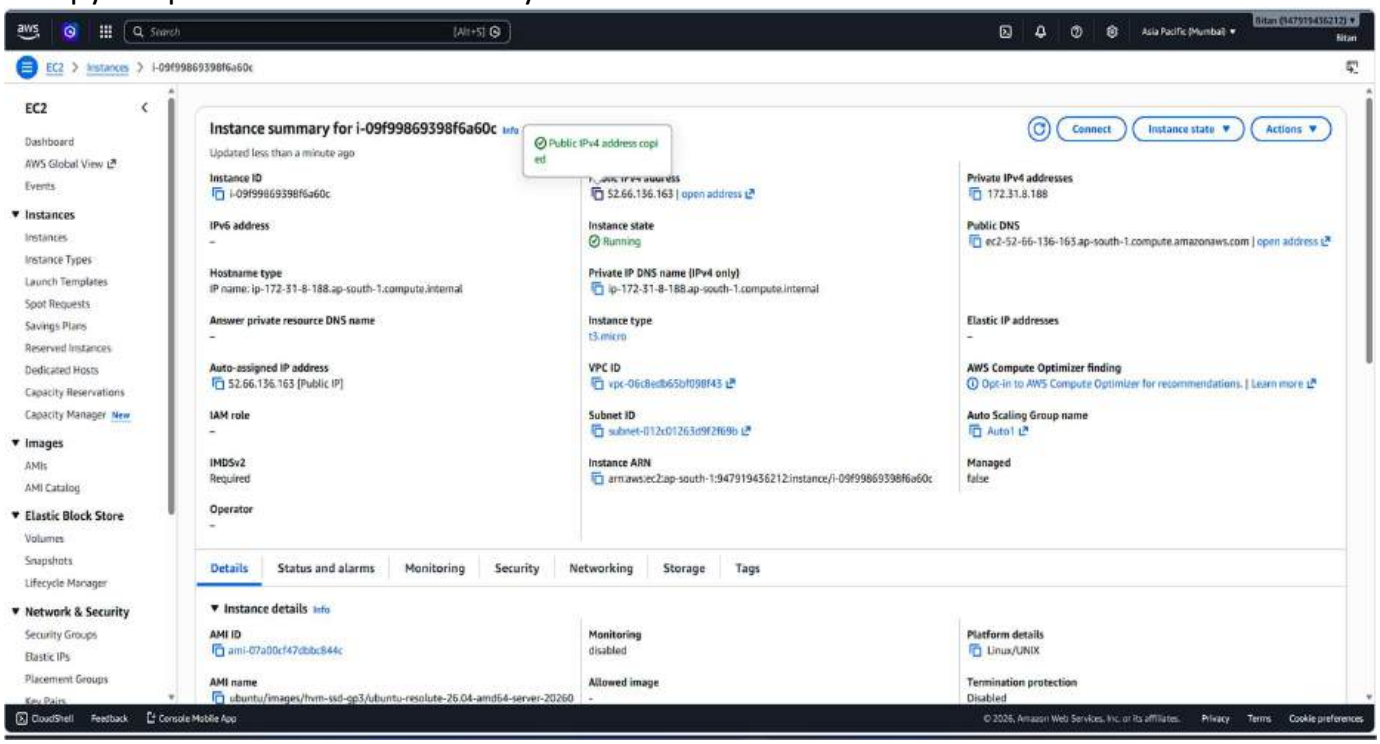
Auto Scaling Group is created.



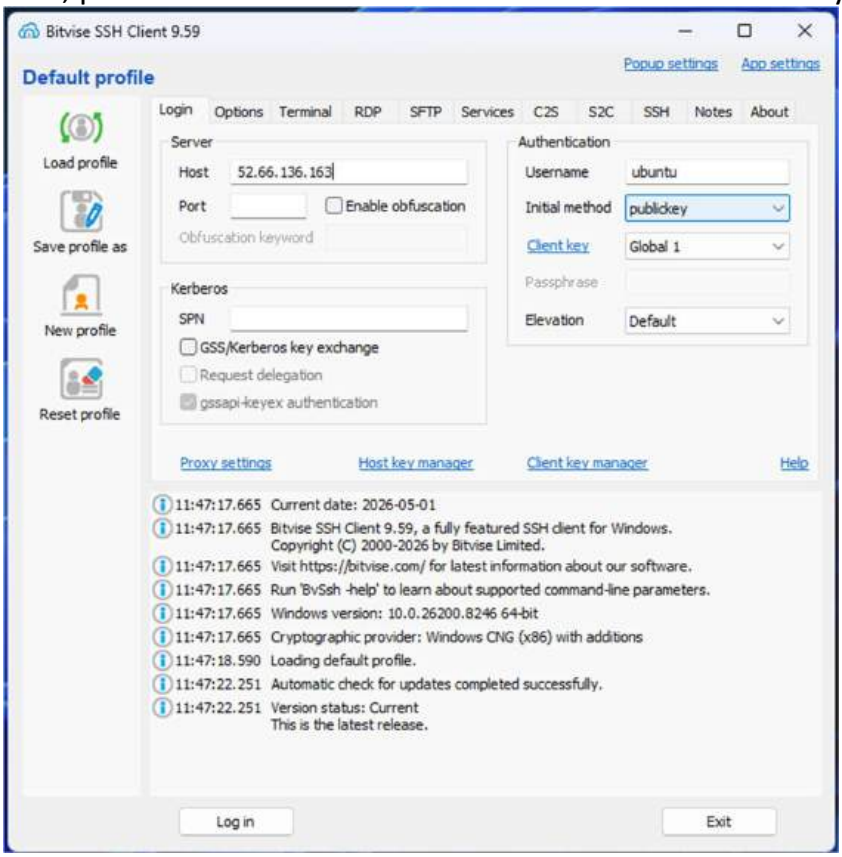
7. Going back to instances we can see two unnamed instances are created.



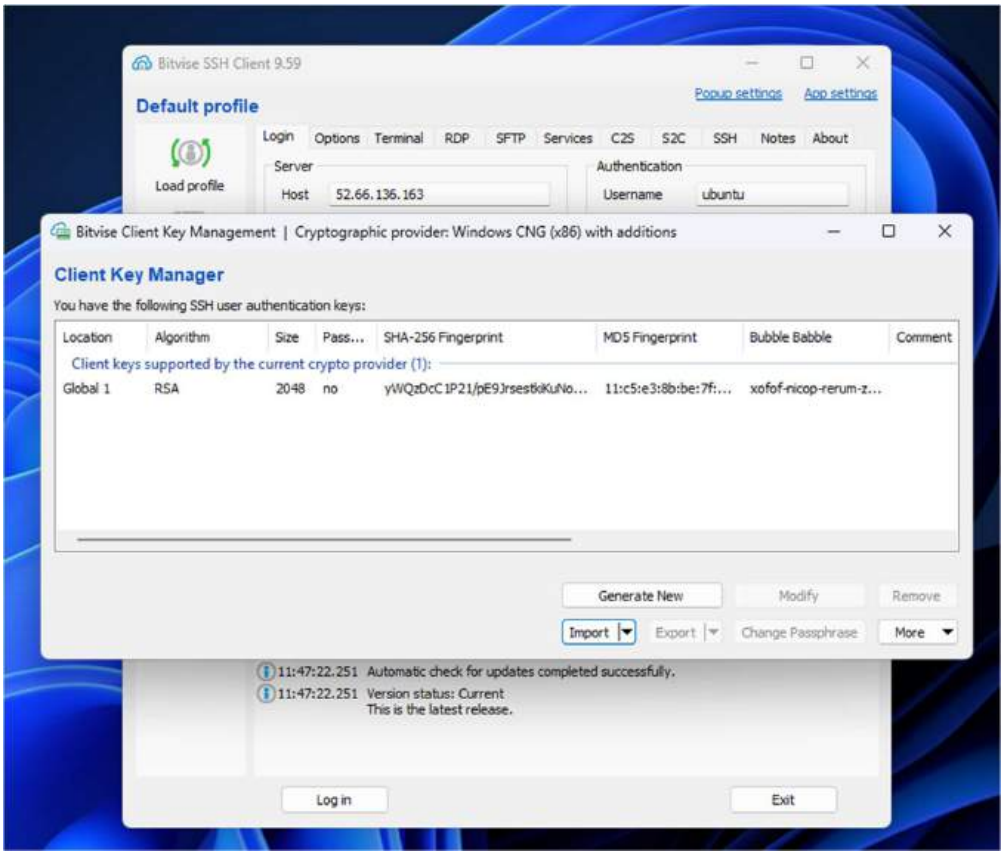
8. Copy the public IPv4 address of any one of the two instance.



9. In Bitvise SSH Client, paste the ‘Public IPV4 address’ and click on ‘Client key Manager’.

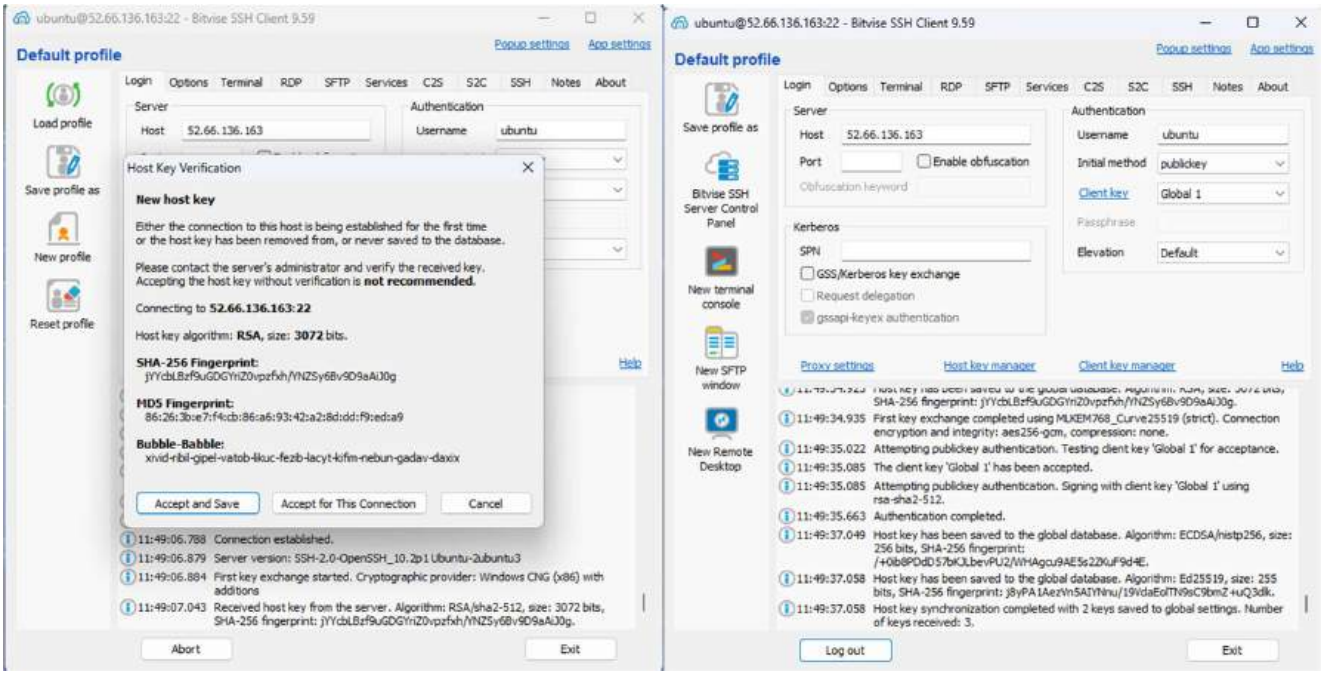


10. Under ‘Client key manager’, if there is any existing key then remove it and click on ‘Import’, select the same key that you used while creating the instance, from your file directory and then again click on ‘Import’ and then the key is successfully imported.



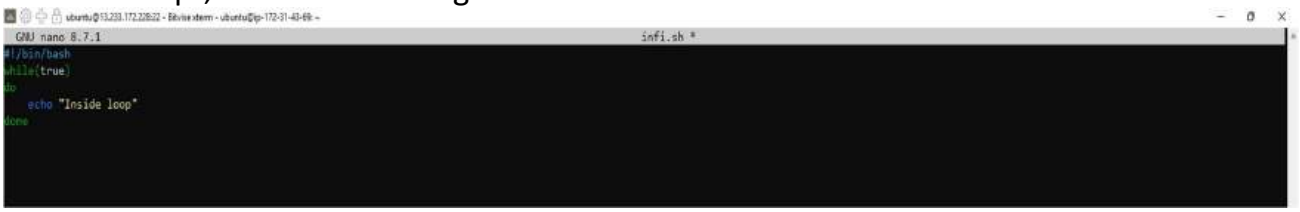
10. In ‘Bitvise SSH Client’, under ‘Default profile’ give the ‘Username’ as ‘ubuntu’ then select ‘Global1’ from ‘Client Key Manager’ then click on ‘Log in’ & ‘Accept and save’.

If the ‘Log In’ button changes to ‘Log Out’, then you are logged in successfully.

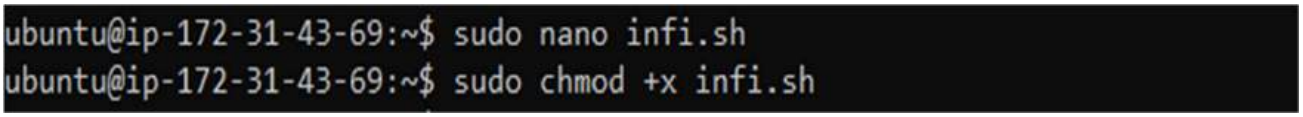


11. Click on ‘New terminal consol’, to access the terminal of your instance.
Once you are inside the terminal of your instance
To create a infinitely running bash script vim infi.sh

In the bash script, add the following code.



To make the script executable
sudo chmod 777 infi.sh

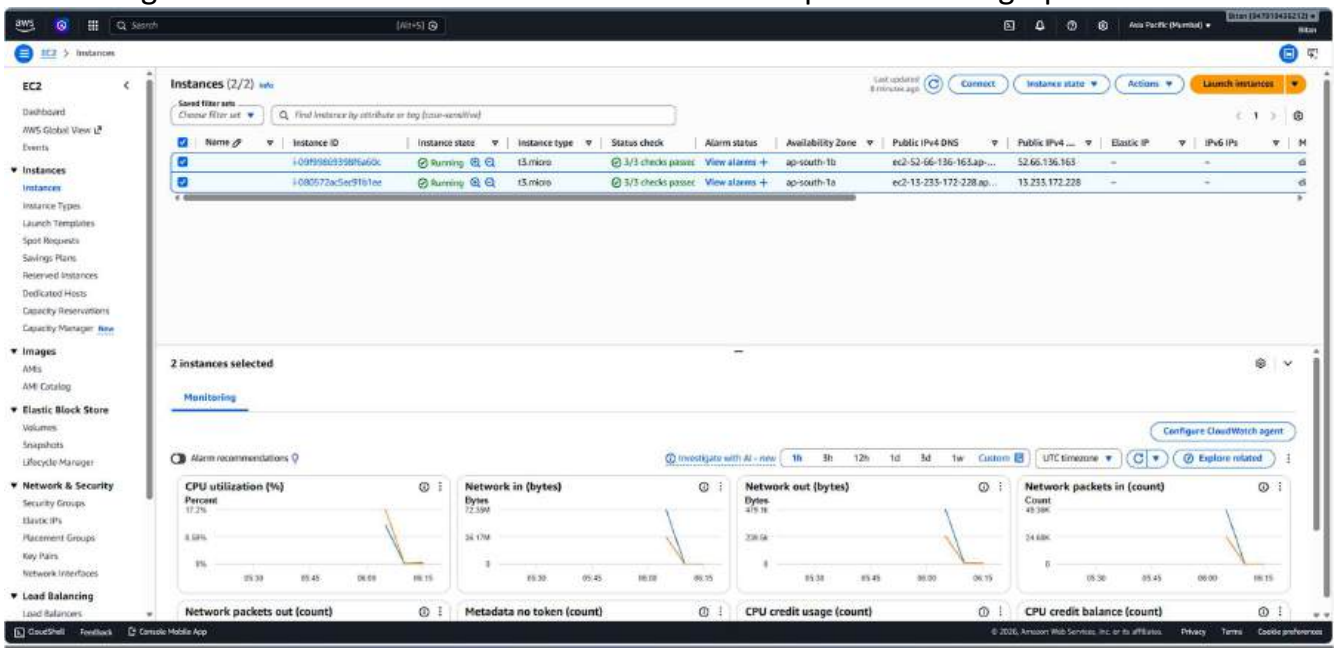


Run the script sh
infi.sh

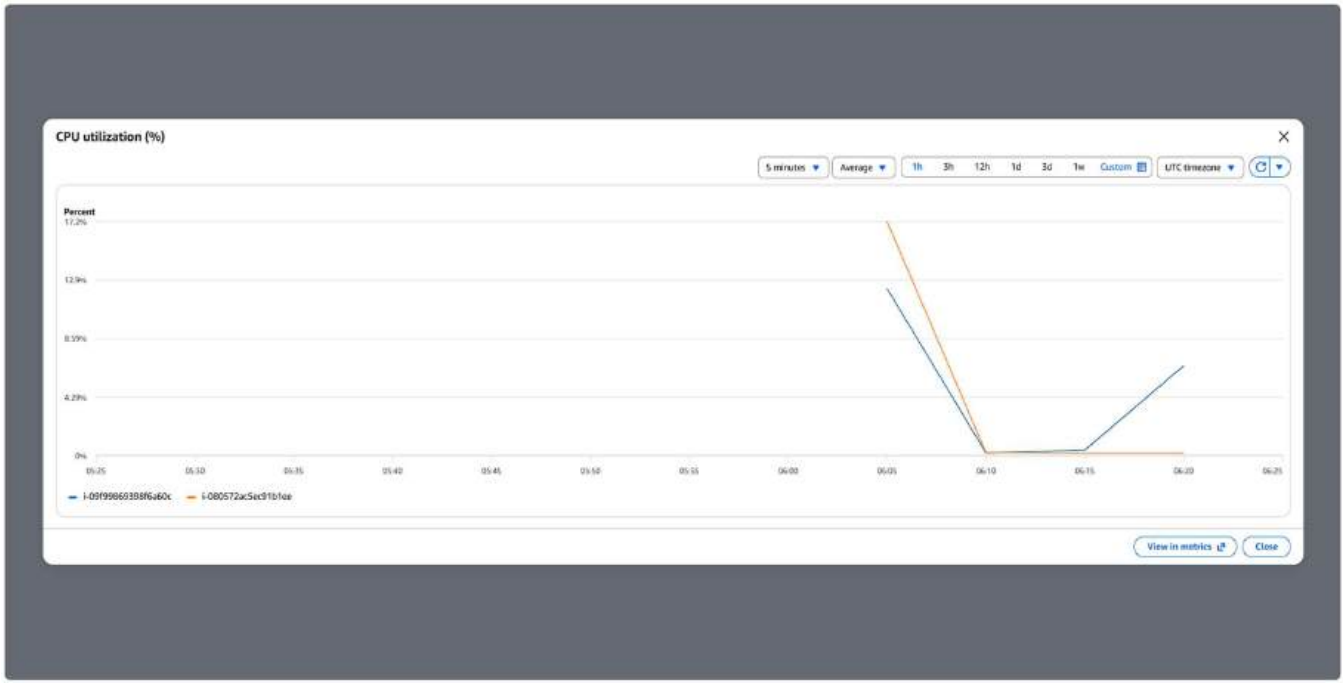
```
ubuntu@ip-172-31-43-69:~$ sh infi.sh
```

12. An infinite loop starts to run, stressing the cpu of the instance.

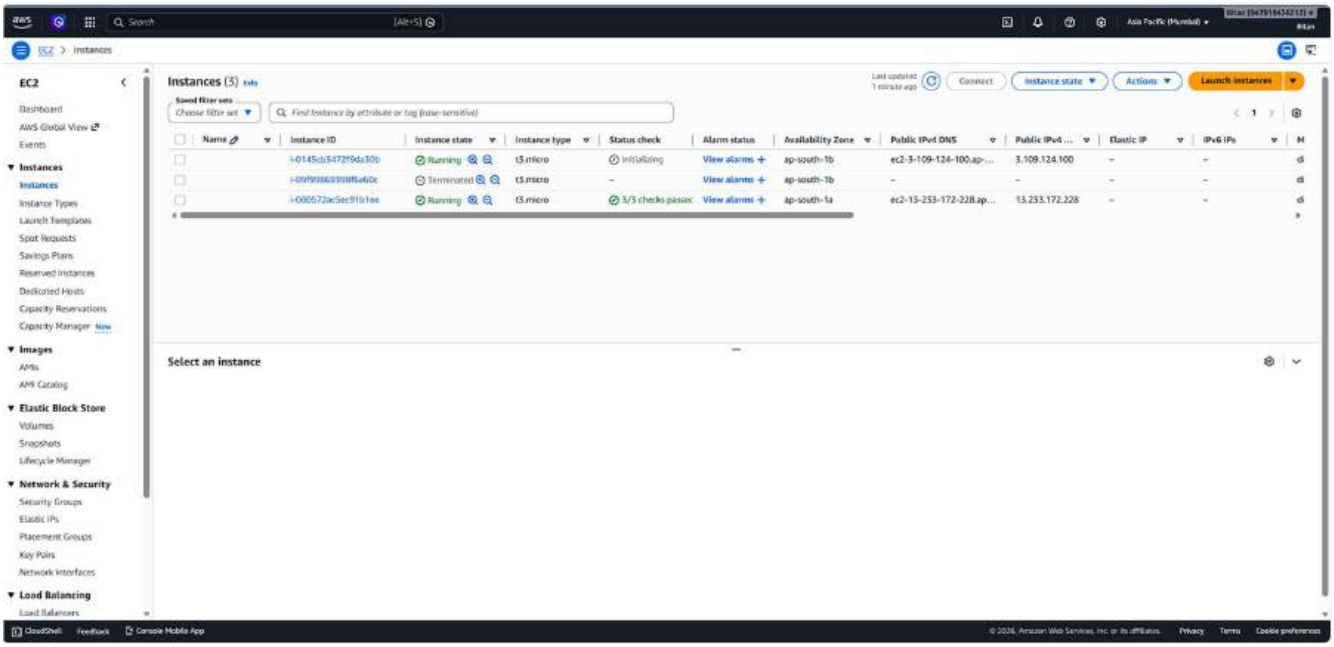
13. Selecting the two instances we can take a look at the cpu utilisation graph.



14. After a while the instance reaches and surpasses it’s 50% CPU utilisation threshold.

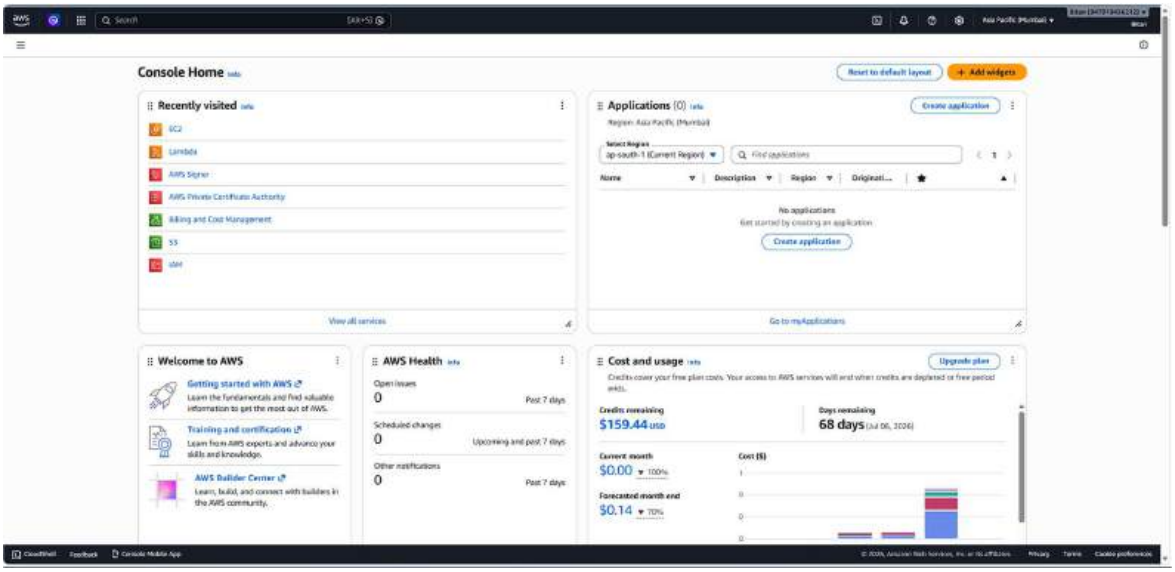


15. So to take that load off the instance, the auto scaler spins up another instance to balance the load.

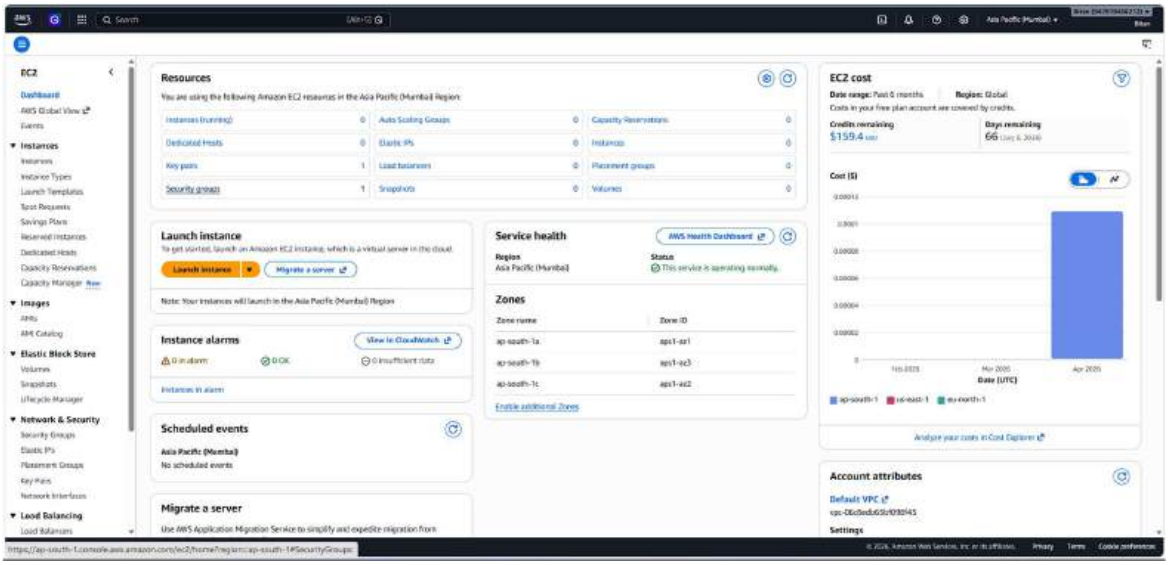


ASSIGNMENT 12 : Deploy and run the project in AWS without using the port.

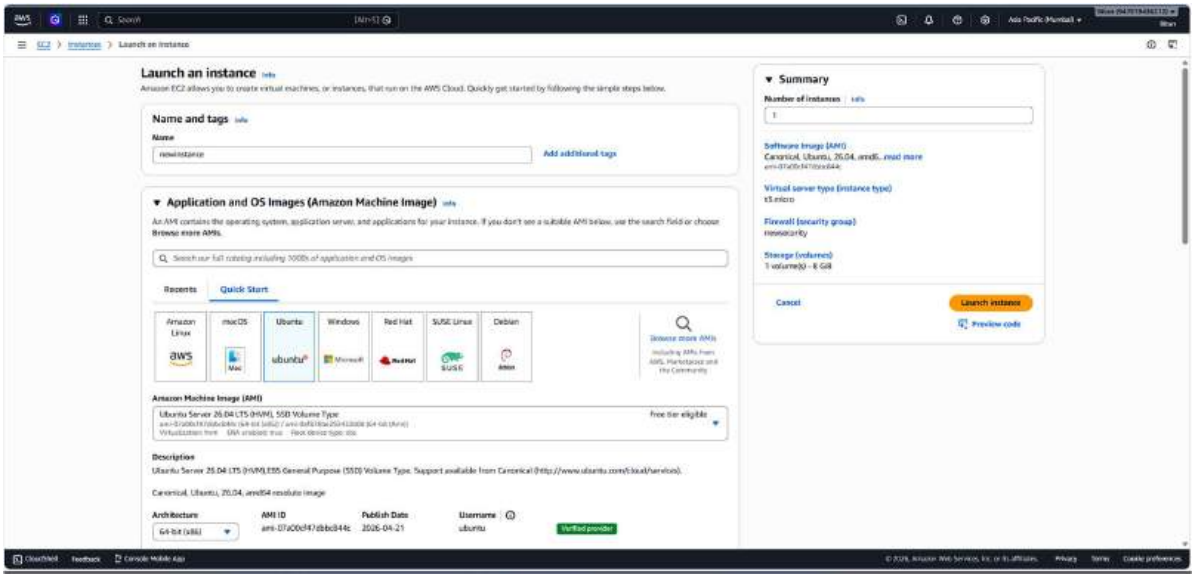
1: Go to the EC2 dashboard.



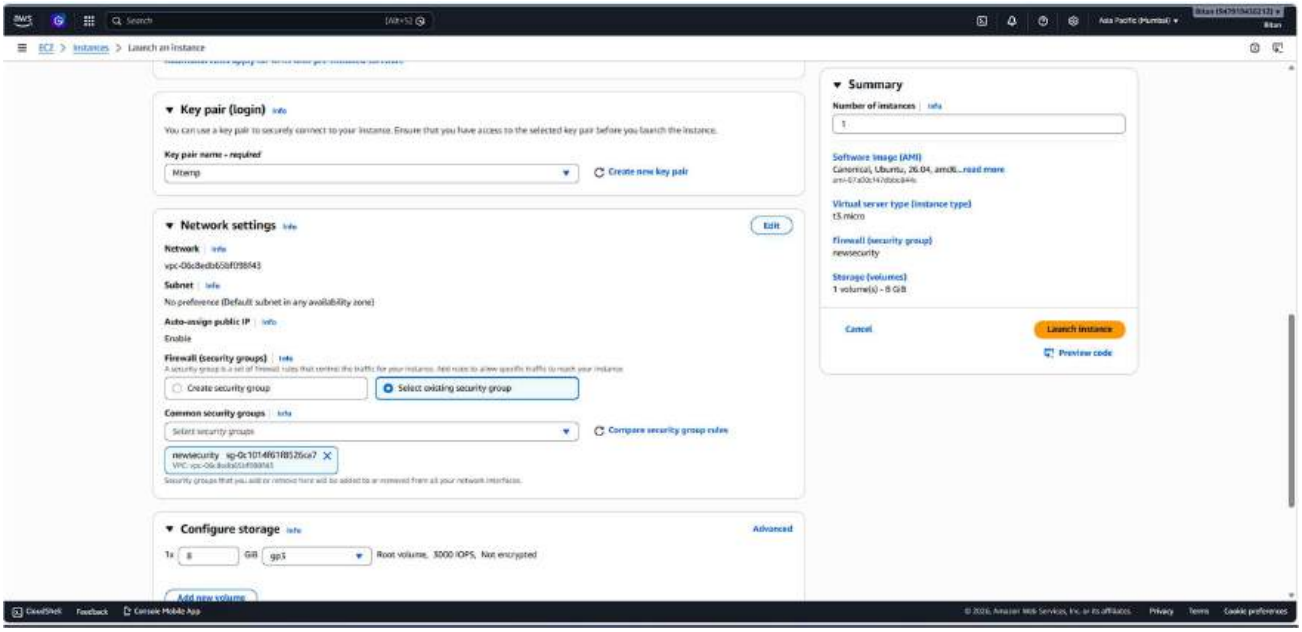
2. Click on Launch Instance.



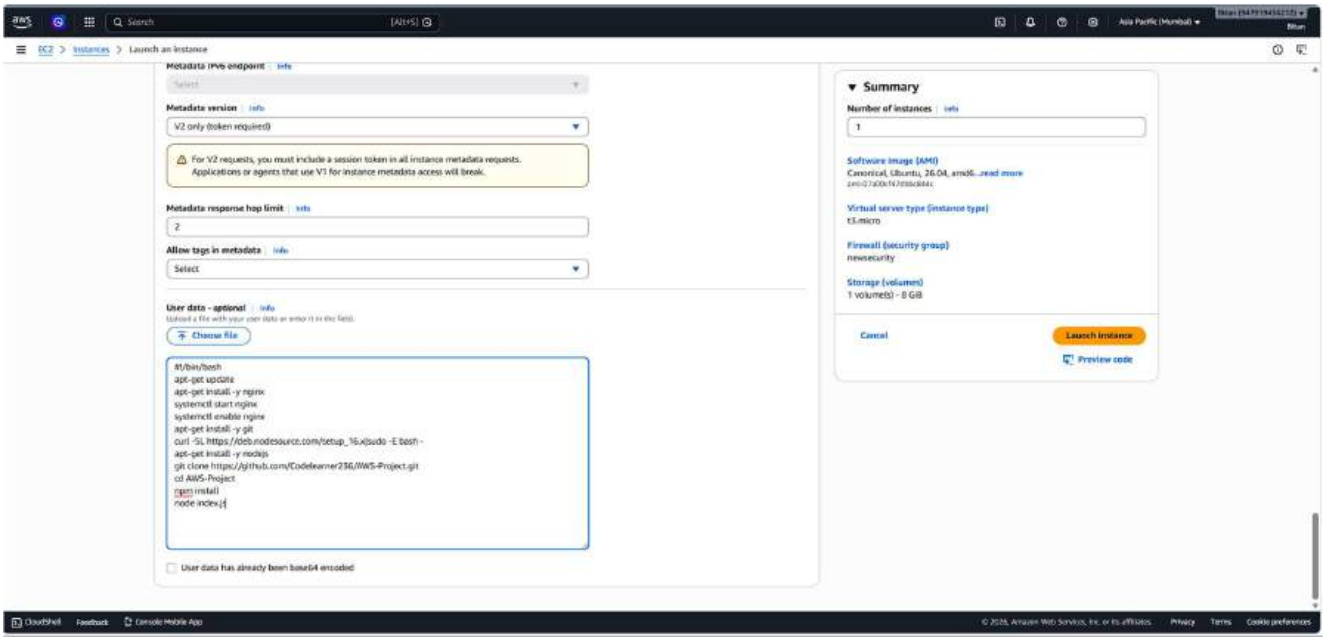
3. Give the name of the instance and select UBUNTU as the OS.



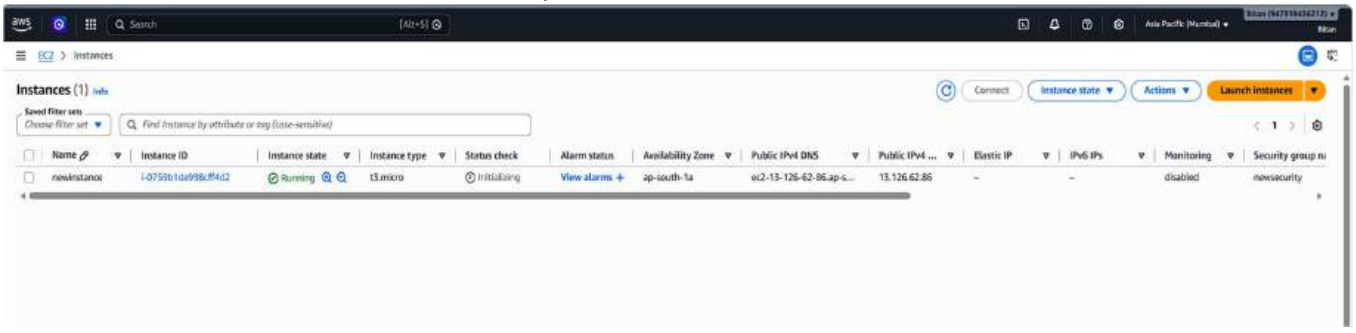
4. Select the keypair and select existing security group. (If not created then create the both).



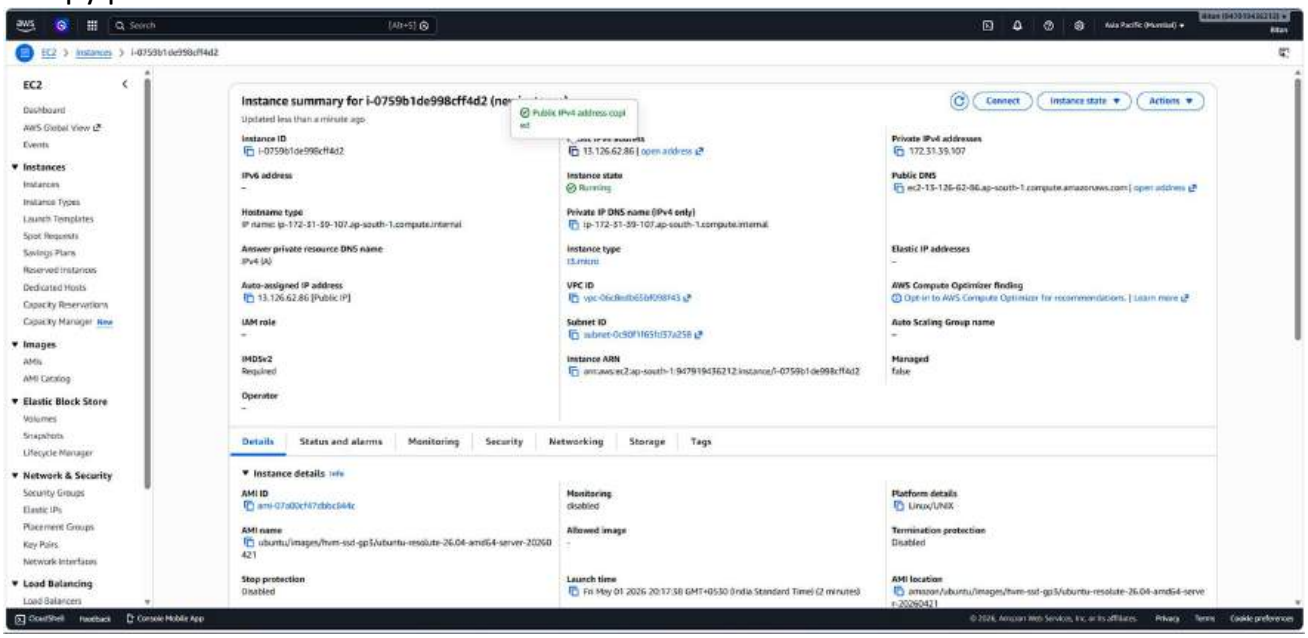
5. In the Advanced settings scroll down and write the code shown in the picture. Then click on “Launch Instance”.



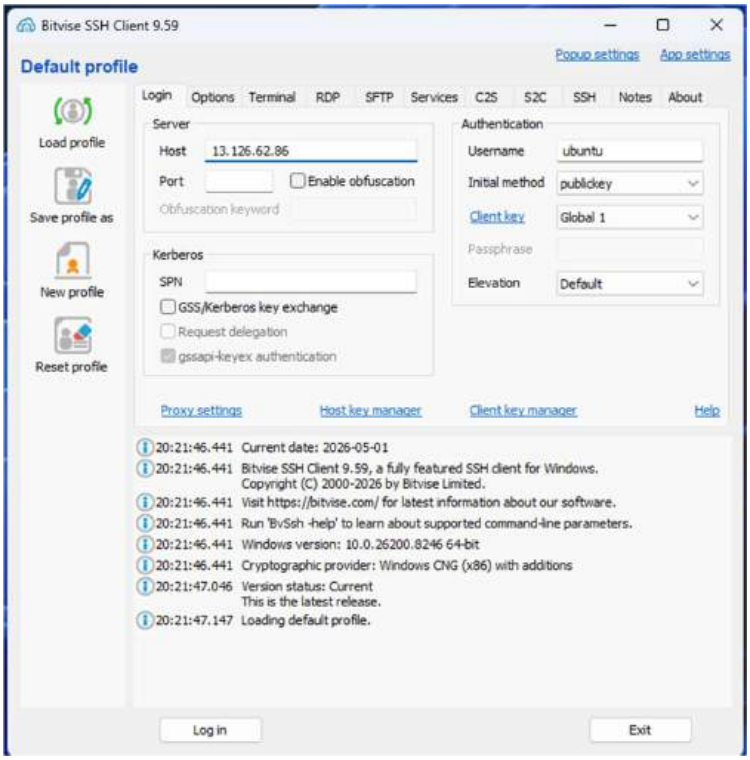
6. The instance is created successfully.



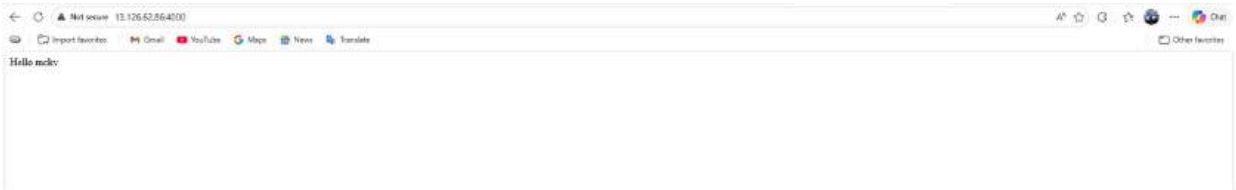
7. Copy paste the Public IPv4 address of the created instance.



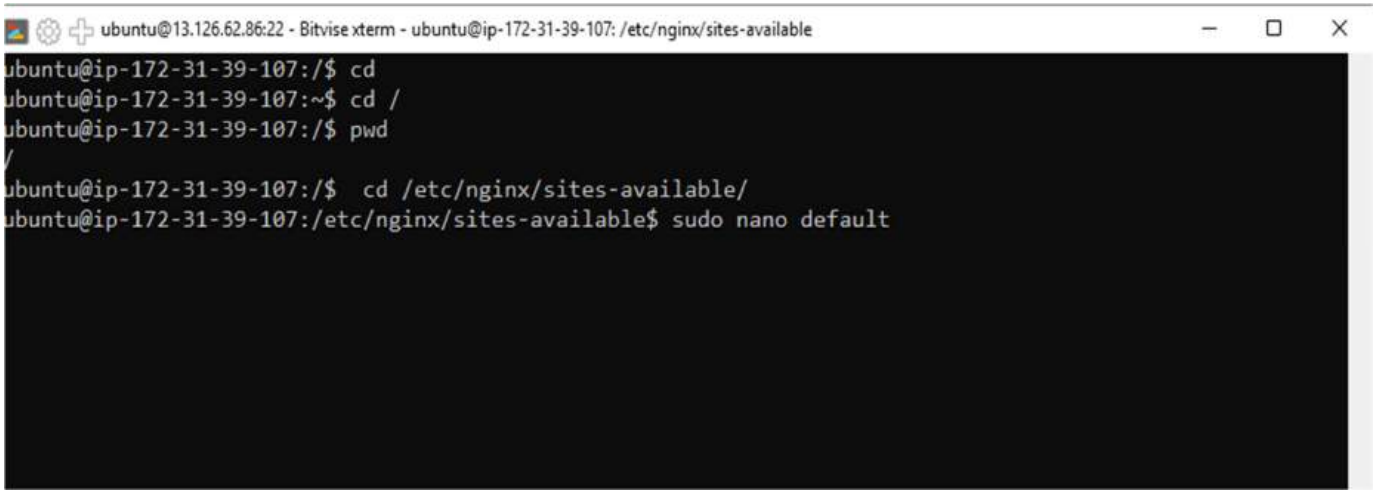
8. Open the Bitvise SSH client and paste the ipv4 address here. Under Authentication select username as “ubuntu”, initial method as “public key” , client key as “Global 1” and in client key manager import the keypair selected while creating the instance. Then click on “Log in” then Accept and Save.



9. Paste the copied ipv4 address on the new tab. Add 4000 (the port number) at the last of the address to see the content of the index.js file.

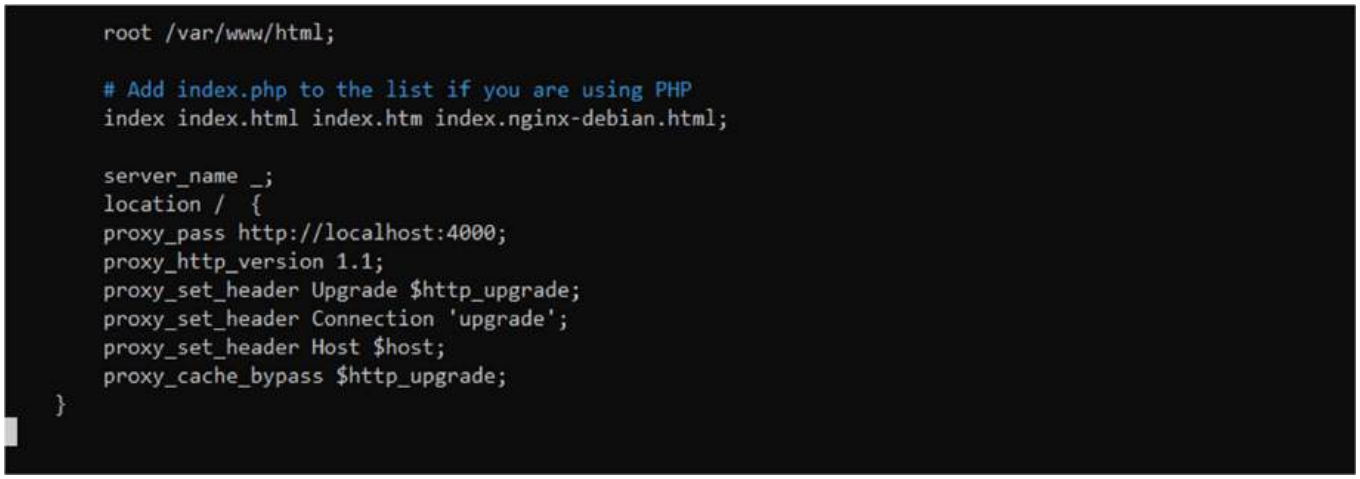


10. Go to the new terminal window from Bitvise SSH client and write the commands shown in the picture.



11. Uncomment the code shown in the picture.

12. Add the following code instead. Then Ctrl + X Y Enter. You will come back to bash terminal.



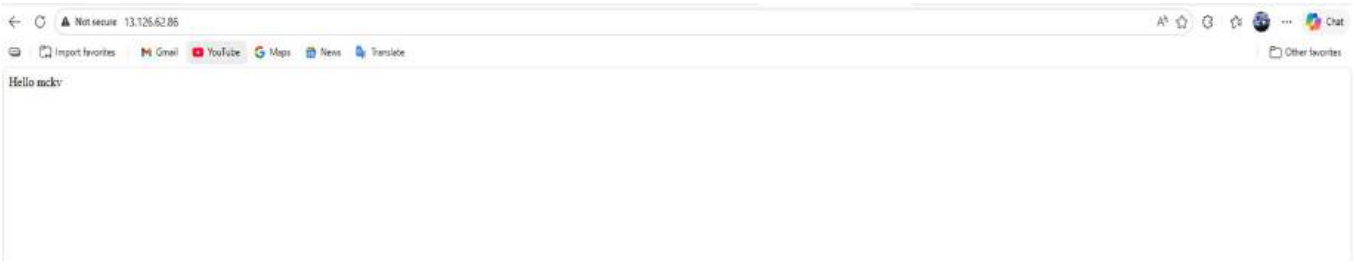
13. Write the code : sudo systemctl restart nginx

Go back to the created instance in AWS and refresh it.

```
ubuntu@ip-172-31-39-107:/etc/nginx/sites-available$ sudo systemctl restart nginx
```

14. Copy paste the ipv4 address again in the new tab without adding the port number 4000 to it.

You will get your desired output.

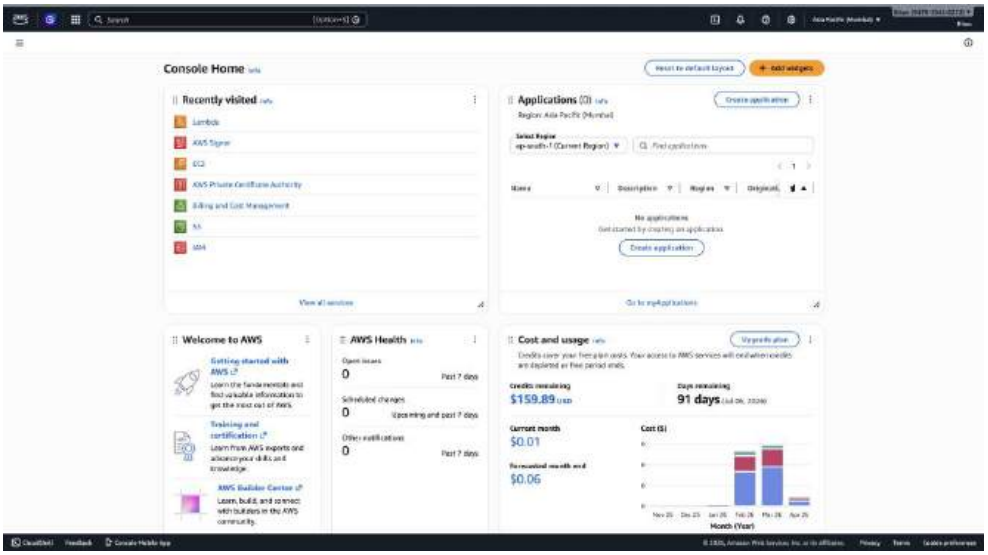


Seeing the output we can conclude that our project is successfully deployed and run in AWS without the help of port number.

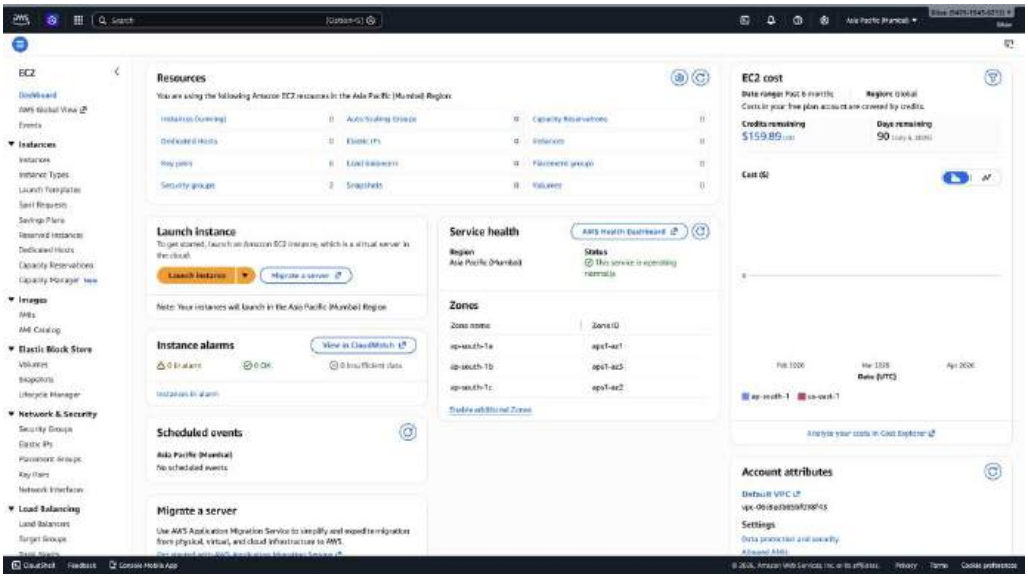
Assignment No. : 14

Problem Statement: Create an Elastic IP for an Instance.

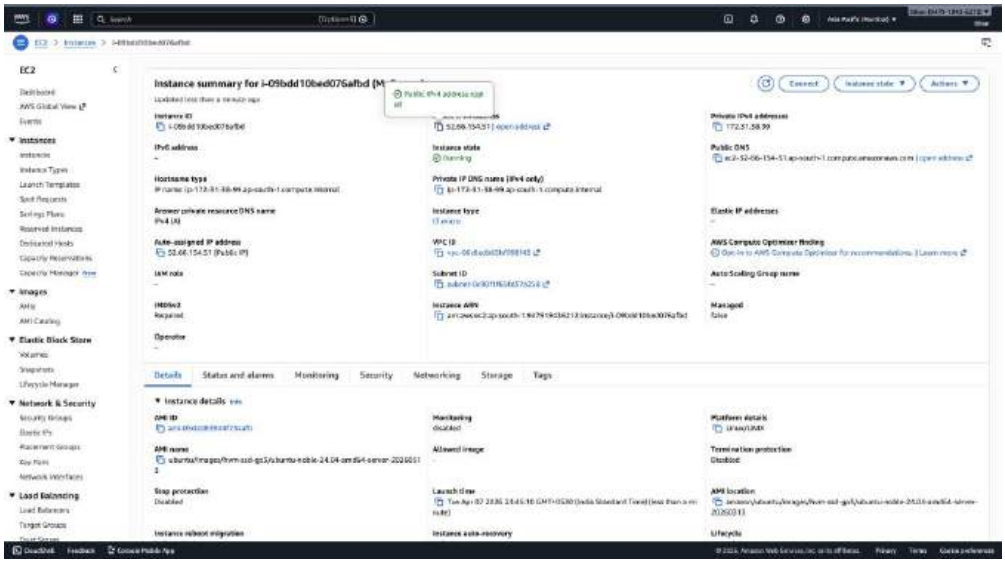
STEP 1-> Go to “EC2” from the AWS home screen.



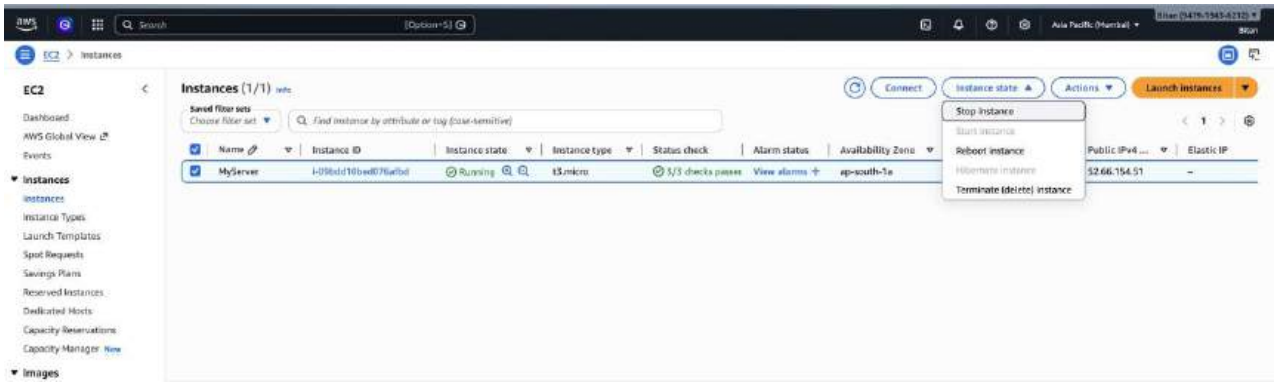
STEP 2-> From the left side menu, go to “Instances”.



STEP 3-> Select an instance or Launch a new one and save its Public IPv4 address in a text file for future comparison.



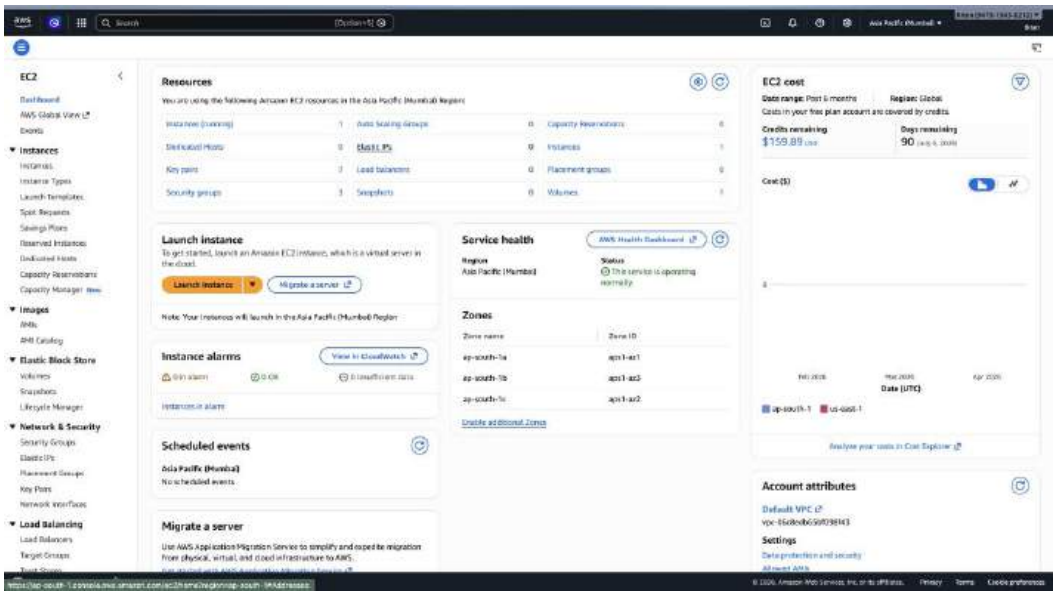
STEP 4-> With the instance selected, open the Instance State dropdown menu and select “Stop Instance” option to temporarily put the instance in the sleep state. Now again start this instance.



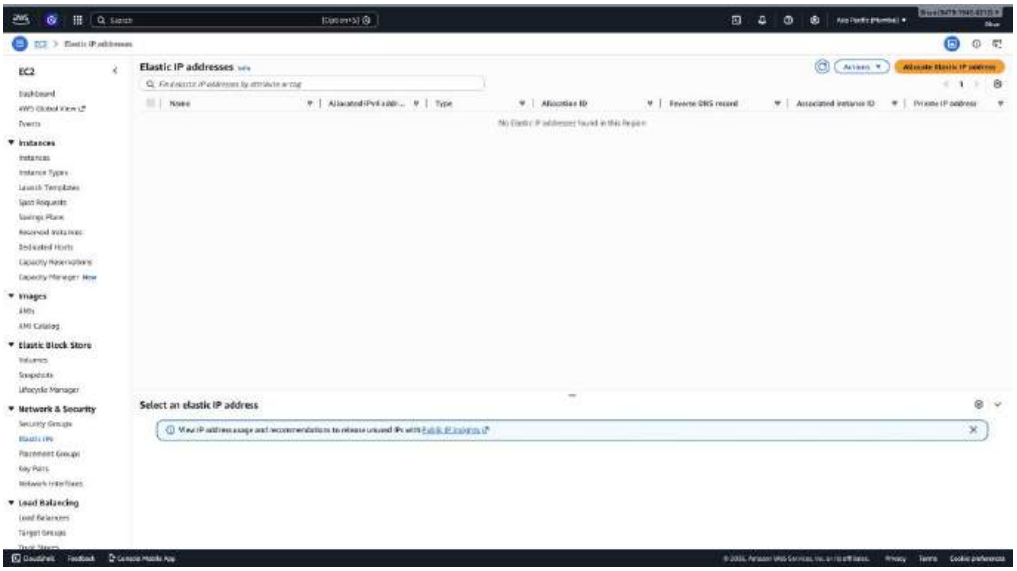
STEP 5-> Now again save the Public IPv4 address in the text file and compare it with the previous IP.

It is evident that the IP address changes every time the instance is launched. To avoid it Elastic IP has to be assigned.

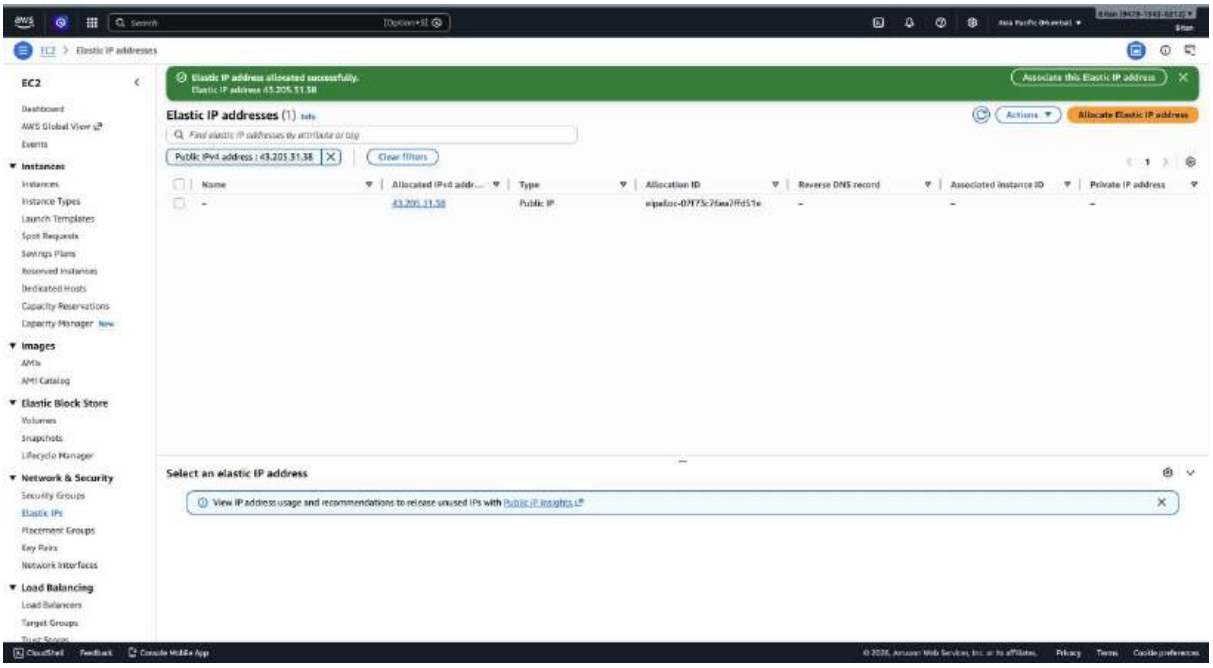
STEP 6-> From the EC2 Dashboard, go to the “Elastic Ips” option.



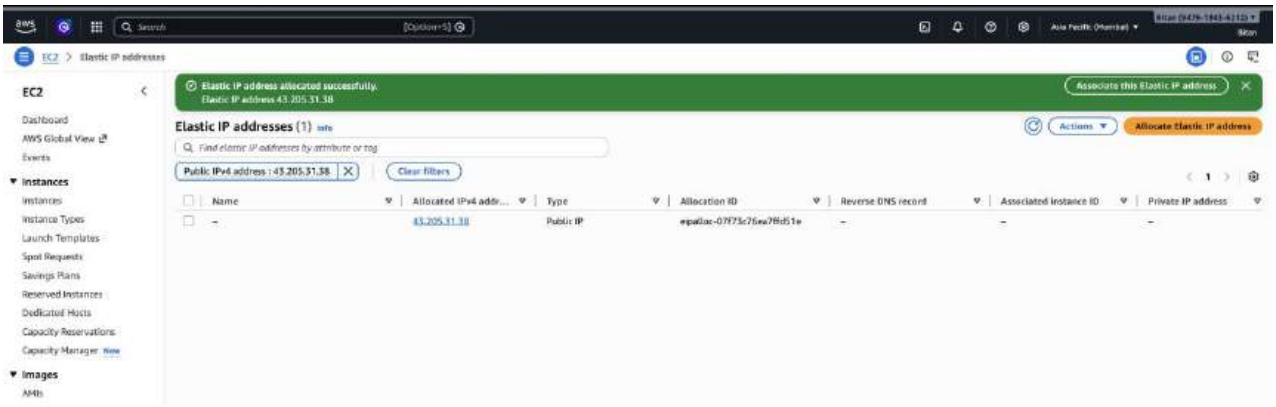
STEP 7-> Click on “Allocate Elastic IP” button on the top right corner of the screen.



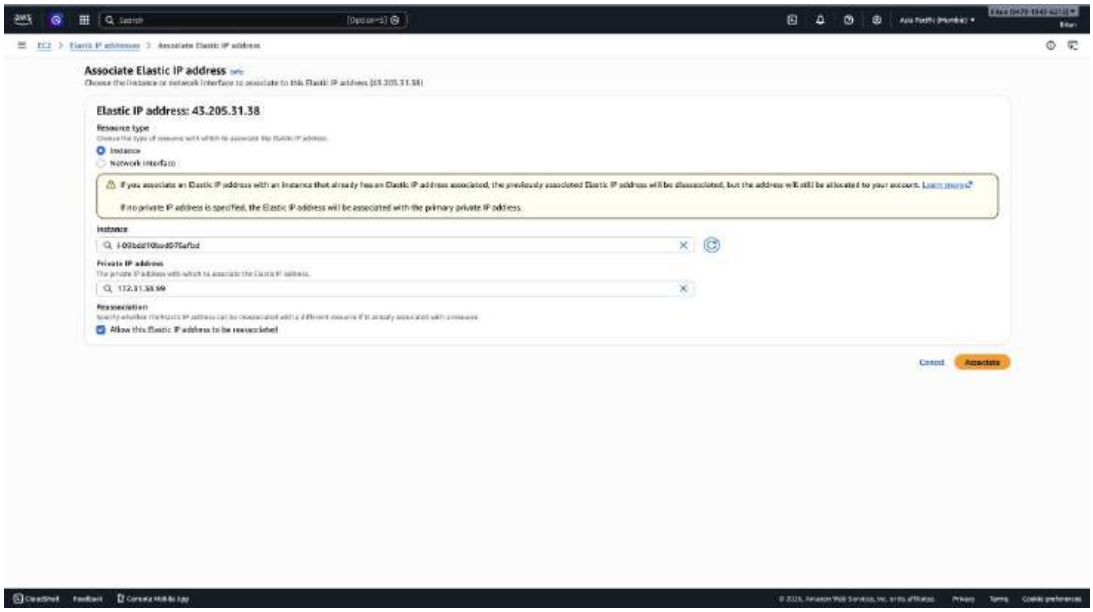
STEP 8-> With “Amazon’s pool of IPv4 Addresses” selected under Public IPv4 Address pool, click on “Allocate”.



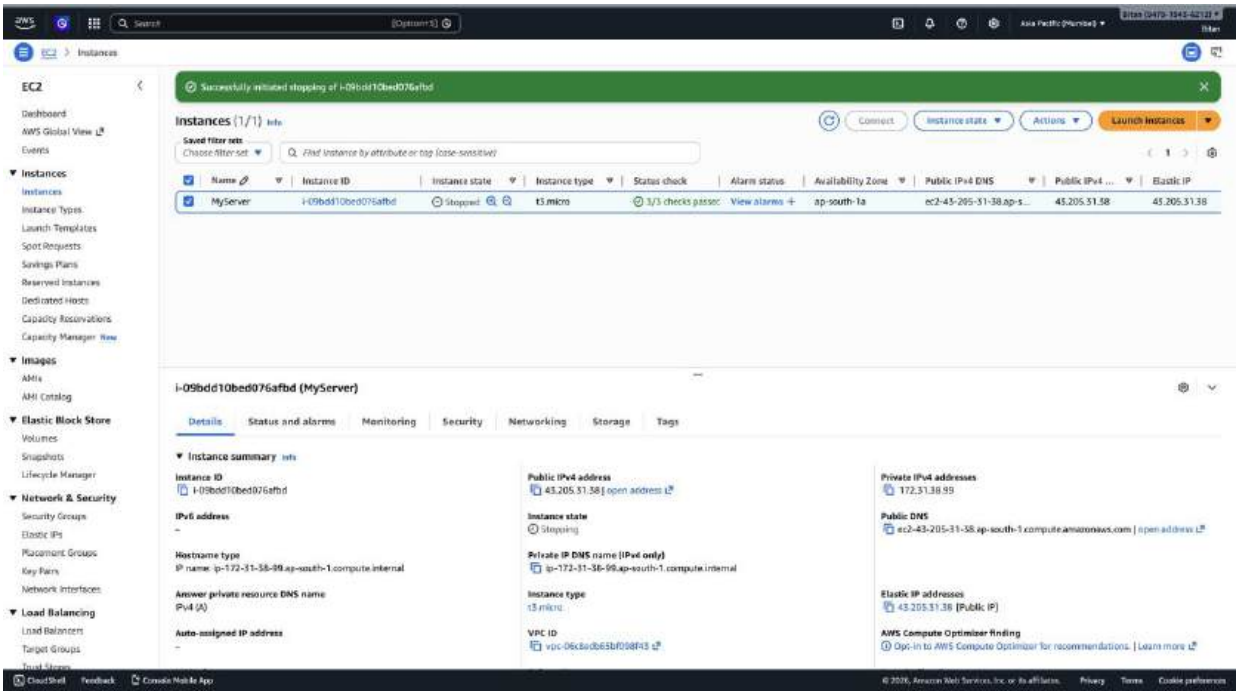
STEP 9-> The Elastic IP has been created. Then Click on “Associate Elastic IP Address” showing in the green box.

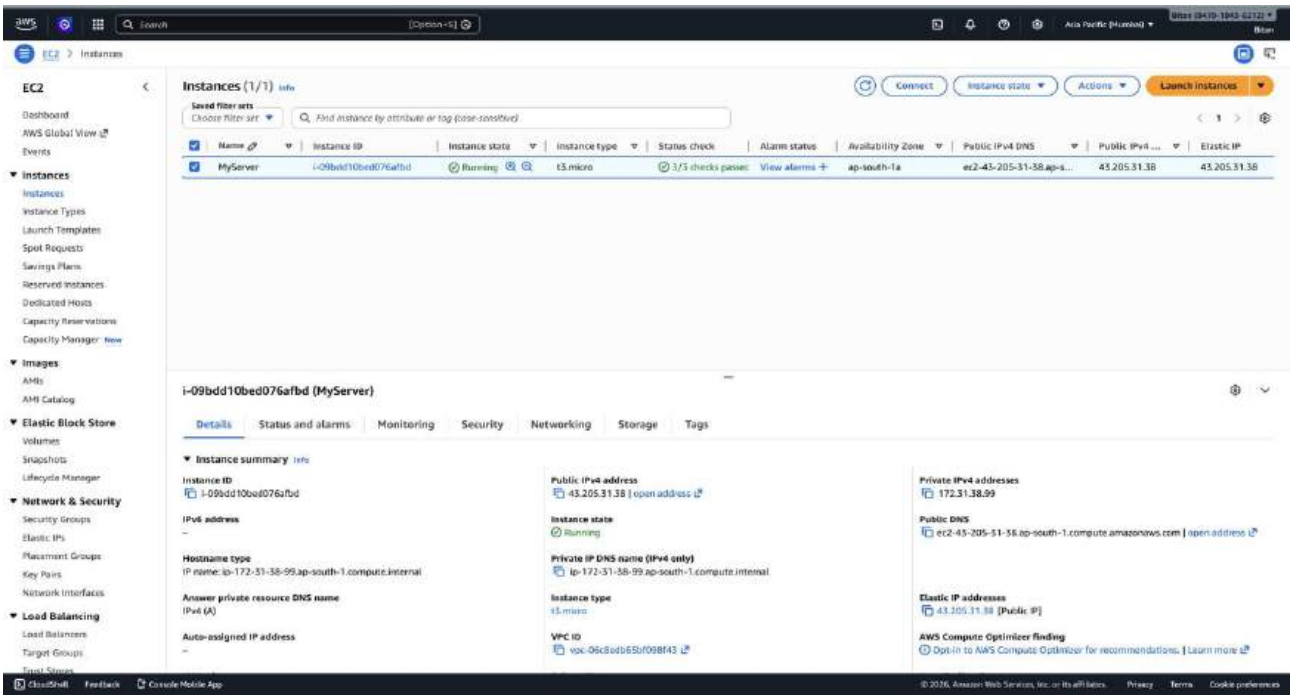


STEP 10-> With resource type selected as “Instance”, select the name of the instance & its private IP address. Check the bottom checkbox to reassociate the Elastic IP. Then click on “Associate”.



STEP 11-> Copy the IPv4 address before selecting “Stop Instance” and again start this instance and copy the IPv4 address.



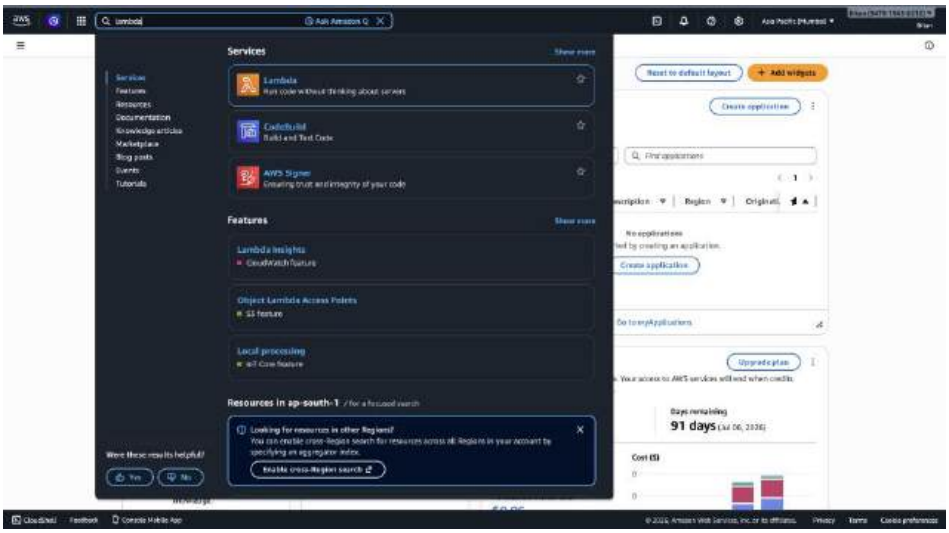


STEP 12-> No changes are observed.

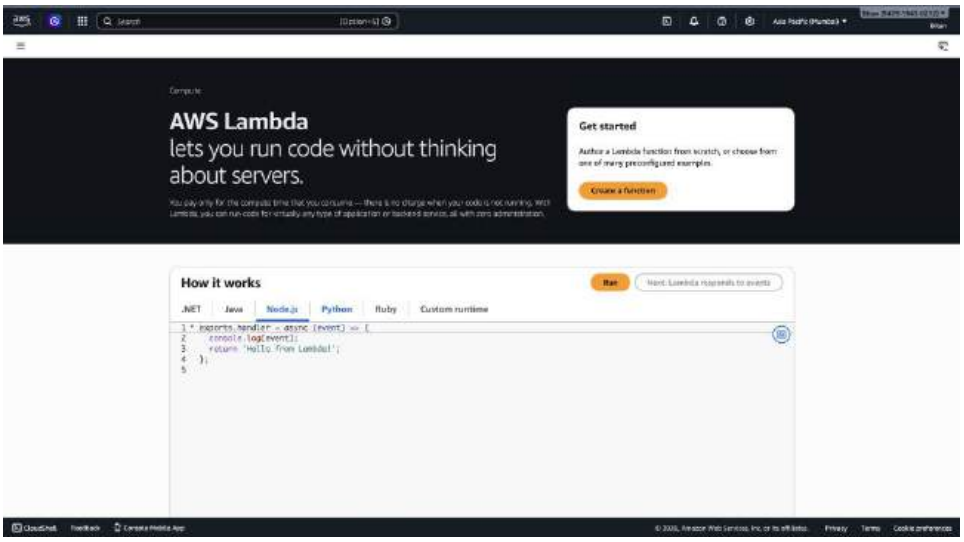
Assignment No. : 15

Problem Statement: Create a serverless computing service.

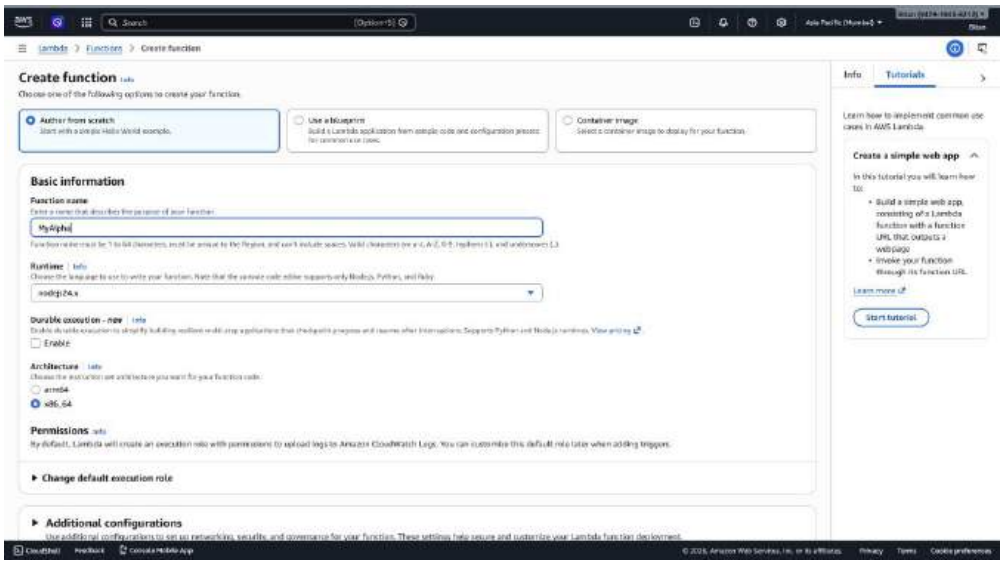
STEP 1-> Search for Lambda and click on it.



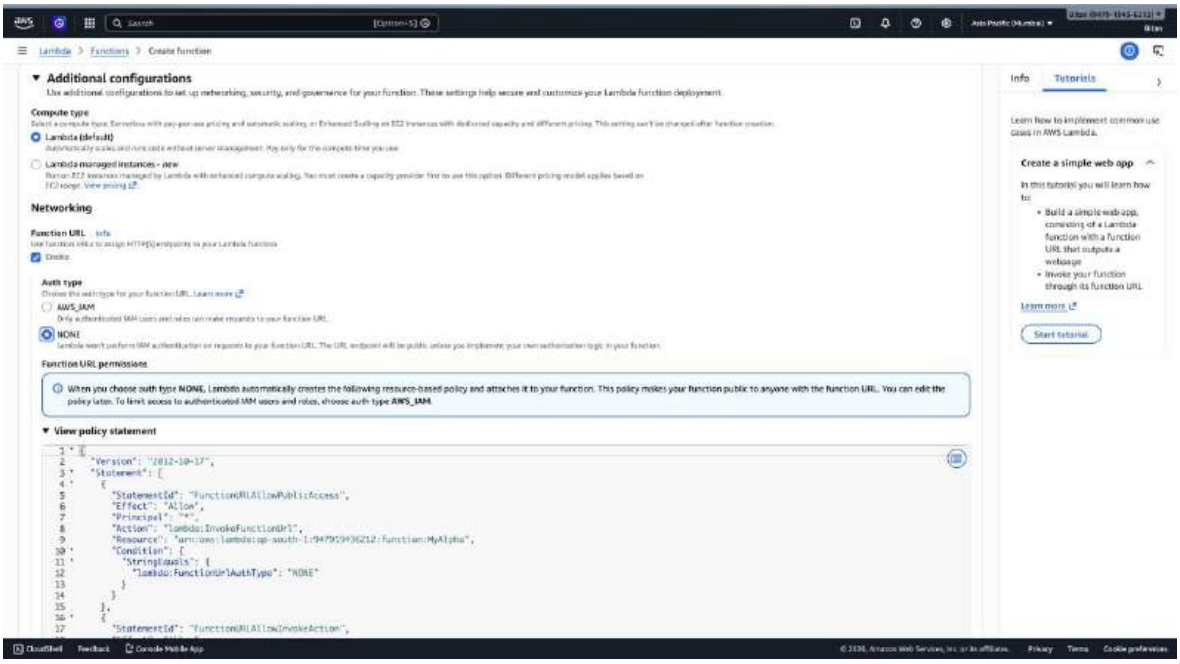
STEP 2-> Click on Create a function button.



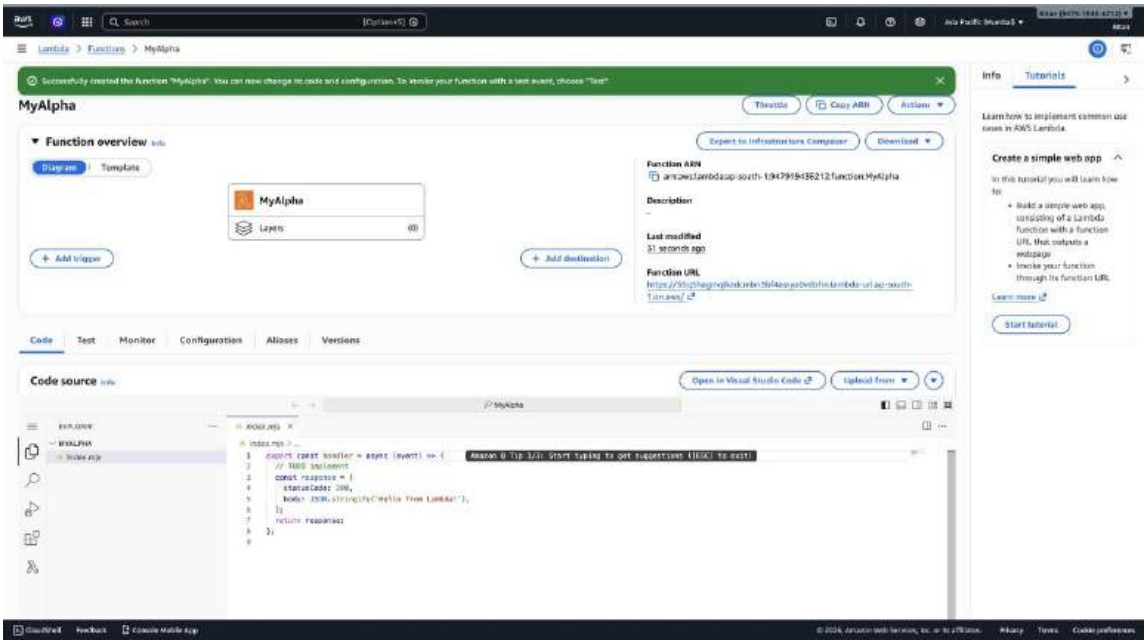
STEP 3-> Give a name to the function here "MyAlpha" and under runtime select “nodejs24.x”.



STEP 4-> Under the Advanced settings, check the Enable Function URL checkbox & under Auth type, select NONE. Then click on Create Function.



STEP 5-> The function has been created. Now copy the function URL and paste it on a new tab.



STEP 6-> The following output should be obtained indicating we have created a serverless computing service.